EXAMTOPICS
- Expert Verified, Online, **Free**.

Question #1 *Topic 1*

The terraform.tfstate file always matches your currently built infrastructure.

A. True

B. False

Suggested Answer: *B*

Reference:

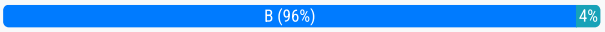
https://www.terraform.io/docs/language/state/index.html

This state is stored by default in a local file named "terraform.tfstate", but it can also be stored remotely, which works better in a team environment.

Terraform uses this local state to create plans and make changes to your infrastructure. Prior to any operation, Terraform does a refresh to update the state with the real infrastructure.

The primary purpose of Terraform state is to store bindings between objects in a remote system and resource instances declared in your configuration. When Terraform creates a remote object in response to a change of configuration, it will record the identity of that remote object against a particular resource instance, and then potentially update or delete that object in response to future configuration changes.

Community vote distribution



fabiomlop Highly Voted 2 years, 11 months ago

It doesn't necessarily always matches the current infrastructure, since one can manually change resources and therefore drift from the state configuration. Terraform has no way to know or track changes made outside of it.

upvoted 37 times

gargaditya 2 years, 2 months ago

Incorrect,your next terraform plan or terraform refresh will inform you that there is change(provided the deployment was initially done via terraform).

tfstate reflects values from actual infrastructure deployed, and it picks drift from actual tf files.

upvoted 1 times

Nunyabiznes 2 years, 1 month ago

Incorrect, question does not mention about "your so called " terraform plan or refresh commands

upvoted 6 times

0kwame Most Recent 1 month, 3 weeks ago

Selected Answer: B

No it doesn't.

This is because; terraform doesn't necessarily reconcile infrastructure automatically.

It is only after you run a terraform command like terraform plan does it reconcile your actual state with your desired state.

upvoted 2 times

eLJoudi 2 months, 3 weeks ago

Selected Answer: B

Answer is B :

Terraform has no way to know or track changes made outside of it.

upvoted 1 times

Soubi 4 months ago

Selected Answer: B

Answer is B

upvoted 1 times

didistars13 4 months, 1 week ago

Selected Answer: B

Agree with others that it is B

upvoted 1 times

Sergiuss95 4 months, 2 weeks ago

Selected Answer: B

ITS B. YOUR INFRA COULD BE CHANGE

upvoted 1 times

clouddevnoob 7 months, 1 week ago

Selected Answer: B

The correct answer is false because Terraform performs a refresh operation before any operation to update the state with the real infrastructure. This means the state file might not always perfectly match the current infrastructure if there are discrepancies between the state and the actual resources.

upvoted 1 times

erif 7 months, 3 weeks ago

Selected Answer: B

The terraform.tfstate file reflects Terraform's understanding of the infrastructure, but it may not always match the actual state of the infrastructure if changes have been made manually (outside of Terraform) or if the state file hasn't been updated after such changes. In those cases, the state file would be outdated or inaccurate.

upvoted 1 times

Deyemzy 7 months, 3 weeks ago

B. False

The terraform.tfstate file reflects the infrastructure as it was last known to Terraform, but changes made outside of Terraform (manually or through other tools) may not be reflected in the state file until a terraform refresh or terraform plan is run to reconcile the differences.

upvoted 1 times

hrajkuma 10 months, 2 weeks ago

Selected Answer: B

upvoted 1 times

Hanu1 1 year, 3 months ago

Selected Answer: A

The terraform.tfstate file maintains the state of your infrastructure as managed by Terraform. It keeps track of the resources that Terraform has provisioned and their current state.

upvoted 1 times

sccampos 1 year, 4 months ago

Selected Answer: B

B is the correct

upvoted 1 times

luxdolorosa 1 year, 6 months ago

Selected Answer: B

Of course, it's B. For example, when you perform tasks in the AWS web console, you cannot see the .tfstate file. Additionally, if we add that, you can update it to the latest state through 'tf refresh.

upvoted 1 times

maze_ 1 year, 8 months ago

Selected Answer: B

The actual provisioned resources may drift, the state represents a snapshot of their status from *the last time* Terraform was run.

upvoted 1 times

hajurbau 1 year, 8 months ago

Selected Answer: B

Question is a bit ambiguous.it says currently built, does it mean to say right after Terraform apply? Anyways feel like B is the answer

upvoted 1 times

Jayanth 1 year, 10 months ago

B. False

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: B

B is the correct answer

upvoted 1 times

Question #2 *Topic 1*

One remote backend configuration always maps to a single remote workspace.

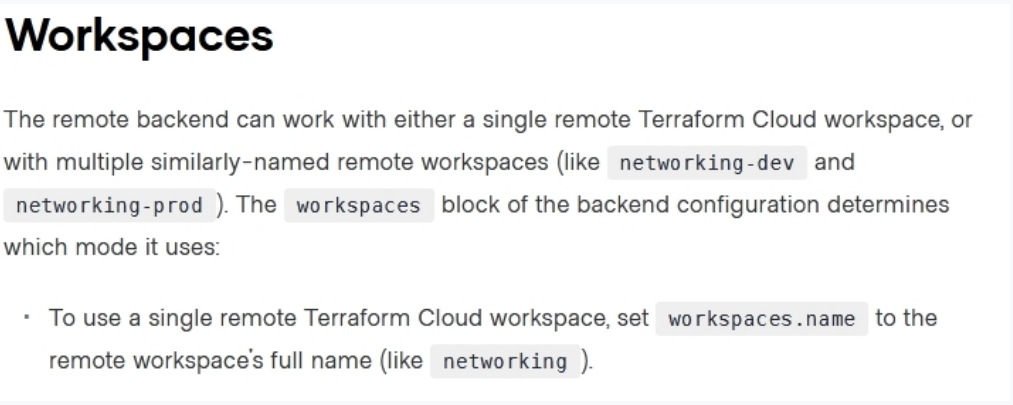
A. True

B. False

Suggested Answer: *A*

Reference:

https://www.terraform.io/docs/language/settings/backends/remote.html



Community vote distribution

B (86%) 14%

Spandrop Highly Voted 1 year, 5 months ago

Selected Answer: B

1 workspace -> 1 backend

1 backend -> multiple workspaces

upvoted 13 times

Zam88 Highly Voted 2 years, 11 months ago

Selected Answer: B

correct B

upvoted 9 times

eLJoudi Most Recent 2 months, 3 weeks ago

Selected Answer: B

The remote backend can work with either a single remote HCP Terraform workspace, or with multiple similarly-named remote workspaces (like networking-dev and networking-prod). The workspaces block of the backend configuration determines which mode it uses:

To use a single remote HCP Terraform workspace, set workspaces.name to the remote workspace's full name (like networking-prod).

upvoted 1 times

erif 7 months, 3 weeks ago

Selected Answer: B

A single remote backend configuration can support multiple remote workspaces. In Terraform, remote backends like Terraform Cloud or remote state storage solutions can manage multiple workspaces, each representing a different environment or set of infrastructure, all within the same backend configuration.

upvoted 1 times

SIAMIANJI 8 months ago

Selected Answer: B

The remote backend can work with either a single remote Terraform Cloud workspace, or with multiple similarly-named remote workspaces (like networking-dev and networking-prod). The workspaces block of the backend configuration determines which mode it uses:

To use a single remote Terraform Cloud workspace, set workspaces.name to the remote workspace's full name (like networking-prod).

To use multiple remote workspaces, set workspaces.prefix to a prefix used in all of the desired remote workspace names. For example, set prefix = "networking-" to use Terraform cloud workspaces with names like networking-dev and networking-prod. This is helpful when mapping multiple

Terraform CLI workspaces used in a single Terraform configuration to multiple Terraform Cloud workspaces.
https://developer.hashicorp.com/terraform/language/settings/backends/remote
upvoted 2 times

purvakdevops91 10 months, 1 week ago
Answer should be B False.

Workspaces (offical documentation --> https://developer.hashicorp.com/terraform/language/settings/backends/remote#workspaces)
The remote backend can work with either a single remote HCP Terraform workspace, or with multiple similarly-named remote workspaces (like networking-dev and networking-prod). The workspaces block of the backend configuration determines which mode it uses:

To use a single remote HCP Terraform workspace, set workspaces.name to the remote workspace's full name (like networking-prod).

To use multiple remote workspaces, set workspaces.prefix to a prefix used in all of the desired remote workspace names. For example, set prefix = "networking-" to use HCP Terraform workspaces with names like networking-dev and networking-prod. This is helpful when mapping multiple Terraform CLI workspaces used in a single Terraform configuration to multiple HCP Terraform workspaces.
upvoted 1 times

hrajkuma 10 months, 2 weeks ago
vote for B. False
upvoted 1 times

090200f 10 months, 3 weeks ago
The remote backend can work with either a single remote HCP Terraform workspace, or with multiple similarly-named remote workspaces, so B is correct answer
upvoted 2 times

liuyomz 1 year, 1 month ago
Selected Answer: B
B. prefix switching
upvoted 1 times

6957dbd 1 year, 2 months ago
Selected Answer: B
The prefix switch is key here:

prefix - (Optional) A prefix used in the names of one or more remote workspaces, all of which can be used with this configuration. The full workspace names are used in Terraform Cloud, and the short names (minus the prefix) are used on the command line for Terraform CLI workspaces. If omitted, only the default workspace can be used. This option conflicts with name.

To use multiple remote workspaces, set workspaces.prefix to a prefix used in all of the desired remote workspace names. For example, set prefix = "networking-" to use Terraform cloud workspaces with names like networking-dev and networking-prod. This is helpful when mapping multiple Terraform CLI workspaces used in a single Terraform configuration to multiple Terraform Cloud workspaces.

The backend configuration requires either name or prefix. Omitting both or setting both results in a configuration error.
upvoted 5 times

TigerInTheCloud 1 year, 5 months ago
Selected Answer: B
https://developer.hashicorp.com/terraform/language/settings/backends/remote
"To use multiple remote workspaces, set workspaces.prefix to a prefix used in all of the desired remote workspace names. For example, set prefix = "networking-" to use Terraform cloud workspaces with names like networking-dev and networking-prod. This is helpful when mapping multiple Terraform CLI workspaces used in a single Terraform configuration to multiple Terraform Cloud workspaces."
upvoted 4 times

luxdolorosa 1 year, 6 months ago
Selected Answer: B
I am working with GitLab, and I can create multiple instances within a single backend space
upvoted 3 times

NKSINGH 1 year, 6 months ago
Correct B
upvoted 1 times

esuarve 1 year, 8 months ago

Selected Answer: B

Your explanation actually states that a remote backend can work with multiple similarly-named remote workspaces (like networking-dev and networking-prod)

The answer A would not allow this

upvoted 2 times

arnabsinha4u 1 year, 8 months ago

Selected Answer: B

correct B

https://developer.hashicorp.com/terraform/language/settings/backends/remote#workspaces

upvoted 1 times

ledjo 1 year, 9 months ago

B.

A remote backend configuration is used to define where the Terraform state is stored, and it can be shared by multiple workspaces, each of which maintains its own separate state.

upvoted 2 times

paromanu007 1 year, 9 months ago

The remote backend can work with either a single remote Terraform Cloud workspace, or with multiple similarly-named remote workspaces (like networking-dev and networking-prod). The workspaces block of the backend configuration determines which mode it uses:

Correct B

upvoted 1 times

Question #3 *Topic 1*

How is the Terraform remote backend different than other state backends such as S3, Consul, etc.?

A. It can execute Terraform runs on dedicated infrastructure on premises or in Terraform Cloud

B. It doesn't show the output of a terraform apply locally

C. It is only available to paying customers

D. All of the above

Suggested Answer: *A*
If you and your team are using Terraform to manage meaningful infrastructure, we recommend using the remote backend with Terraform Cloud or Terraform
Enterprise.
Reference:
https://www.terraform.io/docs/language/settings/backends/index.html

Community vote distribution
A (100%)

softarts Highly Voted 3 years ago
I take A
upvoted 9 times

erif Most Recent 7 months, 3 weeks ago
Selected Answer: A
The correct answer is A. It can execute Terraform runs on dedicated infrastructure on premises or in Terraform Cloud.

The Terraform remote backend differs from other state backends like S3 or Consul in that it provides the ability to execute Terraform runs remotely on dedicated infrastructure, either in Terraform Cloud or on Terraform Enterprise (on-premises). This allows for remote plan and apply operations, unlike backends such as S3 or Consul, which are used solely for storing state files but do not handle the execution of Terraform runs.

Options B and C are incorrect:
B: The output of terraform apply can still be viewed locally even with a remote backend.
C: While certain features of Terraform Cloud require a paid subscription, the remote backend itself is available in both free and paid versions.
upvoted 2 times

hrajkuma 10 months, 2 weeks ago
vote for A
upvoted 1 times

pangchn 1 year ago
Selected Answer: A
The remote backend is unique among all other Terraform backends because it can both store state snapshots and execute operations for HCP Terraform's CLI-driven run workflow. It used to be called an "enhanced" backend
ref
https://developer.hashicorp.com/terraform/language/settings/backends/remote
upvoted 2 times

liuyomz 1 year, 1 month ago
Selected Answer: A
A for sure
upvoted 1 times

enook 1 year, 4 months ago
Selected Answer: A
I think A is correct.
upvoted 1 times

Jayanth 1 year, 10 months ago

A. It can execute Terraform runs on dedicated infrastructure on premises or in Terraform Cloud Most Voted

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: A

i take A

upvoted 1 times

wsyh 2 years ago

Selected Answer: A

Vote A

upvoted 1 times

meriadoc71 2 years, 1 month ago

From ChatGPT seems that the D is the correct answer.

upvoted 1 times

joyboy23 1 year, 10 months ago

Well, answer this: ChatGPT is 100% trusted source. a. False b. False

upvoted 14 times

ericitaquera 2 years, 3 months ago

Selected Answer: A

B and C are incorrect. Correct A

upvoted 1 times

Atta33 2 years, 6 months ago

A is correct

upvoted 2 times

stalos 2 years, 6 months ago

Selected Answer: A

It is A

upvoted 1 times

Anderson01 2 years, 6 months ago

why not C? should be multiple choices, right?

upvoted 1 times

30th 11 months ago

There is a limited free tier.

upvoted 1 times

Atta33 2 years, 7 months ago

Selected Answer: A

A is correct

upvoted 1 times

Janan 2 years, 9 months ago

Selected Answer: A

A is the answer

upvoted 2 times

Eltooth 2 years, 11 months ago

Selected Answer: A

A is correct answer.

upvoted 3 times

Question #4 *Topic 1*

What is the workflow for deploying new infrastructure with Terraform?

A. terraform plan to import the current infrastructure to the state file, make code changes, and terraform apply to update the infrastructure.

B. Write a Terraform configuration, run terraform show to view proposed changes, and terraform apply to create new infrastructure.

C. terraform import to import the current infrastructure to the state file, make code changes, and terraform apply to update the infrastructure.

D. Write a Terraform configuration, run terraform init, run terraform plan to view planned infrastructure changes, and terraform apply to create new infrastructure.

Suggested Answer: *D*

Community vote distribution

D (100%)

softarts Highly Voted 3 years ago

it is D

upvoted 17 times

erif Most Recent 7 months, 3 weeks ago

Selected Answer: D

Without any doubt.

upvoted 1 times

hrajkuma 10 months, 2 weeks ago

vote for D

upvoted 1 times

kambarami 1 year ago

Its DDDD yes!

upvoted 1 times

enook 1 year, 4 months ago

Selected Answer: D

it's DDDDDDDDD

upvoted 1 times

anon1234 1 year, 6 months ago

Selected Answer: D

DDDDDDDDD

upvoted 1 times

Jayanth 1 year, 10 months ago

D. Write a Terraform configuration, run terraform init, run terraform plan to view planned infrastructure changes, and terraform apply to create new infrastructure.

upvoted 1 times

tfdestroy 1 year, 10 months ago

D = Write a Terraform configuration, run terraform init, run terraform plan to view planned infrastructure changes, and terraform apply to create new infrastructure.

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: D

D write init "plan" apply

upvoted 1 times

matthieu_chappaz 1 year, 10 months ago

Selected Answer: D

D is the way

upvoted 1 times

wsyh 2 years ago

Selected Answer: D

Vote D

upvoted 2 times

Chandru1988 2 years ago

Definitely its D

upvoted 1 times

Power123 2 years, 2 months ago

D is correct

upvoted 1 times

Prasad28 2 years, 3 months ago

D is Correct

upvoted 1 times

InformationOverload 2 years, 4 months ago

Selected Answer: D

yes, D

upvoted 1 times

denismaggior8 2 years, 6 months ago

Selected Answer: D

D is the answer

upvoted 1 times

stalos 2 years, 6 months ago

Selected Answer: D

It is D

upvoted 1 times

Question #5 *Topic 1*

A provider configuration block is required in every Terraform configuration.

Example:

```
provider "provider_name" {
   ...
}
```

A. True

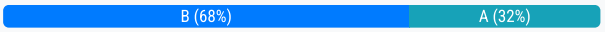
B. False

Suggested Answer: *A*

Reference:

https://github.com/hashicorp/terraform/issues/17928

Community vote distribution



pabrojo Highly Voted 2 years, 9 months ago

Selected Answer: B

It's B. From the official documentation:

Unlike many other objects in the Terraform language, a provider block may be omitted if its contents would otherwise be empty. Terraform assumes an empty default configuration for any provider that is not explicitly configured.

upvoted 47 times

gargaditya 2 years, 2 months ago

Still need atleast 1 provider in the terraform configuration-If I am deploying to Azure, I can skip the AWS provider.
But without the provider block containing details like authentication, how will the deployment actually happen?

upvoted 7 times

marcin3dm 2 years ago

AZ CLI can be used as an authentication source.

upvoted 1 times

Sekir 1 year, 6 months ago

This question is basically asking "is this formatting correct", in which case it is.

upvoted 1 times

softarts Highly Voted 3 years ago

Selected Answer: A

vote A

upvoted 36 times

karamkb Most Recent 3 months, 2 weeks ago

Selected Answer: B

A provider block is not required for all Terraform configurations – it is only required when you need to interact with an external system (e.g., AWS, Azure, Google Cloud).
If your Terraform configuration only uses local resources (like terraform local-exec provisioner or null_resource), then you do not need a provider block.
However, in most real-world Terraform projects, a provider is needed because Terraform is designed to manage infrastructure on external platforms.

Example Where No Provider is Needed
hcl
Copy
Edit
resource "null_resource" "example" {
provisioner "local-exec" {

command = "echo Hello, Terraform!"
}
}
This configuration does not require a provider because it does not interact with any external infrastructure.

False. A provider block is only required when using a provider to manage infrastructure. However, most Terraform configurations do require it.
upvoted 1 times

DataEngDP 5 months, 4 weeks ago

Selected Answer: B

It refers to a BLOCK so this {} that is not mandatory.
upvoted 1 times

erif 7 months, 3 weeks ago

Selected Answer: B

A provider configuration block is not required in every Terraform configuration. In some cases, Terraform can automatically discover providers, or the provider configuration can be inherited from other modules. Additionally, if the provider has been set up globally or within shared modules, it's possible to use the resources without explicitly declaring the provider block in every configuration file.
upvoted 1 times

foreverlearner 8 months ago

Selected Answer: B

I can see the confusion here. https://developer.hashicorp.com/terraform/language/providers/configuration says "all Terraform configurations must declare which providers they require so that Terraform can install and use them", which makes people think the answer is A/True.
The question says "A provider configuration block is required" (emphasis on BLOCK). The above link also says " a provider block may be omitted if its contents would otherwise be empty. Terraform assumes an empty default configuration for any provider that is not explicitly configured.", which confirms that the answer is B/FALSE, a provider BLOCK is not required, Terraform assumes a default one for you.
So a provider is required, a provider BLOCK is not
upvoted 5 times

Bere 8 months ago

Selected Answer: B

Here is an example without using the provider:

terraform {
required_version = ">= 0.13"
}

resource "null_resource" "example_provisioner" {
provisioner "local-exec" {
command = "echo 'Hello, Terraform!' > example.txt"
}
}

In this example, we are using a null_resource with a local-exec provisioner to run a simple shell command that creates a file named "example.txt" with the text "Hello, Terraform!".

Since this configuration doesn't involve any cloud provider resources or external backends, there is no need to include a provider configuration block. Terraform will execute the local-exec provisioner using the local machine's shell without requiring any cloud provider credentials or settings.
upvoted 6 times

samimshaikh 8 months ago

Selected Answer: B

False. A provider configuration block is not required in every Terraform configuration. It is only required when you are using a Terraform provider to interact with a specific infrastructure platform or service.

A provider configuration block typically includes details such as the provider's name, version, and any required authentication or connection information. If you're not using any provider in your Terraform configuration, you may not need a provider configuration block.

Here's an example of a provider configuration block for AWS:

provider "aws" {
region = "us-west-2"
access_key = "your-access-key"
secret_key = "your-secret-key"
}

This block specifies the AWS provider, sets the region, and provides access and secret keys for authentication. If you're not working with AWS or any other provider, you can have a Terraform configuration without a provider block.
upvoted 1 times

hrajkuma 10 months, 2 weeks ago
vote for B. False
A provider configuration block is not required in every Terraform configuration.
Provider configuration blocks are only required when you are using a particular provider to interact with a specific type of infrastructure resource.
If your configuration does not interact with any resources provided by external providers, then you do not need to include a provider configuration block. :)
upvoted 2 times

brundabanm 11 months, 3 weeks ago
Selected Answer: B
Unlike many other objects in the Terraform language, a provider block may be omitted if its contents would otherwise be empty. Terraform assumes an empty default configuration for any provider that is not explicitly configured.

https://developer.hashicorp.com/terraform/language/providers/configuration
upvoted 1 times

Molly1994 11 months, 3 weeks ago
the answer is false. Terraform requires provider. but it does not require specifically to define a provider block {}. Terraform could use the default providers. so the answer is B false.
upvoted 1 times

liuyomz 1 year, 1 month ago
Selected Answer: B
B. i got it wrong but its on docs
upvoted 2 times

Bedmed 1 year, 1 month ago
Each Terraform module must declare which providers it requires, so that Terraform can install and use them. Provider requirements are declared in a required_providers block.

A provider requirement consists of a local name, a source location, and a version constraint:
upvoted 1 times

vibzr2023 1 year, 2 months ago
If your Terraform configuration only includes resources from a single provider and doesn't require any special configuration for that provider, you might not need an explicit provider block. Terraform can automatically download and use the latest version of the required provider based on the resource types used.
Example without an explicit provider block
resource "aws_instance" "example" {
ami = "ami-0c55b159cbfafe1f0"
instance_type = "t2.micro"
}
In this example, Terraform can infer that the AWS provider is needed because of the aws_instance resource. It will use the default configuration for the AWS provider, assuming credentials and region are configured through environment variables or shared credentials files.
upvoted 2 times

vibzr2023 1 year, 2 months ago
Saying the Selected Answer:B
When You Need an Explicit Provider Block:
Example: In scenarios where you need to configure specific settings for a provider, like credentials, region, or aliases for managing resources in multiple regions or with different accounts, you will need an explicit provider block.

```
# Example with an explicit provider block
provider "aws" {
region = "us-west-2"
access_key = "my-access-key"
secret_key = "my-secret-key"
}

resource "aws_instance" "example" {
ami = "ami-0c55b159cbfafe1f0"
instance_type = "t2.micro"
}
```

In this example, the AWS provider is explicitly configured with a specific region and credentials. This is necessary if you're not relying on the default credential chain or if you want to set parameters that differ from the defaults.

upvoted 2 times

6957dbd 1 year, 2 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/providers/configuration

Additionally, all Terraform configurations must declare which providers they require so that Terraform can install and use them. The Provider Requirements page documents how to declare providers so Terraform can install them.

upvoted 2 times

AWSCurt 1 year, 3 months ago

Selected Answer: B

False. A provider configuration block is not required in every Terraform configuration. Provider configuration blocks are only required when you are using a particular provider to interact with a specific type of infrastructure resource. If your configuration does not interact with any resources provided by external providers, then you do not need to include a provider configuration block.

upvoted 2 times

MukeshRattan 1 year, 4 months ago

Selected Answer: B

False.

A provider configuration block is not required in every Terraform configuration. It depends on the specifics of your configuration and the resources you are managing.

In Terraform, provider configuration blocks are used to specify the details of the infrastructure provider you want to use, such as AWS, Azure, Google Cloud, etc. If you are managing resources that don't require a specific provider, or if your configuration relies on provider-agnostic resources, you may not need a provider configuration block.

upvoted 1 times

Question #6 *Topic 1*

You run a local-exec provisioner in a null resource called null_resource.run_script and realize that you need to rerun the script.
Which of the following commands would you use first?

A. terraform taint null_resource.run_script

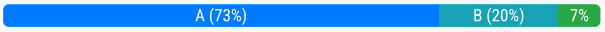
B. terraform apply -target=null_resource.run_script

C. terraform validate null_resource.run_script

D. terraform plan -target=null_resource.run_script

Suggested Answer: *A*

Community vote distribution



alirasouli Highly Voted 2 years, 6 months ago

As discussed, the `taint` command used to be the right choice; however, it is deprecated. The right answer is:
terraform apply -replace="null_resource.run_script"

Reference:
https://developer.hashicorp.com/terraform/cli/commands/taint
upvoted 39 times

Sproket Highly Voted 2 years, 11 months ago

Selected Answer: A

You are all correct that taint has been deprecated and replaced with -replace. But neither D nor any other option here uses the -replace command. Therefore option A is the only valid option given these choices.
upvoted 18 times

Pietjeplukgeluk 2 years, 3 months ago

Not so sure, read Arrash his comment below. If a provisioner fails, it would be marked as tained by default without any user interaction required. This only leaves the apply as a required step....
upvoted 3 times

sam700 Most Recent 2 months, 1 week ago

Selected Answer: B

null_resource - there is nothing getting tracked in the state file, so nothing to taint. Only way to re-run is to apply!
upvoted 1 times

eLJoudi 2 months, 3 weeks ago

Selected Answer: A

As discussed, the `taint` command used to be the right choice; however, it is deprecated. The right answer is:
terraform apply -replace="null_resource.run_script"
upvoted 1 times

YesPlease 3 months, 3 weeks ago

Selected Answer: D

"taint" is deprecated, so this is WRONG

"validate" does not have any options to target files and checks all config files for syntax, so this is WRONG

So that leaves APPLY and PLAN....the question is asking which will you run *FIRST*. Terraform BEST-PRACTICES always wants you to run PLAN before APPLY so that you know exactly what changes will happen and this ensures that you understand and approve the intended changes.
upvoted 1 times

palashpal 4 months, 1 week ago

Selected Answer: A

As the resource is already present in the state file, so terraform will display message No changes. Your infrastructure matches the configuration.

But with taint options, terraform will recreate the resource in next apply as shown below.

terraform taint null_resource.run_script
Resource instance null_resource.run_script has been marked as tainted.
terraform apply
null_resource.run_script: Refreshing state... [id=1726701550426365360]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

null_resource.run_script is tainted, so must be replaced
-/+ resource "null_resource" "run_script" {
~ id = "1726701550426365360" -> (known after apply)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Hence option A is correct
upvoted 2 times

Nunyabiznes 8 months ago
The correct answer is A. terraform taint null_resource.run_script. This command marks the null resource as tainted, which means that Terraform considers the resource to be out-of-date and will recreate it during the next terraform apply run. When Terraform recreates the null resource, it will also rerun the local-exec provisioner.

Option B, terraform apply -target=null_resource.run_script, would work, but it is overkill because it would apply all the resources in the configuration, not just the null resource with the local-exec provisioner.

Option C, terraform validate null_resource.run_script, only checks the syntax of the configuration, and does not affect the state of the resource.

Option D, terraform plan -target=null_resource.run_script, generates a plan for applying changes to the configuration, but does not apply those changes, so it would not rerun the local-exec provisioner.
upvoted 3 times

prabdarq 6 months ago
Hi @Nunyabiznes.... You are right, The correct answer is A. But I would like to clarify the reason as to why Option B wont work. Terraform apply will get executed, But no changes will be made, as they were already run. As we need to rerun the script taint/replace has to be used. Kindly let me know if my understanding is correct
upvoted 1 times

Bere 8 months ago
Selected Answer: A
1) Create main.tf:
terraform {
required_version = ">= 0.13"
}

resource "null_resource" "run_script" {
provisioner "local-exec" {
command = "echo 'Hello, Terraform!' > example.txt"
}
}

2) terraform init
3) terraform apply
4) Modify the content of the example.txt file manually or delete it.
5) terraform taint null_resource.run_script

6) terraform apply
The local-exec provisioner should now run again, and the example.txt file will be recreated with the content specified in the command of the local-exec provisioner block.

As described here:
https://developer.hashicorp.com/terraform/cli/commands/taint
This command is deprecated. For Terraform v0.15.2 and later, we recommend using the -replace option with terraform apply instead

But there is no option that uses the -replace command, so option A as described in my step 5 is the only valid option.
upvoted 2 times

reynaldiekoz 9 months, 1 week ago
Selected Answer: A
im sureee
upvoted 1 times

hrajkuma 10 months, 2 weeks ago
it is option -> A for sure
upvoted 1 times

090200f 10 months, 3 weeks ago
Warning: This command is deprecated. For Terraform v0.15.2 and later, we recommend using the -replace option with terraform apply instead, So { $ terraform apply -replace="aws_instance.example[0]" } but there is no like this apply -replace option in the mentioned options so answer is A : terraform taint .....
upvoted 1 times

enklau 10 months, 3 weeks ago
thats right
upvoted 1 times

gofavad926 1 year, 8 months ago
Selected Answer: A
A. However taint command is deprecated now and we should use terraform apply -destroy
upvoted 2 times

ledjo 1 year, 9 months ago
Selected Answer: A
terraform apply -replace is missing here, therefore the only valid answer, even if deprecated as a command, is the first one A.
upvoted 2 times

BaburTurk 1 year, 9 months ago
Selected Answer: B
B. terraform apply -target=null_resource.run_script

Running the terraform apply -target=null_resource.run_script command will specifically target the null_resource.run_script resource and execute its provisioner again. This is useful when you want to rerun the local-exec provisioner without affecting other resources.
upvoted 3 times

Jayanth 1 year, 10 months ago
A. terraform taint null_resource.run_script
upvoted 1 times

Busi57 1 year, 10 months ago
Selected Answer: A
I think A
upvoted 1 times

Question #7 *Topic 1*

Which provisioner invokes a process on the resource created by Terraform?

A. remote-exec

B. null-exec

C. local-exec

D. file

Suggested Answer: *A*
The remote-exec provisioner invokes a script on a remote resource after it is created.
Reference:
https://www.terraform.io/docs/language/resources/provisioners/remote-exec.html

Community vote distribution
A (100%)

Cloud9er Highly Voted 3 years ago
No. Answer is remote-exec. because...The local-exec provisioner invokes a local executable after a resource is created. This invokes a process on the machine running Terraform, not on the resource. (this is from the terraform web site)
upvoted 26 times

Eltooth Highly Voted 2 years, 10 months ago
Selected Answer: A
A is correct answer : remote-exec.

"The local-exec provisioner invokes a local executable after a resource is created. This invokes a process on the machine running Terraform, not on the resource." https://www.terraform.io/language/resources/provisioners/local-exec

"The remote-exec provisioner invokes a script on a remote resource after it is created."
https://www.terraform.io/language/resources/provisioners/remote-exec
upvoted 5 times

muammer32 Most Recent 5 months, 2 weeks ago
Selected Answer: A
The remote-exec provisioner is the one that invokes a process on the resource created by Terraform, running commands or scripts on the remote machine after it has been provisioned.
upvoted 2 times

Bere 8 months ago
Selected Answer: A
The remote-exec provisioner invokes a script on a remote resource created by Terraform. It connects to the resource using SSH or WinRM and runs the provided inline or script commands. Here's an example of how you might use it:

resource "aws_instance" "example" {
ami = "ami-0c94855ba95c574c8"
instance_type = "t2.micro"

provisioner "remote-exec" {
inline = [
"echo Hello, World! > /home/ubuntu/hello",
"chmod +x /home/ubuntu/hello",
]
}
}

There's no null-exec provisioner in Terraform.

The local-exec provisioner invokes a local executable after a resource is created. It runs on the machine where Terraform is being executed.

The file provisioner is used to copy files or directories from the machine executing Terraform to the newly created resource.

upvoted 3 times

090200f 10 months, 3 weeks ago

A is correct ans : "process on the resource" is key point here. so remote-exec. The local-exec provisioner invokes a local executable after a resource is created. This invokes a process on the machine running Terraform, not on the resource.

upvoted 1 times

Jayanth 1 year, 10 months ago

A. remote-exec

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: A

remote-exec -> A

upvoted 1 times

Mandeeps468 1 year, 11 months ago

A - correct answer : remote-exec

upvoted 1 times

wsyh 2 years ago

Selected Answer: A

Vote A. local-exec provisioner is execute on the local machine.

upvoted 1 times

Chandru1988 2 years ago

Definitely Remote-exec

upvoted 1 times

Power123 2 years, 2 months ago

A is correct

upvoted 1 times

Mal_8 2 years, 3 months ago

Selected Answer: A

A is the correct answer : remote-exec

upvoted 3 times

Zam88 2 years, 11 months ago

The remote-exec provisioner invokes a script on a remote resource after it is created. This can be used to run a configuration management tool, bootstrap into a cluster, etc. To invoke a local process, see the local-exec provisioner instead. The remote-exec provisioner requires a connection and supports both ssh and winrm.

remote-exec correct

upvoted 1 times

subhala 2 years, 11 months ago

I think it is remote-exec, the question is asking about on the resource created by terraform. local-exec runs on the machine where we run terraform.

upvoted 1 times

softarts 3 years ago

I take A remote-exec.

upvoted 3 times

Eniras 3 years ago

The local-exec provisioner invokes a local executable after a resource is created. This invokes a process on the machine running Terraform, not on the resource. See the remote-exec provisioner to run commands on the resource.
https://www.terraform.io/language/resources/provisioners/local-exec

upvoted 3 times

calebvar 3 years, 1 month ago

Isn't local exec also correct?

upvoted 1 times

Ahmad_Terraform 2 years, 11 months ago

Eniras explained well about the Local Exec,

upvoted 1 times

Question #8 *Topic 1*

Which of the following is not true of Terraform providers?

A. Providers can be written by individuals

B. Providers can be maintained by a community of users

C. Some providers are maintained by HashiCorp

D. Major cloud vendors and non-cloud vendors can write, maintain, or collaborate on Terraform providers

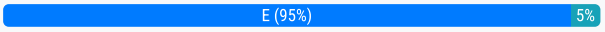
E. None of the above

Suggested Answer: *D*

Reference:

https://jayendrapatil.com/terraform-cheat-sheet/#Terraform_Read_and_write_configuration

Community vote distribution



icedog Highly Voted 3 years ago

Selected Answer: E

E. is the correct answer.

It's obvious that D. is incorrect, of course cloud and non-cloud vendors can collaborate on creating providers just like anyone else

upvoted 38 times

nharaz Highly Voted 8 months ago

Selected Answer: E

E. None of the above

All of the statements are true of Terraform providers.

A. Providers can be written by individuals - Any person or organization can develop and distribute a Terraform provider, allowing them to expand Terraform's capabilities to manage resources that it previously could not.

B. Providers can be maintained by a community of users - Many Terraform providers are open source projects, and the development and maintenance of these providers can be collaborative efforts between multiple individuals and organizations.

C. Some providers are maintained by HashiCorp - HashiCorp, the creators of Terraform, maintain a number of official providers that cover popular infrastructure providers such as AWS, Google Cloud, and Microsoft Azure.

D. Major cloud vendors and non-cloud vendors can write, maintain, or collaborate on Terraform providers - Providers can be developed and maintained by cloud vendors, non-cloud vendors, or a combination of both, to expand Terraform's capabilities and support for different types of infrastructure.

upvoted 13 times

samimshaikh Most Recent 8 months ago

Selected Answer: E

E. None of the above

All of the statements (A, B, C, and D) are true of Terraform providers:

A. Providers can be written by individuals: True. Terraform allows individuals to create custom providers for their specific needs.

B. Providers can be maintained by a community of users: True. Terraform has a community-driven model, and many providers are developed and maintained by the community.

C. Some providers are maintained by HashiCorp: True. HashiCorp, the company behind Terraform, maintains and supports certain official providers.

D. Major cloud vendors and non-cloud vendors can write, maintain, or collaborate on Terraform providers: True. Terraform is designed to work with a

variety of providers, including major cloud providers, on-premises solutions, and other services. Providers can be developed and maintained by the vendors themselves or in collaboration with the community.

upvoted 1 times

starvinboarder 9 months, 1 week ago

Selected Answer: E

Definitely E

upvoted 1 times

hrajkuma 10 months, 2 weeks ago

E is the right answer

upvoted 1 times

Narendra_Sharma 1 year ago

Selected Answer: E

all statement are true

upvoted 1 times

pangchn 1 year ago

Selected Answer: E

https://developer.hashicorp.com/terraform/registry/providers

Official

Partner

Community

upvoted 1 times

galvarado89 1 year, 3 months ago

Selected Answer: D

cloud and non-cloud vendors can collaborate on creating providers just like anyone else

upvoted 1 times

guicane 1 year, 6 months ago

Selected Answer: E

it's obviously E

upvoted 1 times

qs5dq64s5dq5 1 year, 7 months ago

Selected Answer: E

D is obviously true

upvoted 1 times

gofavad926 1 year, 8 months ago

Selected Answer: E

E. None of the above

upvoted 1 times

rajat06 1 year, 8 months ago

E is the correct None of the above

upvoted 1 times

ledjo 1 year, 9 months ago

Selected Answer: E

E. is the correct answer.

upvoted 1 times

Ashutosh_96 1 year, 10 months ago

E, is correct

upvoted 1 times

Jayanth 1 year, 10 months ago

E. None of the above

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: E

all true -> E

upvoted 1 times

Shane_C 1 year, 10 months ago

Selected Answer: E

E is clearly the correct answer

upvoted 1 times

Question #9 *Topic 1*

What command does Terraform require the first time you run it within a configuration directory?

A. terraform import

B. terraform init

C. terraform plan

D. terraform workspace

Suggested Answer: *B*

terraform init command is used to initialize a working directory containing Terraform configuration files.

Reference:

https://www.terraform.io/docs/cli/commands/init.html

Community vote distribution

B (100%)

softarts Highly Voted 3 years ago

it is B

upvoted 8 times

hrajkuma Most Recent 10 months, 2 weeks ago

B is the answer, terraform init

upvoted 1 times

090200f 10 months, 3 weeks ago

B. terraform init is correct answer it initializes all resource config need to current directory (path)

upvoted 1 times

Jayanth 1 year, 10 months ago

B. terraform init

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: B

init b

upvoted 1 times

wsyh 2 years ago

Selected Answer: B

Vote B

upvoted 2 times

Power123 2 years, 2 months ago

B is correct terraform init

upvoted 2 times

chanelno1 2 years, 6 months ago

Selected Answer: B

init downloads things from git so it can be used when you run terraform apply.

upvoted 2 times

Eltooth 2 years, 11 months ago

Selected Answer: B

B is correct answer.

upvoted 2 times

Ahmad_Terraform 2 years, 11 months ago

B is correct , init starts the backend and prepare terraform

upvoted 2 times

javibadillo 3 years ago

Selected Answer: B

Agree with B

upvoted 4 times

Question #10 *Topic 1*

You have deployed a new webapp with a public IP address on a cloud provider. However, you did not create any outputs for your code.
What is the best method to quickly find the IP address of the resource you deployed?

A. Run terraform output ip_address to view the result

B. In a new folder, use the terraform_remote_state data source to load in the state file, then write an output for each resource that you find the state file

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address

D. Run terraform destroy then terraform apply and look for the IP address in stdout

Suggested Answer: *A*
Reference:
https://www.terraform.io/docs/cli/commands/output.html

Community vote distribution
C (100%)

coco10k Highly Voted 3 years, 1 month ago

Selected Answer: C

You can find the info in the state.
not A because you don't have to outputs defined.
upvoted 32 times

fasaleAkshay Most Recent 8 months ago

Selected Answer: C

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including the public IP address.
upvoted 2 times

Serg853377 8 months ago

Answer C
upvoted 1 times

SilentMilli 8 months ago

Selected Answer: C

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address. This option allows you to easily find the public IP address of the resource you deployed, without having to destroy and recreate it. The terraform state list command lists all of the resources that have been added to the state, while the terraform state show command displays the attributes of a specific resource, including its public IP address.
upvoted 1 times

nharaz 8 months ago

Selected Answer: C

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address

If you have not created any outputs in your Terraform code, you can still find the public IP address of the resource by using the terraform state commands. First, you can run terraform state list to see a list of all the resources that Terraform is managing in the state file. Look for the resource with a type that matches your webapp, such as "aws_instance" or "google_compute_instance".

Once you have found the name of the resource, you can run terraform state show <resource_name> to view all of the attributes of the resource, including the public IP address. For example, if the name of the resource is "webapp", you would run terraform state show webapp. This will display all of the attributes of the resource, including the public IP address.
upvoted 2 times

camps 8 months ago

Selected Answer: C

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address is the best method to quickly find the IP address of the resource you deployed.

terraform state list will show you the name of all the resources currently tracked in your Terraform state. You can then use terraform state show followed by the resource name to see all the details of the resource, including any attributes like the public IP address.

upvoted 2 times

samimshaikh 8 months ago

Selected Answer: C

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes, including the public IP address.

Explanation:

Not Option A (terraform output ip_address) assumes that there is an output variable named ip_address in your Terraform configuration, but since you mentioned that you did not create any outputs, this option may not be applicable.

upvoted 2 times

hrajkuma 10 months, 2 weeks ago

vote for option C

upvoted 1 times

brundabanm 11 months, 2 weeks ago

C is the correct answer.

upvoted 1 times

Prat8 1 year, 6 months ago

Is it C or A ?

upvoted 1 times

luke404 1 year, 6 months ago

Selected Answer: C

The question does not talk about any output defined so you can't expect one

upvoted 1 times

gofavad926 1 year, 8 months ago

Selected Answer: C

C for sure

upvoted 1 times

otakuinside 1 year, 8 months ago

Selected Answer: C

A cannot be. You don't have any output declared, so there is no default output or something like that where you could find the ip address. On the other hand, terraform state list, and then show specific, contains the state and this includes the ip address

upvoted 2 times

grogudev 1 year, 8 months ago

Selected Answer: C

C I choose too

upvoted 1 times

Jayanth 1 year, 10 months ago

C. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: C

C i choose

upvoted 1 times

cdechery 1 year, 10 months ago

Selected Answer: C

If there is no output defined for the resource, the "output" command will not show anything. Correct is C.

upvoted 2 times

Question #11 *Topic 1*

Which of the following is not a key principle of infrastructure as code?

A. Versioned infrastructure

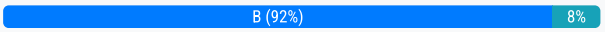
B. Golden images

C. Idempotence

D. Self-describing infrastructure

Suggested Answer: *ABD*
Reference:
https://docs.microsoft.com/en-us/azure/devops/learn/what-is-infrastructure-as-code#:~:text=Idempotence%20is%20a%20principle%20of,of%20the%20environment's%20starting%20state
.

Community vote distribution



ItaloVinodi Highly Voted 3 years ago
Sorry but here is asking which is NOT a key principle. I think the answer is B, golden Image
upvoted 29 times

alexik96 Highly Voted 2 years, 7 months ago
B is correct. Even if you're not sure, you can eliminate the others
A - is a principle (think of the case when you store the state in an S3 bucket and it gets versioned)
B - is NOT a principle (where in terraform documentation is mentioned anything related to images)
C - it is a principle by definition of what terraform does
D - is a principle as you use declarative language -> infrastructure describes itself
upvoted 12 times

nharaz Most Recent 8 months ago
Selected Answer: B
B. Golden images
"Golden images" refers to a specific method of deployment where a pre-configured image of an operating system or application is stored and used as a base for all new deployments. This method is not a key principle of Infrastructure as Code, but it is a common method of deployment in traditional IT environments.
Infrastructure as Code is based on the following key principles:
A. Versioned infrastructure - The infrastructure is treated as code and is version controlled, allowing for auditing, rollback, and collaboration.
C. Idempotence - The infrastructure is provisioned in a repeatable and predictable manner, making it possible to run the provisioning scripts multiple times without creating additional resources or causing changes to existing resources.
D. Self-describing infrastructure - The code used to provision the infrastructure is human-readable and self-documented, making it easier to understand and maintain over time.
upvoted 6 times

Bere 8 months ago
Selected Answer: B
Golden images are not a key principle of infrastructure as code (IaC). Golden images refer to a pre-configured and standardized system image that contains all the necessary software and settings to run a particular application or service. While they are commonly used in traditional server management and virtualization, they are not inherently related to infrastructure as code.
upvoted 3 times

Badii 8 months ago
Selected Answer: B
Golden images are not a key principle of infrastructure as code (IaC). Instead, golden images are a concept related to traditional server provisioning and management.
upvoted 1 times

SilentH 8 months ago

Selected Answer: B

I vote B because not only is a "golden image" not a key principle but, I would argue, in conflict with the idea behind IaC.

upvoted 1 times

samimshaikh 8 months ago

B. Golden images: While golden images are a common concept in traditional infrastructure management, they are not a key principle of infrastructure as code. IaC focuses on defining and managing infrastructure through code rather than relying on pre-configured images.

upvoted 2 times

vibzr2023 8 months ago

Answer:B

B. Golden Images: While golden images (pre-configured images for server deployments) are used in infrastructure management, they are not considered a key principle of IaC. IaC focuses on defining infrastructure through code that can be versioned, reused, and managed as part of application development workflows. Relying solely on golden images does not fully embrace the dynamism and flexibility offered by IaC practices, as it leans more towards a traditional, immutable infrastructure approach.

upvoted 2 times

fasaleAkshay 8 months ago

Selected Answer: B

B. Golden images

Explanation:
Golden images refer to pre-configured, machine images (such as VM snapshots) that contain all necessary software and configurations. While golden images are used in infrastructure provisioning, they are not a key principle of Infrastructure as Code (IaC), especially in Terraform.

upvoted 1 times

fasaleAkshay 8 months, 1 week ago

B. Golden images

Explanation:
Golden images refer to pre-configured, machine images (such as VM snapshots) that contain all necessary software and configurations. While golden images are used in infrastructure provisioning, they are not a key principle of Infrastructure as Code (IaC), especially in Terraform.

upvoted 1 times

starkonbullet 1 year ago

Selected Answer: B

The other 3 options can be associated with the principle.

upvoted 1 times

gofavad926 1 year, 8 months ago

Selected Answer: B

Agree with B

upvoted 1 times

Jayanth 1 year, 10 months ago

B. Golden images

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: B

B Golden image

upvoted 1 times

foreverlearner 1 year, 10 months ago

Selected Answer: B

Golden image is not part of IaC.
For more info see https://developer.hashicorp.com/well-architected-framework/operational-excellence/operational-excellence-tao

upvoted 1 times

Shane_C 1 year, 10 months ago

Selected Answer: B

B is the only answer here that is not a key principle

upvoted 1 times

connecttozee 2 years, 1 month ago

agree B

upvoted 1 times

Question #12 *Topic 1*

Terraform variables and outputs that set the "description" argument will store that description in the state file.

A. True

B. False

Suggested Answer: *A*

Community vote distribution

B (100%)

SilentMilli Highly Voted 2 years, 2 months ago

Selected Answer: B

Terraform variables and outputs that set the "description" argument are not stored in the state file. The "description" argument is used to provide a human-readable description of the variable or output, and it is intended to be used as documentation for other users of the Terraform code.

The state file is used to store the current state of the infrastructure managed by Terraform, including the values of variables and outputs. However, the "description" argument is not part of the state file, and it is not used by Terraform to manage the infrastructure. Instead, it is only used as metadata to describe the variable or output.

upvoted 21 times

antivrillee Highly Voted 2 years, 8 months ago

Selected Answer: B

Answer is B. Descriptions aren't stored in the state file.

upvoted 12 times

Bere Most Recent 8 months ago

Selected Answer: B

1) Create the following Terraform configuration:

```
provider "aws" {
region = "eu-west-1"
}

variable "instance_type" {
description = "The type of instance to create"
default = "t2.micro"
}

resource "aws_instance" "example" {
ami = "ami-096800910c1b781ba" # Replace this with an appropriate AMI ID for your region
instance_type = var.instance_type
}

output "instance_id" {
description = "The ID of the created instance"
value = aws_instance.example.id
}
```

2) After you run terraform apply on this configuration, the state file (terraform.tfstate) will include the value of the instance_type variable ("t2.micro") and the value of the instance_id output (which would be the ID of the created AWS instance),
but it would not include the descriptions of either the variable or the output.

upvoted 9 times

starkonbullet 1 year ago

Selected Answer: B

Validated; it would just show type, value in the terraform state.

upvoted 2 times

mamtak_2008 1 year, 3 months ago

A. True

Terraform variables and outputs that set the "description" argument will indeed store that description in the state file. This allows for better documentation and understanding of the purpose of variables and outputs within the Terraform configuration. However, it's important to note that sensitive information should not be included in descriptions as the state file is stored in plain text and could potentially expose sensitive information if not handled properly.

upvoted 1 times

Lashe 9 months, 3 weeks ago

not true. Terraform state file only stores the actual infrastructure resources and their state

upvoted 1 times

samimshaikh 1 year, 4 months ago

Selected Answer: B

Terraform state files are intended to store the actual configuration and state of the infrastructure, focusing on information that is necessary for Terraform to understand and manage the resources. Descriptions, being informational and not affecting the infrastructure's configuration or behavior, are typically not included in the state file to keep it concise and focused on essential details for Terraform's management.

upvoted 3 times

guicane 1 year, 6 months ago

Selected Answer: B

I don't understand how such a simple question can show the wrong answer by the website.

upvoted 1 times

luke404 1 year, 6 months ago

Selected Answer: B

simply check the output of "terraform state pull" or see inside the tfstate file if using local state, there are no descriptions stored in the state

upvoted 1 times

Mimi666 1 year, 7 months ago

Selected Answer: B

variables are not stored in state

upvoted 1 times

Badii 1 year, 8 months ago

Selected Answer: B

Answer is B

upvoted 1 times

Jayanth 1 year, 10 months ago

B. False

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: B

i think B

upvoted 1 times

Shane_C 1 year, 10 months ago

Selected Answer: B

B is correct

upvoted 1 times

SIAMIANJI 1 year, 11 months ago

Selected Answer: B

The correct answer is B.

upvoted 2 times

Power123 2 years, 2 months ago

Correct Ans is B

upvoted 1 times

Mal_8 2 years, 3 months ago

Selected Answer: B

The answer is B. Descriptions aren't stored in the state file.

upvoted 1 times

Only5 2 years, 4 months ago

Answer is A, I can see description in state file on my environment

upvoted 1 times

guicane 1 year, 6 months ago

no you cannot

upvoted 1 times

Question #13 *Topic 1*

What is the provider for this fictitious resource?

```
resource "aws_vpc" "main" {
    name = "test"
}
```

A. vpc

B. main

C. aws

D. test

Suggested Answer: *C*

Reference:

https://docs.aws.amazon.com/cloudformation-cli/latest/userguide/resource-types.html

Community vote distribution

C (100%)

softarts Highly Voted 3 years ago

C (aws)

upvoted 12 times

Nunyabiznes Highly Voted 8 months ago

In Terraform, a provider is responsible for managing resources of a particular type or a specific cloud service. In the given example, the resource type is aws_vpc which belongs to the AWS cloud platform, so the provider is specified as aws.

Therefore, the correct code snippet with the provider block would be:

provider "aws" {

region = "us-west-2"

}

resource "aws_vpc" "main" {

name = "test"

}

upvoted 9 times

Zatoichi2 Most Recent 2 weeks, 1 day ago

Selected Answer: C

Even if C is correct, the provided is example is not very correct. The aws_vpc resource does not have a name attribute. Instead, you assign a name by tagging the resource.

resource "aws_vpc" "main" {

cidr_block = "10.0.0.0/16"

tags = {

Name = "test"

}

}

upvoted 1 times

eLJoudi 2 months, 3 weeks ago

Selected Answer: C

aws obviously

upvoted 1 times

gargaditya 8 months ago

Selected Answer: C

Each resource block describes one or more infrastructure objects, such as virtual networks, compute instances, or higher-level components such as DNS records.

A resource block declares a resource of a given type ("aws_instance") with a given local name ("web"). The name is used to refer to this resource from elsewhere in the same Terraform module, but has no significance outside that module's scope.

The resource type and name together serve as an identifier for a given resource and so must be unique within a module.

Terraform usually automatically determines which provider to use based on a resource type's name. (By convention, resource type names start with their provider's preferred local name.)

eg.
resource "aws_instance" "web" {
ami = "ami-a1b2c3d4"
instance_type = "t2.micro"
}
upvoted 2 times

Molly1994 12 months ago
Resource block defines all resources from its own provider. The questions is asking the provider. So answer definitely is AWS
upvoted 1 times

enook 1 year, 4 months ago
Answer is C.
aws_vpc is "resource type", and main is "resource name".
upvoted 1 times

gofavad926 1 year, 8 months ago
Selected Answer: C
C for sure
upvoted 1 times

Jayanth 1 year, 10 months ago
C. aws
upvoted 1 times

Busi57 1 year, 10 months ago
Selected Answer: C
i choose C
upvoted 1 times

gspb 2 years ago
Selected Answer: C
C - aws
Here, aws is the provider and aws_vpc is the resource_type
upvoted 1 times

Saifwsm 2 years, 1 month ago
C - AWS
upvoted 1 times

E_aws 2 years, 4 months ago
actually I see no error in the syntax https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpc
upvoted 1 times

E_aws 2 years, 4 months ago
sorry, misunderstood.. C is the correct answer
upvoted 1 times

terraform 2 years, 4 months ago
Selected Answer: C
C , AWS
upvoted 1 times

Sm1ley 2 years, 6 months ago

Selected Answer: C

C, AWS

upvoted 1 times

alifie 2 years, 7 months ago

Selected Answer: C

c, aws

upvoted 1 times

Eltooth 2 years, 11 months ago

Selected Answer: C

C is correct answer.

upvoted 1 times

Question #14 *Topic 1*

If you manually destroy infrastructure, what is the best practice reflecting this change in Terraform?

A. Run terraform refresh

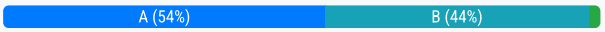
B. It will happen automatically

C. Manually update the state fire

D. Run terraform import

Suggested Answer: *B*

Community vote distribution



tf181 Highly Voted 3 years, 1 month ago

How could change reflect in terraform without doing anything?
I think A is right answer. any thoughts?
upvoted 35 times

G__ Highly Voted 3 years, 1 month ago

Wouldn't it be A?
upvoted 12 times

MeneJohn Most Recent 6 days, 7 hours ago

Selected Answer: A

The terraform refresh command reads the current settings from all managed remote objects and updates the Terraform state to match. This command is deprecated. Instead, add the -refresh-only flag to terraform apply and terraform plan commands.
This does not modify your real remote objects, but it modifies the Terraform state.
upvoted 1 times

eLJoudi 2 months, 3 weeks ago

Selected Answer: A

"Manually destroying infrastructure" means you changed infrastructure outside of Terraform.
What is the best practice reflecting this change in your* Terraform configuration*?
Correct Anwser is [A] : Run terraform refresh

(Answer B is wrong: Because a manual change in remote infrastructure is not automatically parsed into your Terraform configuration)
upvoted 1 times

d759fb3 4 months, 1 week ago

Selected Answer: A

When infrastructure is manually destroyed (outside of Terraform):
 Terraform's state file becomes out of sync with reality

 terraform refresh updates the state file to match the real infrastructure
 No changes are made to your infrastructure, only to the state file

Why it's not "C":
- Dangerous practice
- Easy to corrupt state file
- Can cause inconsistencies
- Not recommended
upvoted 1 times

didistars13 4 months, 1 week ago

Selected Answer: A

Most probably A
upvoted 1 times

Labman32 6 months, 2 weeks ago

A. is correct

but now as a best practice the correct way should be running terraform apply with the -refresh-only, this way you will get a prompt when it compares and see's that the deleted infrastructure is missing and will show you during your plan.

upvoted 1 times

erif 7 months, 3 weeks ago

Selected Answer: C

The best practice when infrastructure is manually destroyed is to manually update the state file (option C). This means the need to update Terraform's state file to reflect that the resource has been destroyed outside of Terraform.

Specifically:

You can use the terraform state rm command to remove the resource from the state file, as it no longer exists.
Alternatively, you could manually edit the state file, but this is risky and generally not recommended.
Other options like terraform refresh will only update the state based on the existing infrastructure, but if the resource was destroyed outside of Terraform, it will still appear in the state file unless manually removed.

upvoted 1 times

nss373 8 months ago

Answer is C. We have to manually remove the the resource from state file.

upvoted 1 times

akkam89 8 months ago

Selected Answer: B

```The terraform refresh command reads the current settings from all managed remote objects and updates the Terraform state to match.

Warning: This command is deprecated, because its default behavior is unsafe if you have misconfigured credentials for any of your providers. See below for more information and recommended alternatives.```

upvoted 2 times

**ravk2321** 8 months ago

**Selected Answer: B**

"The Terraform refresh command reads the current settings from all managed remote objects and updates the Terraform state to match."
Warning: This command is deprecated because its default behaviour is unsafe if you have misconfigured credentials for any of your providers. See below for more information and recommended alternatives.

This won't modify your real remote objects, but it will modify the Terraform state.

"You shouldn't typically need to use this command, because Terraform automatically performs the same refreshing actions as a part of creating a plan in both the Terraform plan and Terraform apply commands. This command is here primarily for backward compatibility, but we don't recommend using it because it provides no opportunity to review the effects of the operation before updating the state."
Link:https://developer.hashicorp.com/terraform/cli/commands/refresh

upvoted 2 times

**fasaleAkshay** 8 months, 1 week ago

**Selected Answer: A**

terraform refresh is used to reconcile the Terraform state with the real-world infrastructure. If you manually destroy infrastructure, the Terraform state will still show that the resource exists. Running terraform refresh updates the state to reflect the current state of the infrastructure, marking resources that were manually destroyed as no longer existing.
B. It will happen automatically: No, Terraform does not automatically update its state when infrastructure is changed outside of Terraform. You need to run a command like terraform refresh.

upvoted 1 times

**Mattt** 8 months, 2 weeks ago

**Selected Answer: B**

he terraform refresh command : You shouldn't typically need to use this command, because Terraform automatically performs the same refreshing actions as a part of creating a plan in both the terraform plan and terraform apply commands.

upvoted 1 times

**bora4motion** 9 months, 1 week ago
```

Selected Answer: A

I'd say it's A but the question is weird and I got tricked. By manually I thought it'd be "terraform destroy" but it seems like manually mean go into the cloud and delete stuff, therefore you need to refresh the file.

upvoted 1 times

enklau 10 months, 1 week ago

A is correct, but now as a best practice the correct way should be running terraform apply with the -refresh-only flag

upvoted 1 times

callmegio 11 months, 2 weeks ago

A is the correct answer.

Terraform says

Warning: This command is deprecated, because its default behavior is unsafe if you have misconfigured credentials for any of your providers.

You shouldn't typically need to use this command, because Terraform automatically performs the same refreshing actions as a part of creating a plan in both the terraform plan and terraform apply commands.

upvoted 1 times

starkonbullet 1 year ago

Selected Answer: A

The command had been deprecated already. Use -refresh-only option along with terraform plan or terraform apply for the same operation.

upvoted 1 times

Question #15 *Topic 1*

What is not processed when running a terraform refresh?

A. State file

B. Configuration file

C. Credentials

D. Cloud provider

Suggested Answer: *CD*

Reference:

https://www.terraform.io/docs/cli/commands/refresh.html

Community vote distribution

B (98%)

wangchung Highly Voted 3 years ago

Selected Answer: B

A. State file - its updated during the refresh

B. Configuration file -

C. Credentials - required to get onto the cloud

D. Cloud provider - required to carry out the refresh of whats in the cloud vs whats in state.

upvoted 39 times

cloudcuckooland Highly Voted 3 years ago

Selected Answer: B

the answer has to be B, as per docs, it's checking remote objects and update the state file, it needs to process the creds to connect, the cloud provider to check objects and the state to update.

"The terraform refresh command reads the current settings from all managed remote objects and updates the Terraform state to match."

upvoted 13 times

fasaleAkshay Most Recent 8 months, 1 week ago

Selected Answer: B

When running terraform refresh, Terraform updates the state file by querying the cloud provider using the appropriate credentials to check the actual state of the infrastructure. However, the configuration file (the .tf files) is not processed during a refresh. Terraform is only concerned with updating the state to match the real-world infrastructure, not applying changes from the configuration.

upvoted 2 times

shubhamlonkar 10 months, 4 weeks ago

Selected Answer: B

Refresh command updates the state file but not the terraform configuration file

upvoted 1 times

Molly1994 12 months ago

Refresh command will update the remote state file to match the current configuration. So configuration file won't change anything. State file and credentials and cloud providers will associate with the refresh action

upvoted 1 times

ravk2321 1 year, 2 months ago

The question is outdated as this command is deprecated by Terraform, and they also recommend against using it because it provides no opportunity to review the effects of the operation before updating the state.

upvoted 1 times

Jayanth 1 year, 10 months ago

B. Configuration file

upvoted 2 times

Busi57 1 year, 10 months ago

Selected Answer: B

i choose B

upvoted 1 times

Busi57 1 year, 10 months ago

I choose B

upvoted 1 times

cdechery 1 year, 10 months ago

Selected Answer: B

Of course it is B. Credentials are used during refresh.

upvoted 1 times

Ibrahim2023 1 year, 10 months ago

The answer is C. Credentials.

When you run terraform refresh, Terraform will read the state file and the configuration file, but it will not read the credentials. This is because the credentials are used to connect to the cloud provider, and Terraform does not need to connect to the cloud provider to refresh the state file.

upvoted 3 times

HMthyl 1 year, 10 months ago

Refresh is about ensuring the state file reflects real world objects: it's not bringing the state file in line with your configuration, it's bringing it into line with the real thing

upvoted 2 times

connecttozee 2 years, 1 month ago

Agree with B. Only confiugration file not processed for provisioning infra. Provider & credential need login to infra & state file will overwrite new config current infra

upvoted 2 times

SilentMilli 2 years, 2 months ago

Selected Answer: B

he configuration file is not processed during terraform refresh. The configuration file is only used during the terraform apply or terraform plan commands to determine the desired state of the infrastructure. terraform refresh does not make any changes to the infrastructure, it only updates the state file to reflect the current state of the resources in the cloud provider.

upvoted 3 times

Mal_8 2 years, 3 months ago

Selected Answer: B

The answer is B.

upvoted 1 times

InformationOverload 2 years, 4 months ago

Selected Answer: B

The terraform refresh command does not create new resources or modify existing resources. It only updates the state to reflect the current configuration. If you want to create new resources or modify existing resources, you need to use the terraform apply command.

upvoted 1 times

Fatoch 2 years, 7 months ago

there is should be two answers, then B and another one?

upvoted 1 times

yogishrb2020 2 years, 7 months ago

Selected Answer: BD

State file and if credentials stored in the state file gets updated.

I chose B and D

upvoted 1 times

Question #16 *Topic 1*

What information does the public Terraform Module Registry automatically expose about published modules?

A. Required input variables

B. Optional inputs variables and default values

C. Outputs

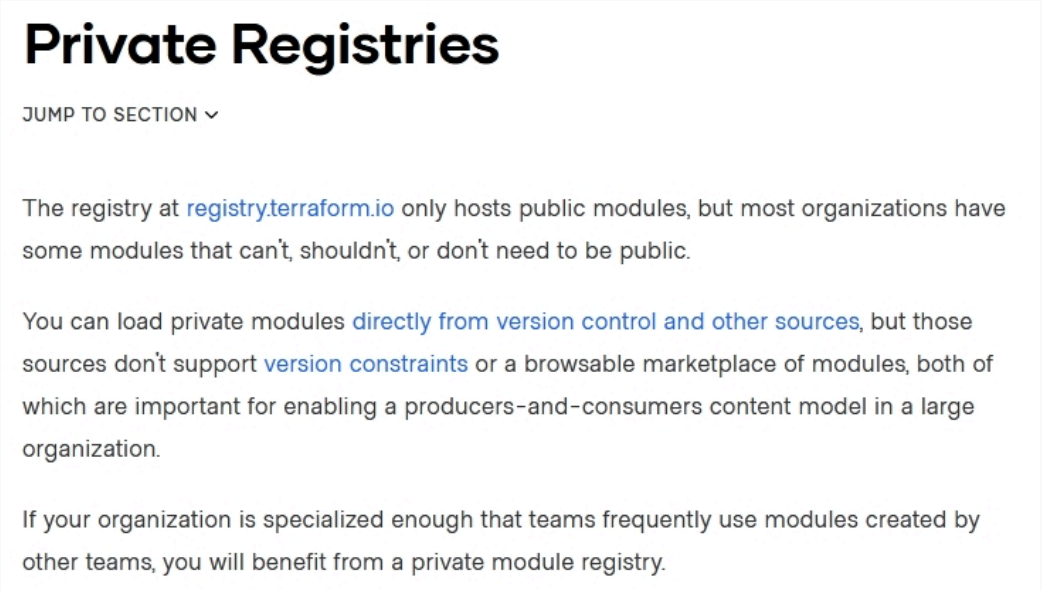
D. All of the above

E. None of the above

Suggested Answer: *E*

Reference:

https://www.terraform.io/docs/registry/modules/publish.html



Community vote distribution

D (98%)

Egger1992 Highly Voted 3 years ago

Selected Answer: D

According to the documentation provided, D is correct.

Extract from documentation:

"The registry extracts information about the module from the module's source. The module name, provider, documentation, inputs/outputs, and dependencies are all parsed and available via the UI or API, as well as the same information for any submodules or examples in the module's source repository."

upvoted 28 times

doodlearmy Most Recent 10 months, 3 weeks ago

Selected Answer: D

All, According to the documentation provided, D is correct.

upvoted 1 times

udenaro 11 months, 1 week ago

Selected Answer: D

the correct answer is D

upvoted 1 times

starkonbullet 1 year ago

Selected Answer: D

Exposed variables, optional variables & outputs are further given to use them in our custom module.

upvoted 1 times

barkadog 1 year, 4 months ago

Either the person choosing the correct answers on this website was drunk, or this website deliberately wants us to fail.
upvoted 4 times

enook 1 year, 4 months ago
Selected Answer: D
DDDDDDD
upvoted 1 times

ktritesh 1 year, 8 months ago
The question is asking for public, not private, lol D is correct
upvoted 1 times

shivamkrsingh 1 year, 8 months ago
I believe the D is the correct answer. The answer marked as correct is for the private modules not for the public modules. It might be a typo or something.
upvoted 1 times

HarshPatel198 1 year, 9 months ago
Selected Answer: D
All of above
upvoted 3 times

Jayanth 1 year, 10 months ago
D. All of the above
upvoted 1 times

Busi57 1 year, 10 months ago
Selected Answer: D
D teams
upvoted 1 times

Shane_C 1 year, 10 months ago
Selected Answer: D
D is correct
upvoted 1 times

IK912 1 year, 11 months ago
Answer Definitely D
upvoted 1 times

Gaboh97 1 year, 11 months ago
Selected Answer: D
D. All of the above
upvoted 1 times

gspb 2 years ago
Selected Answer: D
D. All of the above
upvoted 1 times

camps 2 years, 1 month ago
Selected Answer: D
D. All of the above.

The public Terraform Module Registry automatically exposes information about published modules, including required input variables, optional input variables and their default values, and outputs. This information is displayed on the module's page in the registry, which can be accessed by anyone.
upvoted 4 times

Power123 2 years, 1 month ago
D is correct
upvoted 1 times

Question #17 *Topic 1*

If a module uses a local values, you can expose that value with a terraform output.

A. True

B. False

Suggested Answer: *A*
Output values are like function return values.
Reference:
https://www.terraform.io/docs/language/values/locals.html
https://www.terraform.io/docs/language/values/outputs.html

Community vote distribution
A (100%)

Bere Highly Voted 1 year, 10 months ago

Selected Answer: A
Code example:

variable "input" {
description = "An input variable"
default = "Hello"
}

locals {
local_value = "${var.input}, World!"
}

output "greeting" {
description = "A greeting message"
value = local.local_value
}

Outputs:

greeting = "Hello, World!"
upvoted 13 times

SilentMilli Highly Voted 8 months ago

Selected Answer: A
If a module uses local values, those values can be exposed using the "terraform output" directive. By using the terraform output directive, you can define an output for a module that provides information about the local values used within the module. This information can then be consumed by other parts of the Terraform configuration, allowing you to make use of the values that have been set within the module. To expose a local value, you would define an output block in the module, specifying the name and value of the output, and then reference the output in other parts of your Terraform configuration using the "terraform output" data source.
upvoted 5 times

gargaditya Most Recent 8 months ago

Selected Answer: A
1. Output values make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use. Output values are similar to return values in programming languages.

2.Basic Syntax:
https://drive.google.com/file/d/1td5DcyZ-7iHL3Xhpwep4deEEfKbC7DkT/view?usp=sharing

3.Using Output Values with Modules(logical concept described in diagram below):
https://drive.google.com/file/d/1tbYWsCSev063ibz9Pv9poqViOY6fAAsG/view?usp=sharing

Basically,the question is narrowed down to exposing locals from the root module(can be any resource attribute in general).
upvoted 1 times

HarshPatel198 1 year, 9 months ago

Selected Answer: A

Bere is right
upvoted 1 times

Jayanth 1 year, 10 months ago

A. True
upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: A

Teams A
upvoted 1 times

VSMu 1 year, 10 months ago

Answer A.
Output the local variable as an output value
output "instance_public_ip" {
value = local.instance_public_ip
}
upvoted 1 times

SammySunny 1 year, 11 months ago

Why not B? The documentation says "A local value can only be accessed in expressions within the module where it was declared"
upvoted 1 times

HMthyl 1 year, 10 months ago

Confirmed using a project, A is true. That definition threw me too - but if you explicitly output the local value it can be accessed as expected
upvoted 3 times

IK912 1 year, 11 months ago

Answer Definitely A
upvoted 1 times

karendavtyan 2 years ago

Selected Answer: A

A. true
upvoted 1 times

Tanacet 2 years, 3 months ago

Selected Answer: A

It is A
upvoted 1 times

ssanjayt 2 years, 4 months ago

Selected Answer: A

A is correct
upvoted 1 times

jj22222 2 years, 7 months ago

Selected Answer: A

A is correct
upvoted 1 times

hectordj 2 years, 7 months ago

Selected Answer: A

A is correct
upvoted 1 times

poojithakadiyala 2 years, 8 months ago

A is correct
upvoted 1 times

daaww 2 years, 8 months ago

Selected Answer: A

A is correct

upvoted 3 times

Question #18 *Topic 1*

You should store secret data in the same version control repository as your Terraform configuration.

A. True

B. False

Suggested Answer: *B*

Reference:

https://blog.gruntwork.io/a-comprehensive-guide-to-managing-secrets-in-your-terraform-code-1d586955ace1

Community vote distribution

B (100%)

SilentMilli 8 months ago

Selected Answer: B

It is generally considered insecure to store secret data, such as passwords, API keys, and other sensitive information, in the same version control repository as your Terraform configuration. This is because version control repositories are often publicly accessible, and if sensitive information is stored in the repository it can be easily accessed by unauthorized individuals. Additionally, version control repositories typically have a history of all changes made to files, so even if sensitive information is deleted at a later point, it can still be retrieved from the repository history. To properly secure secret data, it is recommended to store it in a secure and encrypted format, such as in a secure vault or by using a tool specifically designed for storing secrets.

upvoted 4 times

Molly1994 12 months ago

You should store secrets in vaults. And retrieve the secrets when used in your terraform application. So the answer is B false.

upvoted 3 times

Jayanth 1 year, 10 months ago

B. False

upvoted 1 times

tfdestroy 1 year, 10 months ago

Selected Answer: B

B: It is not recommended to store secret data such as passwords, API keys, or other sensitive information in your version control system (VCS). This is a general best practice in software development, not just with Terraform.

upvoted 2 times

Busi57 1 year, 10 months ago

Selected Answer: B

it's no secure B

upvoted 1 times

IK912 1 year, 11 months ago

Answer Definitely B

upvoted 1 times

karendavtyan 2 years ago

Selected Answer: B

B. False

upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: B

B. False

Storing secret data in the same version control repository as Terraform configuration is not recommended as it increases the risk of exposing sensitive information. Version control systems are designed for sharing and collaboration, which means that they may not have robust security mechanisms to protect against unauthorized access.

upvoted 1 times

Nunyabiznes 2 years, 2 months ago

I know it is not recommended, but technically you really "could" store it. SO, this is kinda confusing since it is not asking for Best Security Practices, but just want to check if it is possible or not

upvoted 1 times

Multi_Cloud 2 years, 2 months ago

If you re-read the question it says, You "should" - which means the answer should be B - false.

Had it been "could/might" your argument would have been correct.

upvoted 2 times

Eltooth 2 years, 10 months ago

Selected Answer: B

B is correct answer : false.

upvoted 4 times

fabiomlop 2 years, 11 months ago

Selected Answer: B

If you store secrets in a version control system, they will be exposed in plain text to anyone with access to the repository.

upvoted 4 times

Eniras 3 years ago

It's false. Your secrets are stored in a dedicated secret store that enforces encryption and strict access control.

upvoted 3 times

Question #19 *Topic 1*

Which of the following is not a valid string function in Terraform?

A. split

B. join

C. slice

D. chomp

Suggested Answer: *D*

Reference:

https://www.terraform.io/docs/language/functions/chomp.html

`chomp` removes newline characters at the end of a string.

This can be useful if, for example, the string was read from a file that has a newline character at the end.

Examples

```
> chomp("hello\n")
hello
> chomp("hello\r\n")
hello
> chomp("hello\n\n")
hello
```

Copy

Community vote distribution

C (93%) 7%

testtaker_1885 Highly Voted 3 years, 1 month ago

answer is c

slice not listed in official docs

https://www.terraform.io/language/functions

upvoted 30 times

bp339 3 years ago

https://www.terraform.io/language/functions/slice

upvoted 3 times

Shane_C 1 year, 10 months ago

The question asked for a function that is not a string function.

Slice is a collection function, all other answers are string functions.

Answer is C

upvoted 7 times

doumx 2 years, 12 months ago

string function

upvoted 2 times

samkobadev 2 years, 3 months ago

so the correct answer is?

upvoted 1 times

Shane_C 1 year, 10 months ago

Answer is C

upvoted 2 times

coco10k Highly Voted 3 years, 1 month ago

Selected Answer: C

slice is not in string function list.

upvoted 10 times

b890yc Most Recent 1 month, 3 weeks ago

Selected Answer: C

slice extracts some consecutive elements from within a list.

https://developer.hashicorp.com/terraform/language/functions/slice

upvoted 1 times

gswar 3 months, 1 week ago

Selected Answer: D

terraform does support the other functions.

upvoted 1 times

samirsetc 3 months, 3 weeks ago

Selected Answer: C

slice is a collections function

upvoted 1 times

Selvan1978 8 months ago

Selected Answer: D

A. split: The split function is used to split a string into a list of substrings based on a specified delimiter.

B. join: The join function is used to concatenate a list of strings into a single string with a specified delimiter.

C. slice: The slice function is used to extract a range of elements from a list or tuple.

D. chomp: The chomp function is not a valid string function in Terraform. It is not part of Terraform's standard function set for working with strings.

So, the correct answer is D. chomp.

upvoted 6 times

MisterR0B0T 8 months ago

ANSWER IS D. Chomp is not valid

Here are some examples of how to use these string functions in Terraform:

```
# Split a string into a list of strings, based on a comma delimiter
variable "tags" {
type = string
}

output "tags_list" {
value = split(var.tags, ",")
}

# Join a list of strings into a single string, using a space separator
variable "regions" {
type = list(string)
}

output "regions_string" {
value = join(var.regions, " ")
}
```

```
# Extract a substring from a string, starting at position 5 and ending at position 10
variable "hostname" {
type = string
}

output "hostname_suffix" {
value = slice(var.hostname, 5, 10)
}
```

upvoted 1 times

mohamed1999 1 year, 5 months ago

that is not correct :
chomp removes newline characters at the end of a string.

This can be useful if, for example, the string was read from a file that has a newline character at the end.

https://developer.hashicorp.com/terraform/language/functions/chomp

upvoted 2 times

Molly1994 12 months ago

answer is C slice. slice is collection function. all other three are string function

upvoted 1 times

Venki_dev 1 year, 1 month ago

Selected Answer: C

slice comes under collective function and not string functions. answer is C

upvoted 1 times

samimshaikh 1 year, 4 months ago

Selected Answer: C

slice is collection function and chomp is string function so SLICE is correct answer as not a valid string function "C"

upvoted 1 times

MisterR0B0T 1 year, 6 months ago

chomp is not a valid string function in Terraform. The other three options are all valid string functions:

split splits a string into a list of strings, based on a delimiter.
join joins a list of strings into a single string, using a separator.
slice extracts a substring from a string, starting at a specified position and ending at a specified position or length.

upvoted 1 times

mohamed1999 1 year, 5 months ago

that is not correct :
chomp removes newline characters at the end of a string.

This can be useful if, for example, the string was read from a file that has a newline character at the end.

https://developer.hashicorp.com/terraform/language/functions/chomp

upvoted 1 times

imkhan 1 year, 7 months ago

The correct answer is C. Slice function is not listed in the official documentation of terraform string functions
https://developer.hashicorp.com/terraform/language/functions
slice is a collection function that extracts some consecutive elements from within a list.
https://developer.hashicorp.com/terraform/language/functions/slice

If you are interested to learn about the Terraform string functions you can visit the following link https://bitsto.cloud/terraform-string-functions/

upvoted 1 times

kudakk 1 year, 9 months ago

Selected Answer: C

Slice Function is not part of the string function. Others like join, split, chomp are part of it.

upvoted 2 times

Jayanth 1 year, 10 months ago

C. slice

upvoted 1 times

Busi57 1 year, 10 months ago

Selected Answer: D

i hesitate between c and D , i choose D

upvoted 1 times

Venki_dev 1 year, 1 month ago

slice comes under collective function and not string functions. answer is C

upvoted 1 times

cdechery 1 year, 10 months ago

Slice is not a string function. Why are so many answers here just plain wrong with 99% and sometimes 100% of votes on the correct answer?

upvoted 1 times

Shane_C 1 year, 10 months ago

Selected Answer: C

The question asked for a function that is not a string function.

Slice is a collection function, all other answers are string functions.

Answer is C

upvoted 1 times

Question #20 *Topic 1*

You have provisioned some virtual machines (VMs) on Google Cloud Platform (GCP) using the gcloud command line tool. However, you are standardizing with
Terraform and want to manage these VMs using Terraform instead.
What are the two things you must do to achieve this? (Choose two.)

A. Provision new VMs using Terraform with the same VM names

B. Use the terraform import command for the existing VMs

C. Write Terraform configuration for the existing VMs

D. Run the terraform import-gcp command

Suggested Answer: *BD*
The terraform import command is used to import existing infrastructure.
Import existing Google Cloud resources into Terraform with Terraformer.
Reference:
https://www.terraform.io/docs/cli/import/usage.html
https://cloud.google.com/docs/terraform

Community vote distribution
BC (100%)

tipzzz Highly Voted 3 years, 1 month ago
BC for sure
upvoted 33 times

fabiomlop Highly Voted 2 years, 11 months ago
Selected Answer: BC
You should create the equivalent configuration first, and then run import to load it on the state file.
upvoted 18 times

prabdarq 6 months ago
@fabiomlop... Is it so or either one of the process is sufficient? Kindly clarify
upvoted 1 times

sumaju 3 months, 3 weeks ago
Terraform is yet to find a solution to import the configuration of the existing resources. So you need to write the configuration and terraform import will update the state file only. BC is correct.
upvoted 1 times

SilentMilli Most Recent 8 months ago
Selected Answer: BC
To manage the VMs that were provisioned using the gcloud command line tool using Terraform, you must first use the "terraform import" command to import the existing VMs into Terraform's state file. This allows Terraform to recognize the VMs and manage them as part of your infrastructure.

After importing the VMs, you must then write Terraform configuration for the existing VMs. This includes defining the resources for the VMs and specifying the necessary configuration options, such as the instance type and image, as well as any other desired settings. By doing this, you can manage the VMs using Terraform, which provides a single, unified tool for provisioning, configuring, and managing your infrastructure.
upvoted 1 times

doodlearmy 10 months, 3 weeks ago
Selected Answer: BC
there is no 'import-gcp' in terraform official doc
upvoted 1 times

kingfighers 1 year, 1 month ago
there is no 'import-gcp' in terraform official doc
upvoted 2 times

enook 1 year, 4 months ago

Selected Answer: BC

BCBCBCBCBCBC

upvoted 2 times

Tyler2023 1 year, 7 months ago

There are two ways to manage existing resource using Terraform
You can use 'terraform import' or you can use the import block

So answers are:
B. Use the terraform import command for the existing VMs Most Voted
C. Write Terraform configuration for the existing VMs

https://developer.hashicorp.com/terraform/cli/import/usage
https://developer.hashicorp.com/terraform/language/import

upvoted 3 times

Jayanth 1 year, 10 months ago

B. Use the terraform import command for the existing VMs
C. Write Terraform configuration for the existing VMs

upvoted 2 times

Busi57 1 year, 10 months ago

Selected Answer: BC

i choose BC

upvoted 1 times

Busi57 1 year, 10 months ago

i choose BC

upvoted 2 times

Shane_C 1 year, 10 months ago

Selected Answer: BC

BC are the correct answers here

upvoted 1 times

milan92stankovic 1 year, 11 months ago

Selected Answer: BC

Write configuration and import.

upvoted 1 times

Ni33 2 years ago

Selected Answer: BC

B&C are correct

upvoted 2 times

karendavtyan 2 years ago

Selected Answer: BC

B.True
C. True

upvoted 1 times

AzRNoob 2 years, 1 month ago

BC are the correct options:

B. Use the terraform import command for the existing VMs. This command allows you to import existing infrastructure into your Terraform state file so that Terraform can manage it. You will need to provide the resource type and name, along with any required attributes, for each VM you want to import.

C. Write Terraform configuration for the existing VMs. Once the VMs have been imported into the Terraform state file, you will need to write configuration code that describes the desired state of the VMs. This will typically involve creating a new Terraform module or modifying an existing one to include the imported resources.

Option A is incorrect because provisioning new VMs with the same names would create duplicate resources and could cause conflicts with the

existing VMs.

Option D is also incorrect because there is no terraform import-gcp command in Terraform. The correct command for importing GCP resources is simply terraform import.

upvoted 10 times

connecttozee 2 years, 1 month ago

BC is correct

https://developer.hashicorp.com/terraform/tutorials/state/state-import

upvoted 1 times

Power123 2 years, 1 month ago

B,C - first write the configuration and then import

upvoted 1 times

Question #21 *Topic 1*

You have recently started a new job at a retailer as an engineer. As part of this new role, you have been tasked with evaluating multiple outages that occurred during peak shopping time during the holiday season. Your investigation found that the team is manually deploying new compute instances and configuring each compute instance manually. This has led to inconsistent configuration between each compute instance.
How would you solve this using infrastructure as code?

A. Implement a ticketing workflow that makes engineers submit a ticket before manually provisioning and configuring a resource

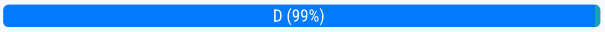
B. Implement a checklist that engineers can follow when configuring compute instances

C. Replace the compute instance type with a larger version to reduce the number of required deployments

D. Implement a provisioning pipeline that deploys infrastructure configurations committed to your version control system following code reviews

Suggested Answer: *A*

Community vote distribution



tipzzz Highly Voted 3 years, 1 month ago

Selected Answer: D

D for sure

upvoted 28 times

Eniras 3 years ago

Yes Answer is D

upvoted 3 times

fabiomlop Highly Voted 2 years, 11 months ago

Selected Answer: D

Obviously D... who picks those answers, FFS...

upvoted 17 times

valli_ Most Recent 1 month, 2 weeks ago

Selected Answer: D

it should definitely be D

upvoted 1 times

Mattt 8 months, 2 weeks ago

Selected Answer: D

D is correct

upvoted 1 times

SilentH 1 year, 1 month ago

Selected Answer: D

C'mon, the answer is obviously D. I'm only on question 21 and have seen at least 5 obviously wrong answers. It's like ExamTopics is trying to fail people.

upvoted 6 times

qs5dq64s5dq5 1 year, 7 months ago

Selected Answer: D

D for sure

upvoted 1 times

shivamkrsingh 1 year, 8 months ago

I think the answer is D. The person who stored the A as correct answer might not want to write terraform code or CICD.😁

upvoted 7 times

pradeepukg 1 year, 9 months ago

Selected Answer: D

D is correct answer

upvoted 1 times

jitusonawane89 1 year, 10 months ago

D is correct

upvoted 1 times

wheelan 1 year, 10 months ago

Selected Answer: D

D - in line with IaC

upvoted 1 times

Shane_C 1 year, 10 months ago

Selected Answer: D

D is the only thing close to a correct answer here

upvoted 1 times

DuboisNicolasDuclair 1 year, 11 months ago

Selected Answer: A

Because the topic said *a sample main.tf* it means that we can have many types of files like *.tf

upvoted 1 times

HMthyl 1 year, 10 months ago

This implies the question has completely changed over the last 3 weeks, huh?

upvoted 2 times

IK912 1 year, 11 months ago

Definitely D

upvoted 1 times

Ni33 2 years ago

Selected Answer: D

D is the correct answer

upvoted 1 times

karendavtyan 2 years ago

Selected Answer: D

D. for sure

upvoted 1 times

rookie1025 2 years, 1 month ago

man... obviously the answer is D.

So many incorrect answers are there in all the dumps. With this website being so inconsistent with their answers they sure are shameless giving us captcha challenge and asking us to buy the pro version every 5 questions.

upvoted 8 times

connecttozee 2 years, 1 month ago

agree D

upvoted 1 times

Question #22 *Topic 1*

terraform init initializes a sample main.tf file in the current directory.

A. True

B. False

Suggested Answer: *A*

Community vote distribution



Jlee7 Highly Voted 1 year, 5 months ago

The answer is False. The terraform init command does not initialize a sample main.tf file in the current directory. It initializes the working directory by downloading and installing provider plugins, downloading modules, and creating a Terraform state file.

If you do not have a main.tf file in your working directory, Terraform will create one for you. However, the file will be empty. You will need to add your Terraform configuration to the main.tf file.

You can find more information about the terraform init command in the Terraform documentation:
https://www.terraform.io/docs/cli/commands/init.html
upvoted 18 times

tbhtp 4 months, 2 weeks ago

init will not create a state file. Only apply does.
upvoted 1 times

vibzr2023 8 months ago

The answer is False but terraform init will not create a empty main.tf file but will still initialize the directory for use with Terraform, setting up the necessary internal data structures and downloading providers specified in any Terraform configuration files that are present. If you run terraform init in an empty directory or a directory without any .tf files, Terraform will complete the initialization process but will indicate that no configuration files were found. You'll need to create main.tf or other .tf files manually. I ran a test

---

PS C:\Users\rajna\OneDrive\Documents\TerraformCI> terraform init
Terraform initialized in an empty directory!

The directory has no Terraform configuration files. You may begin working
with Terraform immediately by creating Terraform configuration files.
PS C:\Users\rajna\OneDrive\Documents\TerraformCI>
upvoted 3 times

aanataliya Highly Voted 1 year, 2 months ago

Selected Answer: A

Answer is A. See below points.
1. Question does not ask if it creates main.tf or not. it asks wheather main.tf initializes or not.
2. Ran terraform init in empty directory and it warns "The directory has no Terraform configuration files." it means it tries to find configuration file to initialize.
3. configuration file can be of any name with .tf extension.
4. ran command with main.tf in directory with extra curly braces and initialization fails. because it try to initialize main.tf
5. rename main.tf with sample.tf with extra curly braces and it still gives same error.
6. Official documentation says "command initializes a working directory containing Terraform configuration files". main.tf is a configuration file.
Again, question never says it creates it. question clearly asks if main.tf is initialized. it means main.tf is already exists w.r.t. this question
Reference: https://developer.hashicorp.com/terraform/cli/commands/init

Please vote for this correct answer
upvoted 7 times

Tyler2023 1 year, 1 month ago

The answer is B
The question says terraform init initializes a sample main.tf file,
think about the word "sample" meaning it creates the file which is terraform will not create that for you
By default, terraform init assumes that the working directory already contains a configuration and will attempt to initialize that configuration. It will never try to create a sample file with sample configuration for you. But the question says it creates that sample file, meaning the direction is empty.
upvoted 2 times

aanataliya 1 year, 2 months ago
Those who are thinking its about creating main.tf. Question should have been "terrafom init creates sample init file" not initializes.
upvoted 2 times

MeneJohn Most Recent 6 days, 7 hours ago
Selected Answer: B
The terraform init command initializes a working directory containing Terraform configuration files. This is the first command you should run after writing a new Terraform configuration or cloning an existing configuration from version control. It is safe to run this command multiple times.
upvoted 1 times

sumaju 3 months, 3 weeks ago
Selected Answer: B
Terraform works based on the file extensions (except *override.tf) and read all the files and then work on the entire code base. The logical separation of files (e.g. variables.tf, output.tf) is only for better readability but does impact the execution if written in a single file. terraform inti is only for initializing the providers and remote backend, if configured.
upvoted 1 times

Molly1994 5 months, 3 weeks ago
main.tf is an option file. it might not exist. terraform init means to download all basic infrastructures from the configuration file. B false.
upvoted 2 times

SilentH 7 months, 2 weeks ago
Selected Answer: B
Even if the answer is somehow A, I'm voting B on principle.
upvoted 2 times

DianaPopal 11 months ago
The correct answer is B. False.
The terraform init command in Terraform is used to initialize a Terraform configuration in the current directory. It downloads the necessary provider plugins and sets up the backend configuration.
However, terraform init does not create a sample main.tf file in the current directory. The main.tf file is the main configuration file in Terraform where you define your infrastructure resources and their configurations. It is typically created manually by the user and contains the desired state of the infrastructure.
When you run terraform init, it looks for an existing main.tf file in the current directory to load the configuration. If the main.tf file does not exist, Terraform will not create it automatically.
upvoted 1 times

Hp45 1 year, 2 months ago
A lot of people are taking this question as init command is creating a main.tf. The context in the question is - if there was a sample main.tf in current folder then init would initialize the file. I vote for A.
upvoted 4 times

Shane_C 1 year, 4 months ago
It absolutely does not generate a main.tf; B is the correct answer
upvoted 2 times

DuboisNicolasDuclair 1 year, 5 months ago
A
Because terraform initialize *a sample main.tf* it means that we can have a multiple types of files like *.tf
So A is a good answer
upvoted 2 times

Shane_C 1 year, 4 months ago
It absolutely does not generate a main.tf; B is the correct answer
upvoted 2 times

karendavtyan 1 year, 6 months ago

Selected Answer: B

B. False

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: B

B is correct. It initializes versions and providers irrespective of name of the tf files.

upvoted 2 times

Rezi 1 year, 7 months ago

A is correct 100%

terraform init initializes any file with the extension ".tf" in the pwd. it says "a" sample main.tf, it didn't say "the" sample main.tf. Please note the difference.

upvoted 2 times

AzRNoob 1 year, 7 months ago

B. False.

The terraform init command initializes a new or existing Terraform working directory by downloading and installing any necessary plugins and modules specified in the configuration. It does not create a main.tf file or any other configuration file in the current directory.

You will need to create a main.tf file (or another configuration file with a different name) manually and write the necessary configuration code for your infrastructure. Once you have done so, you can run terraform init to prepare the working directory for use with Terraform.

upvoted 2 times

Faaizz 1 year, 7 months ago

Selected Answer: B

BBBBBBBB

upvoted 1 times

Power123 1 year, 7 months ago

B for sure

upvoted 1 times

sagunala5 1 year, 8 months ago

By default, terraform init assumes that the working directory already contains a configuration and will attempt to initialize that configuration.

upvoted 2 times

Question #23 *Topic 1*

Which two steps are required to provision new infrastructure in the Terraform workflow? (Choose two.)

A. Destroy

B. Apply

C. Import

D. Init

E. Validate

Suggested Answer: *BD*

Reference:

https://www.terraform.io/guides/core-workflow.html

Community vote distribution

BD (100%)

PLN6302 2 weeks, 1 day ago

Selected Answer: BD

B and D

upvoted 1 times

enook 10 months, 4 weeks ago

Selected Answer: BD

BD for sure

upvoted 2 times

hungran91 1 year, 4 months ago

Selected Answer: BD

Definitely BD

upvoted 1 times

Jlee7 1 year, 5 months ago

Glad to see most popular answer and examtopics answer agree!! :) B&D

upvoted 3 times

karendavtyan 1 year, 6 months ago

Selected Answer: BD

B and D

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: BD

B and D

upvoted 1 times

connecttozee 1 year, 7 months ago

required sure init first & apply later

BD is correct

upvoted 2 times

Power123 1 year, 7 months ago

B & D - init and apply

upvoted 1 times

Nunyabiznes 1 year, 8 months ago

Selected Answer: BD

The two steps required to provision new infrastructure in the Terraform workflow are:

B. Apply: This step applies the changes to the infrastructure by creating or modifying resources in accordance with the configuration described in the

Terraform code.

D. Init: This step initializes a new or existing Terraform working directory by downloading and installing any required providers and modules, as well as setting up the backend.

The other options are:

A. Destroy: This step destroys the resources that are managed by Terraform in the specified configuration.

C. Import: This step allows the user to import existing infrastructure into Terraform's state.

E. Validate: This step checks the syntax and validity of the Terraform configuration files without actually creating or modifying any resources.

upvoted 3 times

Bilalglg93350 1 year, 8 months ago

The two required steps to provision new infrastructure in the Terraform workflow are:

B. Apply - This step applies the changes to the infrastructure by creating, modifying, or deleting resources based on the Terraform configuration.

D. Init - This step initializes a new or existing Terraform configuration by downloading the required provider plugins and setting up the backend to store the state of the infrastructure.

Options A, C, and E are not required steps to provision new infrastructure in the Terraform workflow:

Destroy (option A) is used to destroy the resources created by the Terraform configuration, which is not required to provision new infrastructure.
Import (option C) is used to import an existing infrastructure resource into the Terraform state, which is not a required step to provision new infrastructure.
Validate (option E) is used to validate the syntax and structure of the Terraform configuration but is not a required step to provision new infrastructure.

upvoted 2 times

SilentMilli 1 year, 9 months ago

Selected Answer: BD

The Terraform workflow consists of several steps for provisioning new infrastructure. The two steps that are required to provision new infrastructure in the Terraform workflow are:

B. Apply: The "terraform apply" command is used to provision new infrastructure. This command takes the Terraform configuration files as input and creates or updates the resources specified in the configuration files.

D. Init: The "terraform init" command is used to initialize a Terraform configuration directory. This command must be run before running the "terraform apply" command to ensure that the required Terraform modules and plugins are installed and available.

Note that other steps in the Terraform workflow, such as "terraform destroy" or "terraform import," may also be used depending on your specific needs and the nature of your infrastructure. However, the "terraform apply" and "terraform init" commands are the two steps that are required to provision new infrastructure in the Terraform workflow.

upvoted 1 times

awsguys 1 year, 10 months ago

B , D right

upvoted 1 times

FarziWaliMarzi 1 year, 11 months ago

Selected Answer: BD

Obviously, plan can be skipped.

upvoted 1 times

RVivek 2 years, 2 months ago

Selected Answer: BD

A- is to destrory
C- is to import an existng resource that was manually created . The question is about provisioning a new resource
E - is sjust to valaidate the command structure

upvoted 1 times

cjig 2 years, 4 months ago

Selected Answer: BD

Options B and D are correct.

upvoted 3 times

Eltooth 2 years, 5 months ago

Selected Answer: BD

B and D are correct answers.

upvoted 3 times

BlackZeros 2 years, 5 months ago

Selected Answer: BD

b and d sounds logical

upvoted 2 times

Question #24 *Topic 1*

Why would you use the terraform taint command?

A. When you want to force Terraform to destroy a resource on the next apply

B. When you want to force Terraform to destroy and recreate a resource on the next apply

C. When you want Terraform to ignore a resource on the next apply

D. When you want Terraform to destroy all the infrastructure in your workspace

Suggested Answer: *B*

The terraform taint command manually marks a Terraform-managed resource as tainted, forcing it to be destroyed and recreated on the next apply.

Reference:

https://www.terraform.io/docs/cli/commands/taint.html

Community vote distribution

B (100%)

Eltooth Highly Voted 2 years, 4 months ago

Selected Answer: B

Noe of the answers really explain what taint command does - except B is closest.

"The terraform taint command informs Terraform that a particular object has become degraded or damaged. Terraform represents this by marking the object as "tainted" in the Terraform state, and Terraform will propose to replace it in the next plan you create."

https://www.terraform.io/cli/commands/taint

upvoted 11 times

Alandt Highly Voted 10 months, 1 week ago

Selected Answer: B

IMPORTANT NOTE: taint command is deprecated. For Terraform v0.15.2 and later, Hashicorp recommends using the -replace option with terraform apply instead

upvoted 5 times

instant919 Most Recent 4 months, 1 week ago

Selected Answer: B

B. When you want to force Terraform to destroy and recreate a resource on the next apply

Using the terraform taint command marks a resource as "tainted," which means that Terraform will destroy and then recreate the resource during the next terraform apply operation. This can be useful if you know that a resource is in a bad state and needs to be replaced.

upvoted 1 times

lazy_tortoise89 4 months, 4 weeks ago

Selected Answer: B

The answer is B here.

But I came across to the other question related to taint and it is important to give attention to the small but Important point: Changes will be applied in the NEXT apply, after you run taint command to mark the resource "as delete".

upvoted 1 times

Jerry_m10 1 year ago

Outdated question for v3.0.0 test.

Although B is correct, but we now have terraform -replace="resource_name" for this now.

upvoted 2 times

Halimb 1 year, 2 months ago

Selected Answer: B

Although B is correct here, the question is old and the command is deprecated. Instead use terraform apply -replace="<resource_to_replace>". See https://developer.hashicorp.com/terraform/cli/commands/taint

upvoted 2 times

karendavtyan 1 year, 6 months ago

Selected Answer: B

B. When you want to force Terraform to destroy and recreate a resource on the next apply

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: B

B is correct

upvoted 1 times

Power123 1 year, 7 months ago

This command is now deprecated and suggested option is now terraform apply -replace="state_object" . Ans B

upvoted 4 times

Bilalglg93350 1 year, 8 months ago

The correct answer is C. When you want Terraform to ignore a resource on the next apply.

The terraform taint command is used to mark a resource managed by Terraform as "tainted," which means that it needs to be recreated on the next apply. This is useful when a resource is in an inconsistent state or needs to be recreated for some other reason.

However, the taint command does not destroy or force Terraform to destroy a resource on the next apply (option A and B). Instead, it marks the resource as tainted, indicating that it needs to be recreated. When you run terraform apply after marking a resource as tainted, Terraform will destroy and recreate the resource.

Option D is incorrect because the terraform taint command only marks a single resource as tainted and does not destroy all the infrastructure in your workspace.

upvoted 1 times

SilentMilli 1 year, 9 months ago

Selected Answer: B

The "terraform taint" command is used to mark a resource as "tainted," indicating to Terraform that the resource should be destroyed and recreated on the next "terraform apply" command. This can be useful in cases where you need to make changes to a resource that cannot be updated in place, or if you want to enforce a clean rebuild of a resource for some other reason.

It's important to note that tainting a resource will not immediately destroy the resource, but will instead cause Terraform to plan to destroy and recreate the resource on the next "terraform apply" command. This allows you to review and confirm the changes that Terraform will make before actually applying them.

upvoted 2 times

BlackZeros 2 years, 5 months ago

Selected Answer: B

b it is

upvoted 3 times

javibadillo 2 years, 5 months ago

Selected Answer: B

https://www.terraform.io/cli/commands/taint#recommended-alternative

upvoted 3 times

softarts 2 years, 6 months ago

it is B

upvoted 4 times

d0ug7979 2 years, 6 months ago

This command is now deprecated since v0.15.2 and suggested option is now terraform apply -replace="state_object"
https://www.terraform.io/cli/commands/taint#recommended-alternative

upvoted 3 times

biscuithammer 2 years, 6 months ago

yes but on the exam it's still a valid answer

upvoted 1 times

Question #25 *Topic 1*

Terraform requires the Go runtime as a prerequisite for installation.

A. True

B. False

Suggested Answer: *B*

Reference:

https://www.terraform.io/docs/extend/guides/v1-upgrade-guide.html

As of September 2019, Terraform provider developers importing the Go module `github.com/hashicorp/terraform`, known as Terraform Core, should switch to `github.com/hashicorp/terraform-plugin-sdk`, the Terraform Plugin SDK, instead.

Why a separate module?

While the `helper/*` and other packages in Terraform Core has served us well, in order for provider development to evolve, the SDK needed to break out into its own repository. Terraform Core's versioning has been oriented towards practitioners. With the "unofficial" SDK existing in the core repository, the SDK becomes tied to Core releases and cannot follow semantic versioning. The new standalone SDK github.com/hashicorp/terraform-plugin-sdk follows sematic versioning starting with v1.0.0.

We will use the term "legacy Terraform plugin SDK" when referring to the version of Terraform Core imported and used by providers.

Community vote distribution

B (100%)

Empel Highly Voted 1 year, 5 months ago

Selected Answer: B

Terraform software is written in Go but it is not require to have go installed as you get a binary after installing terraform.

upvoted 13 times

wsyh Most Recent 6 months, 1 week ago

Selected Answer: B

Vote B

upvoted 2 times

Ni33 6 months, 2 weeks ago

Selected Answer: B

BBBBBB

upvoted 2 times

karendavtyan 6 months, 3 weeks ago

Selected Answer: B

B. False

upvoted 2 times

Ni33 6 months, 4 weeks ago

Selected Answer: B

B is correct

upvoted 1 times

Power123 7 months, 4 weeks ago

Ans is B. Terraform is written in Go programming language and doesn't require Go runtime

upvoted 1 times

SilentMilli 9 months, 2 weeks ago

Selected Answer: B

Terraform is written in Go programming language, but it does not require the Go runtime to be installed on the machine where Terraform is installed. You only need to have the Terraform binary installed to be able to use Terraform to provision infrastructure. You can download the appropriate binary for your operating system from the Terraform website, and then add it to your PATH so that you can run Terraform from the command line.

upvoted 3 times

Eltooth 1 year, 4 months ago

Selected Answer: B

B is correct answer : False.

This has been updated with https://www.terraform.io/plugin/sdkv2/guides/v1-upgrade-guide and https://www.terraform.io/plugin/sdkv2/guides/v2-upgrade-guide

upvoted 4 times

amrith501 1 year, 5 months ago

Selected Answer: B

I did not see any documention saying terraform requires go runtime. I will go with false

upvoted 3 times

Question #26 *Topic 1*

When should you use the force-unlock command?

A. You see a status message that you cannot acquire the lock

B. You have a high priority change

C. Automatic unlocking failed

D. You apply failed due to a state lock

Suggested Answer: *C*

Manually unlock the state for the defined configuration.

Reference:

https://www.terraform.io/docs/cli/commands/force-unlock.html

Community vote distribution

C (100%)

Egger1992 Highly Voted 2 years, 6 months ago

Selected Answer: C

Be very careful with this command. If you unlock the state when someone else is holding the lock it could cause multiple writers. Force unlock should only be used to unlock your own lock in the situation where automatic unlocking failed.

Source: https://www.terraform.io/language/state/locking

upvoted 25 times

SilentMilli Highly Voted 1 year, 8 months ago

Selected Answer: C

The force-unlock command should be used when you need to manually release a Terraform state lock that cannot be released by other means. This command is typically used in scenarios where a Terraform command was interrupted, and the lock was not released automatically. Using the force-unlock command will forcibly release the lock, but it may leave the Terraform state in an inconsistent state, so it should be used with caution.

Therefore, the correct answer is:

C. Automatic unlocking failed

upvoted 7 times

valli_ Most Recent 1 month, 2 weeks ago

Selected Answer: C

As unlocking might really be a great advantage and disadvantage. Keep in mind before using it!

upvoted 1 times

enook 10 months, 4 weeks ago

Selected Answer: C

C for sure

upvoted 1 times

karendavtyan 1 year, 6 months ago

Selected Answer: C

C. Automatic unlocking failed

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: C

C is correct

upvoted 1 times

chimons 1 year, 11 months ago

Selected Answer: C

C. Force unlock should not be used under normal circumstances

upvoted 1 times

ceros399 2 years, 2 months ago

Selected Answer: C

answer is c

upvoted 1 times

Eltooth 2 years, 4 months ago

Selected Answer: C

C is correct answer.

https://www.terraform.io/language/state/locking

https://www.terraform.io/cli/commands/force-unlock

upvoted 3 times

Question #27 *Topic 1*

Terraform can import modules from a number of sources `" which of the following is not a valid source?

A. FTP server

B. GitHub repository

C. Local path

D. Terraform Module Registry

Suggested Answer: *A*

Community vote distribution

A (100%)

karendavtyan 6 months, 3 weeks ago

Selected Answer: A

A. FTP server

upvoted 3 times

Ni33 6 months, 4 weeks ago

Selected Answer: A

A is the correct answer.

upvoted 1 times

Bluemoon22 7 months ago

A, ftp server

upvoted 1 times

AzRNoob 7 months, 3 weeks ago

A. FTP server

Terraform can import modules from various sources such as a Git repository, a local file system path, or a public or private module registry. However, an FTP server is not a supported module source in Terraform.

upvoted 1 times

Power123 7 months, 4 weeks ago

A is the correct ans

upvoted 1 times

Eltooth 1 year, 5 months ago

Selected Answer: A

A is correct answer.

upvoted 3 times

Eltooth 1 year, 5 months ago

https://www.terraform.io/language/modules/sources

upvoted 3 times

Ahmad_Terraform 1 year, 5 months ago

A is correct

upvoted 1 times

HungDang 1 year, 5 months ago

Selected Answer: A

A is correct

upvoted 3 times

softarts 1 year, 6 months ago

I choose A

upvoted 4 times

Question #28 *Topic 1*

Which of the following is available only in Terraform Enterprise or Cloud workspaces and not in Terraform CLI?

A. Secure variable storage

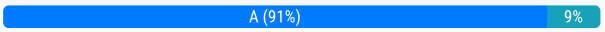
B. Support for multiple cloud providers

C. Dry runs with terraform plan

D. Using the workspace as a data source

Suggested Answer: *A*

Community vote distribution



koneba1309 Highly Voted 2 years, 6 months ago

Selected Answer: A

I think it is A.

upvoted 10 times

DianaPopal Highly Voted 11 months ago

The correct answer is A. Secure variable storage.

Secure variable storage is a feature that is available only in Terraform Enterprise or Terraform Cloud workspaces, and it is not available in the Terraform CLI.

Secure variable storage allows you to store sensitive information, such as API keys, passwords, or other credentials, securely within Terraform. These variables are encrypted and protected, ensuring that they are not exposed in plain text.

upvoted 10 times

IK912 Most Recent 1 year, 5 months ago

Definitely A

upvoted 1 times

Ni33 1 year, 6 months ago

A is correct answer. Terraform Cloud do provide secure storage for variables.

upvoted 3 times

Bluemoon22 1 year, 7 months ago

Answer is A, secure variable storage

upvoted 1 times

AzRNoob 1 year, 7 months ago

A. Secure variable storage is available only in Terraform Enterprise or Cloud workspaces and not in Terraform CLI.

Terraform Enterprise and Cloud workspaces provide additional features and functionality beyond what is available in the open-source Terraform CLI. One of these features is the ability to store sensitive data, such as API keys or passwords, securely within a workspace. This allows teams to share infrastructure code without exposing sensitive information.

upvoted 3 times

Power123 1 year, 7 months ago

Correct answer is A

upvoted 1 times

r1ck 1 year, 9 months ago

https://developer.hashicorp.com/terraform/tutorials/configuration-language/sensitive-variables

still A ?

upvoted 3 times

InformationOverload 1 year, 11 months ago

Selected Answer: A

Its A indeed

upvoted 1 times

RVivek 2 years, 2 months ago

Selected Answer: A

A is available in Terraform cloud and not in Terraform OSS

upvoted 1 times

Optimus 2 years, 2 months ago

Selected Answer: A

It is definitely A

upvoted 4 times

yuvifose 2 years, 4 months ago

Selected Answer: A

I think is A

upvoted 3 times

nhatne 2 years, 4 months ago

Selected Answer: A

it's A

upvoted 4 times

Ahmad_Terraform 2 years, 5 months ago

i think A

upvoted 1 times

bp339 2 years, 6 months ago

Selected Answer: A

A is correct

upvoted 3 times

biscuithammer 2 years, 6 months ago

Selected Answer: A

it should be A

upvoted 2 times

ItaloVinodi 2 years, 6 months ago

Selected Answer: A

I think A is correct

upvoted 2 times

Question #29 *Topic 1*

terraform validate validates the syntax of Terraform files.

A. True

B. False

Suggested Answer: *A*

The terraform validate command validates the syntax and arguments of the Terraform configuration files.

Reference:

https://www.terraform.io/docs/cli/code/index.html

Community vote distribution

A (100%)

kilowd Highly Voted 2 years, 2 months ago

Validate runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state. It is thus primarily useful for general verification of reusable modules, including correctness of attribute names and value types.

upvoted 9 times

enook Most Recent 10 months, 4 weeks ago

Selected Answer: A

AAAAAAAA

upvoted 1 times

Caput 1 year, 3 months ago

Selected Answer: A

It's true.

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

A is the correct answer

upvoted 1 times

Bluemoon22 1 year, 7 months ago

answer is A, true

upvoted 1 times

Power123 1 year, 7 months ago

Ans is A it validates the syntax

upvoted 1 times

Eltooth 2 years, 4 months ago

Selected Answer: A

A is correct answer.

https://www.terraform.io/cli/commands/validate

upvoted 2 times

biscuithammer 2 years, 6 months ago

Selected Answer: A

A is correct

https://www.terraform.io/cli/commands/validate

upvoted 4 times

Question #30 *Topic 1*

You have used Terraform to create an ephemeral development environment in the cloud and are now ready to destroy all the infrastructure described by your
Terraform configuration. To be safe, you would like to first see all the infrastructure that will be deleted by Terraform.
Which command should you use to show all of the resources that will be deleted? (Choose two.)

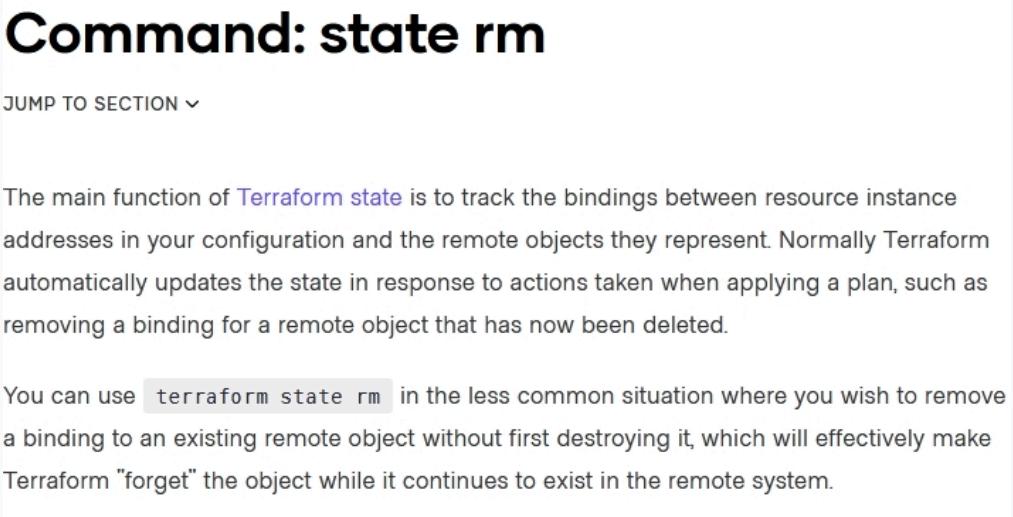
A. Run terraform plan -destroy.

B. This is not possible. You can only show resources that will be created.

C. Run terraform state rm *.

D. Run terraform destroy and it will first output all the resources that will be deleted before prompting for approval.

Suggested Answer: *CD*
Reference:
https://www.terraform.io/docs/cli/commands/state/rm.html



Community vote distribution
AD (94%) 6%

lpergorta Highly Voted 2 years, 6 months ago
The answer is A,D
upvoted 25 times

ItaloVinodi Highly Voted 2 years, 6 months ago
Selected Answer: AD
We need to choose two so AD
upvoted 12 times

ravi135 Most Recent 9 months, 2 weeks ago
answer AD
upvoted 1 times

Shane_C 1 year, 4 months ago
Selected Answer: AD
AD are correct
upvoted 2 times

vj_dhaksh 1 year, 5 months ago
Answer is AD

C - incorrect as running "terraform state rm *" throws error stating variable required (* wont pick all)
upvoted 3 times

karendavtyan 1 year, 6 months ago
D. Run terraform destroy and it will first output all the resources that will be deleted before prompting for approval.

upvoted 1 times

karendavtyan 1 year, 6 months ago

Selected Answer: A

A. Run terraform plan -destroy.

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: AD

A and D are correct.

upvoted 1 times

Bluemoon22 1 year, 7 months ago

A is only to show the resources will be destroyed, terraform plan -destroy
D is a common operation from terraform destroy

upvoted 1 times

AzRNoob 1 year, 7 months ago

A. Run terraform plan -destroy.

This command will generate a plan that shows all of the changes Terraform will make to the infrastructure, including any resources that will be destroyed. The -destroy flag specifies that only the changes that will result in the destruction of resources should be shown.

Option D is also partially correct, as running terraform destroy will output a list of resources that will be destroyed before prompting for approval. However, it is generally recommended to run terraform plan -destroy first to preview the changes before actually executing them with terraform destroy.

Option B is incorrect, as it is possible to show resources that will be deleted using the terraform plan -destroy command.

Option C is also incorrect, as running terraform state rm * will remove all resources from the Terraform state file, effectively "forgetting" about them and making it impossible to manage them with Terraform.

upvoted 6 times

Faaizz 1 year, 7 months ago

Selected Answer: AD

A & D are correct here

upvoted 1 times

connecttozee 1 year, 7 months ago

100% is AD

upvoted 1 times

Power123 1 year, 7 months ago

A and D are correct

upvoted 1 times

r1ck 1 year, 9 months ago

https://developer.hashicorp.com/terraform/cli/commands/destroy
AD

upvoted 1 times

andersonbispos42 1 year, 11 months ago

Selected Answer: AD

The answer is A, D

upvoted 1 times

andersonbispos42 1 year, 11 months ago

Selected Answer: A

The answer is only A

upvoted 1 times

vikramvlr 1 year, 11 months ago

Correct answer - AD

upvoted 1 times

Question #31 *Topic 1*

Which of the following is the correct way to pass the value in the variable num_servers into a module with the input servers?

A. servers = num_servers

B. servers = variable.num_servers

C. servers = var(num_servers)

D. servers = var.num_servers

Suggested Answer: *A*

Community vote distribution

D (100%)

tipzzz Highly Voted 3 years, 1 month ago

D for sure

upvoted 22 times

vitasac Highly Voted 3 years ago

Selected Answer: D

Yes I'm agree D

upvoted 10 times

kanika_sk Most Recent 6 months ago

Selected Answer: D

in terraform we need to use variables defined as var.<variable_name>

upvoted 1 times

kanikachander 9 months, 3 weeks ago

Selected Answer: D

D for sure

upvoted 1 times

Cololand 1 year ago

D - Who even marks the right answers at examtopics?

upvoted 2 times

enook 1 year, 4 months ago

Selected Answer: D

DDDDDDD

upvoted 1 times

ghostGuiggs 1 year, 6 months ago

Selected Answer: D

D is the answer

upvoted 1 times

Kizerfor 1 year, 9 months ago

D for sure

upvoted 1 times

Bere 1 year, 10 months ago

Selected Answer: D

In Terraform, variables are referred to using the syntax var.<variable_name>. Here, var is a special namespace that contains all input variables.

For example, if you have a variable named num_servers in your configuration and you want to pass it to a module, you would do it like this:

variable "num_servers" {
description = "The number of servers to create"
default = 5
}

```
module "server_module" {
source = "app.terraform.io/example/server"
version = "1.0.0"

servers = var.num_servers
}
```

In the above example, the input variable num_servers is defined with a default value of 5. Then, the server_module is being called and the servers argument is being set to the value of var.num_servers, which is 5.

upvoted 9 times

Shane_C 1 year, 10 months ago

Selected Answer: D

D is correct

upvoted 1 times

Ni33 2 years ago

Selected Answer: D

D is the one

upvoted 1 times

Bluemoon22 2 years, 1 month ago

D, servers = var.num_servers

upvoted 2 times

Power123 2 years, 1 month ago

Ans is D

upvoted 1 times

thor7 2 years, 1 month ago

Selected Answer: D

var.num_servers is correct and full answer

upvoted 2 times

Power123 2 years, 2 months ago

D is the ans

upvoted 1 times

kartikjena31 2 years, 4 months ago

I also go with D

upvoted 2 times

lalgebala 2 years, 6 months ago

D for Sure

upvoted 1 times

Question #32 *Topic 1*

A Terraform provisioner must be nested inside a resource configuration block.

A. True

B. False

Suggested Answer: *A*
Most provisioners require access to the remote resource via SSH or WinRM, and expect a nested connection block with details about how to connect.
Reference:
https://www.terraform.io/docs/language/resources/provisioners/connection.html

Community vote distribution
A (66%) B (34%)

quixo Highly Voted 2 years, 10 months ago
A: True.

Example:

resource "aws_instance" "web" {
...

provisioner "local-exec" {
command = "echo The server's IP address is ${self.private_ip}"
}
}
upvoted 16 times

kiran15789 Highly Voted 2 years ago
Selected Answer: A
To use a provisioner in Terraform, you must include it as part of a resource configuration block. This is because a provisioner operates on a resource that has been created or updated by Terraform.
upvoted 6 times

didistars13 Most Recent 4 months, 1 week ago
Selected Answer: A
Definitely A as provisioners are executed as part of the create and destroy phases of the resource lifecycle. They cannot exist independently outside of a resource block. If you decide to define a provisioner outside of a resource block , terraform with throw a syntax error.
upvoted 1 times

090200f 10 months, 3 weeks ago
Answer is A: True
Multiple provisioners can be specified within a resource. Multiple provisioners are executed in the order they're defined in the configuration file.
https://developer.hashicorp.com/terraform/language/resources/provisioners/syntax
resource "aws_instance" "web" {
...

provisioner "local-exec" {
command = "echo first"
}

provisioner "local-exec" {
command = "echo second"
}
}
upvoted 1 times

090200f 10 months, 3 weeks ago

https://developer.hashicorp.com/terraform/language/resources/provisioners/connection
Connection blocks don't take a block label and can be nested within either a resource or a provisioner.

A connection block nested directly within a resource affects all of that resource's provisioners.
A connection block nested in a provisioner block only affects that provisioner and overrides any resource-level connection settings.
in the question they asked 'must' so may be the answer is False : B . I think so please confirm. I am confused now.
upvoted 2 times

ravk2321 1 year, 2 months ago

Selected Answer: A

No Standalone Provisioners: Terraform does not support defining provisioners as standalone elements outside of resource blocks. All provisioner configurations must be part of a resource definition because their execution context is tightly coupled with resources.

Alternatives to Provisioners: For scenarios where you might think about using provisioners in a more decoupled or independent manner, Terraform suggests other mechanisms. For instance, if the goal is to manage configuration or orchestration that seems beyond the scope of what should be tightly coupled to a single resource's lifecycle, tools like Ansible, Chef, Puppet, or Terraform itself (through resource dependencies and proper module design) are recommended to handle such configurations. These tools can be used in conjunction with Terraform but managed through their mechanisms for orchestration or configuration management, rather than through Terraform provisioners.
upvoted 1 times

samimshaikh 1 year, 4 months ago

Selected Answer: A

True provisioner should only placed under the resource block

PS E:\Terraform> terraform.exe validate

```
╷
│ Error: Unsupported block type
│
│ on main.tf line 34:
│ 34: provisioner "local-exec" {
│
│ Blocks of type "provisioner" are not expected here.
╵
```

upvoted 1 times

gold4otas 1 year, 5 months ago

Selected Answer: B

While it's common to place provisioners inside the resource block for clarity and organization, they can also be defined outside of the block at the same level. Both approaches are valid, and the choice depends on your preference and the organization of your Terraform code. The main point is that provisioners are associated with a specific resource and are configured within the resource block or at the same level in the code.
upvoted 5 times

Ramdi1 1 year, 6 months ago

Selected Answer: B

the question is stated provisioner must be. the key word being Must. They do not have to be so I voted B
upvoted 5 times

Ni33 2 years ago

Selected Answer: A

A is the correct answer
upvoted 2 times

Bluemoon22 2 years, 1 month ago

A: true.
for instance,
resource "aws_instance" "testInstance" {
ami = "${var.instance_ami}"
instance_type = "${var.instance_type}"
subnet_id = "${aws_subnet.subnet_public.id}"
vpc_security_group_ids = ["${aws_security_group.sg_22.id}"]

key_name = "${aws_key_pair.ec2key.key_name}"
tags {
"Environment" = "${var.environment_tag}"
}
provisioner "local-exec" {
command = "echo ${aws_instance.testInstance.public_ip} >> public_ip.txt"
}
}
upvoted 2 times

SaadKhamis 2 years, 1 month ago

Selected Answer: B

Examples given in "https://developer.hashicorp.com/terraform/language/resources/provisioners/connection" are not inside a resource configuration block.

provisioner "file" {
source = "conf/myapp.conf"
destination = "/etc/myapp.conf"

connection {
type = "ssh"
user = "root"
password = "${var.root_password}"
host = "${var.host}"
}
}
upvoted 2 times

halfway 2 years ago

That's only a code snippet, not the full configuration, though.
upvoted 5 times

Power123 2 years, 1 month ago

True. Ans is A
upvoted 1 times

vyhak 2 years, 3 months ago

A : True
upvoted 1 times

kartikjena31 2 years, 4 months ago

A- True
upvoted 1 times

Bere 2 years, 5 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/resources/provisioners/syntax

Provisioners are used to execute scripts on a local or remote machine as part of resource creation or destruction. Provisioners can be used to bootstrap a resource, cleanup before destroy, run configuration management, etc.

How to use Provisioners
Note: Provisioners should only be used as a last resort. For most common situations there are better alternatives. For more information, see the sections above.

If you are certain that provisioners are the best way to solve your problem after considering the advice in the sections above, you can add a provisioner block inside the resource block of a compute instance.

resource "aws_instance" "web" {
...

provisioner "local-exec" {

command = "echo The server's IP address is ${self.private_ip}"
}
}
upvoted 3 times

chimons 2 years, 5 months ago

Selected Answer: A

A.
If you need to run provisioners that aren't directly associated with a specific resource, you can associate them with a null_resource.
upvoted 1 times

dv00thay 2 years, 5 months ago

Should be B
Terraform Block
terraform {
required_version = ">= 1.0.0"
required_providers {
azurerm = {
source = "hashicorp/azurerm"
version = ">= 2.0"
}
random = {
source = "hashicorp/random"
version = ">= 3.0"
}
}
}

Provider Block
provider "azurerm" {
features {}
}
upvoted 1 times

VincentMenzel 2 years, 5 months ago

provisioner != provider
upvoted 6 times

ziancom 2 years, 3 months ago

LOL this is funny
upvoted 2 times

Question #33 *Topic 1*

Terraform can run on Windows or Linux, but it requires a Server version of the Windows operating system.

A. True

B. False

Suggested Answer: *B*

Community vote distribution

B (100%)

softarts Highly Voted 2 years ago

choose B

upvoted 16 times

tfdestroy Highly Voted 10 months ago

Selected Answer: B

Terraform is platform-agnostic and it can run on many different systems, including both Windows and Linux. It does not require a Server version of the Windows operating system. You can run it on a standard Windows 10 installation, for example. The only requirement is that you have the Terraform binary installed and that it's in your system's PATH.

upvoted 6 times

Ni33 Most Recent 1 year ago

Selected Answer: B

B is correct

upvoted 2 times

Bluemoon22 1 year, 1 month ago

B, false

upvoted 1 times

Power123 1 year, 1 month ago

B is the correct ans

upvoted 1 times

Eltooth 1 year, 10 months ago

Selected Answer: B

B is correct answer : false.

upvoted 4 times

javibadillo 1 year, 11 months ago

Selected Answer: B

B https://www.terraform.io/downloads

upvoted 5 times

Question #34 *Topic 1*

What does the default "local" Terraform backend store?

A. tfplan files

B. Terraform binary

C. Provider plugins

D. State file

Suggested Answer: *D*

The local backend stores state on the local filesystem, locks that state using system APIs, and performs operations locally.

Reference:

https://www.terraform.io/docs/language/settings/backends/local.html

Community vote distribution

D (100%)

softarts Highly Voted 1 year, 6 months ago

choose D

upvoted 12 times

Eltooth Highly Voted 1 year, 5 months ago

Selected Answer: D

D is correct answer.

https://www.terraform.io/language/settings/backends/local

https://www.terraform.io/language/state

upvoted 9 times

Bluemoon22 Most Recent 7 months ago

D, state files

upvoted 1 times

Power123 7 months, 4 weeks ago

D is correct

upvoted 1 times

camps 7 months, 4 weeks ago

Selected Answer: D

D. State file.

The default "local" Terraform backend stores the state file on the local disk of the machine running Terraform. The state file contains information about the resources managed by Terraform, such as their current state and any dependencies between them. When running Terraform commands, such as terraform plan or terraform apply, Terraform reads the state file to determine the current state of the resources and what changes need to be made to reach the desired state.

upvoted 5 times

vyhak 10 months ago

Selected Answer: D

D is the answer

upvoted 1 times

asat_chil 10 months, 3 weeks ago

Answer: D

upvoted 1 times

Bere 11 months, 1 week ago

Selected Answer: D

https://developer.hashicorp.com/terraform/language/state

Terraform must store state about your managed infrastructure and configuration. This state is used by Terraform to map real world resources to your

configuration, keep track of metadata, and to improve performance for large infrastructures.

This state is stored by default in a local file named "terraform.tfstate", but it can also be stored remotely, which works better in a team environment.
upvoted 3 times

Question #35 *Topic 1*

You have multiple team members collaborating on infrastructure as code (IaC) using Terraform, and want to apply formatting standards for readability.
How can you format Terraform HCL (HashiCorp Configuration Language) code according to standard Terraform style convention?

A. Run the terraform fmt command during the code linting phase of your CI/CD process

B. Designate one person in each team to review and format everyone's code

C. Manually apply two spaces indentation and align equal sign "=" characters in every Terraform file (*.tf)

D. Write a shell script to transform Terraform files using tools such as AWK, Python, and sed

Suggested Answer: *C*
✑ Indent two spaces for each nesting level.
✑ When multiple arguments with single-line values appear on consecutive lines at the same nesting level, align their equals signs.
Reference:
https://www.terraform.io/docs/language/syntax/style.html

Community vote distribution
A (100%)

vitasac Highly Voted 2 years, 7 months ago
Selected Answer: A
Yes Answer A
upvoted 35 times

calebvar Highly Voted 2 years, 7 months ago
correct answer is A
upvoted 12 times

nickyop Most Recent 8 months, 2 weeks ago
Okay, as obvious as the option A is the correct answer, here's one question. What happens if such questions arise in the terraform exam where they have the wrong answers saved as the right ones? For example, in the above question, if we choose the most voted option in the exam, we'll lose 1 mark just because the right answer is C according to the system?? And FYI, I have already undergone one attempt for the exam which I was certain that I would pass because it went pretty well but I failed by 2%. And I know for sure it is only because of such contradictory questions! @Hashicorp please settle such questions once and for all!
upvoted 2 times

AkaAka4 7 months ago
Why are you even tagging Hashicorp here... do you think they will see your comment😂
upvoted 4 times

ghostGuiggs 1 year ago
Selected Answer: A
A is the answer
upvoted 2 times

Tyler2023 1 year, 1 month ago
The answer C is correct
You can manually format your TF configuration to follow the Style Conventions
Option A doesn't make any sense, why you run the terraform fmt in CI/CI process? that will only format the copy of the configuration in the remote agent that is running your configuration files, it doesn't modify your source codes in version control system.
Option C followed the Terraform style convention, but it only do it manually
https://developer.hashicorp.com/terraform/language/syntax/style
upvoted 1 times

Clapton79 1 year ago
Pull Request is a CI/CD process and it can format code and it does modify your version control.
upvoted 1 times

nickyop 8 months, 2 weeks ago

Referring the link you have shared, I think you missed a note where they have mentioned "You can enforce these conventions automatically by running terraform fmt."
upvoted 2 times

gofavad926 1 year, 2 months ago

Selected Answer: A

A for sure
upvoted 1 times

vc1011 1 year, 4 months ago

Selected Answer: A

ITs very evident A
upvoted 1 times

Awadh 1 year, 4 months ago

A is the correct answer, who is writing the answers
upvoted 4 times

Shane_C 1 year, 4 months ago

Selected Answer: A

Now they have to be joking.

The only reasonable answer is A.

Everything else is just nonsense
upvoted 3 times

nickyop 8 months, 2 weeks ago

Yes, thank you!
upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

A is correct
upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

A is the correct answer
upvoted 1 times

Bluemoon22 1 year, 7 months ago

A, by using terraform fmt command
upvoted 1 times

connecttozee 1 year, 7 months ago

A is correct
upvoted 1 times

Power123 1 year, 7 months ago

A is correct
upvoted 1 times

camps 1 year, 7 months ago

Selected Answer: A

A. Run the terraform fmt command during the code linting phase of your CI/CD process.

Terraform provides a command called terraform fmt that can be used to automatically format Terraform HCL code according to the standard Terraform style convention. Running this command on your codebase during the code linting phase of your CI/CD process can ensure that your code is formatted consistently across your team and conforms to the standard Terraform style convention.
upvoted 6 times

hasanuckun 1 year, 8 months ago

answer is a
upvoted 1 times

alexsandroe 1 year, 9 months ago

Selected Answer: A

FMT helped de code

upvoted 1 times

Question #36 *Topic 1*

What value does the Terraform Cloud/Terraform Enterprise private module registry provide over the public Terraform Module Registry?

A. The ability to share modules with public Terraform users and members of Terraform Enterprise Organizations

B. The ability to tag modules by version or release

C. The ability to restrict modules to members of Terraform Cloud or Enterprise organizations

D. The ability to share modules publicly with any user of Terraform

Suggested Answer: *D*

Terraform Registry is an index of modules shared publicly using this protocol. This public registry is the easiest way to get started with Terraform and find modules created by others in the community.

Reference:

https://www.terraform.io/docs/language/modules/sources.html

Community vote distribution

C (100%)

vitasac Highly Voted 3 years ago

Selected Answer: C

for sure C

upvoted 17 times

tipzzz Highly Voted 3 years, 1 month ago

Selected Answer: C

Terraform Cloud's private registry works similarly to the public Terraform Registry and helps you share Terraform providers and Terraform modules across your organization. It includes support for versioning and a searchable list of available providers and modules.

upvoted 10 times

anand0310 Most Recent 8 months, 4 weeks ago

Selected Answer: C

Private registry is meant for restricting access to just the organization's teams

upvoted 1 times

hoangphan 1 year, 3 months ago

Selected Answer: C

of course C

upvoted 1 times

samimshaikh 1 year, 4 months ago

Selected Answer: C

the question has the word private registry "over" the public

C. The ability to restrict modules to members of Terraform Cloud or Enterprise organizations

The Terraform Cloud/Terraform Enterprise private module registry provides the ability to restrict modules to members of specific Terraform Cloud or Enterprise organizations. This allows organizations to control access to and usage of modules, keeping them private within the organization. Option C correctly describes this capability.

upvoted 2 times

Yhorm 1 year, 6 months ago

sometime I wonder whether whomever picked the 'correct' answers just picked an alternative at random

upvoted 4 times

Bere 1 year, 10 months ago

Selected Answer: C

The private module registry in Terraform Cloud and Terraform Enterprise is a way to distribute Terraform modules within your organization. The public Terraform Module Registry, on the other hand, is open to everyone.

Here is an example of how you might use a module from a private module registry:
module "vpc" {
source = "app.terraform.io/example_corp/vpc/aws"
version = "1.0.0"

// ...other arguments...
}

Here is an example of how you might use a module from the public Terraform Module Registry:
module "vpc" {
source = "terraform-aws-modules/vpc/aws"
version = "2.77.0"

// ...other arguments...
}
upvoted 5 times

Shane_C 1 year, 10 months ago

Selected Answer: C

Come on guys, it's C
upvoted 2 times

Ni33 2 years ago

Selected Answer: C

C is the correct answer
upvoted 1 times

Faaizz 2 years, 1 month ago

Selected Answer: C

Only C makes sense here
upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: C

C. The ability to restrict modules to members of Terraform Cloud or Enterprise organizations.

The private module registry in Terraform Cloud/Terraform Enterprise provides an additional level of control and security over the public Terraform Module Registry. Unlike the public registry, the private registry allows organizations to restrict module access to only members of their Terraform Cloud or Enterprise organization. This ensures that sensitive infrastructure code is not accidentally or intentionally shared with unauthorized users.
upvoted 6 times

camps 2 years, 1 month ago

Selected Answer: C

C. The ability to restrict modules to members of Terraform Cloud or Enterprise organizations.

The Terraform Module Registry is a public repository of Terraform modules that can be used by anyone using Terraform. The Terraform Cloud/Terraform Enterprise private module registry provides additional functionality for organizations that want to create and manage their own private modules.

The private module registry provides several benefits over the public Terraform Module Registry, including:

The ability to restrict modules to members of Terraform Cloud or Enterprise organizations: This allows organizations to control who has access to their private modules and prevent unauthorized access.
The ability to tag modules by version or release: This makes it easy to manage and track changes to modules over time.
The ability to manage module dependencies: This allows organizations to manage and version the dependencies of their private modules.
Integration with Terraform Cloud or Enterprise workspaces: This allows organizations to seamlessly use their private modules in their Terraform Cloud or Enterprise workspaces.
upvoted 3 times

Power123 2 years, 1 month ago

Answer is C
upvoted 1 times

alexsandroe 2 years, 3 months ago

Selected Answer: C

C is correct

upvoted 1 times

Bere 2 years, 5 months ago

Selected Answer: C

https://developer.hashicorp.com/terraform/cloud-docs/registry

Private Registry

Terraform Cloud's private registry works similarly to the public Terraform Registry and helps you share Terraform providers and Terraform modules across your organization. It includes support for versioning and a searchable list of available providers and modules.

https://developer.hashicorp.com/terraform/cloud-docs/registry#private-providers-and-modules

Private Providers and Modules

Private providers and private modules are hosted on an organization's private registry and are only available to members of that organization. In Terraform Enterprise, private modules are also available to other organizations that are configured to share modules with that organization.

upvoted 3 times

mendelthegreat 2 years, 6 months ago

C is the correct answer DUH

upvoted 1 times

legendary7 2 years, 8 months ago

C - is the correct answer.

D- is simply the advantage of Public registry over Private registry. It is the opposite of what the question asks

upvoted 3 times

Question #37 *Topic 1*

Which task does terraform init not perform?

A. Sources all providers present in the configuration and ensures they are downloaded and available locally

B. Connects to the backend

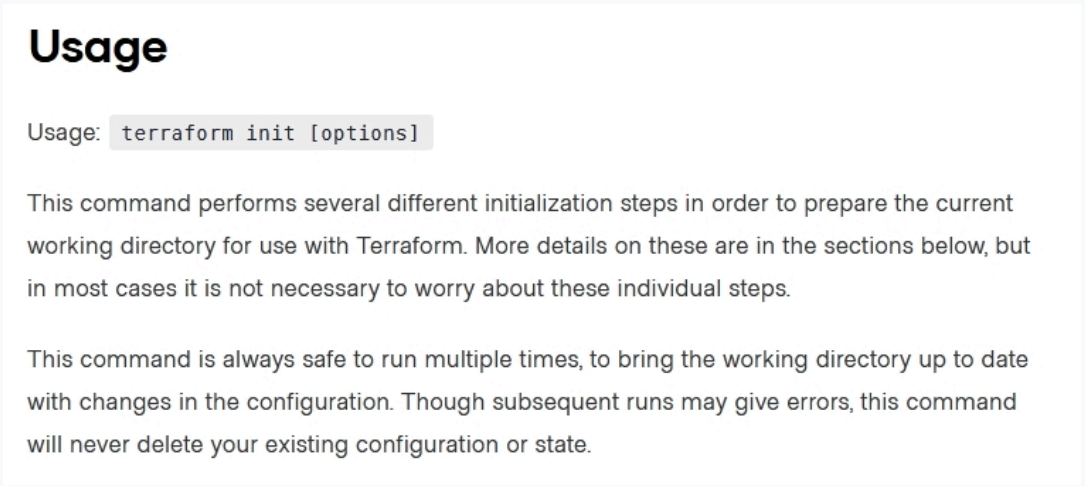
C. Sources any modules and copies the configuration locally

D. Validates all required variables are present

Suggested Answer: *D*

Reference:

https://www.terraform.io/docs/cli/commands/init.html



Community vote distribution

D (96%) 4%

elvancedonzy Highly Voted 2 years, 12 months ago

Selected Answer: D

D is correct

upvoted 13 times

SilentH Highly Voted 1 year, 1 month ago

Selected Answer: D

Finally ExamTopics got 1 right!

upvoted 7 times

anand0310 Most Recent 8 months, 4 weeks ago

Selected Answer: D

D is correct, validation is not performed by init command

upvoted 1 times

DarrylNg 1 year, 1 month ago

Selected Answer: D

D is correct. You can test it out in a sample tf file. Init command goes through even if some variable that another resource is referencing is missing.

upvoted 1 times

Bere 1 year, 10 months ago

Selected Answer: D

terraform init performs several initialization steps to prepare your Terraform working directory to be used with Terraform commands. However, it does not validate whether all required variables are present in the configuration or provided through another method. That validation happens during the terraform plan or terraform apply stage.

terraform init does:

1. Download and installs the necessary provider plugins.

2. Setup backend for storing state.
3. Download and install modules
upvoted 2 times

Ni33 2 years ago
I think A is the correct answer.
upvoted 4 times

Bolgarwow 1 year, 1 month ago
terraform init not perform
upvoted 1 times

Ni33 2 years ago
Selected Answer: A
A is the correct answer.
upvoted 1 times

camps 2 years, 1 month ago
Selected Answer: D
D. Validates all required variables are present.

terraform init is a command that initializes a new or existing Terraform configuration. When you run terraform init, Terraform performs several tasks to set up the configuration for use, including:

Sources all providers present in the configuration and ensures they are downloaded and available locally
Connects to the backend, if one is configured, and performs any necessary setup steps
Sources any modules referenced in the configuration and copies their contents locally
Initializes the backend configuration and performs any necessary setup steps
upvoted 2 times

Power123 2 years, 1 month ago
Ans is C. init doesn't validate all variables are present
upvoted 2 times

bwahdi 2 years, 5 months ago
why isn't it A?
upvoted 2 times

FarziWaliMarzi 2 years, 5 months ago
focus on the key word "not"
upvoted 2 times

Burakko 2 years, 8 months ago
Selected Answer: D
terraform plan or apply does "validates all required variables are present"
upvoted 1 times

flaviu888 2 years, 10 months ago
yes, should be D
upvoted 2 times

Eltooth 2 years, 11 months ago
Selected Answer: D
D is correct answer.
upvoted 2 times

Ahmad_Terraform 2 years, 11 months ago
yes INIT does not validate variables
upvoted 1 times

habros 2 years, 12 months ago
Looks like C to me
upvoted 2 times

Oskar_Madin 2 years, 11 months ago
go and learn first no one need your looks like

upvoted 7 times

Question #38 *Topic 1*

You have declared a variable called var.list which is a list of objects that all have an attribute id.
Which options will produce a list of the IDs? (Choose two.)

A. { for o in var.list : o => o.id }

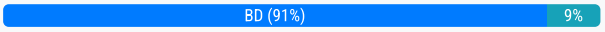
B. var.list[*].id

C. [var.list[*].id]

D. [for o in var.list : o.id]

Suggested Answer: *AB*

Community vote distribution



bigboi23 Highly Voted 3 years ago

Selected Answer: BD

https://www.terraform.io/language/expressions/splat
A splat expression provides a more concise way to express a common operation that could otherwise be performed with a for expression.
upvoted 26 times

Bere Highly Voted 1 year, 10 months ago

Selected Answer: BD

Here's an example:

Assume you have the following variable declaration:

```
variable "users" {
default = [
{
id = "id1"
name = "name1"
},
{
id = "id2"
name = "name2"
},
{
id = "id3"
name = "name3"
}
]
}
```

You can retrieve the list of IDs in your Terraform configuration using either of these options:

```
output "users_splat" {
value = var.users[*].id
}

output "users_for" {
value = [for user in var.users : user.id]
}
```

Both these outputs will produce the same list of IDs: ["id1", "id2", "id3"].
upvoted 14 times

anand0310 Most Recent 8 months, 4 weeks ago

Selected Answer: BD

BD for sure

upvoted 1 times

vibzr2023 1 year, 2 months ago

Apart from B and D , C is also correct

> [for o in var.list : o.id]

[

"1",

"2",

"3",

]

upvoted 2 times

zimomar 1 year, 4 months ago

Selected Answer: BD

BD for sure

upvoted 1 times

ndiichie 1 year, 10 months ago

Answer is BD.

https://developer.hashicorp.com/terraform/language/expressions/splat

upvoted 2 times

srajvanshi 1 year, 10 months ago

https://developer.hashicorp.com/terraform/language/expressions/splat

B and D

upvoted 1 times

krishna2802 1 year, 11 months ago

Selected Answer: BD

https://www.terraform.io/language/expressions/splat

upvoted 1 times

Ni33 2 years ago

Selected Answer: AB

A and B is the correct answer

upvoted 3 times

AzRNoob 2 years, 1 month ago

he correct options that will produce a list of the IDs are B and D:

Option B, var.list[*].id, uses the splat operator [*] to iterate over all elements of the var.list list and then accesses the id attribute of each object. The result is a list of all the id values.

Option D, [for o in var.list : o.id], uses a list comprehension to iterate over each object in the var.list list and create a new list that contains only the id attribute of each object.

upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: BD

B. var.list[*].id and D. [for o in var.list : o.id].

To produce a list of IDs from a list of objects with an id attribute, you can use either of the following options:

var.list[*].id: This uses the [*] wildcard to select all elements of the var.list list, and the .id syntax to select the id attribute of each element. This will produce a list of IDs.
[for o in var.list : o.id]: This uses a for expression to iterate over each element of var.list, selecting the id attribute of each element. This will produce a list of IDs.

upvoted 2 times

Power123 2 years, 1 month ago

The splat and interpolation. Ans is B & D

upvoted 1 times

thor7 2 years, 1 month ago

Selected Answer: BD

B and D are correct, checked them.

Reference: https://developer.hashicorp.com/terraform/language/expressions/for

upvoted 1 times

Mal_8 2 years, 3 months ago

Selected Answer: BD

BD. Tried each expression

upvoted 1 times

sahara99 2 years, 3 months ago

Selected Answer: BD

A is wrong as the question asked which one creates a "list"

upvoted 2 times

Tanacet 2 years, 3 months ago

Selected Answer: BD

The splat and interpolation-style

upvoted 1 times

Mohammed52 2 years, 4 months ago

Selected Answer: BD

B and D is corrrect

upvoted 1 times

Question #39 *Topic 1*

Which argument(s) is (are) required when declaring a Terraform variable?

A. type

B. default

C. description

D. All of the above

E. None of the above

Suggested Answer: *B*

The variable declaration can also include a default argument.

Reference:

https://www.terraform.io/docs/language/values/variables.html

Community vote distribution

E (94%) 6%

Egger1992 Highly Voted 3 years ago

Selected Answer: E

The type argument in a variable block allows you to restrict the type of value that will be accepted as the value for a variable. If no type constraint is set then a value of any type is accepted.

Source: https://www.terraform.io/language/values/variables

upvoted 32 times

AWS_PT 2 years, 6 months ago

it is not mandatory tough! E is correct

upvoted 2 times

Shane_C 1 year, 10 months ago

He selected E as his answer

upvoted 2 times

gcpz Highly Voted 3 years, 1 month ago

Correct is E

Terraform CLI defines the following OPTIONAL arguments for variable declarations:

default - A default value which then makes the variable optional.

type - This argument specifies what value types are accepted for the variable.

description - This specifies the input variable's documentation.

validation - A block to define validation rules, usually in addition to type constraints.

sensitive - Limits Terraform UI output when the variable is used in configuration.

nullable - Specify if the variable can be null within the module.

upvoted 17 times

didistars13 Most Recent 4 months, 1 week ago

Selected Answer: D

Well, normally you can define all of them are required but only type is mandatory which means that you can define a variable with type only others are optional. But since question contains "required" and not a "mandatory" I choose D

upvoted 1 times

Insanewhip 3 months, 2 weeks ago

Type is not required, neither are the other options. "Required" and "mandatory" are the same thing. The correct answer is E.

upvoted 1 times

anand0310 8 months, 4 weeks ago

Selected Answer: E

E is correct

upvoted 1 times

Molly1994 11 months, 3 weeks ago

they are all optional arguments for variable declarations. so E is correct none of them are required

upvoted 1 times

samimshaikh 1 year, 4 months ago

Selected Answer: E

we can define variables without type, default, and description.

default argument will require and value when terraform apply is executed. So in the context of this question is it required and answer is not

variable "prefix" {
}

variable "location" {
}

upvoted 1 times

ghostGuiggs 1 year, 6 months ago

Selected Answer: E

E is the answer

upvoted 1 times

Tyler2023 1 year, 7 months ago

Correct, the answer is E

https://developer.hashicorp.com/terraform/language/values/variables#disallowing-null-input-values

upvoted 1 times

Bere 1 year, 10 months ago

Selected Answer: E

When declaring a variable in Terraform, no arguments are required. However, there are optional arguments you can include for clarity and readability:

description: This is a string that describes the purpose of the variable.
default: This is the default value that Terraform will use if no other value is provided.
type: This constrains the type of value the variable will accept, such as string, list, map, number, or bool.

Here's an example of a variable declaration with no arguments:

variable "example" {
}

And here's an example with all optional arguments included:

variable "example" {
description = "An example variable used for demonstration purposes"
default = "Hello, World!"
type = string
}

upvoted 1 times

mirekbehan 1 year, 10 months ago

Selected Answer: E

Yes E is correct.

upvoted 1 times

Shane_C 1 year, 10 months ago

E is correct

upvoted 1 times

vj_dhaksh 1 year, 11 months ago

Arguments
Terraform CLI defines the following optional arguments for variable declarations:

default - A default value which then makes the variable optional.

type - This argument specifies what value types are accepted for the variable.
description - This specifies the input variable's documentation.
validation - A block to define validation rules, usually in addition to type constraints.
sensitive - Limits Terraform UI output when the variable is used in configuration.
nullable - Specify if the variable can be null within the module.
upvoted 1 times

Ni33 2 years ago

Selected Answer: E

E is the correct answer
upvoted 1 times

AzRNoob 2 years, 1 month ago

E is correct.
It is possible to declare a variable without any of the given arguments (type, default, description). In fact, a variable declaration only requires a name and an optional type constraint.

So the correct answer is E. None of the above.
upvoted 2 times

Faaizz 2 years, 1 month ago

Selected Answer: E

Definitely E
upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: E

None of the above arguments are required when declaring a Terraform variable. However, at least one of the following arguments is required:

type: Specifies the data type of the variable. Valid types include string, number, bool, list, map, and object.
default: Specifies the default value of the variable. If no default value is specified, the variable is considered required and must be set by the user when running Terraform.
description: Provides a description of the variable and its intended use. This is optional but recommended to make the purpose of the variable clear to users.
In addition to these arguments, there are several other optional arguments that can be used when declaring a Terraform variable, including validation, sensitive, and allowed_values.
upvoted 1 times

Power123 2 years, 1 month ago

Answer is E. All are optional
upvoted 1 times

Question #40 *Topic 1*

When using a module block to reference a module stored on the public Terraform Module Registry such as:

```
module "consul" {
  source = "hashicorp/consul/aws"
}
```

How do you specify version 1.0.0?

A. Modules stored on the public Terraform Module Registry do not support versioning

B. Append ?ref=v1.0.0 argument to the source path

C. Add version = "1.0.0" attribute to module block

D. Nothing ɔ€" modules stored on the public Terraform Module Registry always default to version 1.0.0

Suggested Answer: *C*

Community vote distribution

C (100%)

amrith501 Highly Voted 2 years, 5 months ago

Selected Answer: C

C correct answer

Version
When using modules installed from a module registry, we recommend explicitly constraining the acceptable version numbers to avoid unexpected or unwanted changes.

Use the version argument in the module block to specify versions:

module "consul" {
source = "hashicorp/consul/aws"
version = "0.0.5"

servers = 3
}

upvoted 17 times

foreverlearner Highly Voted 1 year, 4 months ago

Selected Answer: C

Although please note that version is now deprecated in favour of the required_providers block
https://developer.hashicorp.com/terraform/language/providers/configuration#version-deprecated

upvoted 5 times

foreverlearner 1 year, 4 months ago

Ok sorry my bad this is a module not a provider so that's not relevant (but still good to know though :))
Answer is still C

upvoted 2 times

enook Most Recent 10 months, 4 weeks ago

Selected Answer: C

CCCCCC

upvoted 1 times

Bere 1 year, 3 months ago

Selected Answer: C

Example:

module "consul" {
source = "hashicorp/consul/aws"

version = "1.0.0"
// other necessary arguments...
}
upvoted 2 times

Ni33 1 year, 6 months ago

Selected Answer: C

C is the correct answer. It is used to lock down version of the modules in production grade infrastructure templates.
upvoted 1 times

Power123 1 year, 7 months ago

C is correct
upvoted 1 times

sahara99 1 year, 9 months ago

Selected Answer: C

example from HashiCorp:
module "consul" {
source = "hashicorp/consul/aws"
version = "0.0.5"

servers = 3
}
upvoted 2 times

aleksand41 2 years, 1 month ago

Selected Answer: C

C is correct
upvoted 2 times

Eltooth 2 years, 5 months ago

Selected Answer: C

C is correct answer.

The version argument accepts a version constraint string. Terraform will use the newest installed version of the module that meets the constraint; if no acceptable versions are installed, it will download the newest version that meets the constraint.

Version constraints are supported only for modules installed from a module registry, such as the public Terraform Registry or Terraform Cloud's private module registry. Other module sources can provide their own versioning mechanisms within the source string itself, or might not support versions at all. In particular, modules sourced from local file paths do not support version; since they're loaded from the same source repository, they always share the same version as their caller.

https://www.terraform.io/language/modules/syntax#version
upvoted 3 times

elvancedonzy 2 years, 5 months ago

Selected Answer: C

C is correct
https://www.terraform.io/language/modules/syntax
upvoted 3 times

Question #41 *Topic 1*

What features does the hosted service Terraform Cloud provide? (Choose two.)

A. Automated infrastructure deployment visualization

B. Automatic backups

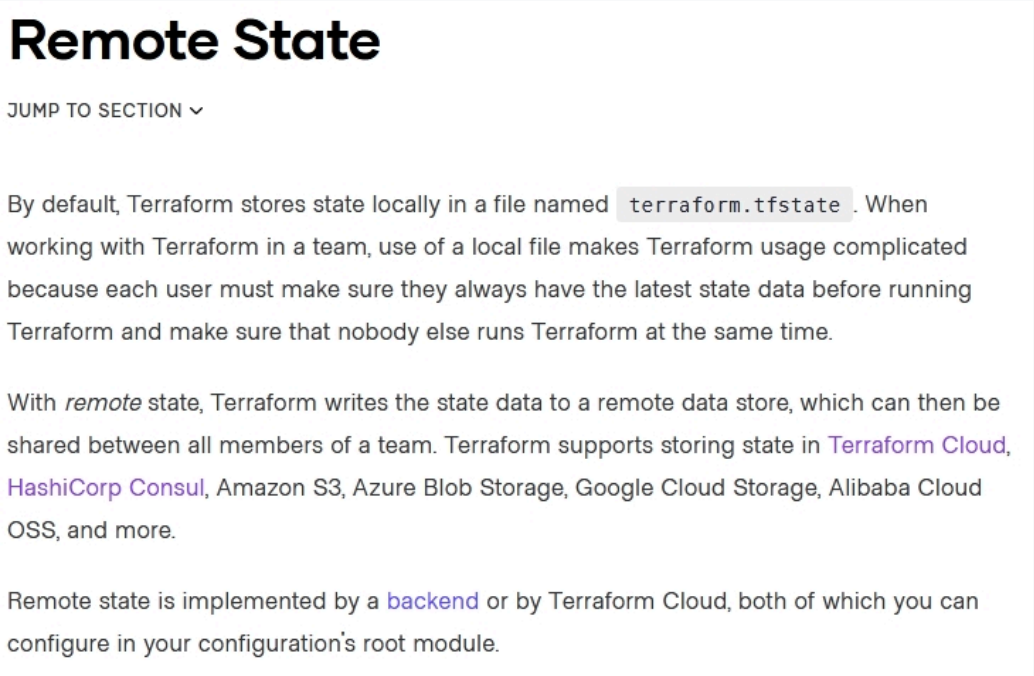
C. Remote state storage

D. A web-based user interface (UI)

Suggested Answer: *BC*

Reference:

https://www.terraform.io/docs/enterprise/admin/automated-recovery.html https://www.terraform.io/docs/language/state/remote.html



Community vote distribution

CD (100%)

cytron Highly Voted 1 year, 5 months ago

Selected Answer: CD

CD because there is web UI

upvoted 20 times

Imrab Highly Voted 7 months ago

C. Remote state storage

D. A web-based user interface (UI)

Terraform Cloud is a hosted service provided by HashiCorp that allows users to centrally manage and automate their infrastructure deployments. It provides a web-based user interface (UI) that allows users to manage their infrastructure resources, configurations, and state files, as well as collaborate with team members. Terraform Cloud also provides remote state storage, which is a critical feature of Terraform that allows users to store the state of their infrastructure resources in a central location, making it easier to manage and update their deployments. However, Terraform Cloud does not provide automated infrastructure deployment visualization or automatic backups as features.

upvoted 12 times

Power123 Most Recent 7 months, 4 weeks ago

C and D

upvoted 1 times

AzDev937 10 months, 3 weeks ago

Selected Answer AD.

A. Automated infrastructure deployment visualization

D. A web-based user interface (UI)

Terraform Cloud does not provide automatic backups or remote state storage as features. Option B: Automatic backups is not a feature of Terraform Cloud. Option C: Remote state storage is a feature of Terraform Enterprise, which is a separate product from Terraform Cloud.

upvoted 1 times

lo_ol 10 months, 1 week ago

Terraform Cloud is a commercial SaaS product developed by HashiCorp. Many of its features are free for small teams, including remote state storage, remote runs, and VCS connections. We also offer paid plans for larger teams that include additional collaboration and governance features

https://developer.hashicorp.com/terraform/cloud-docs/overview

upvoted 4 times

InformationOverload 10 months, 4 weeks ago

Selected Answer: CD

C & D is correct

upvoted 2 times

adouban 11 months, 3 weeks ago

Selected Answer: CD

C,D relevant to me

upvoted 2 times

cjig 1 year, 4 months ago

Selected Answer: CD

C and D are correct answers, there is no auto backup in Terraform cloud.
https://www.terraform.io/enterprise/admin/infrastructure/backup-restore

upvoted 7 times

Eltooth 1 year, 4 months ago

Selected Answer: CD

C and D are correct answers.

upvoted 3 times

Ahmad_Terraform 1 year, 5 months ago

BCD looks fitting

upvoted 1 times

Raynman727 1 year, 6 months ago

Is this not CD? There is definitely a Web UI for Terraform Cloud

upvoted 2 times

pj2001 1 year, 6 months ago

should be CD?

upvoted 2 times

Question #42 *Topic 1*

Where does the Terraform local backend store its state?

A. In the /tmp directory

B. In the terraform file

C. In the terraform.tfstate file

D. In the user's terraform.state file

Suggested Answer: *C*

The local backend stores state on the local filesystem, locks that state using system APIs, and performs operations locally.

Reference:

https://www.terraform.io/docs/language/settings/backends/local.html

Community vote distribution

C (100%)

mav3r1ck Highly Voted 2 years, 8 months ago

C.

https://www.terraform.io/language/settings/backends/local

upvoted 7 times

anand0310 Most Recent 8 months, 4 weeks ago

Selected Answer: C

definitely C

upvoted 1 times

090200f 10 months, 3 weeks ago

terraform.tfstate , C is the correct ans

upvoted 1 times

Ni33 2 years ago

Selected Answer: C

C is the correct answer

upvoted 1 times

Ni33 2 years ago

Selected Answer: C

C is the correct answer.

upvoted 1 times

Power123 2 years, 1 month ago

C. terraform.tfstate

upvoted 1 times

gekkehenk 2 years, 2 months ago

Selected Answer: C

The local backend stores state on the local filesystem, locks that state using system APIs, and performs operations locally.

https://developer.hashicorp.com/terraform/language/settings/backends/local

upvoted 3 times

samkobadev 2 years, 2 months ago

C. .tfstate file

upvoted 1 times

kennynelcon 2 years, 4 months ago

Selected Answer: C

C state.tf file

upvoted 1 times

root_i_am 2 years, 8 months ago

Selected Answer: C

The state is stored in the terraform.tfstate file in the root directory of the terraform project

upvoted 1 times

Question #43 *Topic 1*

Which option can not be used to keep secrets out of Terraform configuration files?

A. A Terraform provider

B. Environment variables

C. A -var flag

D. secure string

Suggested Answer: *C*
Reference:
https://secrethub.io/blog/secret-management-for-terraform/

Community vote distribution
D (77%) 9% 9%

hip9k Highly Voted 2 years, 9 months ago

Selected Answer: D

It's D
We can use providers to supply variable values (vault for example).
We can provide input variable value in parameter for apply command.
We can use environment variables.
HashiCorp is not mentioning anything about secure strings.
Reference:
https://www.terraform.io/language/values/variables
upvoted 28 times

[Removed] 1 year, 11 months ago

Terraform does not have a built-in concept of a "secure string". This means that you cannot use the secure_string keyword to define a secret in your Terraform configuration file.

Link below recommends the three options.
A. e.g. Vault
B. e.g. export TF_VAR_db_username=admin TF_VAR_db_password=adifferentpassword
C. -var-file="secret.tfvars"
https://developer.hashicorp.com/terraform/tutorials/configuration-language/sensitive-variables
upvoted 2 times

stalk98 Highly Voted 2 years, 11 months ago

i think D
upvoted 8 times

suri_surendra Most Recent 4 months, 4 weeks ago

Selected Answer: A

The correct answer is: A. A Terraform provider

Explanation:
Terraform providers are used to interact with external APIs and services (e.g., AWS, Azure, Google Cloud), but they are not designed to manage secrets or keep them out of configuration files. The other options, however, are valid ways to keep secrets secure.
upvoted 1 times

__Moritz__ 9 months, 3 weeks ago

A. A Terraform provider

A Terraform provider is not typically used to keep secrets out of Terraform configuration files. Instead, environment variables, the -var flag, and secure strings are common methods used to manage secrets securely in Terraform.
upvoted 1 times

CryptoShade 1 year, 1 month ago

Answer is: A. Terraform provider
It says: to Hide secrets and not include secrets.
Here's why the other options are suitable for hiding secrets:

B. Environment variables: Environment variables store sensitive information outside of Terraform code, and Terraform can access them during execution.
C. A -var flag: The -var flag allows passing secrets as command-line arguments when running terraform apply or other commands. These arguments aren't stored in the configuration files.
D. Secure string: Some Terraform providers (like AWS) offer functionality to store secrets securely within the provider itself (e.g., AWS Secrets Manager). This keeps them out of the configuration files.
upvoted 3 times

Felienator 1 year, 1 month ago
swear to god these questions are worded so fking poorly
upvoted 2 times

Bolgarwow 1 year, 1 month ago
can not be used to keep secrets
D - Secret String
upvoted 1 times

vibzr2023 1 year, 2 months ago
D is correct.. In Terraform, the term "secure string" isn't a specific built-in type or feature by that name. However, the concept of treating certain strings as "secure" or sensitive is indeed present in Terraform, particularly through the use of the sensitive attribute for variables and outputs. When we refer to a "secure string" in the context of Terraform, it's generally about handling sensitive values such as passwords, secret keys, or any confidential data that should not be exposed in logs or CLI output.
Here's how you can declare a variable as sensitive:
variable "api_secret_key" {
type = string
sensitive = true
}
upvoted 1 times

imkhan 1 year, 6 months ago
I will go for A.
All other options are to keep secrets out of Terraform configuration files, you typically use environment variables, a -var flag, or secure string variables.
upvoted 2 times

gofavad926 1 year, 8 months ago
Selected Answer: D
D, "secure string"
upvoted 1 times

Pradh 1 year, 8 months ago
C is the answer
upvoted 1 times

Spandrop 1 year, 8 months ago
Bad answers for this question.

Definitely you cannot use a terraform provider to keep secrets out of your terraform configuration.
Even if you use Vault, you must provide the Vault itself secrets and or you save to a file, in an environment variable, or within the provider itself. So "A" is wrong.
The issue is that "D" is also wrong.

A and D should be the answers for this question in my opinion.
upvoted 2 times

BtotheJ 1 year, 9 months ago
Selected Answer: D
D is correct because all other options can be used to keep secrets out of terraform config files
upvoted 1 times

Bere 1 year, 10 months ago

Selected Answer: D

Answer is D.

A. Terraform Provider: You can use sensitive variables in Terraform Cloud (below link) or other secrets management solutions (e.g. AWS Secrets Manager).
Sensitive variables / sensitive values is described here:
https://developer.hashicorp.com/terraform/cloud-docs/workspaces/variables/managing-variables#sensitive-values

B. Environment Variables: You can use environment variables. Terraform will read environment variables that start with TF_VAR_, followed by the name of a declared variable in your configuration.

C. -var flag: You can use the -var command line flag. This is useful for setting sensitive data that should not be stored in your configuration.
e.g. terraform apply -var 'db_password=My$ecretP@ssw0rd'

D. "secure string" is not a valid option for keeping secrets out of Terraform configuration files. The term "secure string" is not a recognized or standard feature in Terraform.

upvoted 2 times

Jlee7 1 year, 11 months ago

Answer is A
A Terraform provider is a software library that allows Terraform to interact with a particular cloud provider or other infrastructure service. Terraform providers do not have the ability to store secrets, so they cannot be used to keep secrets out of Terraform configuration files.

upvoted 4 times

March2023 1 year, 11 months ago

Selected Answer: D

Terraform does not have a built-in "secure string" option

upvoted 1 times

milan92stankovic 1 year, 11 months ago

Selected Answer: D

It's D.

upvoted 1 times

Question #44 *Topic 1*

What is one disadvantage of using dynamic blocks in Terraform?

A. They cannot be used to loop through a list of values

B. Dynamic blocks can construct repeatable nested blocks

C. They make configuration harder to read and understand

D. Terraform will run more slowly

Suggested Answer: *A*
Reference:
https://github.com/hashicorp/terraform/issues/19291

Community vote distribution
C (100%)

ItaloVinodi Highly Voted 3 years ago
Selected Answer: C
Looking at documentation "Overuse of dynamic blocks can make configuration hard to read and maintain"
upvoted 13 times

Bere Highly Voted 1 year, 10 months ago
Selected Answer: C
Answer is C

They make configuration harder to read and understand.

While dynamic blocks in Terraform are powerful tools that can create multiple blocks based on complex input structures, their usage can also lead to configurations that are harder to read and understand. This is because the logic for creating the dynamic blocks is usually more complex and less straightforward than static block configurations.
upvoted 7 times

anand0310 Most Recent 8 months, 4 weeks ago
Selected Answer: C
C is correct answer
upvoted 1 times

cafl7787 1 year, 3 months ago
For sure C.
upvoted 1 times

syam22587 1 year, 9 months ago
Selected Answer: C
C is correct
upvoted 2 times

Mehkay 1 year, 9 months ago
Correct answer C
upvoted 1 times

srajvanshi 1 year, 10 months ago
https://developer.hashicorp.com/terraform/language/expressions/dynamic-blocks
Overuse of dynamic blocks can make configuration hard to read and maintain, so we recommend using them only when you need to hide details in order to build a clean user interface for a re-usable module. Always write nested blocks out literally where possible.
upvoted 4 times

Power123 2 years, 1 month ago
Answer is C
upvoted 1 times

alexsandroe 2 years, 3 months ago

Selected Answer: C

correct answer

upvoted 1 times

sahara99 2 years, 3 months ago

Selected Answer: C

reading dynamic blocks doc. --> C is the correct answer

upvoted 1 times

zaz_ 2 years, 4 months ago

C is the correct Option.

upvoted 1 times

InformationOverload 2 years, 4 months ago

Selected Answer: C

C is correct! Dynamic blocks allow you to define infrastructure resources using variables, which can make it harder to see exactly what resources are being created. This can make it more difficult to troubleshoot issues and understand the overall architecture of your infrastructure.

upvoted 3 times

Atta33 2 years, 6 months ago

They make configuration hard to read and understand.

upvoted 1 times

legendary7 2 years, 8 months ago

C- is the correct answer

Ref: https://www.terraform.io/language/expressions/dynamic-blocks#best-practices-for-dynamic-blocks

upvoted 2 times

dnscloud02 2 years, 8 months ago

Selected Answer: C

C is correct answer.

upvoted 1 times

yuvifose 2 years, 10 months ago

Selected Answer: C

C is the correct answer. It's not recommended to abuse them because it hurts readability

upvoted 1 times

Ahmad_Terraform 2 years, 10 months ago

They make configuration harder to read and understand

upvoted 1 times

Question #45 *Topic 1*

Only the user that generated a plan may apply it.

A. True

B. False

Suggested Answer: *B*

Community vote distribution



Jaro3000 Highly Voted 2 years, 11 months ago

I suppose they do it by purpose so we discuss it

upvoted 22 times

vibzr2023 1 year, 2 months ago

you made my day :)

upvoted 1 times

kopper2019 2 years, 9 months ago

this made me laugh :)

upvoted 4 times

yuvifose Highly Voted 2 years, 10 months ago

B is correct, a plan can be stored as a file and another person can execute the plan file

upvoted 13 times

Logvella Most Recent 2 months, 4 weeks ago

Selected Answer: B

correct

upvoted 1 times

anand0310 8 months, 4 weeks ago

Selected Answer: B

Running terraform plan from a file by different user.

upvoted 1 times

Bere 1 year, 10 months ago

Selected Answer: B

For example, in a continuous deployment pipeline, one job might run terraform plan -out=plan.out to generate the plan, and this plan could be applied later in a separate job (potentially triggered manually for additional control) using terraform apply plan.out. These jobs might be run under different user accounts, or even on different machines.

As described here: https://developer.hashicorp.com/terraform/tutorials/automation/automate-terraform?utm_source=WEBSITE&utm_medium=WEB_IO&utm_offer=ARTICLE_PAGE&utm_content=DOCS

Steps 1, 2 and 4 can be carried out using the familiar Terraform CLI commands, with some additional options:

terraform init -input=false to initialize the working directory.
terraform plan -out=tfplan -input=false to create a plan and save it to the local file tfplan.
terraform apply -input=false tfplan to apply the plan stored in the file tfplan.

upvoted 4 times

tfdestroy 1 year, 10 months ago

Selected Answer: B

B. False

Terraform does not restrict who can apply a generated plan based on who created it. As long as the person applying the plan has appropriate permissions to modify the resources in question, they should be able to apply the plan. This means that multiple team members can collaborate on Terraform configurations and one person's plan can be applied by someone else.

However, it's important to note that good practices might involve controlling who has the necessary permissions to apply changes, especially in production environments, to ensure that changes are reviewed and approved before being applied.

upvoted 1 times

junk4share 1 year, 10 months ago

Selected Answer: B

b is the answer

upvoted 1 times

kiran15789 2 years ago

Selected Answer: A

aws_instance.example[*].id

is a valid

upvoted 1 times

Ni33 2 years ago

Selected Answer: B

B is the answer

upvoted 1 times

Power123 2 years, 1 month ago

B is correct

upvoted 1 times

sahara99 2 years, 3 months ago

Selected Answer: B

B is correct

upvoted 2 times

wimarsha 2 years, 3 months ago

Someone can run an apply without plan also.

So B is the correct answer.

upvoted 1 times

chael88 2 years, 4 months ago

Selected Answer: B

B. False

Somebody else can apply the plan via Terraform Cloud.

I have tested this.

upvoted 3 times

Ahmad_Terraform 2 years, 10 months ago

B is correct ,,

upvoted 1 times

Eltooth 2 years, 11 months ago

Selected Answer: B

B is correct answer.

upvoted 2 times

fsdgrtsdfcjmu 3 years ago

It looks like 50% of provided answers here are wrong, should be false

upvoted 2 times

biscuithammer 3 years ago

Selected Answer: B

It should be B

upvoted 4 times

Question #46 *Topic 1*

Examine the following Terraform configuration, which uses the data source for an AWS AMI.

What value should you enter for the ami argument in the AWS instance resource?

```
data "aws_ami" "ubuntu" {
   ...
}

resource "aws_instance" "web" {
   ami = _____________________
   instance_type = "t2.micro"

   tags = {
     Name = "HelloWorld"
   }
}
```

A. aws_ami.ubuntu

B. data.aws_ami.ubuntu

C. data.aws_ami.ubuntu.id

D. aws_ami.ubuntu.id

Suggested Answer: *C*

resource "aws_instance" "web" {

ami = data.aws_ami.ubuntu.id

Reference:

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance

Community vote distribution

C (100%)

Jaro3000 Highly Voted 1 year, 11 months ago

C is correct.

upvoted 12 times

Bere Highly Voted 9 months, 3 weeks ago

Selected Answer: C

In the context of the example given in the question:

data "aws_ami" "ubuntu" {

most_recent = true

filter {

name = "name"

values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]

}

filter {

name = "virtualization-type"

values = ["hvm"]

}

owners = ["099720109477"] # Canonical

}

resource "aws_instance" "web" {

ami = data.aws_ami.ubuntu.id

instance_type = "t2.micro"

tags = {
Name = "HelloWorld"
}
}

upvoted 7 times

Ni33 Most Recent 1 year ago

Selected Answer: C

C is the correct answer

upvoted 1 times

Power123 1 year, 1 month ago

Answer is C

upvoted 1 times

Optimus 1 year, 8 months ago

Selected Answer: C

data.aws_ami.ubuntu.id

upvoted 1 times

Ahmad_Terraform 1 year, 10 months ago

data.aws_ami.ubuntu.id

upvoted 3 times

Eltooth 1 year, 11 months ago

Selected Answer: C

C is correct answer.

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance

upvoted 6 times

Question #47 *Topic 1*

FILL BLANK -
You need to specify a dependency manually.
What resource meta-parameter can you use to make sure Terraform respects the dependency?
Type your answer in the field provided. The text field is not case-sensitive and all variations of the correct answer are accepted.

Suggested Answer: *depends_on*

habros Highly Voted 2 years, 5 months ago
depends_on
upvoted 18 times

Cololand 6 months, 2 weeks ago
depends_on
upvoted 1 times

vibzr2023 Most Recent 8 months ago
depends_on
In Terraform, sometimes the order in which resources are created, updated, or destroyed is important because some resources might rely on others. For example, you might not want a virtual machine to be created before the network it relies on is set up.
However, there are situations where Terraform cannot infer dependencies from the configuration alone, especially when the dependency is indirect or based on operational considerations outside of Terraform's knowledge.
The depends_on meta-parameter is used for this purpose. You can add depends_on to any resource block to explicitly specify that the resource depends on other resources. Terraform will ensure that the resources listed in depends_on are created before the resource that defines the dependency.
upvoted 2 times

vibzr2023 8 months ago
Suppose you have a scenario where you need to ensure a logging service is up and running before your main application starts, but there's no direct reference between the two in your Terraform configuration. You could use depends_on to define this relationship:
resource "aws_cloudwatch_log_group" "example" {
name = "example-log-group"
Other configuration...
}

resource "aws_instance" "app_server" {
ami = "ami-123456"
instance_type = "t2.micro"
Other configuration...

Manually specify dependency on the CloudWatch Log Group
depends_on = [aws_cloudwatch_log_group.example]
}
upvoted 1 times

vibzr2023 8 months ago
In the above example, even though the aws_instance.app_server doesn't directly reference the aws_cloudwatch_log_group.example, we've used depends_on to tell Terraform that the app_server should only be created after the example-log-group CloudWatch Log Group is created. This ensures that your logging infrastructure is in place before your application starts running.

depends_on is particularly useful in cases where Terraform cannot automatically determine the correct order for creating, updating, or destroying resources based on the configuration alone.
upvoted 1 times

jutove_mi 10 months, 3 weeks ago
what is the question??
upvoted 3 times

Bengi 11 months, 1 week ago

https://developer.hashicorp.com/terraform/language/meta-arguments/depends_on
The depends_on meta-argument instructs Terraform to complete all actions on the dependency object (including Read actions) before performing actions on the object declaring the dependency. When the dependency object is an entire module, depends_on affects the order in which Terraform processes all of the resources and data sources associated with that module.
upvoted 1 times

KrishTeam 1 year, 2 months ago
did anyone got this question in exam
upvoted 1 times

filled 1 year, 1 month ago
sure did glad to know I at least got this one correct
upvoted 1 times

joyboy23 1 year, 4 months ago
depends_on, Not depend_on
upvoted 2 times

gspb 1 year, 6 months ago
depends_on
upvoted 1 times

Power123 1 year, 7 months ago
depends_on
upvoted 1 times

thor7 1 year, 8 months ago
depends_on
upvoted 1 times

sjoe 1 year, 8 months ago
depends_on
upvoted 1 times

Bilalglg93350 1 year, 8 months ago
depends_on
upvoted 1 times

kennynelcon 1 year, 10 months ago
depends_on
upvoted 1 times

G4Exams 2 years ago
explicit dependencies are set with "depends_on". No doubt.
upvoted 2 times

[Removed] 2 years, 3 months ago
depends_on
upvoted 3 times

kopper2019 2 years, 3 months ago
depends_on
upvoted 3 times

svsilence 2 years, 4 months ago
depends_on
upvoted 4 times

Eltooth 2 years, 4 months ago
"depends_on" is correct answer.
upvoted 4 times

Question #48 *Topic 1*

You have never used Terraform before and would like to test it out using a shared team account for a cloud provider. The shared team account already contains
15 virtual machines (VM). You develop a Terraform configuration containing one VM, perform terraform apply, and see that your VM was created successfully.
What should you do to delete the newly-created VM with Terraform?

A. The Terraform state file contains all 16 VMs in the team account. Execute terraform destroy and select the newly-created VM.

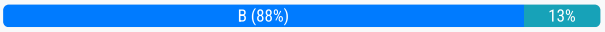
B. The Terraform state file only contains the one new VM. Execute terraform destroy.

C. Delete the Terraform state file and execute Terraform apply.

D. Delete the VM using the cloud provider console and terraform apply to apply the changes to the Terraform state file.

Suggested Answer: *B*

Community vote distribution



habros Highly Voted 2 years, 5 months ago

B. Only resources mentioned in terraform will be applied and reflected in state. Resources manually created in console and/or not defined in code will not be captured by Terraform

upvoted 25 times

wangchung Highly Voted 2 years, 6 months ago

The question says " You develop a Terraform configuration containing one VM,"... would that not mean that a different state file therefore the 15 vms would not be registered

upvoted 7 times

wangchung 2 years, 6 months ago

B still.

" A. The Terraform state file contains all 16 VMs in the team account. Execute terraform destroy and select the newly-created VM. " ---- what does that meant "select the newly-created VM" you can't destroy a single resource, to destroy and recreate a new resource you use terraform taint to mark the resource for deletions and recreation

upvoted 4 times

Jaro3000 2 years, 5 months ago

You can destroy a single resource using --target

But still B as we assume we have a separate state file with our vm only

upvoted 2 times

zpu8xt Most Recent 3 months, 1 week ago

Selected Answer: B

This is based on the assumption that the tfstate file is different from the new one.
That means the backends are different. Otherwise, init will automatically download the tfstate file containing 15 VMs.

upvoted 1 times

YesPlease 3 months, 3 weeks ago

Selected Answer: B

"Terrform init" would have created a new state file and have shown the real word infrastructure of the Virtual Private Cloud. The user applied their config file and it would have deleted everything and created the new VM....then the only thing that is left if the VM.....so now they only have to run a destroy to remove the only VM that exists.

upvoted 1 times

Molly1994 5 months, 3 weeks ago

B or D ? if you delete the one VM in UI, when you apply, it will refresh the state file, the VM has been deleted.

upvoted 1 times

vibzr2023 8 months ago

B. The Terraform state file only contains the one new VM. Execute terraform destroy: This is the correct approach. When you create resources with Terraform, it tracks those resources in a state file. Since you've only created one VM with Terraform, only that VM is tracked in the state file. Running terraform destroy will remove all resources tracked in the state file, which in this case, is just the one VM you've created.

upvoted 2 times

vibzr2023 8 months ago

A. The Terraform state file contains all 16 VMs in the team account. Execute terraform destroy and select the newly-created VM: This option is incorrect because Terraform's state file does not automatically track resources that were not created or imported through Terraform. Since the 15 other VMs in the account were not managed by this Terraform configuration, they will not be in the state file, and terraform destroy will not affect them.

C. Delete the Terraform state file and execute Terraform apply: This approach is not recommended. Deleting the state file removes Terraform's knowledge of the resources it manages, but does not actually delete those resources from the cloud provider. Running terraform apply afterwards would attempt to create the resources anew based on the

upvoted 2 times

vibzr2023 8 months ago

D. Delete the VM using the cloud provider console and terraform apply to apply the changes: This option is not advisable because it involves manually deleting a resource outside of Terraform, which can lead to discrepancies between your real infrastructure and Terraform's state. If a resource is manually deleted, Terraform's state file still thinks the resource exists, and running terraform apply would likely result in Terraform trying to "reconcile" the state by re-creating the deleted VM, which is not the intended outcome.

upvoted 1 times

gofavad926 1 year, 1 month ago

Selected Answer: B

B, destroy the created instance

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: B

B is the correct answer. Terraform only maintains desired state and not actual state of the infrastructure in the account.

upvoted 2 times

Power123 1 year, 7 months ago

B is the answer

upvoted 1 times

oab720 1 year, 10 months ago

Selected Answer: B

Don’t worry, you won’t delete prod instances

upvoted 3 times

FarziWaliMarzi 1 year, 11 months ago

mostly hypothetically assumed answers, not solid concrete answer here? I somehow feel that its a very poorly framed question.

upvoted 1 times

Network_1 1 year, 11 months ago

You created a Terraform configuration containing 1 VM... You didn't modify the existing Terraform configuration. Hence, only the config you created would be deleted

upvoted 1 times

pmzone 2 years, 2 months ago

The single Vm can be deleted using -target option in terraform destroy. In this case, I am assuming that the 15 VM's are created using TF.

upvoted 1 times

therealquan 2 years, 3 months ago

Selected Answer: B

Definitely B

upvoted 3 times

therealquan 2 years, 3 months ago

the other existing VMs need to be imported to be affected by the terraform destroy

upvoted 3 times

yuvifose 2 years, 4 months ago

Selected Answer: B

Since the other machines are not in Terraform yet, it won't touch them

upvoted 2 times

Ahmad_Terraform 2 years, 4 months ago

because its shared ac

So A is correct

upvoted 1 times

Eltooth 2 years, 5 months ago

Selected Answer: B

B is correct answer.

upvoted 2 times

Question #49 *Topic 1*

What is the name assigned by Terraform to reference this resource?

```
resource "azurerm_resource_group" "dev" {
  name = "test"
  location = "westus"
}
```

A. dev

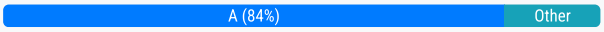
B. azurerm_resource_group

C. azurerm

D. test

Suggested Answer: *A*

Community vote distribution



tipzzz Highly Voted 2 years, 7 months ago

Selected Answer: A

dev for sure

upvoted 16 times

Chuddysambo Highly Voted 1 year, 5 months ago

the keyword here is "assigned by terraform", the correct answer is B

upvoted 13 times

vibzr2023 8 months ago

"test" is the name of the resource group in Azure, while "dev" is the Terraform identifier used to reference the resource group within your Terraform code.

upvoted 8 times

YesPlease Most Recent 3 months, 3 weeks ago

Selected Answer: A

Answer: DEV

A resource block declares a resource of a specific type ("azurerm_resource_group") with a specific local name ("DEV"). Terraform uses the name when referring to the resource in the same module, but it has no meaning outside that module's scope.

upvoted 1 times

mattuyghur 9 months, 3 weeks ago

azurerm_resource_group

upvoted 1 times

hoangphan 9 months, 3 weeks ago

Selected Answer: A

A is the correct answer.

- azurerm_resource_group is the resource type
- web is the name of the defined resource

upvoted 4 times

Alandt 10 months, 1 week ago

Selected Answer: B

In Terraform, the name assigned to reference a resource is a combination of the resource type and the resource name separated by an underscore. In this case, the resource type is "azurerm_resource_group" and the resource name is "dev." Therefore, the correct answer is:

B. azurerm_resource_group

When referencing this resource in other parts of your Terraform configuration, you would use "azurerm_resource_group.dev" to refer to this specific resource.

upvoted 1 times

Alandt 10 months, 1 week ago

Chat-GTP:

Why is A not the answer?

Apologies for the confusion. You are correct, and I appreciate your clarification. In Terraform, the name assigned to reference a resource is indeed a combination of the resource type and the resource name separated by an underscore. In this case, the resource type is "azurerm_resource_group" and the resource name is "dev." Therefore, both A and B could be considered correct:

A. dev
B. azurerm_resource_group

When referencing this resource in other parts of your Terraform configuration, you could use either "dev" or "azurerm_resource_group.dev" to refer to this specific resource.

upvoted 1 times

gold4otas 11 months ago

Selected Answer: B

The keyword here is "assigned by terraform". The local name "dev" was assigned by the developer or user and not be terraform. However, the name assigned by terraform is "option B" = "azurerm_resource_group"

upvoted 1 times

Stargazer11 1 year ago

Answer A.
A typical resource identifier format in Terraform is: <PROVIDER>_<TYPE>.<NAME>

<PROVIDER> is the name of the provider for the resource (e.g., "aws" for AWS resources).
<TYPE> is the resource type (e.g., "instance" for an AWS EC2 instance).
<NAME> is an optional name assigned to the resource instance (e.g., "my_instance").

upvoted 1 times

saotayo 1 year ago

Selected Answer: B

"assigned by terraform', not user

upvoted 1 times

gofavad926 1 year, 1 month ago

Selected Answer: A

A, dev

upvoted 1 times

AWS_cert2023 1 year, 5 months ago

A is the answer.
https://developer.hashicorp.com/terraform/language/resources/syntax

upvoted 2 times

Sergiuss95 4 months, 2 weeks ago

Yes, this is correct:

In the following example, the aws_instance resource type is named web. The resource type and name must be unique within a module because they serve as an identifier for a given resource.

resource "aws_instance" "web" {
ami = "ami-a1b2c3d4"
instance_type = "t2.micro"
}
SO:
resource "resource_type" "resource_name" {
Configuration settings for the resource
attribute1 = value1

attribute2 = value2
...
}
upvoted 1 times

Beast_Hollow 1 year, 5 months ago

Selected Answer: A

Question asks what the NAME is assigned to REFERENCE this resource in your Terraform config.

Resource block consists of a resource type and a reference name hence the answer is A: dev
upvoted 3 times

Ni33 1 year, 6 months ago

Selected Answer: A

A is the correct answer.
upvoted 2 times

karendavtyan 1 year, 6 months ago

Selected Answer: D

D. for sure
upvoted 1 times

Rezi 1 year, 7 months ago

B for sure.

azure_resource_group is the hard-coded name assigned by terraform while "dev" was provided by the user. Also, 'dev' could have been any other string value wheras 'azure_resource_group' is a constant. That's how terraform recognizes that resource.
upvoted 4 times

Power123 1 year, 7 months ago

Answer is A - dev
upvoted 1 times

Midas_Tepes 1 year, 7 months ago

The answer is B, The azure_resource_group is the name given by terraform. Dev is a managed name...basically what you are calling this instance
upvoted 1 times

Question #50 *Topic 1*

Setting the TF_LOG environment variable to DEBUG causes debug messages to be logged into syslog.

A. True

B. False

Suggested Answer: *A*
Reference:
https://www.terraform.io/docs/internals/debugging.html

Debugging Terraform

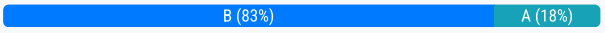
> ***Hands-on:*** *Try the Create Dynamic Expressions tutorial on HashiCorp Learn.*

Terraform has detailed logs which can be enabled by setting the `TF_LOG` environment variable to any value. This will cause detailed logs to appear on stderr.

You can set `TF_LOG` to one of the log levels `TRACE`, `DEBUG`, `INFO`, `WARN` or `ERROR` to change the verbosity of the logs.

Setting `TF_LOG` to `JSON` outputs logs at the `TRACE` level or higher, and uses a parseable JSON encoding as the formatting.

Community vote distribution



Oskar_Madin Highly Voted 2 years, 5 months ago
please stop writing or suggestions if you don't know the answer or are not sure!
upvoted 68 times

esandips Highly Voted 2 years, 7 months ago
B
TF_LOG_PATH needs to be set as well
upvoted 23 times

nhatne 2 years, 4 months ago
B is correct because
TF_LOG_PATH IS NOT REQUIRED, in the docs, they do not mention HAVE TO SET TF_LOG_PATH, it is optional,
therefore without TF_LOG_PATH will cause detailed logs to appear on stderr.
upvoted 5 times

scepticemia 2 years, 6 months ago
Incorrect. A is the correct answer. TF_LOG_PATH just enables persistent logging to be appended to a specific file.
From the docs:
To persist logged output you can set TF_LOG_PATH in order to force the log to always be appended to a specific file when logging is enabled.

https://www.terraform.io/internals/debugging
upvoted 4 times

bigboi23 2 years, 6 months ago
No.
A is correct.
To persist logged output you can set TF_LOG_PATH in order to force the log to always be appended to a specific file when logging is enabled. Note that even when TF_LOG_PATH is set, TF_LOG must be set in order for any logging to be enabled.
upvoted 2 times

anirudhgunda Most Recent 2 months, 2 weeks ago

Selected Answer: B

The correct answer is:

B. False

Explanation:
Setting the TF_LOG environment variable to DEBUG enables detailed debugging messages in Terraform, but these messages are printed to standard error (stderr), not to syslog.

Terraform does not automatically log messages to syslog. If you want to capture Terraform logs in syslog, you would need to manually redirect the output, for example:

sh
Copy
Edit
TF_LOG=DEBUG terraform apply 2>&1 | tee /var/log/terraform.log
Alternatively, you can configure external logging mechanisms to capture Terraform's debug output if needed.

upvoted 2 times

starkonbullet 6 months, 1 week ago

Selected Answer: B

B. False.

Setting the TF_LOG environment variable to "DEBUG" does not cause debug messages to be logged into syslog. It actually causes the Terraform command-line tool to emit detailed debug output to the console (standard error output, specifically).

upvoted 6 times

kingfighers 7 months, 3 weeks ago

B
Terraform has detailed logs that you can enable by setting the TF_LOG environment variable to any value. Enabling this setting causes detailed logs to appear on stderr.
https://developer.hashicorp.com/terraform/internals/debugging

upvoted 1 times

vibzr2023 8 months ago

B is correct
Setting the TF_LOG environment variable to DEBUG does indeed enable detailed debug messages from Terraform, but these messages are not automatically logged to syslog. By default, when TF_LOG is set, Terraform logs messages to stderr. To direct these logs to a file or another logging destination like syslog, you would need to manually redirect the output or use additional tools or settings specific to your operating system or environment.

For example, in a Unix-like environment, you could redirect the Terraform command's stderr to a file or to syslog using command-line redirection or tools like logger. But this is not something Terraform does automatically just by setting TF_LOG to DEBUG.

To log Terraform's output to a file, you might run a command like:
TF_LOG=DEBUG terraform apply 2>terraform-debug.log

upvoted 2 times

Siva_7282 10 months ago

Selected Answer: B

B. False

Setting the TF_LOG environment variable to DEBUG does not automatically log debug messages into syslog. Instead, it causes Terraform to print debug messages to the standard error (stderr) output. If you want to capture or redirect these debug messages, you need to handle the stderr output accordingly, such as redirecting it to a file or using other logging mechanisms.

upvoted 8 times

SauravAmuze 10 months ago

Terraform has detailed logs that you can enable by setting the TF_LOG environment variable to any value. Enabling this setting causes detailed logs to appear on 'stderr' .There is no mention of 'syslog' in the manual. So the answer is B (False)

upvoted 1 times

DianaPopal 11 months ago

B. False

Setting the TF_LOG environment variable to DEBUG in Terraform does not cause debug messages to be logged into syslog. The TF_LOG environment variable is used to control the logging level of Terraform itself, not the system's syslog service. When TF_LOG is set to DEBUG, Terraform will output detailed debug messages to the standard error output (stderr) or the log file specified in the Terraform configuration.

upvoted 3 times

MisterR0B0T 1 year ago

The answer is False. Setting the TF_LOG environment variable to DEBUG causes debug messages to be logged to the standard error stream (stderr). It does not log messages to syslog.

Syslog is a standard logging system that is used by many Unix-like operating systems to collect and store system log messages. Syslog messages are typically stored in a file called /var/log/syslog. Terraform does not log messages to syslog by default.

To log Terraform messages to syslog, you can use a third-party logging tool, such as Logstash or Fluentd. These tools can be configured to collect logs from stderr and send them to syslog.

Here is an example of how to use Logstash to send Terraform logs to syslog:

```
input {
stdin {
type => "plain"
filter => {
remove_field => ["message.level"]
add_field => { "message.level" => "debug" }
}
}
}

output {
syslog {
host => "localhost"
}
}
```

upvoted 8 times

gofavad926 1 year, 1 month ago

Selected Answer: A

A, it appears in stderr. As documentation says: "Terraform has detailed logs that you can enable by setting the TF_LOG environment variable to any value. Enabling this setting causes detailed logs to appear on stderr."

upvoted 2 times

Halimb 1 year, 2 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/cli/config/environment-variables

Quote "Enables detailed logs to appear on stderr which is useful for debugging."

By default, regardless of the log level, Terraform writes its log output to the standard error (stderr) stream. This behavior is useful for capturing log messages in the terminal or console where you are running Terraform commands, allowing you to see any warnings or errors in real-time.

If you wish to redirect the log output to a file or another destination, you can use standard shell redirection as shown in my previous response. This allows you to save the log information for later analysis or to separate it from the terminal output.

upvoted 3 times

Spandrop 1 year, 2 months ago

Correct answer is B

"Terraform has detailed logs that you can enable by setting the TF_LOG environment variable to any value. Enabling this setting causes detailed logs to appear on stderr."

There is nothing to do with syslog

https://developer.hashicorp.com/terraform/internals/debugging

upvoted 2 times

debabrata6983 1 year, 3 months ago

Selected Answer: B

B is the right answer and stop confusing people

upvoted 1 times

AWS_cert2023 1 year, 5 months ago

Enabling this setting causes detailed logs to appear on stderr.
Uses TF_LOG_PATH to change the path of log info.

upvoted 1 times

nirosha1 1 year, 7 months ago

Selected Answer: A

Terraform has detailed logs that you can enable by setting the TF_LOG environment variable to any value. Enabling this setting causes detailed logs to appear on stderr. https://developer.hashicorp.com/terraform/internals/debugging

upvoted 2 times

AzRNoob 1 year, 7 months ago

FALSE

Setting the TF_LOG environment variable to DEBUG causes debug messages to be logged to standard error (stderr) output, not to syslog.

upvoted 8 times

Question #51 *Topic 1*

Where in your Terraform configuration do you specify a state backend?

A. The terraform block

B. The resource block

C. The provider block

D. The datasource block

Suggested Answer: *A*
Backends are configured with a nested backend block within the top-level terraform block.
Reference:
https://www.terraform.io/docs/language/settings/backends/configuration.html

Community vote distribution
A (100%)

Eltooth Highly Voted 1 year, 11 months ago

Selected Answer: A
A is correct answer.
"To configure a backend, add a nested backend block within the top-level terraform block. The following example configures the remote backend."

https://www.terraform.io/language/settings/backends/configuration
https://www.terraform.io/language/settings/backends/configuration#using-a-backend-block
upvoted 8 times

debabrata6983 Most Recent 9 months ago

Selected Answer: A
A is Correct Answer
upvoted 1 times

Bere 9 months, 3 weeks ago

Selected Answer: A
terraform {
backend "remote" {
hostname = "app.terraform.io"
organization = "YourOrganization"

workspaces {
name = "your_workspace"
}
}
}
upvoted 4 times

arunrkaushik 9 months, 3 weeks ago

terraform {
backend "consul" {
address = "demo.consul.io"
scheme = "https"
path = "example_app/terraform_state"
}
}
upvoted 1 times

Ni33 1 year ago

Selected Answer: A
A for sure.

upvoted 1 times

Power123 1 year, 1 month ago

A is the correct ans

upvoted 1 times

oab720 1 year, 4 months ago

Selected Answer: A

Tested in labs - terraform block

upvoted 1 times

Zam88 1 year, 11 months ago

A. The terraform block

upvoted 2 times

CHRIS12722222 1 year, 11 months ago

terraform block

upvoted 1 times

Question #52 *Topic 1*

In Terraform 0.13 and above, outside of the required_providers block, Terraform configurations always refer to providers by their local names.

A. True

B. False

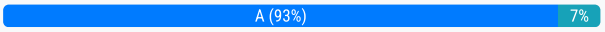
Suggested Answer: *A*

Outside of the required_providers block, Terraform configurations always refer to providers by their local names.

Reference:

https://www.terraform.io/docs/language/providers/requirements.html

Community vote distribution



Bere Highly Voted 1 year, 3 months ago

Selected Answer: A

Here's an example that demonstrates this:

```
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "~> 3.0"
}
}
}

provider "aws" {
region = "us-west-2"
}

resource "aws_instance" "example" {
ami = "ami-0123456789abcdef0"
instance_type = "t2.micro"
}
```

In this example, the required_providers block specifies the source and version constraints for the AWS provider. In the rest of the configuration, the provider is referred to by its local name aws, such as in the provider "aws" block and the aws_instance resource block.

upvoted 12 times

Eltooth Highly Voted 2 years, 5 months ago

Selected Answer: A

A is correct answer.

"Local names are module-specific, and are assigned when requiring a provider. Local names must be unique per-module.
Outside of the required_providers block, Terraform configurations always refer to providers by their local names."

https://www.terraform.io/language/providers/requirements#local-names

upvoted 9 times

BJ5 Most Recent 7 months, 3 weeks ago

Selected Answer: B

What about aliases

```
provider "aws" {
region="us-west-1"
alias = "west"
```

}

resource "aws_instance" "ec2" {
provider = aws.west
}
upvoted 1 times

chaoscreater 6 months ago
that's still referencing the provider name "aws" first
upvoted 1 times

Power123 1 year, 7 months ago
A is correct
upvoted 1 times

camps 1 year, 7 months ago
Selected Answer: A
A. True.

In Terraform 0.13 and above, provider references are always specified using their local name, rather than the legacy name-based syntax used in earlier versions. The provider's local name is specified in the required_providers block, and this local name is used in all provider references in the configuration.
upvoted 1 times

AShahine21 1 year, 11 months ago
Selected Answer: B
what about alias ? I am going to say false
upvoted 1 times

Chehh 2 years, 3 months ago
"Outside of the required_providers block, Terraform configurations always refer to providers by their local names."
https://www.terraform.io/language/providers/requirements#local-names
upvoted 3 times

bigboi23 2 years, 6 months ago
Selected Answer: A
Correct!
https://www.terraform.io/language/providers/requirements
upvoted 4 times

Question #53 *Topic 1*

What command should you run to display all workspaces for the current configuration?

A. terraform workspace

B. terraform workspace show

C. terraform workspace list

D. terraform show workspace

Suggested Answer: *C*
terraform workspace list
The command will list all existing workspaces.
Reference:
https://www.terraform.io/docs/cli/commands/workspace/list.html

Community vote distribution
C (100%)

bicycle Highly Voted 1 year, 5 months ago
Selected Answer: C
The terraform workspace list command is used to list all existing workspaces.
upvoted 8 times

kcw6 Highly Voted 1 year, 3 months ago
terraform workspace -help
Usage: terraform [global options] workspace

new, list, show, select and delete Terraform workspaces.

Subcommands:
delete Delete a workspace
list List Workspaces
new Create a new workspace
select Select a workspace
show Show the name of the current workspace
upvoted 6 times

Power123 Most Recent 7 months, 4 weeks ago
C is the answer
upvoted 1 times

r1ck 9 months ago
https://developer.hashicorp.com/terraform/cli/commands/workspace/list : C
upvoted 1 times

Eltooth 1 year, 5 months ago
Selected Answer: C
C is correct answer.
"The terraform workspace list command is used to list all existing workspaces."

https://www.terraform.io/cli/commands/workspace/list
upvoted 3 times

Question #54 *Topic 1*

Terraform providers are always installed from the Internet.

A. True

B. False

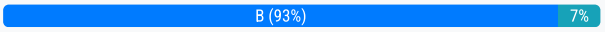
Suggested Answer: *B*

Terraform configurations must declare which providers they require, so that Terraform can install and use them.

Reference:

https://www.terraform.io/docs/language/providers/configuration.html

Community vote distribution



amrith501 Highly Voted 2 years, 5 months ago

Selected Answer: B

For air gapped systems, we can bundle provider along with the Terraform binaries. These Bundle can be installed without internet on air gapped system.

NOTE: For downloading the bundle we need internet access, this can be done on a system which as internet access and later transfer to the air gapped system

upvoted 8 times

bigboi23 Highly Voted 2 years, 6 months ago

Terraform CLI finds and installs providers when initializing a working directory. It can automatically download providers from a Terraform registry, or load them from a local mirror or cache. If you are using a persistent working directory, you must reinitialize whenever you change a configuration's providers.

upvoted 7 times

Tyler2023 Most Recent 1 year, 1 month ago

Downloading a plugin directly from its origin registry is not always appropriate, though. For example, the system where you are running Terraform may not be able to access an origin registry due to firewall restrictions within your organization or your locality.

To allow using Terraform providers in these situations, there are some alternative options for making provider plugins available to Terraform which we'll describe in the following sections.

https://developer.hashicorp.com/terraform/language/providers/requirements#in-house-providers

https://developer.hashicorp.com/terraform/cli/config/config-file#provider-installation

upvoted 2 times

debabrata6983 1 year, 3 months ago

Selected Answer: B

terraform can be used in AirGap region too

upvoted 1 times

Bere 1 year, 3 months ago

Selected Answer: B

Terraform providers are not always installed from the Internet. While Terraform does fetch providers from the HashiCorp Terraform Registry by default, it also allows for the usage of providers from other sources or even local sources.

Here's an example:

terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "2.70.0"
}

```
custom-provider = {
source = "/path/to/local/custom-provider"
}
}
}

provider "aws" {
region = "us-west-2"
}

provider "custom-provider" {
# Configuration for the custom provider
}
```

In this example, the AWS provider is fetched from the HashiCorp Terraform Registry, while the "custom-provider" is loaded from a local path. The local provider must be properly compiled and placed in the specified path.

upvoted 3 times

LunarPhobia 1 year, 4 months ago

B because you can do air gap installs for networks without internet

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

A. Terraform Cloud and Terraform Enterprise install providers as part of every run. Terraform CLI finds and installs providers when initializing a working directory.

upvoted 1 times

vibzr2023 8 months ago

incorrect: think of military/defense systems, nuclear power plants they do air-gapped which they don't rely on intenet.

upvoted 1 times

Power123 1 year, 7 months ago

The answer is B

upvoted 1 times

vikramvlr 1 year, 11 months ago

Answer is B.

Terraform CLI finds and installs providers when initializing a working directory. It can automatically download providers from a Terraform registry, or load them from a local mirror or cache. If you are using a persistent working directory, you must reinitialize whenever you change a configuration's providers.

upvoted 3 times

Ahmad_Terraform 2 years, 4 months ago

B is correct,

upvoted 1 times

Eltooth 2 years, 4 months ago

Selected Answer: B

B is correct answer.

upvoted 2 times

Question #55 *Topic 1*

Which of these is the best practice to protect sensitive values in state files?

A. Blockchain

B. Secure Sockets Layer (SSL)

C. Enhanced remote backends

D. Signed Terraform providers

Suggested Answer: *C*

Use of remote backends and especially the availability of Terraform Cloud, there are now a variety of backends that will encrypt state at rest and will not store the state in cleartext on machines running.

Reference:

https://www.terraform.io/docs/extend/best-practices/sensitive-state.html

Community vote distribution

C (100%)

chimons Highly Voted 1 year, 11 months ago

Selected Answer: C

The best practice for protecting sensitive values in Terraform state files is to use enhanced remote backends, such as those that support encryption at rest and in transit.

A remote backend allows you to store your Terraform state in a central location, such as an S3 bucket or a HashiCorp Consul server. This allows you to share state files between team members and across multiple environments, such as staging and production.

Using an enhanced remote backend with encryption at rest and in transit helps to protect the sensitive values in your state file from unauthorized access or tampering.

Option C, enhanced remote backends, is the correct answer.

Options A and B, blockchain and SSL, are not directly related to protecting sensitive values in Terraform state files.

Option D, signed Terraform providers, is a security feature that allows you to verify the authenticity of a Terraform provider, but it is not directly related to protecting sensitive values in state files.

upvoted 21 times

Aakashj Most Recent 4 months, 3 weeks ago

Selected Answer: C

c is the correct answer

upvoted 1 times

enook 10 months, 4 weeks ago

Selected Answer: C

I choose C

upvoted 1 times

Power123 1 year, 7 months ago

Correct answer is C

upvoted 1 times

dkp 1 year, 11 months ago

c is the correct answer

upvoted 1 times

Ahmad_Terraform 2 years, 4 months ago

Enhanced remote backends

upvoted 3 times

Eltooth 2 years, 4 months ago

Selected Answer: C

C is correct answer.

upvoted 2 times

HDDH 2 years, 5 months ago

C for course

upvoted 1 times

jouyang 2 years, 6 months ago

encrypted backend

upvoted 1 times

Question #56 *Topic 1*

When does terraform apply reflect changes in the cloud environment?

A. Immediately

B. However long it takes the resource provider to fulfill the request

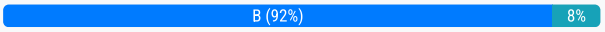
C. After updating the state file

D. Based on the value provided to the -refresh command line argument

E. None of the above

Suggested Answer: *B*

Community vote distribution



Nunyabiznes Highly Voted 1 year, 8 months ago

Selected Answer: B

When you execute terraform apply, Terraform creates a new execution plan by comparing the current state file to the desired state declared in the configuration. After creating the execution plan, Terraform presents the proposed changes and asks for confirmation to apply them. Once you confirm the changes, Terraform updates the state file with the new state reflecting the changes that were made. Terraform then submits the change requests to the resource provider to make the desired changes in the cloud environment. The amount of time it takes for the resource provider to fulfill the requests can vary depending on the resources being modified.

upvoted 17 times

liuyomz 7 months ago

yeah, B is not exactly correct, but the best one.

upvoted 2 times

vitasac Highly Voted 2 years, 6 months ago

Selected Answer: B

B for sure

upvoted 11 times

foreverlearner Most Recent 1 year, 4 months ago

From https://developer.hashicorp.com/terraform/tutorials/cli/apply

When you apply this configuration, Terraform will:

1) Lock your project's state

2) Create a plan, and wait for you to approve it.

3) Execute the steps defined in the plan using the providers you installed when you initialized your configuration. Terraform executes steps in parallel when possible, and sequentially when one resource depends on another.

4) Update your project's state file with a snapshot of the current state of your resources.

5) Unlock the state file.

6) Print out a report of the changes it made, as well as any output values defined in your configuration.

Changes in the cloud environment are made on step 3, before updating the state file

upvoted 6 times

zanhsieh 1 year, 6 months ago

Selected Answer: B

I vote for B. The difference between B and C is that we have to count on timeout cases in the real world, e.g. provision GCP DataProc cluster, MongoDB Atlas, etc. It's possible the provisioner timeout but the provider keeps going till the job finish. In this case the local terraform state file does not sync with the physical infra - terraform did try to update but timeout. Then we run `terraform apply` again and the state file reconciles with the physical infra.

upvoted 2 times

Ni33 1 year, 6 months ago

Selected Answer: B

BBBBBBBBBB

upvoted 2 times

Power123 1 year, 7 months ago

B is the correct ans

upvoted 1 times

Chaks1985 1 year, 10 months ago

Ans: C ("WHEN" is key in this question. Its not asking how long will it take but "WHEN"

upvoted 2 times

InformationOverload 1 year, 10 months ago

Selected Answer: B

If you are creating a new virtual machine using Terraform, it may take a few minutes for the virtual machine to be created and for it to become available for use. During this time, Terraform will continue to report on the progress of the creation process and will display any errors or issues that may arise. Once the virtual machine has been successfully created, the changes will be reflected in your cloud environment. I go with B

upvoted 2 times

FarziWaliMarzi 1 year, 11 months ago

Given it has more than 4 options, I think it should have multiple valid answers, and hence both B & C should be selected. Again, this is assumption based as this is missing in question.

upvoted 1 times

faltu1985 2 years, 2 months ago

Selected Answer: B

B for sure

upvoted 1 times

Nightducky 2 years, 1 month ago

Why B? You have "when" not "how long" in the question.

upvoted 1 times

RVivek 2 years, 2 months ago

Selected Answer: C

When does terraform apply reflect changes in the cloud environment?
Terraform will apply chnages to the cloud, then it will update the state file. After it updates the statefile only it "reflects" those changes to the screen

upvoted 1 times

pas77 2 years ago

This is not what the question is asking. The question is not asking when it will reflect the changes in the "screen". It is asking when it will reflect the changes in the "cloud environment". And this will be whenever it finishes deploying the resources. "B" is the correct answer for sure.

upvoted 2 times

nhatne 2 years, 4 months ago

Selected Answer: C

I vote C because the question is "When" and from the docs:

Terraform plan and apply operations run an implicit in-memory refresh as part of their functionality, reconciling any drift from your state file before suggesting infrastructure changes.

upvoted 2 times

Eltooth 2 years, 4 months ago

Selected Answer: B

B is correct answer.

upvoted 1 times

Zam88 2 years, 5 months ago

B is correct

upvoted 1 times

AzureGurl 2 years, 5 months ago

B is the correct answer.

upvoted 1 times

temp111 2 years, 6 months ago

I think, the answer is B.

upvoted 4 times

Ipergorta 2 years, 6 months ago

Why no B?

upvoted 5 times

Question #57 *Topic 1*

How would you reference the "name" value of the second instance of this fictitious resource?

```
resource "aws_instance" "web" {
  count = 2
  name = "terraform-${count.index}"
}
```

A. element(aws_instance.web, 2)

B. aws_instance.web[1].name

C. aws_instance.web[1]

D. aws_instance.web[2].name

E. aws_instance.web.*.name

Suggested Answer: *A*

Reference:

https://www.terraform.io/docs/configuration-0-11/interpolation.html

Community vote distribution

B (97%)

franksoul Highly Voted 2 years, 11 months ago

Dear ALL, tested in Lab and Answer is : B

upvoted 28 times

bigboi23 Highly Voted 3 years ago

Selected Answer: B

B!!!!!!!!

upvoted 15 times

nmnm22 Most Recent 9 months, 3 weeks ago

why is it index number AND name, though? i would have thought that specifying the index number is enough

upvoted 1 times

nhaastrup 1 year ago

To reference the "name" value of the second instance of the fictitious resource "aws_instance" named "server", you can use the element index notation in Terraform. The element index notation allows you to access elements in a list or collection by their index.

Given that the index in Terraform starts from 0, the second instance has an index of 1. Therefore, to reference the "name" value of the second instance = B

upvoted 4 times

kingfighers 1 year, 1 month ago

A. element(aws_instance.web, 2)

is correct if we want to access the 3rd one, but there is no "the third one"

upvoted 2 times

samimshaikh 1 year, 4 months ago

Selected Answer: B

because the index starts from 0 so second instance name will be at 1

upvoted 4 times

gofavad926 1 year, 8 months ago

Selected Answer: B

B for sure

upvoted 1 times

debabrata6983 1 year, 9 months ago

Selected Answer: B

This is the very basics of terraform resource referencing

upvoted 3 times

Bere 1 year, 9 months ago

Selected Answer: B

Example:
resource "aws_instance" "web" {
count = 2
name = "terraform-${count.index}"
}

output "second_instance_name" {
value = aws_instance.web[1].name
}

upvoted 8 times

Bluemoon22 2 years, 1 month ago

The answer is B, aws_instance.web[1].name

upvoted 5 times

Power123 2 years, 1 month ago

B is correct

upvoted 2 times

thor7 2 years, 1 month ago

Selected Answer: B

B is a correct answer, checked.

upvoted 1 times

Nunyabiznes 2 years, 2 months ago

Selected Answer: B

The count.index starts from 0. Therefore, the second instance would have an index of 1. In that case, the correct answer would be B. aws_instance.web[1].name.

upvoted 9 times

Ell89 2 years, 2 months ago

Selected Answer: B

its B - people who answer incorrectly with such conviction and belief are dangerous.

upvoted 4 times

oab720 2 years, 4 months ago

Selected Answer: B

Tried and tested, definitely B

upvoted 1 times

Bebins 2 years, 5 months ago

The output of A won't return the name of the 2nd instance. So here the answer is "B"

upvoted 1 times

adouban 2 years, 5 months ago

Selected Answer: B

B is the correct answer

upvoted 1 times

Question #58 *Topic 1*

A Terraform provider is not responsible for:

A. Understanding API interactions with some service

B. Provisioning infrastructure in multiple clouds

C. Exposing resources and data sources based on an API

D. Managing actions to take based on resource differences

Suggested Answer: *D*
Reference:
https://www.terraform.io/docs/configuration-0-11/providers.html

Providers are responsible in Terraform for managing the lifecycle of a resource: create, read, update, delete.

Most providers require some sort of configuration to provide authentication information, endpoint URLs, etc. Where explicit configuration is required, a `provider` block is used within the configuration as illustrated in the following sections.

By default, resources are matched with provider configurations by matching the start of the resource name. For example, a resource of type `vsphere_virtual_machine` is associated with a provider called `vsphere`.

Community vote distribution



amrith501 Highly Voted 2 years, 11 months ago

Selected Answer: B

The answer should be B
A terraform can only provision resouce in one Cloud not multiple cloud
upvoted 22 times

Nunyabiznes 2 years, 1 month ago

No, it can provision in multiple clouds:
provider "aws" {
region = "us-west-2"
}

provider "azurerm" {
features {}
}

resource "aws_instance" "example" {
ami = "ami-0c55b159cbfafe1f0"
instance_type = "t2.micro"
}

resource "azurerm_resource_group" "example" {
name = "example"
location = "East US"
}
upvoted 7 times

jerikoo 2 years ago

those are 2 providers, each provider can only deploy resources in one cloud.. several providers, in multicloud.. so "A Terraform provider" can only provision in one Cloud... B!
upvoted 27 times

Spandrop Highly Voted 1 year, 5 months ago

Selected Answer: D

I'm not a native English speaker, but I think that in the context of the question, "A Terraform provider" is referring to the concept of providers in Terraform as a whole, not a specific individual provider.

I would go with D

"Terraform providers are a plugin for Terraform that makes a collection of related resources available. A provider plugin is responsible for understanding API interactions and exposing resources. Providers generally are IaaS (like AWS, GCP, Microsoft Azure, OpenStack), PaaS (like Heroku), or SaaS services (like Terraform Cloud, DNSimple, CloudFlare).

However, the management of actions based on resource differences (i.e., what to create, update, or delete) is handled by Terraform's core engine, not by the providers. The providers simply inform Terraform's core about what resources they can manage and how to manage them."
upvoted 10 times

vl67 Most Recent 2 weeks ago

Selected Answer: B

Provisioning infrastructure in multiple clouds — that's the role of Terraform configurations, which can use multiple providers in a single project.

The provider itself focuses only on one service/cloud.
upvoted 1 times

GiorgioGss 2 weeks ago

Selected Answer: B

The provider can't be multi-cloud. (eg AWS)
upvoted 1 times

cat018 2 weeks, 5 days ago

Selected Answer: B

Right answer is B:
A. Understanding API interactions - Providers translate Terraform configurations into API calls for the target service (e.g., AWS API for aws_instance).
C. Exposing resources/data sources - Providers define what resources (e.g., aws_vpc) and data sources (e.g., aws_ami) are available.
D. Managing actions based on differences - Providers determine how to create, update, or delete resources to match the desired state (e.g., scaling an EC2 instance).

B. Provisioning infrastructure in multiple clouds:
A single provider is designed for one specific platform (e.g., aws for AWS, azurerm for Azure).
To manage multi-cloud infrastructure, you must use multiple providers (e.g., aws + google + azurerm) in the same configuration.
No provider natively handles cross-cloud provisioning; this is orchestrated by Terraform Core (not the provider itself).
upvoted 1 times

sultaz 3 months ago

Selected Answer: B

While Terraform itself supports managing infrastructure across multiple clouds, individual providers are designed to focus on a single API or domain (e.g., AWS, Azure). They do not inherently provision infrastructure across multiple clouds; this is achieved by combining multiple providers in a Terraform configuration.
upvoted 1 times

YesPlease 3 months, 3 weeks ago

Selected Answer: B

Answer B

Poorly worded question, but a single Terraform provider is designed to work with only one specific cloud platform. For example, you can not ask AWS to create an infrastructure on Azure and vice versa.
upvoted 1 times

d759fb3 3 months, 4 weeks ago

Selected Answer: B

It should be B. Multiple Clouds (NOT a Provider Responsibility):

```
# Use multiple providers for multiple clouds
provider "aws" {
region = "us-west-2"
}
provider "azurerm" {
features {}
}
```

Managing Resource Differences (IS a Provider Responsibility):

```
# Provider handles:
# - Detecting changes
# - Planning updates
# - Applying changes

resource "aws_instance" "example" {
instance_type = "t2.micro" # Provider detects if this changes
tags = {
Name = "example" # Provider manages tag updates
}
}
```

upvoted 1 times

cerepx 4 months, 1 week ago

Selected Answer: D

Terraform itself is responsible for comparing the desired configuration (in .tf files) with the current state (in the .tfstate file) and determining the appropriate actions (create, update, delete) to achieve the desired state. Providers execute those actions once Terraform decides what needs to be done, but they are not responsible for managing those actions based on resource differences. Terraform itself handles the planning and state comparison logic. Therefore, this is the correct answer as something a provider is not responsible for.

upvoted 1 times

rraabbbind 4 months, 4 weeks ago

Selected Answer: B

A terraform can only provision resouce in one Cloud not multiple cloud

upvoted 1 times

suri_surendra 4 months, 4 weeks ago

Selected Answer: D

The correct answer is: D. Managing actions to take based on resource differences

Explanation:
In Terraform, providers are responsible for interacting with APIs, provisioning resources, and exposing resources and data sources. However, the task of managing actions based on resource differences (such as determining what actions to take when the desired state differs from the current state) is handled by Terraform itself during the plan and apply phases, not by the provider.

upvoted 2 times

pyxgzm 5 months ago

Selected Answer: D

The correct answer is D. Managing actions to take based on resource differences.
However, managing actions to take based on resource differences is the responsibility of Terraform Core, not the provider. Terraform Core is responsible for:

- Determining the desired state of the infrastructure based on the configuration
- Comparing the desired state to the current state of the infrastructure
- Determining the actions needed to reconcile any differences between the desired and current states

So, while providers are responsible for interacting with external services and managing resources, Terraform Core is responsible for managing the overall provisioning process and determining the actions needed to achieve the desired state.

upvoted 2 times

Greysi 5 months ago

Selected Answer: D

A resource provider may provision to multiple clouds. you can write your own resource provider which covers multiple clouds.
It's not part of resource provider to managing actions based on differences between existing resource and configuration. Resource provider will only execute actions. Managing actions and triggering resource provider to execute this action is part of terraform itself.

upvoted 2 times

shant_haghverdian 5 months, 1 week ago

Selected Answer: B

each provider is dedicated to its related infrastructure, you cant provision azure resources using was provider

upvoted 1 times

muammer32 5 months, 2 weeks ago

Selected Answer: D

The provider's role is focused on API interactions, provisioning infrastructure, and exposing resources. Managing actions based on resource differences is handled by Terraform's core logic during the terraform plan and terraform apply processes, not by the provider itself. Thus, D is the correct answer.

upvoted 1 times

khvan84 7 months, 1 week ago

B. Provisioning infrastructure in multiple clouds: Providers enable provisioning across different cloud platforms, so this is also a responsibility of the provider .

upvoted 1 times

a45e482 8 months, 2 weeks ago

The answer is B
Tricky question a provider can only provision resources in one cloud platform, you will need multiple providers to provision in multiple cloud.

upvoted 1 times

Question #59 *Topic 1*

Terraform provisioners can be added to any resource block.

A. True

B. False

Suggested Answer: *A*
Reference:
https://www.terraform.io/docs/language/resources/provisioners/syntax.html

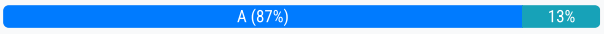
Terraform includes the concept of provisioners as a measure of pragmatism, knowing that there will always be certain behaviors that can't be directly represented in Terraform's declarative model.

However, they also add a considerable amount of complexity and uncertainty to Terraform usage. Firstly, Terraform cannot model the actions of provisioners as part of a plan because they can in principle take any action. Secondly, successful use of provisioners requires coordinating many more details than Terraform usage usually requires: direct network access to your servers, issuing Terraform credentials to log in, making sure that all of the necessary external software is installed, etc.

The following sections describe some situations which can be solved with provisioners in principle, but where better solutions are also available. We do not recommend using provisioners for any of the use-cases described in the following sections.

Even if your specific use-case is not described in the following sections, we still recommend attempting to solve it using other techniques first, and use provisioners only if there is no other option.

Community vote distribution



GopinathM Highly Voted 2 years, 2 months ago

A is correct
https://www.terraform.io/language/resources/provisioners/syntax
resource "aws_instance" "web" {
...

provisioner "local-exec" {
command = "echo The server's IP address is ${self.private_ip}"
}
}
upvoted 16 times

agmesas Highly Voted 1 year, 9 months ago

Selected Answer: A

the key of this question is "could you use a local-exec provisioner in any type of resource?", yes, because the execution of this provisioner is on my local so we couldnuse local-exec in any resource type because it is independent of the type of resource. Answer is A "true"
upvoted 10 times

suri_surendra Most Recent 4 months, 4 weeks ago

Selected Answer: B

The correct answer is: B. False

Explanation:
Terraform provisioners are not automatically supported in all resource blocks. They are designed to be used with specific types of resources,

primarily those that involve remote execution or configuration (such as virtual machines or instances). However, not every resource type is compatible with provisioners.

upvoted 1 times

samimshaikh 11 months ago

Selected Answer: A

A. True

Terraform provisioners can be added to any resource block. Provisioners are used to execute scripts or commands on a local or remote machine as part of resource creation or destruction. They are defined within a resource block and allow you to perform actions such as initializing, configuring, or setting up resources after they have been created.

upvoted 2 times

Pradh 1 year, 2 months ago

answer should be "B"

Can you add a provisioner to below resource block ?

==========

resource "azurerm_network_interface" "Nic-1"

==========

upvoted 3 times

ilmi70 9 months, 1 week ago

resource "azurerm_network_interface" "Nic-1" {

...

provisioner "local-exec" {

command = "echo 'Network interface created'"

}

}

upvoted 4 times

debabrata6983 1 year, 3 months ago

Selected Answer: A

Please stop confusing people. Basics of Provisioner block : always bind with resource block

upvoted 7 times

Bere 1 year, 3 months ago

Selected Answer: A

Since the local-exec provisioner executes on the machine running Terraform and not on the resource itself, it can indeed be added to any resource block. The actions taken by the local-exec provisioner are not dependent on the type of resource, so it's possible to use this provisioner with any resource in Terraform.

upvoted 4 times

modarov 1 year, 3 months ago

The answer is False. Terraform provisioners can only be added to resource blocks that support them

upvoted 2 times

LunarPhobia 1 year, 4 months ago

The new exam doesn't ask about provisioners anymore

upvoted 3 times

Ha_Baruh_Architect13 1 year, 4 months ago

The correct answer is B. False.

Terraform provisioners are not added to resource blocks directly. Instead, provisioners are added as separate blocks within a resource block. Provisioners are used to execute scripts or commands on a resource after it has been created or updated. They are defined outside of the resource block and are associated with the resource using the "provisioner" keyword followed by the specific provisioner type CHATGPT

upvoted 1 times

Jlee7 1 year, 5 months ago

The answer is False.

Terraform provisioners can only be added to resource blocks that support them. For example, the AWS provider supports the remote-exec provisioner, but the Azure provider does not.

upvoted 1 times

joyboy23 1 year, 4 months ago

But what about the local-exec ?

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

AAAAAAAA

upvoted 2 times

Bluemoon22 1 year, 7 months ago

The answer is A

upvoted 2 times

sylvergorilla 1 year, 7 months ago

Selected Answer: A

Yes, Terraform provisioners can be added to any resource block. Provisioners are used to execute scripts or commands on a resource after it has been created or updated.

upvoted 3 times

Power123 1 year, 7 months ago

A is correct

upvoted 1 times

Nunyabiznes 1 year, 8 months ago

Selected Answer: A

Provisioners can be added to any resource block in Terraform configuration, but it is not recommended to use them for configuration management as it goes against the declarative approach of Terraform. Provisioners should only be used as a last resort when no other Terraform resource types are available to handle a specific task.

upvoted 3 times

Tanacet 1 year, 10 months ago

Selected Answer: A

Provisioner is an arbitrary command executed by the terraform when a resource is created/destroyed. It is not related to a resource itself. The creation/destruction of a resource is just a trigger.

upvoted 3 times

Question #60 *Topic 1*

What is terraform refresh intended to detect?

A. Terraform configuration code changes

B. Empty state files

C. State file drift

D. Corrupt state files

Suggested Answer: *C*

Reference:

https://www.hashicorp.com/blog/detecting-and-managing-drift-with-terraform

Prior to a plan or apply operation, Terraform does a refresh to update the state file with real-world status. You can also do a refresh any time with `terraform refresh`:

```
$ terraform refresh
aws_instance.example: Refreshing state... (ID: i-011a9893eff09ede1)
```

What Terraform is doing here is reconciling the resources tracked by the state file with the real world. It does this by querying your infrastructure providers to find out what's actually running and the current configuration, and updating the state file with this new information. Terraform is designed to co-exist with other tools as well as manually provisioned resources and so it only refreshes resources under its management.

The output for a refresh is minimal. Terraform lists each resource it is refreshing along with its internal ID. Running `refresh` does not modify infrastructure, but does modify the state file. If the state has drifted from the last time Terraform ran, `refresh` allows that drift to be detected.

Community vote distribution

C (100%)

camps Highly Voted 1 year, 1 month ago

Selected Answer: C

C. State file drift.

The terraform refresh command is used to reconcile the state Terraform has stored in the state file with the real-world infrastructure. When you run terraform apply, Terraform updates the state file to reflect the current state of the infrastructure it manages. However, if changes are made to the infrastructure outside of Terraform, such as via the web console or API, the state file will become out-of-date and will not accurately reflect the current state of the infrastructure.

upvoted 6 times

LunarPhobia Most Recent 10 months ago

This command is now deprecated

upvoted 4 times

Power123 1 year, 1 month ago

Answer is C

upvoted 1 times

Nunyabiznes 1 year, 2 months ago

Selected Answer: C

Terraform refresh is intended to detect state file drift. It updates the Terraform state file with the real-world state of the resources being managed, which allows Terraform to detect any changes that might have been made outside of Terraform. This is useful when the state file is out of sync with the actual resources in the cloud provider, which can happen if someone manually modifies a resource outside of Terraform. By running terraform refresh, Terraform can identify those differences and update the state file to reflect the real-world state of the resources being managed.

upvoted 3 times

chimons 1 year, 5 months ago

Selected Answer: C

C. State file drift

terraform refresh is a command in the Terraform infrastructure as code tool that is used to update the state file with the current real-world infrastructure. It compares the resources described in the state file to the actual resources in the infrastructure and updates the state file to match the infrastructure. One of the things that terraform refresh is intended to detect is state file drift, which is a difference between the infrastructure and the resources described in the state file.

upvoted 1 times

lucitama 1 year, 11 months ago

Selected Answer: C

C is correct

upvoted 4 times

Eltooth 1 year, 11 months ago

Selected Answer: C

C is the closet correct answer, however this command is no longer in use.

"The terraform refresh command reads the current settings from all managed remote objects and updates the Terraform state to match.

Warning: This command is deprecated, because its default behavior is unsafe if you have misconfigured credentials for any of your providers. See below for more information and recommended alternatives."

https://www.terraform.io/cli/commands/refresh

upvoted 3 times

amrith501 1 year, 11 months ago

Selected Answer: C

State file drift for sure

upvoted 4 times

Question #61 *Topic 1*

FILL BLANK -
Which flag would you add to terraform plan to save the execution plan to a file?
Type your answer in the field provided. The text field is not case-sensitive and all variations of the correct answer are accepted.

Suggested Answer: *-out=FILENAME*
Reference:
https://www.terraform.io/docs/cli/commands/plan.html

You can use the optional `-out=FILE` option to save the generated plan to a file on disk, which you can later execute by passing the file to `terraform apply` as an extra argument. This two-step workflow is primarily intended for when running Terraform in automation.

If you run `terraform plan` without the `-out=FILE` option then it will create a *speculative plan*, which is a description of the effect of the plan but without any intent to actually apply it.

Eltooth Highly Voted 2 years, 5 months ago
Answer is correct: terraform plan -out=FILE

"You can use the optional -out=FILE option to save the generated plan to a file on disk, which you can later execute by passing the file to terraform apply as an extra argument. This two-step workflow is primarily intended for when running Terraform in automation.

If you run terraform plan without the -out=FILE option then it will create a speculative plan, which is a description of the effect of the plan but without any intent to actually apply it."

https://www.terraform.io/cli/commands/plan
upvoted 9 times

secdaddy Highly Voted 1 year, 11 months ago
Question asks for the flag and the flag is -out
=FILE is the parameter for the flag
No idea if they'll just parse this for -out or if =FILE will be useful
terraform plan -out
| Error: Failed to parse command-line flags
| flag needs an argument: -out
upvoted 5 times

Canarys_Automations Most Recent 10 months, 3 weeks ago
-out=FILEPATH
upvoted 1 times

modarov 1 year, 3 months ago
terraform plan -out plan.out
upvoted 1 times

Power123 1 year, 7 months ago
terraform plan -out =FILE
upvoted 2 times

thor7 1 year, 8 months ago
terraform plan -out=path/to/the/file
upvoted 4 times

samkobadev 1 year, 9 months ago
-out=path
Write a plan file to the given path. This can be used asinput to the "apply" command.
upvoted 2 times

Dsmo 1 year, 11 months ago

Would -out still count as correct?
upvoted 1 times

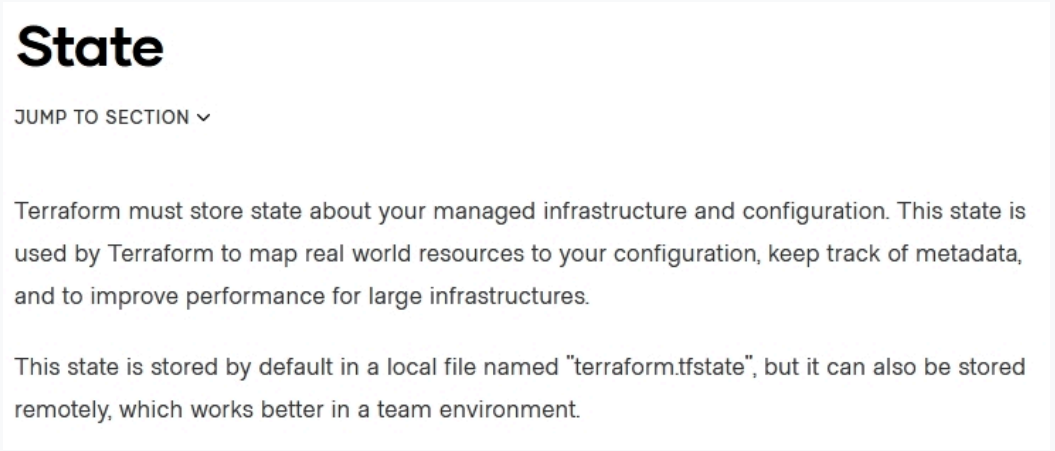
kopper2019 2 years, 3 months ago
terraform plan -out=FILE
upvoted 2 times

Honeywealth 2 years, 4 months ago
-out=FILE
upvoted 3 times

Question #62 *Topic 1*

FILL BLANK -
What is the name of the default file where Terraform stores the state?
Type your answer in the field provided. The text field is not case-sensitive and all variations of the correct answer are accepted.

Suggested Answer: *Terraform.tfstate*
Reference:
https://www.terraform.io/docs/language/state/index.html



Eltooth Highly Voted 2 years, 11 months ago
Answer is correct : terraform.tfstate

"This state is stored by default in a local file named "terraform.tfstate", but it can also be stored remotely, which works better in a team environment."

https://www.terraform.io/language/state
upvoted 8 times

nmnm22 Most Recent 9 months, 3 weeks ago
terraform.tfstate
upvoted 1 times

090200f 10 months, 2 weeks ago
terraform.tfstate , which is stored in our local workspace
upvoted 1 times

NPE_ 1 year, 5 months ago
The answer is wrong. terraform.tfstate not Terraform.tfstate, with t not T.
upvoted 1 times

joyboy23 1 year, 10 months ago
terraform.tfstate
upvoted 1 times

Power123 2 years, 1 month ago
terraform.tfstate
upvoted 1 times

thor7 2 years, 1 month ago
terraform.tfstate
upvoted 1 times

Bilalglg93350 2 years, 2 months ago
terraform.tfstate
upvoted 2 times

titoTitoApo 2 years, 3 months ago
terraform.tfstate
upvoted 2 times

kopper2019 2 years, 9 months ago

terraform.tfstate

upvoted 4 times

Question #63 *Topic 1*

A Terraform local value can reference other Terraform local values.

A. True

B. False

Suggested Answer: *A*

Reference:

https://www.terraform.io/docs/configuration-0-11/locals.html

The `locals` block defines one or more local variables within a module. Each `locals` block can have as many locals as needed, and there can be any number of `locals` blocks within a module.

The names given for the items in the `locals` block must be unique throughout a module. The given value can be any expression that is valid within the current module.

The expression of a local value can refer to other locals, but as usual reference cycles are not allowed. That is, a local cannot refer to itself or to a variable that refers (directly or indirectly) back to it.

It's recommended to group together logically-related local values into a single block, particularly if they depend on each other. This will help the reader understand the relationships between variables. Conversely, prefer to define *unrelated* local values in *separate* blocks, and consider annotating each block with a comment describing any context common to all of the enclosed locals.

Community vote distribution

A (100%)

Eltooth Highly Voted 2 years, 5 months ago

Answer is correct : True

"The expressions in local values are not limited to literal constants; they can also reference other values in the module in order to transform or combine them, including variables, resource attributes, or other local values:"

https://www.terraform.io/language/values/locals#declaring-a-local-value

upvoted 13 times

Molly1994 Most Recent 5 months, 3 weeks ago

A

```
locals {
# Ids for multiple sets of EC2 instances, merged together
instance_ids = concat(aws_instance.blue.*.id, aws_instance.green.*.id)
}
locals {
# Common tags to be assigned to all resources
common_tags = {
Service = local.service_name
Owner = local.owner
}
}
```

upvoted 2 times

ilmi70 9 months, 1 week ago

```
locals {
common_tags = {
Owner = "team-name"
Service = "customer-service"
}
extra_tags = merge(local.common_tags, { Environment = "production" })
}
```

upvoted 2 times

debabrata6983 1 year, 3 months ago

Selected Answer: A

The right answer is A as local value can also reference other terraform local value(s)

upvoted 1 times

Ragque 1 year, 3 months ago

Selected Answer: A

A is the correct answer

upvoted 1 times

camps 1 year, 7 months ago

Selected Answer: A

A. True.

A Terraform local value can reference other Terraform local values. Local values in Terraform are used to create intermediate values that can be used to simplify complex expressions or avoid duplicating values in the code. They can reference other resources and values within the Terraform configuration.

upvoted 1 times

Abuu 1 year, 9 months ago

Selected Answer: A

True; Terraform local values can reference other Terraform local values. This is useful for sharing common values between resources, such as defining a common name or set of tags. When referencing other local values, Terraform will automatically look for the referenced value and substitute it in the configuration. This helps avoid repetition and makes Terraform configurations easier to maintain.

upvoted 1 times

eduvar4 2 years, 2 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/values/locals#declaring-a-local-value

upvoted 1 times

Question #64 *Topic 1*

Which of the following is not a valid Terraform collection type?

A. list

B. map

C. tree

D. set

Suggested Answer: *C*

Reference:

https://www.terraform.io/docs/language/expressions/type-constraints.html

The three kinds of collection type in the Terraform language are:

- `list(...)`: a sequence of values identified by consecutive whole numbers starting with zero.

  The keyword `list` is a shorthand for `list(any)`, which accepts any element type as long as every element is the same type. This is for compatibility with older configurations; for new code, we recommend using the full form.

- `map(...)`: a collection of values where each is identified by a string label.

  The keyword `map` is a shorthand for `map(any)`, which accepts any element type as long as every element is the same type. This is for compatibility with older configurations; for new code, we recommend using the full form.

  Maps can be made with braces ({}) and colons (:) or equals signs (=): { "foo": "bar", "bar": "baz" } OR { foo = "bar", bar = "baz" }. Quotes may be omitted on keys, unless the key starts with a number, in which case quotes are required. Commas are required between key/value pairs for single line maps. A newline between key/value pairs is sufficient in multi-line maps.

  Note: although colons are valid delimiters between keys and values, they are currently ignored by `terraform fmt` (whereas `terraform fmt` will attempt vertically align equals signs).

- `set(...)`: a collection of unique values that do not have any secondary identifiers or ordering.

Community vote distribution

C (100%)

Asif_draxi 4 months, 1 week ago

Selected Answer: C

C. tree

Here's why:

Valid Terraform Collection Types:

list: An ordered collection of values.
map: An unordered collection of key-value pairs.
set: An unordered collection of unique values.
tree: Terraform does not have a built-in collection type called "tree." Trees are a data structure typically used in other programming languages for hierarchical data representation.

upvoted 1 times

godie44 7 months, 1 week ago

Selected Answer: C

C is the right answer

upvoted 3 times

debabrata6983 1 year, 3 months ago

Selected Answer: C

the tree is not a valid collection type in terrafrom

upvoted 2 times

SairamObili 1 year, 3 months ago

we dont a have tree type in terraform

upvoted 1 times

Bluemoon22 1 year, 7 months ago

answer is C, tree

upvoted 2 times

Power123 1 year, 7 months ago

Answer is C - tree

upvoted 1 times

Ignotus 2 years, 4 months ago

my bad, is NOT a valid... answer C is correct

upvoted 1 times

Ignotus 2 years, 4 months ago

tree isn't even on that page..

upvoted 2 times

Ahmad_Terraform 2 years, 4 months ago

C is the right answer

upvoted 4 times

Eltooth 2 years, 5 months ago

Selected Answer: C

C is correct answer. Tree

https://www.terraform.io/language/expressions/type-constraints#collection-types

upvoted 4 times

Question #65 *Topic 1*

When running the command terraform taint against a managed resource you want to force recreation upon, Terraform will immediately destroy and recreate the resource.

A. True

B. False

Suggested Answer: *B*

Reference:

https://www.devopsschool.com/blog/terraform-taint-and-untaint-explained-with-example-programs-and-tutorials/

Community vote distribution

B (100%)

HereToReAssureMyself Highly Voted 2 years, 11 months ago

Selected Answer: B

its not immediately

The terraform taint command informs Terraform that a particular object has become degraded or damaged. Terraform represents this by marking the object as "tainted" in the Terraform state, and Terraform will propose to replace it in the next plan you create.

upvoted 21 times

Eltooth Highly Voted 2 years, 11 months ago

Selected Answer: B

B is correct answer. False.

"The terraform taint command informs Terraform that a particular object has become degraded or damaged. Terraform represents this by marking the object as "tainted" in the Terraform state, and Terraform will propose to replace it in the next plan you create."

FYI - This command is deprecated. For Terraform v0.15.2 and later, we recommend using the -replace option with terraform apply instead. For Terraform v0.15.2 and later, we recommend using the -replace option with terraform apply to force Terraform to replace an object even though there are no configuration changes that would require it.

https://www.terraform.io/cli/commands/taint

upvoted 15 times

muammer32 Most Recent 5 months, 2 weeks ago

Selected Answer: B

The terraform taint command does not immediately destroy and recreate the resource. The resource will only be destroyed and recreated when you run terraform apply after tainting it.

upvoted 1 times

bora4motion 9 months, 1 week ago

Selected Answer: B

taint > plan> apply

I go with B.

upvoted 1 times

Molly1994 11 months, 3 weeks ago

false. taint is marking status, terraform apply make it destroy and recreate

upvoted 2 times

ilmi70 1 year, 3 months ago

B. False

The terraform taint command only marks a Terraform-managed resource as tainted, forcing it to be destroyed and recreated on the next apply. However, it does not immediately destroy and recreate the resource. The actual destruction and recreation happen when you run terraform apply.

upvoted 4 times

debabrata6983 1 year, 9 months ago

Selected Answer: B

It doesn't apply immediately. It will replace the managed resource, which is marked as tainted in the state file, during the next execution cycle (aka terraform apply)

upvoted 2 times

BennaniHaythem 2 years, 1 month ago

True :

When you use the terraform taint command, it marks a resource as "tainted", indicating that it needs to be recreated on the next terraform apply run. Once a resource is marked as tainted, Terraform will destroy and recreate it as part of the next apply operation. This behavior is intended to ensure that the resource is recreated from scratch and that any changes to its configuration are applied properly. Therefore, the statement "When running the command terraform taint against a managed resource you want to force recreation upon, Terraform will immediately destroy and recreate the resource" is true.

upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: B

B. False.

When running the command terraform taint against a managed resource in Terraform, you are marking the resource as tainted, which means that it will be recreated during the next terraform apply run. Tainting a resource does not immediately destroy and recreate the resource, but it causes Terraform to consider the resource as out-of-date and recreate it when the configuration is next applied.

upvoted 2 times

oab720 2 years, 4 months ago

Selected Answer: B

Can only be changed after 'Terraform apply'

upvoted 1 times

Ahmad_Terraform 2 years, 10 months ago

next apply.

upvoted 4 times

fifi1907 2 years, 11 months ago

Not A?

The terraform taint command manually marks a Terraform-managed resource as tainted, forcing it to be destroyed and recreated on the next apply.

upvoted 2 times

bp339 2 years, 11 months ago

Terraform will immediately destroy and recreate the resource.. tf doesn't immediately destroy it.. it will just mark it as tainted and in the next plan/apply it will recreate..so answer is B

upvoted 8 times

Question #66 *Topic 1*

All standard backend types support state storage, locking, and remote operations like plan, apply and destroy.

A. True

B. False

Suggested Answer: *A*
Reference:
https://www.terraform.io/docs/language/settings/backends/remote.html

Community vote distribution
B (92%) 8%

vadeemkaa Highly Voted 1 year, 11 months ago
B. False

Backend types:
Enhanced Backend – Additional operations like plan, apply, etc. on remote.
Standard backend – Simple State file storage and lock facility
upvoted 14 times

BaburTurk Highly Voted 1 year, 3 months ago
Selected Answer: B
The answer is False.

Not all standard backend types support state locking and remote operations. The following standard backend types do not support state locking:

AzureRM
AzureKeyVault
Consul
Docker
Google Cloud Storage
Kubernetes
MySQL
Oracle
PostgreSQL
Vault
The following standard backend types support state locking but not remote operations:

AWS S3
upvoted 9 times

krusty93 1 year, 1 month ago
Don't know the others, but AzureRM supports state locking: https://developer.hashicorp.com/terraform/language/settings/backends/azurerm
upvoted 3 times

kingfighers Most Recent 7 months, 3 weeks ago
Note: In Terraform versions before 1.1.0, we classified backends as standard or enhanced. The enhanced label differentiated the remote backend, which could both store state and perform Terraform operations. This classification has been removed.
https://developer.hashicorp.com/terraform/language/settings/backends/configuration another deprecated feature and question..
upvoted 1 times

NashP 9 months, 4 weeks ago
A. True

Explanation:
All standard backend types in Terraform support state storage, locking, and remote operations such as plan, apply, and destroy. Terraform backends are responsible for storing the Terraform state file, managing concurrent access to the state, and facilitating collaboration among team members.

Different backend types may have additional features or considerations, but these fundamental capabilities are common across all standard backends.

upvoted 1 times

itstammy 10 months, 2 weeks ago

https://digitalvarys.com/complete-terraform-tutorial-part-6-terraform-backends/ its B

upvoted 1 times

MisterR0B0T 1 year ago

Terraform

Documentation - Not all backends support locking. The documentation for each backend includes details about whether it supports locking or not.

upvoted 2 times

foreverlearner 1 year, 4 months ago

This question is outdated https://developer.hashicorp.com/terraform/language/settings/backends/configuration : In Terraform versions before 1.1.0, we classified backends as standard or enhanced. The enhanced label differentiated the remote backend, which could both store state and perform Terraform operations. This classification has been removed. "

"Some of these backends act like plain remote disks for state files, while others support locking the state while operations are being performed"

upvoted 2 times

Ni33 1 year, 6 months ago

Selected Answer: A

AAAAAAAAAAAA

upvoted 2 times

jerikoo 1 year, 6 months ago

yes, it's A.

SOME of these backends act like plain remote disks for state files, while OTHERS support locking the state while operations are being performed.

SO not ALL are LOCKING... A!

upvoted 1 times

jerikoo 1 year, 6 months ago

sorry, dyslexia, its B! :))

upvoted 2 times

Power123 1 year, 7 months ago

Answer is B

upvoted 2 times

agmesas 1 year, 9 months ago

B. Artefactory is a standard backend and has not locking feature. Also, etcd has not locking feature (only etcdv3).

upvoted 4 times

kopper2019 2 years, 3 months ago

Selected Answer: B

not all of them so False

upvoted 4 times

Ggggg23 2 years, 4 months ago

https://www.terraform.io/language/settings/backends/configuration

"Some of these backends act like plain remote disks for state files, while others support locking the state while operations are being performed. This helps prevent conflicts and inconsistencies. The built-in backends listed are the only backends. You cannot load additional backends as plugins."

upvoted 3 times

Ahmad_Terraform 2 years, 4 months ago

Not all backend encrypt , B false = correct

upvoted 1 times

Eltooth 2 years, 5 months ago

Selected Answer: B

B is correct answer : False.

"By default, Terraform uses a backend called local, which stores state as a local file on disk. You can also configure one of the built-in backends listed in the documentation sidebar.

Some of these backends act like plain remote disks for state files, while others support locking the state while operations are being performed. This helps prevent conflicts and inconsistencies. The built-in backends listed are the only backends. You cannot load additional backends as plugins."

https://www.terraform.io/language/settings/backends/configuration#available-backends

upvoted 4 times

HDDH 2 years, 5 months ago

Selected Answer: B

What Backends Do

Backends primarily determine where Terraform stores its state. Terraform uses this persisted state data to keep track of the resources it manages. Since it needs the state in order to know which real-world infrastructure objects correspond to the resources in a configuration, everyone working with a given collection of infrastructure resources must be able to access the same state data.

By default, Terraform implicitly uses a backend called local to store state as a local file on disk. Every other backend stores state in a remote service of some kind, which allows multiple people to access it. Accessing state in a remote service generally requires some kind of access credentials, since state data contains extremely sensitive information.

Some backends act like plain "remote disks" for state files; others support locking the state while operations are being performed, which helps prevent conflicts and inconsistencies.

upvoted 4 times

stalk98 2 years, 5 months ago

answer is B

upvoted 2 times

bigboi23 2 years, 6 months ago

Selected Answer: B

Some backends act like plain "remote disks" for state files; others support locking the state while operations are being performed, which helps prevent conflicts and inconsistencies.

upvoted 2 times

Question #67 *Topic 1*

How can terraform plan aid in the development process?

A. Validates your expectations against the execution plan without permanently modifying state

B. Initializes your working directory containing your Terraform configuration files

C. Formats your Terraform configuration files

D. Reconciles Terraform's state against deployed resources and permanently modifies state using the current status of deployed resources

Suggested Answer: *A*
Reference:
https://github.com/hashicorp/terraform/issues/19235

Community vote distribution
A (86%) 14%

Eltooth Highly Voted 2 years, 5 months ago

Selected Answer: A

A is correct answer : Validates your expectations against the execution plan without permanently modifying state

"The terraform plan command creates an execution plan, which lets you preview the changes that Terraform plans to make to your infrastructure. By default, when Terraform creates a plan it:
Reads the current state of any already-existing remote objects to make sure that the Terraform state is up-to-date.
Compares the current configuration to the prior state and noting any differences.
Proposes a set of change actions that should, if applied, make the remote objects match the configuration."

"The plan command alone will not actually carry out the proposed changes, and so you can use this command to check whether the proposed changes match what you expected before you apply the changes or share your changes with your team for broader review.

If Terraform detects that no changes are needed to resource instances or to root module output values, terraform plan will report that no actions need to be taken."

https://www.terraform.io/cli/commands/plan

upvoted 8 times

Molly1994 Most Recent 5 months, 3 weeks ago

A is right. D is wrong. terraform plan did not change anything. It only give you an execution plan that shows your changes.

upvoted 1 times

rga91 1 year, 2 months ago

Selected Answer: D

D. When you execute a Terraform Plan, a refresh is produced and the state is modified.
Easy example: you have created a Virtual Machine using Terraform and then you manually deleted using for example the AWS console. In the state, it will appear that Virtual Machine exists, but after running Terraform plan, it will detect no longer exists, and it will update the state.

You can try it yourself. It is a very simple test.

PD: the only way to avoid this refresh, is using the flag -refresh=false in terraform plan, but since it does not say anything about it, this is not a option

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

AAAAAAAAAAAA

upvoted 1 times

InformationOverload 1 year, 10 months ago

Selected Answer: A

Its A, no question.

upvoted 4 times

chimons 1 year, 11 months ago

Selected Answer: A

A. Validates your expectations against the execution plan without permanently modifying state

Terraform's plan command is used to create an execution plan that outlines the steps that Terraform will take to reach your desired infrastructure state. It allows you to preview and validate the changes that will be made to your infrastructure before actually making those changes. This can be helpful in the development process because it allows you to see exactly what will be changed and ensure that it aligns with your expectations before you apply those changes.

upvoted 3 times

lxgywil 1 year, 11 months ago

Selected Answer: D

It's D.

`terraform plan` reads the current state of any already-existing remote objects to MAKE SURE that the Terraform state is up-to-date.

`-refresh-only` - creates a plan whose goal is ONLY to update the Terraform state and any root module output values to match changes made to remote objects outside of Terraform.

`-refresh=false` - disables the DEFAULT behavior of synchronizing the Terraform state with remote objects before checking for configuration changes.

Reference: https://developer.hashicorp.com/terraform/cli/commands/plan

upvoted 2 times

MadMarc 1 year, 8 months ago

D. Reconciles Terraform's state against deployed resources and permanently modifies state using the current status of deployed resources

Terraform plan does not, by any means, modify the state file permanently. You could run tf plan a hundred times and it will show you the same differences.

The correct answer, and the only logical answer possible, is A.

upvoted 2 times

raf314 1 year, 7 months ago

Your answer (D) is wrong... Terraform plan does not modify the state plan, which is what D states.

upvoted 2 times

eduvar4 2 years, 2 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/cli/commands/plan

upvoted 2 times

tf_user 2 years, 2 months ago

Selected Answer: A

A for sure

upvoted 1 times

Ahmad_Terraform 2 years, 4 months ago

Validates your expectations

upvoted 1 times

CHRIS12722222 2 years, 5 months ago

A looks good

upvoted 1 times

Zam88 2 years, 5 months ago

A correct.

upvoted 1 times

Question #68 *Topic 1*

You would like to reuse the same Terraform configuration for your development and production environments with a different state file for each. Which command would you use?

A. terraform import

B. terraform workspace

C. terraform state

D. terraform init

Suggested Answer: *B*

Community vote distribution

B (100%)

amrith501 Highly Voted 2 years, 11 months ago

Selected Answer: B

Workspace is the right answer

upvoted 10 times

Ahmad_Terraform Highly Voted 2 years, 10 months ago

workspace = different state file

upvoted 7 times

090200f Most Recent 10 months, 2 weeks ago

terraform workspace new dev (or prod) , terraform apply -var-file dev.tfvars

so terraform workspace is the answer

upvoted 2 times

Ni33 2 years ago

Selected Answer: B

BBBBBBBBB

upvoted 1 times

eduvar4 2 years, 8 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/language/state/workspaces#when-to-use-multiple-workspaces

upvoted 2 times

Eltooth 2 years, 11 months ago

Selected Answer: B

B is correct answer : Workspaces.

https://www.terraform.io/language/state/workspaces#when-to-use-multiple-workspaces

upvoted 4 times

Question #69 *Topic 1*

What is the name assigned by Terraform to reference this resource?

```
mainresource “google_compute_instance” “main” {
      name = “test”
}
```

A. compute_instance

B. main

C. google

D. teat

Suggested Answer: *B*

Community vote distribution

B (100%)

Eltooth Highly Voted 2 years, 5 months ago

Selected Answer: B

B is correct answer : main

See Topic 1 #Question49 for similar question.

upvoted 6 times

TigerInTheCloud Most Recent 11 months, 1 week ago

invalid terraform config :-)

upvoted 1 times

IK912 1 year, 5 months ago

B is correct

upvoted 1 times

Power123 1 year, 7 months ago

Main . B is correct

upvoted 1 times

zecch 1 year, 8 months ago

Selected Answer: B

answer is B. Main is the name of resource block.

upvoted 1 times

Ahmad_Terraform 2 years, 4 months ago

Main

is the name of the resources

upvoted 3 times

Question #70 *Topic 1*

You're building a CI/CD (continuous integration/ continuous delivery) pipeline and need to inject sensitive variables into your Terraform run. How can you do this safely?

A. Pass variables to Terraform with a ג€"var flag

B. Copy the sensitive variables into your Terraform code

C. Store the sensitive variables in a secure_vars.tf file

D. Store the sensitive variables as plain text in a source code repository

Suggested Answer: *B*

Community vote distribution

A (100%)

Bere Highly Voted 1 year, 9 months ago

Selected Answer: A

The safest method to inject sensitive variables into your Terraform run in a CI/CD pipeline is:
A. Pass variables to Terraform with a -var flag.

When running Terraform in your CI/CD pipeline, you would pass the sensitive variables like this:

terraform apply -var="database_password=mysecretpassword" -var="api_key=1234567890abcdef"

For enhanced security, in a CI/CD environment, you'd typically use environment variables or secret management tools that are supported by your CI/CD platform to pass these sensitive values, so they are never exposed in logs or stored in insecure locations.

For instance, using environment variables:

export TF_VAR_database_password=mysecretpassword
export TF_VAR_api_key=1234567890abcdef
terraform apply

upvoted 15 times

rfd Highly Voted 2 years, 11 months ago

Selected Answer: A

Definitely A.

upvoted 7 times

akki_0108 Most Recent 8 months, 1 week ago

Selected Answer: A

A is Correct for passing sensitive vars to CI/CD

upvoted 1 times

samimshaikh 1 year, 4 months ago

Selected Answer: A

The safest option among the choices provided is to use the -var flag to pass sensitive variables to Terraform. This allows you to provide variable values at runtime without hardcoding them directly into the Terraform code or exposing them in the source code repository.

upvoted 5 times

Tlakmini 1 year, 9 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/values/variables

upvoted 2 times

cloudznj 1 year, 10 months ago

I see that is A but since its on B. Will it be B on the exam? Or that option, even though everyone is on A. Just curious.

upvoted 1 times

LunarPhobia 1 year, 10 months ago

The answers on here aren't always correct. Typically go with the highest rated one

upvoted 2 times

Faaizz 2 years, 1 month ago

Selected Answer: A

Obviously A

upvoted 2 times

camps 2 years, 1 month ago

Selected Answer: A

A. Pass variables to Terraform with a "var" flag.

When running Terraform in a CI/CD pipeline, it is important to securely pass sensitive variables to Terraform. One way to pass sensitive variables to Terraform is to use the -var flag to supply the value of the variable at runtime.

upvoted 3 times

Power123 2 years, 1 month ago

A is correct

upvoted 1 times

thor7 2 years, 1 month ago

Selected Answer: A

A is a correct answer

Reference: https://developer.hashicorp.com/terraform/language/values/variables

upvoted 1 times

eduvar4 2 years, 8 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/values/variables

upvoted 1 times

tf_user 2 years, 8 months ago

Selected Answer: A

A is correct

upvoted 1 times

Tomcrui1234589 2 years, 9 months ago

A for sure

upvoted 1 times

Ahmad_Terraform 2 years, 10 months ago

A= correct

Var

upvoted 1 times

Eltooth 2 years, 11 months ago

Selected Answer: A

A is correct answer : -var

See Topic 1 # Question 43

https://blog.gruntwork.io/a-comprehensive-guide-to-managing-secrets-in-your-terraform-code-1d586955ace1

upvoted 3 times

bp339 2 years, 11 months ago

Selected Answer: A

A is the answer

upvoted 2 times

biscuithammer 3 years ago

Selected Answer: A

I vote A

upvoted 1 times

Question #71 *Topic 1*

Your security team scanned some Terraform workspaces and found secrets stored in a plaintext in state files.
How can you protect sensitive data stored in Terraform state files?

A. Delete the state file every time you run Terraform

B. Store the state in an encrypted backend

C. Edit your state file to scrub out the sensitive data

D. Always store your secrets in a secrets.tfvars file.

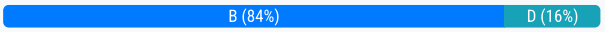
Suggested Answer: *B*
Reference:
https://www.terraform.io/docs/language/state/sensitive-data.html

Storing state remotely can provide better security. As of Terraform 0.9, Terraform does not persist state to the local disk when remote state is in use, and some backends can be configured to encrypt the state data at rest.

For example:

- Terraform Cloud always encrypts state at rest and protects it with TLS in transit. Terraform Cloud also knows the identity of the user requesting state and maintains a history of state changes. This can be used to control access and track activity. Terraform Enterprise also supports detailed audit logging.
- The S3 backend supports encryption at rest when the `encrypt` option is enabled. IAM policies and logging can be used to identify any invalid access. Requests for the state go over a TLS connection.

Community vote distribution



Asif_draxi 4 months, 1 week ago

Selected Answer: B

Your security team scanned some Terraform workspaces and found secrets stored in a plaintext in state files.
How can you protect sensitive data stored in Terraform state files?

A. Delete the state file every time you run Terraform
B. Store the state in an encrypted backend
C. Edit your state file to scrub out the sensitive data
D. Always store your secrets in a secrets.tfvars file.

upvoted 1 times

NashP 9 months, 4 weeks ago

B. Store the state in an encrypted backend

Explanation:
Storing the Terraform state in an encrypted backend is the recommended approach to protect sensitive data. Terraform supports various backends, and using an encrypted backend ensures that the state file is encrypted at rest, adding an extra layer of security.

upvoted 2 times

shefulacertificari 10 months, 1 week ago

Selected Answer: B

B is the answer.

upvoted 1 times

gofavad926 1 year, 1 month ago

Selected Answer: B

B, got answer:
To protect sensitive data stored in Terraform state files, you should use an encrypted backend. Storing state in an encrypted backend helps secure sensitive information, such as secrets and access keys, that might be present in the state file. This prevents unauthorized access to sensitive data.

D. Always store your secrets in a secrets.tfvars file: Storing secrets in separate variable files is a common practice, but it doesn't directly address the security of the Terraform state. Even if secrets are in a separate file, protecting the state file is still important.
upvoted 3 times

aanataliya 1 year, 2 months ago

Selected Answer: B

Confusion between B and D? Check this.
Terraform will still record sensitive values in the state, and so anyone who can access the state data will have access to the sensitive values in cleartext
Ref: https://developer.hashicorp.com/terraform/language/values/variables#suppressing-values-in-cli-output

Variables in any file is still stored in state as plaintext. So D cannot be correct answer. Please vote for correct answer to help others.
upvoted 2 times

Ni33 1 year, 6 months ago

Selected Answer: D

Why not D? Setting values with a .tfvars file allows you to separate sensitive values from the rest of your variable values, and makes it clear to people working with your configuration which values are sensitive. However, it requires that you maintain and share the secret.tfvars file with only the appropriate people. You must also be careful not to check .tfvars files with sensitive values into version control.
upvoted 3 times

joyboy23 1 year, 4 months ago

Wouldn't it still be rendered as plain text in your state files ?
upvoted 2 times

Chrisler 1 year, 2 months ago

I disagree, setting values with a .tfvar file is also an option but least recommended. Securing your state file by encryption would only give access to the person or a team that needs it.
upvoted 1 times

DKwork 1 year, 5 months ago

It cannot be D because the problem is that the secrets are plaintext within your secrets.tfvars file. Think about how you would secure that secrets.tfvars within your group of appropriate people
upvoted 3 times

Power123 1 year, 7 months ago

B is correct
upvoted 1 times

camps 1 year, 7 months ago

Selected Answer: B

B. Store the state in an encrypted backend.

Terraform state files can contain sensitive information such as access keys, passwords, and private keys. To protect this information from unauthorized access, it is important to store the state file securely.
upvoted 3 times

vadeemkaa 1 year, 11 months ago

Definitely B
upvoted 1 times

nakikoo 1 year, 11 months ago

Selected Answer: B

agreeee
upvoted 1 times

eduvar4 2 years, 2 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/language/state/sensitive-data

upvoted 3 times

Ahmad_Terraform 2 years, 4 months ago

B

store in encrypted backend , E.g S3

upvoted 3 times

Eltooth 2 years, 5 months ago

Selected Answer: B

B is correct answer : Store in encrypted backend.

upvoted 3 times

Question #72 *Topic 1*

In contrast to Terraform Open Source, when working with Terraform Enterprise and Cloud Workspaces, conceptually you could think about them as completely separate working directories.

A. True

B. False

Suggested Answer: *A*

Community vote distribution

A (100%)

Eltooth Highly Voted 1 year, 4 months ago

Selected Answer: A

Very poorly worded question however I believe they are refering to the difference between OSS (directories) and Cloud/Ent that uses workspaces, which includes separate working directories.

https://www.terraform.io/cloud-docs/workspaces

upvoted 18 times

Eltooth 1 year, 4 months ago

"When run locally, Terraform manages each collection of infrastructure with a persistent working directory, which contains a configuration, state data, and variables. Since Terraform CLI uses content from the directory it runs in, you can organize infrastructure resources into meaningful groups by keeping their configurations in separate directories.

Terraform Cloud manages infrastructure collections with workspaces instead of directories. A workspace contains everything Terraform needs to manage a given collection of infrastructure, and separate workspaces function like completely separate working directories."

upvoted 8 times

devopschamp Highly Voted 1 year, 4 months ago

Selected Answer: A

Answer is A:

From the doc: "Terraform Cloud manages infrastructure collections with workspaces instead of directories. A workspace contains everything Terraform needs to manage a given collection of infrastructure, and separate workspaces function like completely separate working directories."

https://www.terraform.io/cloud-docs/workspaces

upvoted 5 times

Power123 Most Recent 7 months, 4 weeks ago

A is correct

upvoted 1 times

kopper2019 1 year, 3 months ago

Selected Answer: A

A, True

upvoted 3 times

Ahmad_Terraform 1 year, 4 months ago

A

is correct.

upvoted 1 times

yufei 1 year, 5 months ago

Answer A is correct.

upvoted 1 times

CHRIS12722222 1 year, 5 months ago

I think the answer should be FALSE. Terraform enterprise is essentially same as terraform cloud with additional features like logging etc.

upvoted 1 times

Oskar_Madin 1 year, 4 months ago

you compared Terraform enterprise and terraform cloud but how about Terraform Open Source?

upvoted 1 times

CHRIS12722222 1 year, 4 months ago

I misunderstood the question, Answer is True

upvoted 1 times

Question #73 *Topic 1*

You want to know from which paths Terraform is loading providers referenced in your Terraform configuration (*.tf files). You need to enable debug messages to find this out.
Which of the following would achieve this?

A. Set the environment variable TF_LOG=TRACE

B. Set verbose logging for each provider in your Terraform configuration

C. Set the environment variable TF_VAR_log=TRACE

D. Set the environment variable TF_LOG_PATH

Suggested Answer: *A*
Reference:
https://www.terraform.io/docs/cli/config/environment-variables.html

Terraform refers to a number of environment variables to customize various aspects of its behavior. None of these environment variables are required when using Terraform, but they can be used to change some of Terraform's default behaviors in unusual situations, or to increase output verbosity for debugging.

TF_LOG

Enables detailed logs to appear on stderr which is useful for debugging. For example:

```
export TF_LOG=trace
```

Copy

Community vote distribution
A (100%)

Eltooth Highly Voted 1 year, 5 months ago
Selected Answer: A
A is correct answer. TF_LOG=TRACE

Although this will only output to stderr and if you need to review log file you will need to include TF_LOG_PATH=pathtofile

https://www.terraform.io/internals/debugging
upvoted 13 times

yufei Highly Voted 1 year, 5 months ago
A. TRACE is the most verbose option available and should show what the user is looking for.
TRACE will show TRACE, DEBUG, INFO, WARN and ERROR messages in the output.
upvoted 6 times

Power123 Most Recent 7 months, 4 weeks ago
TF_LOG=TRACE is correct. Ans A
upvoted 1 times

chimons 11 months, 1 week ago
Selected Answer: A
A. Set the environment variable TF_LOG=TRACE

To enable debug messages in Terraform, you can set the environment variable TF_LOG to the value TRACE. This will cause Terraform to print detailed debug information, including the paths from which it is loading providers.
upvoted 3 times

Ahmad_Terraform 1 year, 4 months ago

Set the environment variable TF_LOG=TRACE

upvoted 3 times

Question #74 *Topic 1*

How is terraform import run?

A. As a part of terraform init

B. As a part of terraform plan

C. As a part of terraform refresh

D. By an explicit call

E. All of the above

Suggested Answer: *D*

Community vote distribution

D (100%)

Eltooth Highly Voted 1 year, 11 months ago

Selected Answer: D

I'm going with D based on this doc.

"The current implementation of Terraform import can only import resources into the state. It does not generate configuration. A future version of Terraform will also generate configuration.

Because of this, prior to running terraform import it is necessary to write manually a resource configuration block for the resource, to which the imported object will be mapped.

While this may seem tedious, it still gives Terraform users an avenue for importing existing resources."

https://www.terraform.io/cli/import/usage

upvoted 9 times

Eltooth 1 year, 10 months ago

https://www.terraform.io/cli/commands/import

Usage: terraform import [options] ADDRESS ID

Import will find the existing resource from ID and import it into your Terraform state at the given ADDRESS.

ADDRESS must be a valid resource address. Because any resource address is valid, the import command can import resources into modules as well as directly into the root of your state.

ID is dependent on the resource type being imported. If the ID is invalid, you'll just receive an error message.

Warning: Terraform expects that each remote object it is managing will be bound to only one resource address, which is normally guaranteed by Terraform itself having created all objects. If you import existing objects into Terraform, be careful to import each remote object to only one Terraform resource address. If you import the same object multiple times, Terraform may exhibit unwanted behavior.

upvoted 1 times

Clapton79 Most Recent 6 months, 2 weeks ago

Selected Answer: D

Latest version of Terraform can import as part of apply in case you have import {} clauses.

Since this is not part of the answers, explicit call remains, answer D.

upvoted 1 times

Power123 1 year, 1 month ago

Correct answer is D

upvoted 1 times

Optimus 1 year, 8 months ago

Selected Answer: D

D is correct. I have tested it.

upvoted 2 times

Ahmad_Terraform 1 year, 10 months ago

By an explicit call

upvoted 3 times

Question #75 *Topic 1*

You have a simple Terraform configuration containing one virtual machine (VM) in a cloud provider. You run terraform apply and the VM is created successfully.
What will happen if you delete the VM using the cloud provider console, and run terraform apply again without changing any Terraform code?

A. Terraform will remove the VM from state file

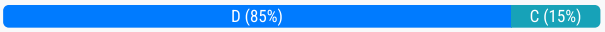
B. Terraform will report an error

C. Terraform will not make any changes

D. Terraform will recreate the VM

Suggested Answer: *D*

Community vote distribution



vitasac Highly Voted 2 years, 6 months ago

Selected Answer: D

for sure response D

upvoted 18 times

Tyler2023 Highly Voted 1 year ago

I tried this,
I created the storage account first by running terraform apply
Then I manually deleted the storage account through azure portal
Then reran the terraform apply
- It refresh the state, which detects that the storage account was gone
- Then it re-creates storage account with the same name but without the data from previous instance

upvoted 6 times

Asif_draxi Most Recent 4 months, 1 week ago

Selected Answer: D

The correct answer is **D. Terraform will recreate the VM**.

When you delete the VM manually via the cloud provider's console, the actual infrastructure no longer exists, but Terraform's state file still has a record of the VM. When you run `terraform apply` again, Terraform detects that the VM is missing from the actual infrastructure and will attempt to recreate it to match the configuration defined in the Terraform code.

upvoted 1 times

chaoscreater 6 months ago

The people who answered C are just doomed to fail this exam. This is such a basic foundational knowledge and it's the very core of Terraform and you can't even get this simple one right? This is such a freebie question.

upvoted 3 times

vibzr2023 8 months ago

D. Terraform will recreate the VM

When you delete a resource like a VM directly through the cloud provider's console (outside of Terraform), the Terraform state file still believes the resource exists, as it's unaware of any changes made outside its management. The next time you run terraform apply, Terraform compares the desired state (defined in your Terraform configuration) with the actual state (as recorded in the state file and observed in the cloud environment).

Since the actual VM no longer exists but your Terraform configuration still defines it, Terraform detects this discrepancy and takes action to reconcile the difference by creating a new VM to match the desired state defined in your Terraform configuration. Terraform's goal is always to make the real-world infrastructure match the configuration.

upvoted 2 times

gofavad926 1 year, 1 month ago

Selected Answer: D

D, the instance will be recreated

upvoted 1 times

Halimb 1 year, 2 months ago

Selected Answer: D

D. Stop messing around.

This behavior applies to any resource created and managed by Terraform, not just virtual machines, e.g. simple resources like resource groups in Azure (like I've tested with) or any other cloud provider's resources.

upvoted 1 times

Bluemoon22 1 year, 7 months ago

answer is D

upvoted 1 times

lucasbg 1 year, 7 months ago

Selected Answer: C

it`s C for sure. Since the VM was deleted using the cloud web/console, the tfstate will still contain the VM information there, hence nothing will be done. This is drift, and the steps here will be to run terraform -apply | plan -refresh-only, the tfsate will update that the VM was removed and then you will run apply again.

upvoted 3 times

Mridul31792 1 year, 7 months ago

Correct answer is D as terraform apply implicitly runs terraform refresh before applying the changes which will clear the data of VM from the state file and plan will show to create a new VM.

upvoted 2 times

Power123 1 year, 8 months ago

D is correct

upvoted 1 times

robertninho 1 year, 10 months ago

Correct answer is D. Terraform will recreate the VM.

In Terraform, the state file is used to store the current state of your infrastructure. When you run terraform apply, Terraform compares the state of your infrastructure as defined in the configuration files with the state recorded in the state file, and then makes any necessary changes to bring the infrastructure into compliance with the configuration.

upvoted 4 times

vadeemkaa 1 year, 11 months ago

Definitely the answer is D

upvoted 1 times

chimons 1 year, 11 months ago

Selected Answer: D

A refresh will be made before apply, therefore terraform will detect the VM is missing, and will update state accordingly.
Then, it will create a new one to match configuration

upvoted 5 times

adouban 1 year, 11 months ago

Selected Answer: D

I have tested this on my lab on oracle cloud,
1- created a VM using TF
2- Deleted VM manually
3- on Terraform apply, attempted to recreate the instance

D is the correct answer

upvoted 3 times

Raghav_123 1 year, 11 months ago

The ans is C

upvoted 1 times

secdaddy 1 year, 11 months ago

Not sure how you can get C
I just did this :

1. created a VM on AWS using terraform apply
2. used the AWS console to delete the VM
3. ran terraform apply again without changing any Terraform code

Result : terraform recreated the VM (D)

upvoted 3 times

GHOST1985 2 years ago

Selected Answer: C

Of course C

When you delete from the console you don't remove the VM from the state file so in the next plan or apply terrafom will not apply any changes because from based on the state file the VM is already created.

upvoted 2 times

Question #76 *Topic 1*

Which of these options is the most secure place to store secrets foe connecting to a Terraform remote backend?

A. Defined in Environment variables

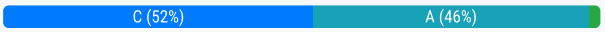
B. Inside the backend block within the Terraform configuration

C. Defined in a connection configuration outside of Terraform

D. None of above

Suggested Answer: *A*

Community vote distribution



zyxphreez Highly Voted 2 years, 9 months ago

Selected Answer: A

Definitely is: A

https://www.terraform.io/language/settings/backends/configuration#credentials-and-sensitive-data

Warning: We recommend using environment variables to supply credentials and other sensitive data. If you use -backend-config or hardcode these values directly in your configuration, Terraform will include these values in both the .terraform subdirectory and in plan files. This can leak sensitive credentials.

upvoted 33 times

Alandt 1 year, 4 months ago

Authentication outside of Terraform is more secure than environment variables. Your environment variables can still refer to a file or the definition of your variables inside terraform. So I would go for C.

upvoted 3 times

deepeshukla 1 year, 11 months ago

I will select C. In option A, any debugging will still disclose data.

upvoted 1 times

Gomjaba 1 year, 8 months ago

I presume they are hinting at vault here.

upvoted 1 times

CHRIS12722222 Highly Voted 2 years, 10 months ago

Selected Answer: C

I will go for option C. Whenever possible, it is best to authenticate outside of terraform to keep secrets out of state file

upvoted 18 times

Alandt 1 year, 4 months ago

I agree with this.

upvoted 1 times

KG8 Most Recent 2 months, 3 weeks ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/backend

Warning: We recommend using environment variables to supply credentials and other sensitive data. If you use -backend-config or hardcode these values directly in your configuration, Terraform will include these values in both the .terraform subdirectory and in plan files. This can leak sensitive credentials.

upvoted 2 times

Asif_draxi 4 months, 1 week ago

Selected Answer: A

The correct answer is **A. Defined in Environment variables**.

Environment variables are considered the most secure place to store secrets for connecting to a Terraform remote backend because:

- They keep sensitive information out of source control (such as in `.tf` files or other configuration files).
- Secrets stored in environment variables can be managed securely by the operating system or deployment environment and can be encrypted or otherwise protected.

Storing secrets inside the backend block (option B) or in a connection configuration outside Terraform (option C) can expose them to version control or make them less secure. Therefore, it's best practice to use environment variables for storing sensitive credentials securely.
upvoted 1 times

suri_surendra 4 months, 4 weeks ago

Selected Answer: A

The correct answer is: A. Defined in Environment variables

Explanation:
When storing secrets for connecting to a Terraform remote backend, environment variables are the most secure option. Here's why:

Environment Variables:
Secure: Environment variables can be securely managed outside of Terraform configuration files and are not checked into version control.
Flexibility: You can use tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault to inject secrets as environment variables dynamically.
Common Practice: Many services (e.g., AWS, Azure, Google Cloud) support using environment variables for storing sensitive credentials like access keys, secret keys, and tokens.
upvoted 1 times

Dimedrol1 7 months ago

I've answered D - "None of the above", because the most secure way is e.g. Azure Key Vault, when working with Azure, of course.
In this case - we're not revealing any of the secrets values.
Terraform can refer them via "data" operators.
(and, yes, this approach is not a "connection configuration")
upvoted 1 times

Algol 8 months, 2 weeks ago

Selected Answer: A

The most secure option for storing secrets when connecting to a Terraform remote backend is A. Defined in environment variables. This approach keeps sensitive information, such as API keys or access tokens, out of your version-controlled configuration files. Defining secrets in environment variables reduces the risk of accidental exposure or leakage through your Terraform configuration. Using external secret management tools (like HashiCorp Vault) to manage environment variables can further enhance security.
upvoted 1 times

premnick 10 months ago

Question is which one is MOST secure. Environment variable is not secure. You type env command from the host and easily view the secret value.
upvoted 2 times

SureNot 11 months, 2 weeks ago

Selected Answer: C

Let's imagine use AWS S3 bas a backend. Credentials to S3 Bucket are stored in ~/.aws/credentials file - Outside of terraform, most secure way.
upvoted 2 times

Molly1994 11 months, 3 weeks ago

C vault as example
upvoted 1 times

deepakpamban 1 year ago

Option C
upvoted 2 times

Venki_dev 1 year, 1 month ago

Selected Answer: C

C. Defined in a connection configuration outside of Terraform (Most Secure)

This is the most secure option. Here, you store your secrets in a separate dedicated location outside of your Terraform configuration. There are several ways to achieve this:

Secret Management Tools: Utilize tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault to store and manage your secrets securely. These tools offer access control and encryption mechanisms.

Encrypted Files: Store secrets in an encrypted file outside your Terraform configuration directory. Terraform can access these secrets during execution by referencing the decrypted content of the file.

upvoted 4 times

kingfighers 1 year, 1 month ago

choose A:

when we use vault, we still need to download it into a file,here is official doc:

- **File**: A configuration file may be specified via the `init` command line. To specify a file, use the `-backend-config=PATH` option when running `terraform init`. If the file contains secrets it may be kept in a secure data store, such as [Vault](https://www.vaultproject.io/), in which case it must be downloaded to the local disk before running Terraform.

https://developer.hashicorp.com/terraform/language/settings/backends/configuration#credentials-and-sensitive-data

upvoted 2 times

aksliveswithaws 1 year, 1 month ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/settings/backends/configuration#credentials-and-sensitive-data:~:text=and%20apply%20steps.-,backend%20types,-The%20block%20label

upvoted 1 times

AntonyPeter7 1 year, 2 months ago

Selected Answer: C

Authentication outside of Terraform is more secure than environment variables. Like using terraform vault or cloud

upvoted 1 times

Kaname93 1 year, 3 months ago

Selected Answer: A

From the documentation :
Warning: We recommend using environment variables to supply credentials and other sensitive data. If you use -backend-config or hardcode these values directly in your configuration, Terraform will include these values in both the .terraform subdirectory and in plan files. This can leak sensitive credentials.

So it's A

upvoted 1 times

Alandt 1 year, 4 months ago

Selected Answer: C

Definitely C. Authentication outside of Terraform is the most secure way.

upvoted 1 times

Question #77 *Topic 1*

Your DevOps team is currently using the local backend for your Terraform configuration. You would like to move to a remote backend to begin storing the state file in a central location.
Which of the following backends would not work?

A. Amazon S3

B. Artifactory

C. Git

D. Terraform Cloud

Suggested Answer: *A*
Reference:
https://secrethub.io/blog/secret-management-for-terraform/

Community vote distribution
C (63%) B (37%)

Reet Highly Voted 1 year, 9 months ago

Selected Answer: B

Note: We removed the artifactory, etcd, etcdv3, manta, and swift backends in Terraform v1.3. Information about their behavior in older versions is still available in the Terraform v1.2 documentation. For migration paths from these removed backends, refer to Upgrading to Terraform v1.3.
https://developer.hashicorp.com/terraform/language/settings/backends/configuration#available-backends
upvoted 13 times

ray320x 12 months ago

With the removal of support for Artifactory in Terraform v1.3 and Git never being a supported backend, both options B and C would not work. However, Git has never been a supported backend for Terraform state files, making it the primary correct answer.

The correct answer is:
C. Git
upvoted 8 times

Tyler2023 1 year, 6 months ago

Thanks for this, as I question why artifactory
upvoted 2 times

nez15 Highly Voted 3 years ago

Selected Answer: C

https://www.terraform.io/cdktf/concepts/remote-backends
upvoted 8 times

KG8 Most Recent 2 months, 3 weeks ago

Selected Answer: C

It is explicit that we do not store the state files in the Version Control Systems, so Git will be the answer.
upvoted 1 times

Aakashj 4 months, 2 weeks ago

Selected Answer: C

Git is where we do not store state files
upvoted 2 times

muammer32 5 months, 2 weeks ago

Selected Answer: C

Git is the only option that does not work as a remote backend for storing Terraform state files. The other options (Amazon S3, Artifactory, and Terraform Cloud) are valid backends for remote state storage.
upvoted 1 times

Algol 8 months, 2 weeks ago

Selected Answer: C

The backend that would not work for storing Terraform state files is C. Git. Git is not designed to handle the dynamic state file that Terraform requires for tracking infrastructure. Terraform needs backends that support locking and consistent state management, such as A. Amazon S3, B. Artifactory, or D. Terraform Cloud, which are suitable for remote state storage and ensure safe concurrent access to the state file.

upvoted 1 times

ray320x 12 months ago

With the removal of support for Artifactory in Terraform v1.3 and Git never being a supported backend, both options B and C would not work. However, Git has never been a supported backend for Terraform state files, making it the primary correct answer.

The correct answer is:
C. Git

upvoted 2 times

Absence379 1 year, 2 months ago

B
Note: We removed the artifactory, etcd, etcdv3, manta, and swift backends in Terraform v1.3. Information about their behavior in older versions is still available in the Terraform v1.2 documentation. For migration paths from these removed backends, refer to Upgrading to Terraform v1.3.

upvoted 1 times

KLIERKO 1 year, 8 months ago

Selected Answer: C

The anwer is C stop kliering

upvoted 1 times

gofavad926 1 year, 8 months ago

Selected Answer: C

C. git is not supported

upvoted 1 times

Bere 1 year, 9 months ago

Selected Answer: C

Supported backends:
https://developer.hashicorp.com/terraform/cdktf/concepts/remote-backends#supported-backends

Git is not a supported backend for Terraform state. While it's possible to store state files in Git, doing so is not recommended because state files can contain sensitive information, and it's not designed for concurrent state management.

upvoted 3 times

Tlakmini 1 year, 9 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/language/settings/backends/configuration#available-backends

upvoted 2 times

sumnear 1 year, 9 months ago

Selected Answer: C

C is the answer

upvoted 1 times

wheelan 1 year, 10 months ago

Selected Answer: B

Answer is B. I am using Git as a backend.

Artifactory is no longer a valid backend.
https://developer.hashicorp.com/terraform/language/settings/backends/configuration#available-backends

upvoted 2 times

kingfighers 1 year, 1 month ago

git is not official support now, check the doc: https://developer.hashicorp.com/terraform/language/settings/backends/local

upvoted 1 times

cedry87 2 years, 1 month ago

answer : B

upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: C

C. Git.

Git is not a suitable remote backend for Terraform. While it is possible to store Terraform state files in Git, it is not recommended because Git is not designed to manage state files and does not provide the necessary locking mechanisms to prevent concurrent access.

upvoted 2 times

ArnaldoW 2 years, 1 month ago

If Everyone knows, that the answer is C why didn't the admin fix it yet?

upvoted 7 times

kapara 1 year, 9 months ago

I believe the administrator is aware of the solution. However, if they merely provide us with answers without encouraging exploration, individuals might resort to memorizing the responses. This could potentially lead to a situation where the company detects and takes action against the website for distributing exam dumps. It's essential for learning and growth that we understand concepts thoroughly through exploration rather than relying solely on pre-supplied answers.

upvoted 12 times

Question #78 *Topic 1*

Which backend does the Terraform CLI use by default?

A. Terraform Cloud

B. Consul

C. Remote

D. Local

Suggested Answer: *D*

Reference:
https://www.terraform.io/docs/language/settings/backends/configuration.html



Community vote distribution

D (100%)

Eltooth Highly Voted 1 year, 11 months ago

Selected Answer: D

D is correct answer : local

"By default, Terraform implicitly uses a backend called local to store state as a local file on disk. Every other backend stores state in a remote service of some kind, which allows multiple people to access it. Accessing state in a remote service generally requires some kind of access credentials, since state data contains extremely sensitive information."

https://www.terraform.io/language/settings/backends

upvoted 7 times

Pavan6516 Most Recent 5 months ago

Selected Answer: D

The answer is definitely D - Local and its filename is terraform.tfstate

upvoted 1 times

shehreen 10 months, 1 week ago

Selected Answer: D

Answer is : D

upvoted 1 times

Power123 1 year, 1 month ago

Answer is D- local

upvoted 1 times

SilentMilli 1 year, 2 months ago

Selected Answer: D

The default backend used by the Terraform CLI is the local backend. This means that the state file is stored on the local file system in a file named "terraform.tfstate".

upvoted 3 times

vadeemkaa 1 year, 5 months ago

Answer D. By default in Local backend

upvoted 1 times

Ahmad_Terraform 1 year, 10 months ago

D. Local

upvoted 3 times

Zam88 1 year, 11 months ago

By default, Terraform uses a backend called local, which stores state as a local file on disk. You can also configure one of the built-in backends listed in the documentation sidebar.

D correct

upvoted 4 times

Question #79 *Topic 1*

When you initialize Terraform, where does it cache modules from the public Terraform Module Registry?

A. On disk in the /tmp directory

B. In memory

C. On disk in the .terraform sub-directory

D. They are not cached

Suggested Answer: *C*
Reference:
https://www.terraform.io/docs/language/modules/sources.html

Community vote distribution
C (100%)

Eltooth Highly Voted 2 years, 5 months ago
Selected Answer: C
C is correct answer : hidden terraform directory

"A hidden .terraform directory, which Terraform uses to manage cached provider plugins and modules, record which workspace is currently active, and record the last known backend configuration in case it needs to migrate state on the next run. This directory is automatically managed by Terraform, and is created during initialization."

https://www.terraform.io/cli/init
upvoted 12 times

amrith501 Highly Voted 2 years, 5 months ago
Selected Answer: C
on disk for sure for sure
upvoted 5 times

Eltooth 2 years, 4 months ago
On disk in the .terraform sub-directory - just to clarify.
upvoted 1 times

shefulacertificari Most Recent 10 months, 1 week ago
Answer - C. It's stored in the .terraform directory.
upvoted 1 times

Ni33 1 year, 6 months ago
Selected Answer: C
C is the correct option
upvoted 1 times

Power123 1 year, 8 months ago
Answer is C
upvoted 1 times

vadeemkaa 1 year, 11 months ago
Answer C. On disk in the .terraform sub-directory
upvoted 1 times

eduvar4 2 years, 2 months ago
Selected Answer: C
https://developer.hashicorp.com/terraform/language/modules/sources
upvoted 1 times

Ahmad_Terraform 2 years, 4 months ago
C. On disk in the .terraform sub-directory
upvoted 1 times

Question #80 *Topic 1*

You write a new Terraform configuration and immediately run terraform apply in the CLI using the local backend.
Why will the apply fail?

A. Terraform needs you to format your code according to best practices first

B. Terraform needs to install the necessary plugins first

C. The Terraform CLI needs you to log into Terraform cloud first

D. Terraform requires you to manually run terraform plan first

Suggested Answer: *C*

Community vote distribution

B (100%)

nez15 Highly Voted 2 years, 7 months ago

Selected Answer: B

Need to run Terraform Init first to install the plugins

upvoted 26 times

camps Highly Voted 1 year, 7 months ago

Selected Answer: B

B. Terraform needs to install the necessary plugins first.

When running terraform apply for a new Terraform configuration, Terraform needs to install the necessary provider plugins and any dependencies before it can create or modify any resources. Without the necessary plugins, Terraform cannot fully understand the resources that are defined in the configuration file.

upvoted 6 times

godie44 Most Recent 7 months, 1 week ago

Selected Answer: B

Has to run terraform init first

upvoted 1 times

shefulacertificari 10 months, 1 week ago

Answer - B. You need to run 'terraform init' command first to initialize the terraform plugins.

upvoted 3 times

LunarPhobia 1 year, 4 months ago

Selected Answer: B

B. You need to run init for required plugins

upvoted 3 times

Ni33 1 year, 6 months ago

Selected Answer: B

B for sure.

upvoted 2 times

Power123 1 year, 8 months ago

B is correct

upvoted 2 times

Rybitska 1 year, 8 months ago

Selected Answer: B

it's B

upvoted 2 times

vadeemkaa 1 year, 11 months ago

Answer B. terraform init

upvoted 2 times

adouban 1 year, 11 months ago

Selected Answer: B

B is the correct, terraform init to download required plugins

upvoted 1 times

karapet 2 years ago

First of all: terraform init

upvoted 2 times

kopper2019 2 years, 3 months ago

it is B

upvoted 1 times

Ahmad_Terraform 2 years, 4 months ago

init required

Terraform needs to install the necessary plugins first

upvoted 2 times

Ahmad_Terraform 2 years, 4 months ago

B plugins first

upvoted 2 times

Eltooth 2 years, 5 months ago

Selected Answer: B

B is correct answer.

You need to run terraform init before running apply.

upvoted 3 times

rfd 2 years, 5 months ago

Selected Answer: B

terraform init needs to be run first so it can download required modules and setup backend.

upvoted 1 times

bp339 2 years, 5 months ago

Selected Answer: B

B for sure

upvoted 2 times

Question #81 *Topic 1*

What features stops multiple admins from changing the Terraform state at the same time?

A. Version control

B. Backend types

C. Provider constraints

D. State locking

Suggested Answer: *D*

Reference:

https://blog.gruntwork.io/how-to-manage-terraform-state-28f5697e68fa

> 2. **Locking:** Most version control systems do not provide any form of locking that would prevent two team members from running `terraform apply` on the same state file at the same time.

Community vote distribution

D (100%)

Eltooth Highly Voted 1 year, 5 months ago

Selected Answer: D

D is correct answer.

Somewhat ambiguous question however the key phrase is "feature".
You need a remote backend first with a State Locking feature available to avoid this scenario.

https://blog.gruntwork.io/how-to-manage-terraform-state-28f5697e68fa

upvoted 6 times

Ni33 Most Recent 6 months, 2 weeks ago

Selected Answer: D

D is correct option.

upvoted 1 times

Power123 8 months ago

Answer is D

upvoted 1 times

vadeemkaa 11 months, 1 week ago

Answer D. state-lock feature

upvoted 1 times

Ahmad_Terraform 1 year, 4 months ago

D. State locking

upvoted 3 times

amrith501 1 year, 5 months ago

Selected Answer: D

State locking

upvoted 3 times

Question #82 *Topic 1*

A fellow developer on your team is asking for some help in refactoring their Terraform code. As part of their application's architecture, they are going to tear down an existing deployment managed by Terraform and deploy new. However, there is a server resource named aws_instance.ubuntu[1] they would like to keep to perform some additional analysis.
What command should be used to tell Terraform to no longer manage the resource?

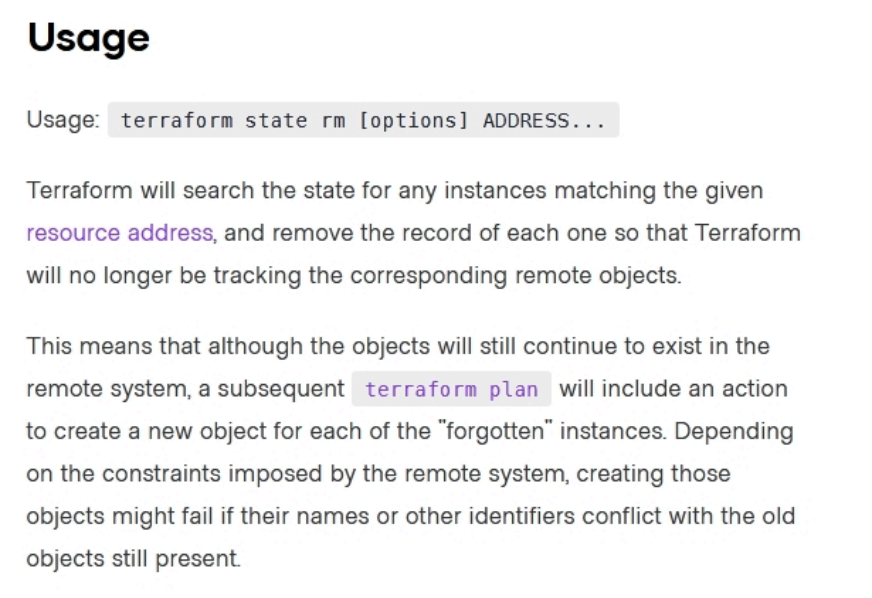
A. terraform apply rm aws_instance.ubuntu[1]

B. terraform state rm aws_instance.ubuntu[1]

C. terraform plan rm aws_instance.ubuntu[1]

D. terraform delete aws_instance.ubuntu[1]

Suggested Answer: *B*
Reference:
https://www.terraform.io/docs/cli/commands/state/rm.html



Community vote distribution
B (100%)

Eltooth Highly Voted 1 year, 11 months ago
Selected Answer: B
B is correct answer : terraform state rm

"You can use terraform state rm in the less common situation where you wish to remove a binding to an existing remote object without first destroying it, which will effectively make Terraform "forget" the object while it continues to exist in the remote system."

https://www.terraform.io/cli/commands/state/rm
upvoted 13 times

Tyler2023 Most Recent 6 months, 3 weeks ago
The answer is correct based on this https://developer.hashicorp.com/terraform/cli/commands/state/rm

You can use terraform state rm in the less common situation where you wish to remove a binding to an existing remote object without first destroying it, which will effectively make Terraform "forget" the object while it continues to exist in the remote system.
upvoted 2 times

Ni33 1 year ago
Selected Answer: B
BBBBBBBBBB
upvoted 1 times

camps 1 year, 1 month ago

Selected Answer: B

B. terraform state rm aws_instance.ubuntu[1]

To tell Terraform to no longer manage a specific resource, you can use the terraform state rm command. This command removes the specified resource from the Terraform state file, effectively telling Terraform that the resource no longer exists.

upvoted 3 times

Power123 1 year, 1 month ago

B is correct. terraform state rm

upvoted 1 times

eduvar4 1 year, 8 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/cli/commands/state/rm

upvoted 1 times

habbypat123 1 year, 11 months ago

The answer is B for sure

upvoted 2 times

Zam88 1 year, 11 months ago

state rm

The terraform state rm command is used to remove items from the Terraform state. This command can remove single resources, single instances of a resource, entire modules, and more

B

upvoted 3 times

Question #83 *Topic 1*

Terraform can only manage resource dependencies if you set them explicitly with the depends_on argument.

A. True

B. False

Suggested Answer: *A*
Reference:
https://learn.hashicorp.com/tutorials/terraform/dependencies?in=terraform/0-13

Community vote distribution
B (93%) 7%

vitasac Highly Voted 2 years ago
Selected Answer: B
I think it's B for sure
upvoted 12 times

gofavad926 8 months ago
"I think", and "for sure"... are not compatible :D
upvoted 24 times

Ni33 Highly Voted 1 year ago
B is the correct answer. Terraform do manage implicit dependencies using the attribute values mentioned in the configuration of the resources.
upvoted 7 times

gofavad926 Most Recent 8 months ago
Selected Answer: B
B, false
upvoted 2 times

dhez46 1 year, 1 month ago
Selected Answer: B
https://developer.hashicorp.com/terraform/tutorials/0-13/dependencies?in=terraform%2F0-13

Implicit dependencies are the primary way that Terraform understands the relationships between your resources. Sometimes there are dependencies between resources that are not visible to Terraform, however. The depends_on argument is accepted by any resource or module block and accepts a list of resources to create explicit dependencies for.
upvoted 4 times

camps 1 year, 1 month ago
Selected Answer: B
B. False.

Terraform can manage resource dependencies automatically without explicitly setting them using the depends_on argument.
upvoted 3 times

Power123 1 year, 1 month ago
B is correct. Implicit as well
upvoted 2 times

resnef 1 year, 4 months ago
Selected Answer: B
you have implicit as well, therefore B
upvoted 1 times

vikramvlr 1 year, 5 months ago
Answer is B.
Most of the time, Terraform infers dependencies between resources based on the configuration given, so that resources are created and destroyed in the correct order. Occasionally, however, Terraform cannot infer dependencies between different parts of your infrastructure, and you will need to

create an explicit dependency with the depends_on argument.
https://developer.hashicorp.com/terraform/tutorials/0-13/dependencies?in=terraform%2F0-13
upvoted 3 times

eduvar4 1 year, 8 months ago
Answer: B
"Most of the time, Terraform infers dependencies between resources based on the configuration given, so that resources are created and destroyed in the correct order"
-> https://learn.hashicorp.com/tutorials/terraform/dependencies
upvoted 2 times

j1809 1 year, 8 months ago
Terraform automatically infers when one resource depends on another by studying the resource attributes used in interpolation expressions.
upvoted 1 times

GopinathM 1 year, 8 months ago
A is correct we need pass explicitly depends_on
upvoted 1 times

kopper2019 1 year, 9 months ago
Selected Answer: B
It is B
upvoted 1 times

Ahmad_Terraform 1 year, 10 months ago
B. False
implicit = hidden dependencies Trfm can can manage.
upvoted 2 times

Ahmad_Terraform 1 year, 11 months ago
B is correct
implicit decencies trfm can manage without depends on command, for example in AWS eip resource will be dependent on the Ec2 creation, so trfm will first create the Ec2 and then eip will be allocated/created. and eip creation will be after the creation of Ec2.
https://learn.hashicorp.com/tutorials/terraform/dependencies?in=terraform/0-13
upvoted 1 times

Eltooth 1 year, 11 months ago
Selected Answer: A
I believe A is the correct answer.

"Use the depends_on meta-argument to handle hidden resource or module dependencies that Terraform cannot automatically infer. You only need to explicitly specify a dependency when a resource or module relies on another resource's behavior but does not access any of that resource's data in its arguments."

https://www.terraform.io/language/meta-arguments/depends_on
upvoted 2 times

RVivek 1 year, 8 months ago
You voted A and the comment is in support of B ?
upvoted 2 times

Zam88 1 year, 11 months ago
Manage explicit dependencies
Implicit dependencies are the primary way that Terraform understands the relationships between your resources. Sometimes there are dependencies between resources that are not visible to Terraform, however. The depends_on argument is accepted by any resource or module block and accepts a list of resources to create explicit dependencies for.

B
upvoted 1 times

Zam88 1 year, 11 months ago
I was wrong i have checked it again - Terraform cannot infer dependencies between different parts of your infrastructure, and you will need to create an explicit dependency with the depends_on argument.
A is correct answer

upvoted 2 times

rfd 1 year, 11 months ago

Selected Answer: B

Terraform is smart enough to manage resource dependencies to an extent, without any explicit instructions.

upvoted 3 times

Question #84 *Topic 1*

A terraform apply can not _____ infrastructure.

A. change

B. destroy

C. provision

D. import

Suggested Answer: *D*

Community vote distribution

D (100%)

alejandro_torres Highly Voted 1 year, 9 months ago

deprecated question https://www.hashicorp.com/blog/terraform-1-5-brings-config-driven-import-and-checks

upvoted 7 times

SureNot Most Recent 11 months, 2 weeks ago

Terraform v1.5.0 and later supports import blocks...

A bit old question...

upvoted 1 times

LunarPhobia 1 year, 10 months ago

Selected Answer: D

D as importing requires a separate command from Apply

upvoted 4 times

iDiddy 1 year, 11 months ago

import is an explicit command

upvoted 3 times

Clapton79 1 year, 6 months ago

Not any more, you can have an import {} clause so I guess this question is no longer valid.

https://www.hashicorp.com/blog/terraform-1-5-brings-config-driven-import-and-checks

upvoted 2 times

Power123 2 years, 1 month ago

Answer is D

upvoted 1 times

vadeemkaa 2 years, 5 months ago

Answer D. terraform import command

upvoted 1 times

root_i_am 2 years, 8 months ago

Selected Answer: D

The answer is D.

While terraform allows you import existing infrastructure, it is not through the apply command.

upvoted 3 times

dnscloud02 2 years, 8 months ago

Not sure --> Terraform can import existing infrastructure resources. This functionality allows you take resources you created by some other means and bring them under Terraform management.

upvoted 1 times

antivrillee 2 years, 7 months ago

The question is about the command "terraform apply" and not terraform

upvoted 1 times

antivrillee 2 years, 8 months ago

Selected Answer: D

https://www.educative.io/answers/what-is-the-command-to-destroy-infrastructure-in-terraform

upvoted 3 times

Question #85 *Topic 1*

You need to constrain the GitHub provider to version 2.1 or greater.
Which of the following should you put into the Terraform 0.12 configuration's provider block?

A. version >= 2.1

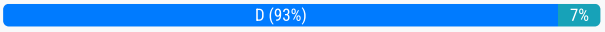
B. version ~> 2.1

C. version = ɹ2.1 =>€ɹ€

D. version = ɹ2.1 =<€ɹ€

Suggested Answer: *B*
Reference:
https://github.com/hashicorp/terraform-provider-null/issues/31

Community vote distribution



19kilo Highly Voted 2 years, 10 months ago

Selected Answer: D

I found cleaner version of the multiple choices.

A- version >= 2.1
B- version ~> 2.1
C- version = "<= 2.1"
D- version = ">= 2.1"

The answer is D ">= 2.1". Requires quotes I believe.

upvoted 66 times

Chrisler Highly Voted 1 year, 8 months ago

A- version >= 2.1

This option is not valid in Terraform's provider block. Terraform uses a specific version string format to specify provider versions, and this format is not compatible with the >= operator.

B- version ~> 2.1

This option uses a version constraint that allows Terraform to use any version that is compatible with version 2.1, but it does not enforce using version 2.1 or greater. The ~> operator specifies that it should use a version greater than or equal to 2.1 but less than the next major version.

C- version = "<= 2.1"

This option constrains the provider to versions less than or equal to 2.1, which is the opposite of what you want. It limits Terraform to use versions up to and including 2.1 but not greater.

D- version = ">= 2.1"

This option correctly specifies that Terraform should use the GitHub provider version 2.1 or any version greater than 2.1, which matches your requirement of constraining it to version 2.1 or greater.

upvoted 8 times

090200f Most Recent 10 months, 2 weeks ago

version = ɹ2.1 =<€ɹ€
why option D showing like this? not version = ">=2.1" ? any idea.
even in exam they will give like this lambda exp?

upvoted 2 times

smitttt 1 year, 1 month ago

in 0.12 terraform version
>=2.1, <3.0.0

in 0.13 or later terraform version
~> 2.1
upvoted 1 times

gofavad926 1 year, 8 months ago

Selected Answer: D

D- version = ">= 2.1"
upvoted 2 times

Hp45 1 year, 8 months ago

The votes seem incorrect. Correct answer should be the ~ sign. See the provider block on registry page:-
aws = {
source = "hashicorp/aws"
version = "~> 5.0"
upvoted 2 times

Bere 1 year, 9 months ago

Selected Answer: D

A- version >= 2.1
B- version ~> 2.1
C- version = "<= 2.1"
D- version = ">= 2.1"

Example:

provider "github" {
version = ">= 2.1"
Other configurations for the provider
}
upvoted 4 times

Mikhael1984 1 year, 9 months ago

For 0.12 version (1.1.x and earlier), exactly, refer https://developer.hashicorp.com/terraform/language/v1.1.x/expressions/version-constraints.

= (or no operator): Allows only one exact version number. Cannot be combined with other conditions.

!=: Excludes an exact version number.

>, >=, <, <=: Comparisons against a specified version, allowing versions for which the comparison is true. "Greater-than" requests newer versions, and "less-than" requests older versions.

~>: Allows only the rightmost version component to increment. For example, to allow new patch releases within a specific minor release, use the full version number: ~> 1.0.4 will allow installation of 1.0.5 and 1.0.10 but not 1.1.0. This is usually called the pessimistic constraint operator.
upvoted 2 times

Ni33 2 years ago

Selected Answer: D

D is the correct option
upvoted 1 times

kiran15789 2 years ago

Selected Answer: D

based on 19kilo cleaner version
upvoted 2 times

camps 2 years, 1 month ago

Selected Answer: D

. version = ">= 2.1"

In Terraform, the provider block is used to configure a provider plugin that Terraform uses to interact with a specific cloud or service. The version argument in the provider block is used to constrain the version of the provider plugin that Terraform should use.

upvoted 2 times

Power123 2 years, 1 month ago

D is correct .version = ">= 2.1"

upvoted 1 times

Nunyabiznes 2 years, 2 months ago

Selected Answer: D

To constrain the GitHub provider to version 2.1 or greater in a Terraform 0.12 configuration, you should use the following constraint in the provider block:

D. version = ">= 2.1"

This constraint indicates that the provider version must be greater than or equal to 2.1. Here's an example of how to configure this in the provider block:

```
provider "github" {
version = ">= 2.1"
...
}
```

This ensures that the GitHub provider used will be at least version 2.1, allowing for any version greater than or equal to 2.1.

upvoted 3 times

Ahmed_Elmelegy 2 years, 2 months ago

Selected Answer: D

= ">= "

upvoted 1 times

agmesas 2 years, 3 months ago

Selected Answer: D

D. version = ">= 2.1" (must be a literal string)

upvoted 2 times

InformationOverload 2 years, 4 months ago

Selected Answer: D

version = ">= 2.1"

upvoted 3 times

yaza 2 years, 7 months ago

A is false you should have version = ...

upvoted 2 times

Question #86 *Topic 1*

You just scaled your VM infrastructure and realized you set the count variable to the wrong value. You correct the value and save your change. What do you do next to make your infrastructure match your configuration?

A. Run an apply and confirm the planned changes

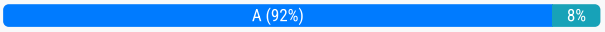
B. Inspect your Terraform state because you want to change it

C. Reinitialize because your configuration has changed

D. Inspect all Terraform outputs to make sure they are correct

Suggested Answer: *A*

Community vote distribution



vitasac Highly Voted 3 years ago

Selected Answer: A

I'm aggree it's A

upvoted 12 times

090200f Most Recent 10 months, 2 weeks ago

A: run an apply is correct ans

upvoted 1 times

Power123 2 years, 1 month ago

A is correct

upvoted 1 times

RVivek 2 years, 8 months ago

Selected Answer: A

A-- is required to deploy the the changed count

B- Not required

C-- checked in my Lab for the count change it worked with out init

D Not required

upvoted 4 times

Eltooth 2 years, 11 months ago

Selected Answer: A

A is correct answer : apply

You need your deployment to match your config, so only way to implement changes is through terraform apply.

upvoted 4 times

leonelferrari 2 years, 11 months ago

A is correct

upvoted 2 times

stalk98 2 years, 11 months ago

A is the answer

upvoted 1 times

wangchung 3 years ago

D? Its like throughout all these questions for this exam the answers are often not just wrong, but the obvious wrong one.

A

upvoted 1 times

Sunrayk 3 years ago

It should be A as the plan will show you what will change and then you can confirm the apply

upvoted 3 times

Sunrayk 3 years ago

Actually no, it should be C as the config has changed.
https://www.terraform.io/cli/commands/init

upvoted 1 times

vikramvlr 2 years, 5 months ago

initialize is used to refer the plug-ins

upvoted 1 times

Parthasarathi 3 years ago

Selected Answer: C

It should be C as the configuration file is changed

upvoted 2 times

Maaran07 3 years ago

Configuration change don't require init

upvoted 3 times

nez15 3 years ago

Selected Answer: A

Not sure why D. Should be A

upvoted 4 times

Question #87 *Topic 1*

Terraform provisioners that require authentication can use the _____ block.

A. connection

B. credentials

C. secrets

D. ssh

Suggested Answer: *B*

Community vote distribution

A (100%)

BaburTurk Highly Voted 1 year, 3 months ago

Selected Answer: A

The answer is A. connection.

The connection block is used to configure the authentication settings for a Terraform provisioner. This includes the username, password, and SSH key that will be used to connect to the remote resource.

The credentials and secrets blocks are used to store sensitive information, such as passwords and SSH keys. These blocks are not directly used by provisioners, but they can be referenced by the connection block.

The ssh block is used to configure the SSH client that Terraform will use to connect to the remote resource. This block is not typically used by provisioners, as the connection block can be used to configure the SSH client for all provisioners.

Here is an example of a connection block for a provisioner that requires SSH authentication:

```
connection {
host = "example.com"
user = "root"
password = "my-password"
}
```

upvoted 10 times

NashP Most Recent 9 months, 4 weeks ago

```
resource "aws_instance" "example" {
ami = "ami-0c55b159cbfafe1f0"
instance_type = "t2.micro"

connection {
type = "ssh"
user = "ec2-user"
private_key = file("~/.ssh/private_key.pem")
}

provisioner "remote-exec" {
inline = [
"echo 'Hello, World!' > hello.txt",
]
}
}
```

A connection

upvoted 3 times

Tlakmini 1 year, 3 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/cli/config/config-file

upvoted 1 times

Power123 1 year, 8 months ago

A is correct. Connection block

upvoted 2 times

tgaos 1 year, 8 months ago

Selected Answer: A

https://developer.hashicorp.com/terraform/language/resources/provisioners/connection

Most provisioners require access to the remote resource via SSH or WinRM and expect a nested connection block with details about how to connect.

upvoted 4 times

resnef 1 year, 10 months ago

Selected Answer: A

connection block: https://developer.hashicorp.com/terraform/language/resources/provisioners/connection#connection-block

upvoted 1 times

adouban 1 year, 11 months ago

Selected Answer: A

A is correct

upvoted 1 times

asudhin 2 years, 1 month ago

Selected Answer: A

It is A

upvoted 1 times

Burakko 2 years, 2 months ago

Selected Answer: A

Must be the connection block.

upvoted 1 times

aadi1121 2 years, 4 months ago

Selected Answer: A

https://www.terraform.io/language/resources/provisioners/connection

upvoted 3 times

Eltooth 2 years, 5 months ago

Selected Answer: A

A is correct answer : connection.

"Most provisioners require access to the remote resource via SSH or WinRM and expect a nested connection block with details about how to connect."

"Connection blocks don't take a block label and can be nested within either a resource or a provisioner."

https://www.terraform.io/language/resources/provisioners/connection

upvoted 3 times

Zam88 2 years, 5 months ago

Most provisioners require access to the remote resource via SSH or WinRM and expect a nested connection block with details about how to connect.

A

upvoted 2 times

Question #88 *Topic 1*

Terraform validate reports syntax check errors from which of the following scenarios?

A. Code contains tabs indentation instead of spaces

B. There is missing value for a variable

C. The state files does not match the current infrastructure

D. None of the above

Suggested Answer: *B*

Reference:
http://man.hubwiz.com/docset/Terraform.docset/Contents/Resources/Documents/docs/commands/validate.html

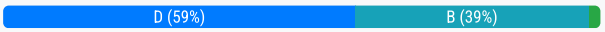
The `terraform validate` command is used to validate the syntax of the terraform files. Terraform performs a syntax check on all the terraform files in the directory, and will display an error if any of the files doesn't validate.

This command **does not** check formatting (e.g. tabs vs spaces, newlines, comments etc.).

The following can be reported:

- invalid HCL syntax (e.g. missing trailing quote or equal sign)
- invalid HCL references (e.g. variable name or attribute which doesn't exist)
- same `provider` declared multiple times
- same `module` declared multiple times
- same `resource` declared multiple times
- invalid `module` name
- interpolation used in places where it's unsupported (e.g. `variable`, `depends_on`, `module.source`, `provider`)
- missing value for a variable (none of `-var foo=...` flag, `-var-file=foo.vars` flag, `TF_VAR_foo` environment variable, `terraform.tfvars`, or default value in the configuration)

Community vote distribution



Eltooth Highly Voted 2 years, 4 months ago

Selected Answer: B

B is correct answer.

The terraform validate command is used to validate the syntax of the terraform files. Terraform performs a syntax check on all the terraform files in the directory, and will display an error if any of the files doesn't validate.

This command does not check formatting (e.g. tabs vs spaces, newlines, comments etc.).

The following can be reported:
invalid HCL syntax (e.g. missing trailing quote or equal sign)
invalid HCL references (e.g. variable name or attribute which doesn't exist)
same provider declared multiple times
same module declared multiple times
same resource declared multiple times
invalid module name
interpolation used in places where it's unsupported (e.g. variable, depends_on, module.source, provider)
missing value for a variable (none of -var foo=... flag, -var-file=foo.vars flag, TF_VAR_foo environment variable, terraform.tfvars, or default value in the configuration)

https://www.typeerror.org/docs/terraform/commands/validate
https://learning-ocean.com/tutorials/terraform/terraform-validate
upvoted 29 times

Pietjeplukgeluk 1 year, 9 months ago
Just tested this in lab, and "missing value for a variable (none of -var foo=... flag, -var-file=foo.vars flag, TF_VAR_foo environment variable, terraform.tfvars, or default value in the configuration)" is untrue, it is required to declare the variable type (but a value is not required to pass the "terraform validate")e.g. validates if referred somewhere else :

variable "ami_type" {
type = string
}
upvoted 5 times

Tyler2023 1 year ago
Agree on this, D is the correct answer
Terraform Validate runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state.

To verify configuration in the context of a particular run (a particular target workspace, input variable values, etc), use the terraform plan command instead, which includes an implied validation check.

https://developer.hashicorp.com/terraform/cli/commands/validate
upvoted 3 times

deepeshukla 1 year, 5 months ago
Agree. D is the correct answer. I tested in Lab. Missing variable value will not be reported in terraform valid, because it can be passed while running code (hence not invalid code).
upvoted 5 times

joyboy23 1 year, 4 months ago
I tested the same too. VSCode is detecting it but not terraform validate.From exam point of view, I wonder which one is the correct answer
upvoted 1 times

aanataliya Highly Voted 1 year, 2 months ago
Selected Answer: D
Confusion between B and D.
B was correct in older version of terraform. it was correct until v0.12.9 but then it was removed in later version. see below documentation links.
As per latest documentation B is incorrect when we have option D available.
https://github.com/hashicorp/terraform/blob/v0.12.9/website/docs/commands/validate.html.markdown
https://github.com/hashicorp/terraform/blob/v0.12.10/website/docs/commands/validate.html.markdown
please vote for correct answer to help others.
upvoted 17 times

AntonyPeter7 Most Recent 8 months, 3 weeks ago
Selected Answer: D
D is the correct answer. Without value for variable, if we run terraform apply, it will ask for value via CLI.
upvoted 4 times

erhard 9 months ago
Selected Answer: D
https://developer.hashicorp.com/terraform/cli/commands/validate
... regardless of any provided variables or existing state
upvoted 1 times

Alandt 10 months, 1 week ago
Selected Answer: D
Newer version of Terraform does not require you to give up values.
upvoted 2 times

TigerInTheCloud 11 months, 1 week ago
Selected Answer: D

$ cat main.tf
variable "cmd" {}
resource "null_resource" "cmd" {
provisioner "local-exec" {
command = "${var.cmd}"
}
}
$ terraform validate
Success! The configuration is valid.
upvoted 2 times

Ramdi1 1 year ago

Selected Answer: B

I think it is B
upvoted 1 times

ghostGuiggs 1 year ago

Selected Answer: B

B is the answer
upvoted 1 times

MisterR0B0T 1 year ago

D -
The terraform validate command is used to validate the syntax of the terraform files. Terraform performs a syntax check on all the terraform files in the directory, and will display an error if any of the files doesn't validate.

This command does not check formatting (e.g. tabs vs spaces, newlines, comments etc.)
upvoted 1 times

MisterR0B0T 1 year ago

A
terraform validate is a command that validates the configuration files in a directory, referring only to the configuration and not accessing any remote services such as remote state, provider APIs, etc1. It runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state 1. Therefore, it reports syntax check errors if there are any syntax errors in the code 1.

Therefore, the correct answer is A. Code contains tabs indentation instead of spaces.
upvoted 1 times

MauroSoli 1 year, 1 month ago

Also input variable of a root module is a variable.
So if a variable of an used module haven't a default value terraform validate return error.
Answer is B
upvoted 1 times

Aman726 1 year, 1 month ago

Selected Answer: D

D is correct
upvoted 1 times

Alex1atd 1 year, 1 month ago

Selected Answer: D

You can have missing value on variable, so D is the answer
upvoted 2 times

adityanarayan 1 year, 1 month ago

i have checked practiacally with none variable value output say its valid so option D
upvoted 1 times

gofavad926 1 year, 1 month ago

Selected Answer: B

B. There is missing value for a variable

Terraform validate will check for syntax errors in your Terraform configuration, including the following:

Missing or invalid variable values
Invalid resource references
Invalid attribute values
Invalid data source references
Invalid module references
Terraform validate will not check for the following:

Code indentation
State file consistency
Other configuration errors, such as logical errors or errors in the desired state of your infrastructure
upvoted 1 times

Halimb 1 year, 2 months ago

Selected Answer: D

D. None of the above.

Ref: https://developer.hashicorp.com/terraform/cli/commands/validate
upvoted 1 times

Spandrop 1 year, 2 months ago

D is the correct answer

"Validate runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state. It is thus primarily useful for general verification of reusable modules, including correctness of attribute names and value types."

https://developer.hashicorp.com/terraform/cli/commands/validate
upvoted 2 times

Question #89 *Topic 1*

Which of the following is allowed as a Terraform variable name?

A. count

B. name

C. source

D. version

Suggested Answer: *B*

Reference:
https://www.terraform.io/docs/language/values/variables.html

> The label after the `variable` keyword is a name for the variable, which must be unique among all variables in the same module. This name is used to assign a value to the variable from outside and to reference the variable's value from within the module.
>
> The name of a variable can be any valid identifier *except* the following: `source` , `version` , `providers` , `count` , `for_each` , `lifecycle` , `depends_on` , `locals` .
>
> These names are reserved for meta-arguments in module configuration blocks, and cannot be declared as variable names.

Community vote distribution

B (100%)

Eltooth Highly Voted 1 year, 5 months ago

Selected Answer: B

B is correct answer : name

"The name of a variable can be any valid identifier except the following: source, version, providers, count, for_each, lifecycle, depends_on, locals."

https://www.terraform.io/language/values/variables

upvoted 13 times

Zam88 Highly Voted 1 year, 5 months ago

The name of a variable can be any valid identifier except the following: source , version , providers , count , for_each , lifecycle , depends_on , locals .
These names are reserved for meta-arguments in module configuration blocks, and cannot be declared as variable names.

B

upvoted 5 times

Ni33 Most Recent 6 months, 2 weeks ago

Selected Answer: B

B is correct one.

upvoted 1 times

Power123 8 months ago

B is correct. name

upvoted 1 times

kopper2019 1 year, 3 months ago

Selected Answer: B

name is OK

upvoted 4 times

Ahmad_Terraform 1 year, 4 months ago

name is right

upvoted 2 times

Question #90 *Topic 1*

What type of block is used to construct a collection of nested configuration blocks?

A. for_each

B. repeated

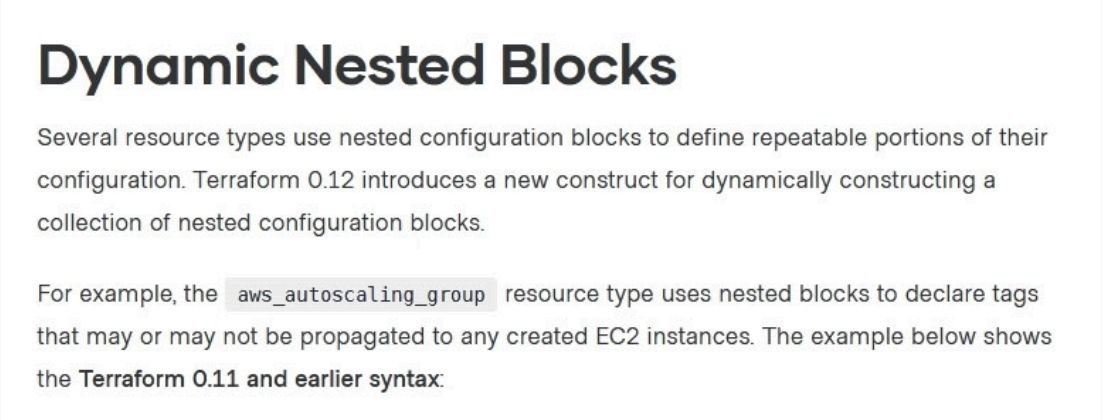
C. nesting

D. dynamic

Suggested Answer: *D*

Reference:

https://www.hashicorp.com/blog/hashicorp-terraform-0-12-preview-for-and-for-each



Community vote distribution

D (80%) A (20%)

bp339 Highly Voted 2 years, 5 months ago

Selected Answer: D

Dynamic is true...

upvoted 8 times

GiorgioGss Most Recent 2 weeks ago

Selected Answer: D

```
resource "aws_security_group" "example" {
name = "example"

dynamic "ingress" {
for_each = var.ingress_rules
content {
from_port = ingress.value.from_port
to_port = ingress.value.to_port
protocol = ingress.value.protocol
cidr_blocks = ingress.value.cidr_blocks
}
}
}
```

upvoted 1 times

biprodatta 1 year, 4 months ago

Selected Answer: A

dynamic doesn't create nested block, for_each does

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: A

A is the correct answer.

upvoted 1 times

Power123 1 year, 8 months ago

Dynamic block. D

upvoted 1 times

Abhijeet2904 1 year, 9 months ago

Selected Answer: A

A. for_each

In Terraform, you can use the "for_each" block to construct a collection of nested configuration blocks that are repeated based on an input list or map data structure. The "for_each" block allows you to define reusable code that can be applied to multiple resources, similar to a loop in a programming language. The block takes a data structure as an argument and creates one block of nested configuration for each element in the data structure, allowing you to manage multiple similar resources in a more streamlined way.

upvoted 2 times

Nunyabiznes 1 year, 8 months ago

The dynamic block in Terraform is used to construct nested configuration blocks dynamically based on a collection of values, such as a list or a map. This block type allows you to create multiple instances of the same resource or module, without having to duplicate the configuration code.

The for_each argument is used with the dynamic block to specify the collection of values to iterate over. The content block inside the dynamic block is used to define the configuration for each iteration.

upvoted 11 times

eduvar4 2 years, 2 months ago

Selected Answer: D

https://developer.hashicorp.com/terraform/language/expressions/dynamic-blocks

upvoted 4 times

alen995454 10 months, 2 weeks ago

form page shared above:

"A dynamic block acts much like a for expression, but produces nested blocks instead of a complex typed value. It iterates over a given complex value, and generates a nested block for each element of that complex value."

"The for_each argument provides the complex value to iterate over."

To me this says "D" is correct

upvoted 1 times

Eltooth 2 years, 5 months ago

Selected Answer: D

D is correct answer : dynamic

https://www.terraform.io/language/expressions/dynamic-blocks

upvoted 4 times

Question #91 *Topic 1*

Module variable assignments are inherited from the parent module and do not need to be explicitly set.

A. True

B. False

Suggested Answer: *B*
Reference:
https://github.com/hashicorp/terraform/issues/15818

Community vote distribution



stalk98 Highly Voted 1 year, 11 months ago

Modules do not inherit variables from the parent module. All modules are self-contained units. So you have to explicitly define variables in the child module, and then explicit set these variables in the parent module, when you instantiate the child module.

upvoted 20 times

Duleep 4 months, 2 weeks ago

variable assignments is
name = "ABC" // this is assignments and it will inherit and not need to define and assign again in child module
variable "name" {
type = string
default = "t2.micro" # This is an optional default value.
}

upvoted 1 times

Bere Most Recent 7 months, 1 week ago

Selected Answer: B

Module variable assignments must be explicitly set by the parent module when it instantiates the child module.

Variable Declaration in Child Module (modules/vm/main.tf):

```
variable "instance_type" {
type = string
default = "t2.micro" # This is an optional default value.
}

resource "aws_instance" "example" {
instance_type = var.instance_type
ami = "ami-abc123"
subnet_id = "subnet-1234abcd"
}
```

Variable Assignment in Parent Module (main.tf):

```
module "vm" {
source = "./modules/vm"

instance_type = "t3.micro" # Assigning a value to the variable.
}
```

In this example, instance_type is declared as a variable in the child module and is assigned a value by the parent module when it instantiates the vm module. The value "t3.micro" assigned by the parent module will override the default value "t2.micro" declared in the child module.

upvoted 4 times

modarov 9 months, 3 weeks ago

The answer is False.

Module variable assignments are not inherited from the parent module and do need to be explicitly set.

upvoted 2 times

cracit 11 months, 3 weeks ago

Modules do not inherit variables from the parent module. All modules are self-contained units. So you have to explicitly define variables in the child module, and then explicit set these variables in the parent module, when you instantiate the child module.

upvoted 2 times

DineshSG 1 year ago

I think we are confused between variable assignment and variable declaration. Variable need to be declared in child module but not necessarily be assigned. We can pass values from parent module

upvoted 4 times

Ni33 1 year ago

Selected Answer: A

I think A is the correct answer. Variable value assignments can be passed from root module variables to child module.

upvoted 2 times

Eltooth 1 year, 10 months ago

Selected Answer: B

B is correct answer : false.

upvoted 4 times

amrith501 1 year, 11 months ago

Selected Answer: B

Modules do not inherit variables from the parent module

upvoted 2 times

Question #92 *Topic 1*

If writing Terraform code that adheres to the Terraform style conventions, how would you properly indent each nesting level compared to the one above it?

A. With four spaces

B. With a tab

C. With three spaces

D. With two spaces

Suggested Answer: *D*

Reference:

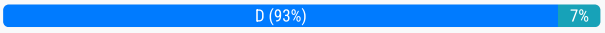
https://www.terraform.io/docs/language/syntax/style.html

The Terraform parser allows you some flexibility in how you lay out the elements in your configuration files, but the Terraform language also has some idiomatic style conventions which we recommend users always follow for consistency between files and modules written by different teams. Automatic source code formatting tools may apply these conventions automatically.

Note: You can enforce these conventions automatically by running `terraform fmt`.

- Indent two spaces for each nesting level.
- When multiple arguments with single-line values appear on consecutive lines at the same nesting level, align their equals signs:

Community vote distribution



Eltooth Highly Voted 2 years, 4 months ago

Selected Answer: D

D is correct answer: Indent two spaces for each nesting level.

https://www.terraform.io/language/syntax/style#style-conventions

upvoted 14 times

Power123 Most Recent 1 year, 7 months ago

2 spaces. Answer is D

upvoted 1 times

aone 1 year, 8 months ago

The Terraform parser allows you some flexibility in how you lay out the elements in your configuration files, but the Terraform language also has some idiomatic style conventions which we recommend users always follow for consistency between files and modules written by different teams. Automatic source code formatting tools may apply these conventions automatically.

Note: You can enforce these conventions automatically by running terraform fmt.

Indent two spaces for each nesting level.

When multiple arguments with single-line values appear on consecutive lines at the same nesting level, align their equals signs:

upvoted 1 times

zecch 1 year, 8 months ago

definitely D

upvoted 1 times

Abhijeet2904 1 year, 9 months ago

Selected Answer: A

A. With four spaces

When writing Terraform code that adheres to the Terraform style conventions, each nesting level should be indented four spaces compared to the level above it. This helps to visually distinguish between different levels of nesting and improves readability of the code.

In Terraform, a common use case for nested blocks is to define resources that depend on other resources. For example, you might have a resource that creates a network security group, and then another resource that creates a virtual machine that references the network security group. In this case, the virtual machine resource would be indented four spaces compared to the network security group resource.

Using four spaces for indentation is a widely accepted standard in the software development community, and it is recommended to use four spaces for indentation in Terraform code as well.

upvoted 1 times

alen995454 10 months, 2 weeks ago

"Compared to one above" .. not the parent

upvoted 1 times

David_C_90 1 year, 8 months ago

dont paste chat gpt answers

upvoted 5 times

vadeemkaa 1 year, 11 months ago

Answer D for sure

upvoted 1 times

Question #93 *Topic 1*

Which of the following is not an action performed by terraform init?

A. Create a sample main.tf file

B. Initialize a configured backend

C. Retrieve the source code for all referenced modules

D. Load required provider plugins

Suggested Answer: *A*

Reference:

https://www.terraform.io/docs/cli/init/index.html

Community vote distribution

A (100%)

Eltooth Highly Voted 1 year, 4 months ago

Selected Answer: A

Of options, only A is correct answer : Create a sample main.tf file

B. Initialize a configured backend : https://www.terraform.io/cli/commands/init#backend-initialization

C. Retrieve the source code for all referenced modules : https://www.terraform.io/cli/commands/init#child-module-installation

D. Load required provider plugins : https://www.terraform.io/cli/commands/init#plugin-installation

upvoted 9 times

keiffo2 Highly Voted 1 year, 2 months ago

init doen't create an main.tf file.

upvoted 5 times

Power123 Most Recent 7 months, 4 weeks ago

Correct answer is A

upvoted 1 times

Nzudin 8 months, 3 weeks ago

Selected Answer: A

A of course :')

upvoted 1 times

Question #94 *Topic 1*

HashiCorp Configuration Language (HCL) supports user-defined functions.

A. True

B. False

Suggested Answer: *B*

Reference:

https://www.packer.io/docs/templates/hcl_templates/functions

The HCL language includes a number of built-in functions that you can call from within expressions to transform and combine values. The general syntax for function calls is a function name followed by comma-separated arguments in parentheses:

```
max(5, 12, 9)
```

Copy

For more details on syntax, see *Function Calls* on the Expressions page.

The HCL language does not support user-defined functions, and so only the functions built in to the language are available for use. The navigation for this section includes a list of all of the available built-in functions.

Community vote distribution

B (100%)

kopper2019 Highly Voted 1 year, 3 months ago

Selected Answer: B

False

https://www.terraform.io/language/functions

The Terraform language does not support user-defined functions, and so only the functions built into the language are available for use

upvoted 8 times

Power123 Most Recent 7 months, 4 weeks ago

Answer is B

upvoted 2 times

aiya 1 year, 3 months ago

B - false

The HCL language does not support user-defined functions, and so only the functions built in to the language are available for use.

upvoted 4 times

Question #95 *Topic 1*

How can you trigger a run in a Terraform Cloud workspace that is connected to a Version Control System (VCS) repository?

A. Only Terraform Cloud organization owners can set workspace variables on VCS connected workspaces

B. Commit a change to the VCS working directory and branch that the Terraform Cloud workspace is connected to

C. Only members of a VCS organization can open a pull request against repositories that are connected to Terraform Cloud workspaces

D. Only Terraform Cloud organization owners can approve plans in VCS connected workspaces

Suggested Answer: *B*

Reference:
https://www.terraform.io/docs/cloud/vcs/index.html

Terraform Cloud is more powerful when you integrate it with your version control system (VCS) provider. Although you can use many of Terraform Cloud's features without one, a VCS connection provides additional features and improved workflows. In particular:

- When workspaces are linked to a VCS repository, Terraform Cloud can automatically initiate Terraform runs when changes are committed to the specified branch.
- Terraform Cloud makes code review easier by automatically predicting how pull requests will affect infrastructure.
- Publishing new versions of a private Terraform module is as easy as pushing a tag to the module's repository.

We recommend configuring VCS access when first setting up an organization, and you might need to add additional VCS providers later depending on how your organization grows.

Configuring a new VCS provider requires permission to manage VCS settings for the organization. (More about permissions.)

Community vote distribution

B (100%)

Eltooth Highly Voted 1 year, 10 months ago

Selected Answer: B

B is correct answer.

"In a workspace linked to a VCS repository, runs start automatically when you merge or commit changes to version control.

A workspace is linked to one branch of a VCS repository and ignores changes to other branches. You can specify which files and directories within your repository trigger runs. "

https://www.terraform.io/cloud-docs/run/ui#automatically-starting-runs

upvoted 15 times

modarov Most Recent 9 months, 3 weeks ago

The answer is B. Commit a change to the VCS working directory and branch that the Terraform Cloud workspace is connected to.

When you commit a change to a VCS repository that is connected to a Terraform Cloud workspace, Terraform Cloud will automatically trigger a run in the workspace. The run will use the latest state file in the workspace to determine what changes need to be made to the infrastructure.

upvoted 1 times

Ni33 1 year ago

Selected Answer: B

B is the correct answer

upvoted 1 times

Power123 1 year, 1 month ago

Answer is B

upvoted 1 times

samkobadev 1 year, 3 months ago

Commit = Push to VCS. So, the correct answer is B.

When we commit to VCS Repository thats automatically trigger run on the terraform cloud workspace

upvoted 1 times

Ahmad_Terraform 1 year, 10 months ago

commit mean saving code into the VSC, it will trigger run in the terraform

upvoted 1 times

Question #96 *Topic 1*

Terraform and Terraform providers must use the same major version number in a single configuration.

A. True

B. False

Suggested Answer: *B*

Reference:

https://www.terraform.io/docs/language/expressions/version-constraints.html

Community vote distribution

B (100%)

Eltooth Highly Voted 1 year, 10 months ago

Selected Answer: B

I think B is correct answer : false.

https://www.terraform.io/language/expressions/version-constraints#terraform-core-and-provider-versions

upvoted 11 times

chessmaster5000 Most Recent 2 months, 2 weeks ago

Selected Answer: B

You can use an older version in your environment although there is a newer version available from the provider.

upvoted 1 times

Busi57 8 months, 1 week ago

Selected Answer: B

I think b

upvoted 1 times

Power123 1 year, 1 month ago

Answer is B. False

upvoted 1 times

zecch 1 year, 2 months ago

Definitely B. terraform version and providers version can be different.

upvoted 3 times

nakikoo 1 year, 4 months ago

Version can be set to what is working with your environment resource

upvoted 1 times

Zam88 1 year, 11 months ago

B is correct

upvoted 3 times

Question #97 *Topic 1*

Which statement describes a goal of infrastructure as code?

A. An abstraction from vendor specific APIs

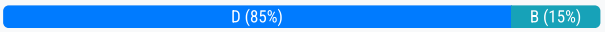
B. Write once, run anywhere

C. A pipeline process to test and deliver software

D. The programmatic configuration of resources

Suggested Answer: *D*

Community vote distribution



amrith501 Highly Voted 1 year, 11 months ago

Selected Answer: D

Looks like D

upvoted 10 times

Eltooth Highly Voted 1 year, 11 months ago

Selected Answer: D

D is correct answer.

upvoted 6 times

modarov Most Recent 9 months, 3 weeks ago

D. The programmatic configuration of resources.

upvoted 2 times

Power123 1 year, 1 month ago

D is correct

upvoted 1 times

agmesas 1 year, 3 months ago

Selected Answer: D

A, IaC is not for "specific APIs". B (WORE) an example is JVM or container, no matter where you run your code because there is an abstraction layer like docker, java virtual machine, etc.. where the code run fine. For Terraform you need to write a specific terraform configuration file based on which cloud you will deploy the infra (we can not deploy a tf file write for AWS in Azure changing only the provider, right?. C, IaC is a part of the pipeline but not the entire pipeline. The correct answer is D, "The programmatic configuration of resources" in a human-readable code.

upvoted 6 times

mifune 1 year, 11 months ago

I think that is B because one of the main features of Terraform is being a "Cloud agnostic" solution which best matches with: "Write once, run anywhere".

upvoted 1 times

mifune 1 year, 11 months ago

Sorry, I misunderstood the question... it's D, true.

upvoted 4 times

Zam88 1 year, 11 months ago

The purpose of infrastructure as code is to enable developers or operations teams to automatically manage, monitor and provision resources, rather than manually configure discrete hardware devices and operating systems. Infrastructure as code is sometimes referred to as programmable or software-defined infrastructure.

D

upvoted 3 times

stalk98 1 year, 11 months ago

maybe is B?

upvoted 2 times

Roro_Brother 2 years ago

Selected Answer: B

Sure, it's B

upvoted 4 times

bigboi23 2 years ago

I think it is B.

The purpose of infrastructure as code is to enable developers or operations teams to automatically manage, monitor and provision resources, rather than manually configure discrete hardware devices and operating systems.

upvoted 2 times

tbhtp 1 year, 9 months ago

So you're argument is supporting Answer D

B states "run anywhere". Like any cloud provider. That is just not the case. Still, migrating to another cloud provider may be less of an headache with IAC.

upvoted 1 times

Question #98 *Topic 1*

When using Terraform to deploy resources into Azure, which scenarios are true regarding state files? (Choose two.)

A. When a change is made to the resources via the Azure Cloud Console, the changes are recorded in a new state file

B. When a change is made to the resources via the Azure Cloud Console, Terraform will update the state file to reflect them during the next plan or apply

C. When a change is made to the resources via the Azure Cloud Console, the current state file will not be updated

D. When a change is made to the resources via the Azure Cloud Console, the changes are recorded in the current state file

Suggested Answer: *AC*

Community vote distribution



AzureGurl Highly Voted 2 years, 11 months ago

IT should be B and C, however the wording is incorrect, Terraform Refresh will only update the state file for new changes via Az Console

Terraform Plan does not change the State file, It will however say the diff in the state.

The Terraform apply will make the changes to the state file to keep it consistent.

So the wording of Option B is bit ambiguous.

upvoted 26 times

alirasouli 2 years, 5 months ago

Option B's wording is too fuzzy.

upvoted 1 times

msingh20 1 year, 9 months ago

Terraform plan / apply runs a terraform refresh to update the state with infrastructure so B makes sense.

upvoted 2 times

Tyler2023 1 year, 6 months ago

I don't think so, terraform will not generate the configuration changes you've maid manually, terraform apply will remove the configuration you've made that is not included in your tf files

upvoted 1 times

Nikita74 Highly Voted 2 years, 7 months ago

Selected Answer: C

Only C

upvoted 12 times

chessmaster5000 Most Recent 2 months, 2 weeks ago

Selected Answer: C

that state file needs to be updated as per the changes made in the console or terraform refresh to update

upvoted 1 times

YesPlease 3 months, 3 weeks ago

Selected Answer: B

Answer: B and C

if you make a change outside of Terraform, then the state file will not get updated until you run apply/plan in terraform to pull the latest infrastructure info and then save it to the state file.

upvoted 1 times

090200f 10 months, 2 weeks ago

B, C are correct answers

upvoted 1 times

vibzr2023 1 year, 1 month ago

I believe only option C is correct. I tested this by spinning up an EC2 instance in AWS using Terraform and then manually updating the "Name" tag in the AWS console. Immediately after that, when I ran terraform plan, it indicated that the tag would be changed from the Terraform-assigned value ("TerraformCI") to the manually applied value ("Rajesh"). This demonstrates that Terraform detects changes made outside of its management and plans to update the infrastructure accordingly.

upvoted 2 times

vibzr2023 1 year, 1 month ago

terraform plan output

```
# aws_instance.ec2_example will be updated in-place
 ~ resource "aws_instance" "ec2_example" {
    id                = "i-037feb3cac6a58f05"
  ~ tags              = {
      ~ "Name" = "TerraformCI" -> "Rajesh"
    }
  ~ tags_all          = {
      ~ "Name" = "TerraformCI" -> "Rajesh"
    }
```

terraform apply output

update tag from TerraformCI to Rajesh

upvoted 2 times

vibzr2023 1 year, 1 month ago

While prevailing sentiment leans towards options B and C, as evidenced by the popularity of ChatGPT's response, I posit that only option C is accurate. Definitive proof of option B's validity hinges on the demonstration of program output reflecting a successful state drift reconciliation. Here is the chatgpt answer when i demonstrated only C is correct.

Thank you for providing detailed information about your test and findings. Based on your description, it seems there was a misunderstanding regarding how Terraform interacts with external changes.

continued.....

upvoted 1 times

vibzr2023 1 year, 1 month ago

When you manually changed the "Name" tag in the AWS console and then ran terraform plan, Terraform compared the actual infrastructure state (with the "Name" tag set to "Rajesh") against what's defined in your Terraform configuration (where the tag might be set to "TerraformCI"). Seeing the discrepancy, Terraform planned to update the resource to match the configuration—changing the tag back to "TerraformCI". This behavior is expected and demonstrates Terraform's goal to ensure the infrastructure matches the configuration specified in your .tf files.

continued....

upvoted 1 times

Absence379 1 year, 2 months ago

Selected Answer: BC

B and C

upvoted 2 times

mattuyghur 1 year, 3 months ago

Selected Answer: BC

should be B and C

upvoted 1 times

Tyler2023 1 year, 6 months ago

I don't think B is an answer here

When you change any configuration manually outside your terraform configuration

The next time your run terraform apply, terraform will revert your manual changes

and follow what is defined in your configuration files

So answer i C

upvoted 1 times

brax404 1 year, 7 months ago

Selected Answer: BC

B. This is true, but with an important distinction. Terraform doesn't automatically detect changes made outside of it. If you modify resources directly in Azure (outside of Terraform), during the next terraform plan or terraform apply, there might be a discrepancy between the state file and the actual infrastructure. Terraform will then propose to reconcile this difference, which could involve changing or recreating the affected resources.

C. This is true. The Terraform state file isn't automatically updated when changes are made outside of Terraform, such as directly in the Azure Cloud Console.

upvoted 2 times

Pikopo 1 year, 7 months ago

the correct answer are B and C

upvoted 1 times

gofavad926 1 year, 8 months ago

Selected Answer: BC

BC

When a change is made to the resources via the Azure Cloud Console, the current state file will not be updated.

Terraform will update the state file to reflect the changes made to the resources via the Azure Cloud Console during the next plan or apply.

upvoted 1 times

debabrata6983 1 year, 9 months ago

Selected Answer: BC

The correct answer is B&C

upvoted 1 times

Rajmane 1 year, 9 months ago

Selected Answer: BC

The right answer is B & C

upvoted 1 times

modarov 1 year, 9 months ago

B. When a change is made to the resources via the Azure Cloud Console, Terraform will update the state file to reflect them during the next plan or apply

C. When a change is made to the resources via the Azure Cloud Console, the current state file will not be updated

upvoted 1 times

March2023 1 year, 11 months ago

Selected Answer: BC

b and c

upvoted 1 times

milan92stankovic 1 year, 11 months ago

Selected Answer: BC

B & C are the correct answer.

upvoted 1 times

Question #99 *Topic 1*

You need to deploy resources into two different cloud regions in the same Terraform configuration. To do that, you declare multiple provider configurations as follows:

```
provider “aws” {
  region = “us-east-1”
}

provider “aws” {
  alias = “west”
  region = “us-west-2”
}
```

What meta-argument do you need to configure in a resource block to deploy the resource to the `us-west-2` AWS region?

A. alias = west

B. provider = west

C. provider = aws.west

D. alias = aws.west

Suggested Answer: *C*

Reference:

https://github.com/hashicorp/terraform/issues/451

Community vote distribution



bp339 Highly Voted 1 year, 11 months ago

Selected Answer: C

https://www.terraform.io/language/providers/configuration

upvoted 14 times

Bere Most Recent 6 months, 3 weeks ago

Selected Answer: C

For us-east-1:

resource "aws_instance" "example_east" {

ami = "ami-abc12345"

instance_type = "t2.micro"

// No provider meta-argument specified, so it uses the default (us-east-1)

}

For us-west-2:

resource "aws_instance" "example_west" {

ami = "ami-xyz67890"

instance_type = "t2.micro"

provider = aws.west // This directs Terraform to use the 'us-west-2' region provider configuration

}

With this setup, Terraform will deploy the example_east instance in the us-east-1 region and the example_west instance in the us-west-2 region.

upvoted 2 times

Ni33 1 year ago

Selected Answer: C

CCCCCCCCCC

upvoted 2 times

Power123 1 year, 1 month ago

Answer is C

upvoted 1 times

thor7 1 year, 1 month ago

Selected Answer: C

Definitely, C is the correct answer.
https://www.terraform.io/language/providers/configuration

upvoted 2 times

Nzudin 1 year, 2 months ago

https://developer.hashicorp.com/terraform/language/providers/configuration#selecting-alternate-provider-configurations

upvoted 1 times

Eltooth 1 year, 11 months ago

Selected Answer: C

C is correct answer.

upvoted 1 times

Zam88 1 year, 11 months ago

alias is not meta-argument
c is correct

upvoted 2 times

elvancedonzy 1 year, 11 months ago

Selected Answer: C

C is Correct

upvoted 2 times

concepts015 2 years ago

I think the right answer is D alias is used in the argument and AWS.WEST is the right way to write the argument

upvoted 3 times

CHRIS12722222 1 year, 11 months ago

No, C is correct

upvoted 2 times

Roro_Brother 2 years ago

Selected Answer: C

C is correct

upvoted 3 times

bigboi23 2 years ago

Selected Answer: C

Confirmed. C is correct.

upvoted 2 times

Orpheus123 2 years ago

C - is the correct answer

upvoted 4 times

traceme 2 years ago

answer is C

upvoted 1 times

Question #100 *Topic 1*

You have declared an input variable called environment in your parent module. What must you do to pass the value to a child module in the configuration?

A. Add node_count = var.node_count

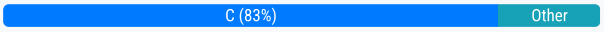
B. Declare the variable in a terraform.tfvars file

C. Declare a node_count input variable for child module

D. Nothing, child modules inherit variables of parent module

Suggested Answer: *C*

Community vote distribution



tbhtp Highly Voted 2 years, 9 months ago

Selected Answer: C

C is correct as the question is about passing a variable. So if you want the parent to pass its output to its child you need to configure an input variable for that.
The name of the input variable makes no sense but maybe that is to emphasize on the fact that the name of the input variable is arbitrary.

"That module may call other modules and connect them together by passing output values from one to input values of another."
https://www.terraform.io/language/modules/develop
upvoted 22 times

Tyler2023 Highly Voted 1 year, 6 months ago

All given options are wrong, they don't make any sense
You have declared a variable called "environment"
A. where did you get the var.node_count, is it related to the question?
B. You can do that, but it is not what you need for the question
C. Then what? node_count variable should accept number, not a string. Assuming "environment" variable will hold string value like Dev, UAT, Prod
D. Child module will not automatically inherit variables from parent module
They don't make any sense
upvoted 16 times

d759fb3 Most Recent 4 months ago

Selected Answer: A

C, Just declaring isn't enough, need to pass the value
upvoted 1 times

7b5b962 11 months, 1 week ago

module "child_module" {
source = "./path_to_child_module"
environment = var.environment
}
Therefore, the correct answer is:
A. Add node_count = var.node_count
upvoted 2 times

TigerInTheCloud 1 year, 5 months ago

Selected Answer: A

You need to pass the value.
(The child module should have the variable, that the child module developer's work)
upvoted 1 times

arunrkaushik 1 year, 9 months ago

Mr. Nunyabusines, may you like to rewrite the answer as:
module "child_module" {
source = "./parent_module"

environment = var.environment
}
upvoted 5 times

modarov 1 year, 9 months ago
C. Declare a node_count input variable for child module.

When you declare an input variable in a parent module, the child module will not automatically inherit the value of the variable. You must explicitly declare the variable in the child module.
upvoted 3 times

milan92stankovic 1 year, 11 months ago
Selected Answer: C
I vote for C
upvoted 1 times

Nunyabiznes 2 years, 1 month ago
Selected Answer: C
module "child_module" {
source = "./child_module"

environment = var.environment
}
upvoted 4 times

camps 2 years, 1 month ago
Selected Answer: C
C. Declare a node_count input variable for the child module.

In Terraform, input variables are used to parameterize Terraform configurations. When using child modules, it is necessary to pass values for the input variables defined in the child module. The values for the input variables can be passed using different methods such as using default values, command-line flags, environment variables, variable files, and so on.
upvoted 1 times

thor7 2 years, 1 month ago
Selected Answer: C
https://www.terraform.io/language/modules/develop
C is a correct answer
upvoted 1 times

pyro7 2 years, 3 months ago
C. Declare a node_count input variable for child module.

When passing variables from a parent module to a child module in Terraform, you need to explicitly declare the variables in both the parent and the child modules. In this case, you would need to declare an input variable for the child module that corresponds to the input variable declared in the parent module. So, the correct option is C.

Option A is incorrect because it is using an example variable that is not related to the input variable mentioned in the question. Option B is incorrect because it refers to how to set the value of the variable, but not how to pass the value from the parent to the child module. Option D is also incorrect because child modules do not automatically inherit the variables of their parent modules.
upvoted 3 times

gekkehenk 2 years, 4 months ago
Selected Answer: C
C, declaration of variables is not done in terraform.tfvars.
upvoted 2 times

DerekKey 2 years, 5 months ago
C - correct
for every variable used in terraform module (root/child) you must first declare it.
Without declaration in child module the variable will not be recognized.
upvoted 2 times

robertninho 2 years, 4 months ago

```
# Parent module
module "child" {
source = "./child"
node_count = var.environment
}

# Child module
variable "node_count" {
type = number
}
```

upvoted 7 times

yogishrb2020 2 years, 7 months ago

Selected Answer: C

Declare the variable in a terraform.tfvars file --> doesnot apply to child module
-->https://www.terraform.io/language/values/variables

upvoted 2 times

nani1709 2 years, 7 months ago

A is the correct answer.

upvoted 1 times

Linus11 2 years, 7 months ago

Searched in google " terraform node_count"
no result.
What is this node_count do?

upvoted 2 times

Question #101 *Topic 1*

If a module declares a variable with a default, that variable must also be defined within the module.

A. True

B. False

Suggested Answer: *A*

Community vote distribution

B (100%)

govs Highly Voted 1 year, 1 month ago

Selected Answer: B

If a module declares a variable with a default value, it is not necessary to define the variable within the module calling the module. The module will automatically use the default value if a value is not explicitly assigned to the variable in the calling module

upvoted 11 times

Eltooth Highly Voted 1 year, 11 months ago

Selected Answer: B

B is correct answer.

upvoted 7 times

Clapton79 Most Recent 6 months, 2 weeks ago

This question does not make sense in its current form. (Very poor English, sorry)

You declare the variable with variable "my_var" {type=string} and in there you may add a default: variable "my_var" {
type=string
default="a"
}

SETTING a variable in a statement calling a module that contains this variable is not compulsory.
module "my_module" {
source= "./this_module"
my_var = "b" # <-- you can omit this in which case the value will be "a"
}

upvoted 5 times

umavaja 5 months, 2 weeks ago

Agree, very poor English, same issue even for UDEMY courses as well Terraform documentation

upvoted 3 times

otakuinside 8 months ago

The variable must be declared aswell, another thing is that you have to assign a value to it. But the declaration is mandatory.
At least this is my understanding of what the question says.

upvoted 1 times

Alex1778 1 year, 1 month ago

B is correct answer.

upvoted 1 times

Power123 1 year, 1 month ago

Bis correct

upvoted 1 times

Nunyabiznes 1 year, 2 months ago

Selected Answer: B

False.

If a module declares a variable with a default value, it does not necessarily need to be defined within the module. The default value will be used if the variable is not set in the calling module.

For example, consider the following module:
```
variable "instance_type" {
description = "The instance type to use"
default = "t2.micro"
}

resource "aws_instance" "example" {
ami = "ami-0c55b159cbfafe1f0"
instance_type = var.instance_type
}
```
upvoted 5 times

Nunyabiznes 1 year, 2 months ago
In this module, a variable called instance_type is declared with a default value of t2.micro. The default value will be used if the variable is not set in the calling module.

If the calling module does not set the instance_type variable, the default value of t2.micro will be used. If the calling module does set the instance_type variable, the value from the calling module will be used instead of the default.

It is important to note that if a variable is declared without a default value and is not set in the calling module, Terraform will prompt the user to enter a value for that variable.
upvoted 6 times

SilentMilli 1 year, 2 months ago
Selected Answer: B
A module can declare a variable with a default value without defining it within the module. This is because the variable can be used as-is with the default value, or it can be assigned a new value by code that imports the module.
upvoted 2 times

bukake 1 year, 9 months ago
Where are 102-103-104 Questions?
upvoted 4 times

kopper2019 1 year, 9 months ago
no idea, I am taking exam tomorrow
upvoted 2 times

hello2022 1 year, 8 months ago
How did you do? How relevant were these questions.
upvoted 1 times

gsx 1 year, 11 months ago
Selected Answer: B
when no value is passed, default value will be automatically be taken.
upvoted 4 times

Maaran07 2 years ago
It is not mandatory to define inside module. default can be in module itself.
upvoted 1 times

traceme 2 years ago
Why not A . When we define the variable as default , we need to pass tha value for the variables right
upvoted 2 times

habros 1 year, 11 months ago
It already had a variable. Worse case it knows where to get if it's not manually defined
upvoted 1 times

Question #102 *Topic 1*

Which option cannot be used to keep secrets out of Terraform configuration files?

A. Environment Variables

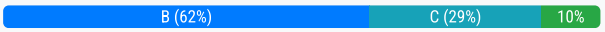
B. Mark the variable as sensitive

C. A Terraform provider

D. A -var flag

Suggested Answer: *D*

Community vote distribution



rotimislaw Highly Voted 2 years, 3 months ago

Selected Answer: B

It's B

The sensitive flag only prevents secret from showing up in the CLI outputs, but the value itself is still present in the configuration files.

upvoted 19 times

Chrisler 1 year, 8 months ago

This is true, you can still use the answer D. A terraform provider (e.g. Vault) to keep the secrets out of the terraform configuration.

upvoted 2 times

MauroSoli 1 year, 7 months ago

The question is "Which option CANNOT be used"

upvoted 5 times

Tyler2023 1 year, 6 months ago

I accidentally click the like button :D

I agree with rotimislaw, the answer is B

upvoted 1 times

zaaath Highly Voted 2 years, 1 month ago

Selected Answer: B

It's B. I think ChatGPT gets confused with this question, and people end up posting its response here.

A. Environment Variables
- keeps the value out of configuration and state
B. Mark the variable as sensitive
- the value is still in the configuration and state, but not in the console output
C. A Terraform provider
- keeps the value out of configuration, but not the state (like a provider's data resource)
D. A -var flag
- keeps the value out of configuration, but not the state

upvoted 8 times

zaaath 2 years, 1 month ago

Correction:

D. A -var flag

- keeps the value out of configuration and state

upvoted 2 times

vl67 Most Recent 1 week, 5 days ago

Selected Answer: C

✓ A. Environment Variables

Yes, you can use environment variables (e.g., TF_VAR_secret) to pass secrets without hardcoding them into .tf files.

✓ B. Mark the variable as sensitive
Marking a variable as sensitive does not keep it out of the config files, but it prevents it from being displayed in CLI output or logs, which helps reduce accidental exposure, although the secret must still be provided via secure means.

✓ D. A -var flag
You can pass secrets via the -var flag when running terraform apply or plan, e.g.:

bash
Copy
Edit
terraform apply -var="db_password=secret123"
This allows keeping secrets out of config files.

✗ C. A Terraform provider
Terraform providers are used to interact with APIs/services — they are not a mechanism for passing or hiding secrets. While some providers may require credentials, they don't inherently keep secrets out of your configuration.
upvoted 2 times

premnick 10 months ago
Selected Answer: B
Answer is B, this is because marking variable as sensitive is only to mask the value in the logs during tf apply. Programmer can always open the terraform file and view the value.
upvoted 1 times

a54b16f 1 year, 3 months ago
Selected Answer: B
C refers to Vault, which is valid
upvoted 1 times

vipulchoubisa 1 year, 4 months ago
D--VAR OPTION IS OUTSIDE .TF FILE.
A-CAN BE SET IF YOU ARE USING TERRAFORM CLOUD
B-CAN BE SET UNDER VARIABLE.TF FILE
C-CAN BE SET UNDER PROVIDER BLOCK WHICH IS ALSO ONE OF THE .TF FILE

ONLY -VAR FLAG IS OUTSIDE TERRAFROM, HENCE THIS IS CORRECT.
upvoted 1 times

jutove_mi 1 year, 4 months ago
Using Option B we can keep secrets out of Terraform configuration files, but question asked which option "can not be used" ? so I suppose C (also chatgpt reply C)
upvoted 1 times

Arshad011294 1 year, 5 months ago
Selected Answer: B
I will go for B, for anyone thinking its C, there is a video in this official terraform documentation where a lady is explaining best practices of using provider vault, where we can set secrets to expire and hence the actual secrets are kept from getting revealed in the terraform state file.
https://registry.terraform.io/providers/hashicorp/vault/latest/docs
upvoted 2 times

Bere 1 year, 6 months ago
Selected Answer: B
Answer is B. Mark the variable as sensitive.

A. Environment Variables: This method keeps secrets out of configuration files by setting them externally.
B. Mark the variable as sensitive: While this marks the variable as sensitive in Terraform's state and prevents it from being displayed in CLI outputs, the secret can still be present in the configuration files and state file.
C. A Terraform provider: This doesn't inherently store or hide secrets in configuration files.
D. A -var flag: This method can be used to pass variables at runtime, thus keeping them out of configuration files.
upvoted 2 times

MisterR0B0T 1 year, 6 months ago

B

Terraform documentation

When you mark a variable as sensitive in Terraform, it does not keep the secrets out of the configuration files . Instead, it makes the variable write-only and prevents all users from viewing its value in the Terraform Cloud UI or reading it through the Variables API endpoint . Users with permission to read and write variables can set new values for sensitive variables, but you must delete and recreate the variable to edit its other attributes.

upvoted 1 times

brax404 1 year, 7 months ago

Selected Answer: C

C. A Terraform provider: Terraform providers are plugins that are responsible for managing the lifecycle of resources. They don't inherently offer a mechanism to keep secrets out of Terraform configuration files.

upvoted 4 times

debabrata6983 1 year, 9 months ago

Selected Answer: B

Secret flag prevents showing up the value in CLI o/p

upvoted 1 times

BaburTurk 1 year, 9 months ago

Selected Answer: C

C. A Terraform provider.

Terraform providers are used to interact with specific cloud providers or other APIs. They do not have the ability to access or store secrets.

The other options, environment variables, marking the variable as sensitive, and the -var flag, can all be used to keep secrets out of Terraform configuration files.

Environment variables are a common way to pass secrets to Terraform. They are stored outside of the Terraform configuration files, so they are not visible to anyone who can read the Terraform code.
Marking the variable as sensitive tells Terraform to encrypt the variable value when it is stored in the state file. This helps to protect the secret value from being exposed.
The -var flag can be used to pass secret values to Terraform from the command line. This is useful for one-off operations, but it is not a recommended way to manage secrets.

upvoted 3 times

Tyler2023 1 year, 6 months ago

I think Terraform will not encrypt your secrets in your state file, unless you are using Terraform cloud or enterprise

upvoted 1 times

kudakk 1 year, 9 months ago

Selected Answer: D

GPT:

D. A -var flag

Using the -var flag when running terraform apply or terraform plan from the command line requires you to specify variable values directly in the command, which could expose sensitive information in the command line history, system process list, or in logs.

While the -var flag is a way to set variables in Terraform, it's not a recommended way to handle sensitive information due to the exposure risks associated with command line arguments.

upvoted 1 times

Chrisler 1 year, 8 months ago

But based on the question, "secrets out of Terraform configuration files" Yes it is in the CLI history, sys process list or in logs, but not the terraform configuration.

upvoted 1 times

arunrkaushik 1 year, 9 months ago

Try to understand the question, it is important :
What is one method that cannot be utilized to prevent secrets from appearing in Terraform configuration files?
What approach does not work to exclude confidential data from being included in Terraform config files?
Which technique is not valid for keeping sensitive information out of the configs managed by Terraform?

What is an invalid solution for ensuring private credentials do not get embedded in a Terraform configuration?

Which choice is not viable for stopping secret keys and access tokens from being added to Terraform's setup files?

What method fails to protect classified details from being inserted into the configuration code used by Terraform?

upvoted 1 times

milan92stankovic 1 year, 11 months ago

Selected Answer: B

I vote for B

upvoted 1 times

sdm13168 2 years ago

Selected Answer: B

tested

upvoted 1 times

Question #103 *Topic 1*

Which of the following arguments are required when declaring a Terraform output?

A. sensitive

B. description

C. default

D. value

Suggested Answer: *D*

Community vote distribution

D (100%)

Nunyabiznes Highly Voted 8 months ago

Selected Answer: D

The only required argument when declaring a Terraform output is the value argument, which specifies the value of the output:

```
output "example" {
value = "example output value"
}
```

The description argument is optional and can be used to provide additional information about the output:

```
output "example" {
value = "example output value"
description = "An example output"
}
```

upvoted 10 times

Nunyabiznes 8 months ago

The sensitive argument is also optional and can be used to mark the output as sensitive, which prevents its value from being displayed in the output of Terraform commands, logs, and state files:

```
output "example" {
value = "example output value"
sensitive = true
}
```

The default argument is not a valid argument for an output. It is used for input variables and specifies a default value to be used if a value is not set in the configuration or passed in through the command line.

upvoted 4 times

Ni33 Most Recent 6 months, 2 weeks ago

Selected Answer: D

DDDDDDDDD

upvoted 1 times

Power123 7 months, 4 weeks ago

D is correct

upvoted 1 times

nakikoo 10 months, 2 weeks ago

Selected Answer: D

https://developer.hashicorp.com/terraform/language/values/outputs

upvoted 3 times

Gaby999 1 year, 2 months ago

yes, correct answer is D

upvoted 1 times

Burakko 1 year, 2 months ago

Selected Answer: D

There has to be a value for sure.

upvoted 2 times

Question #104 *Topic 1*

Your risk management organization requires that new AWS S3 buckets must be private and encrypted at rest. How can Terraform Enterprise automatically and proactively enforce this security control?

A. With a Sentinel policy, which runs before every apply

B. By adding variables to each TFE workspace to ensure these settings are always enabled

C. With an S3 module with proper settings for buckets

D. Auditing cloud storage buckets with a vulnerability scanning tool

Suggested Answer: *B*

Community vote distribution

A (100%)

camps Highly Voted 1 year, 1 month ago

Selected Answer: A

A. With a Sentinel policy, which runs before every apply.

Terraform Enterprise can enforce security controls through the use of Sentinel policies. Sentinel is a policy as code framework that integrates with Terraform Enterprise and can be used to enforce specific security controls. In this case, the Sentinel policy could check that all new S3 buckets are set to be private and encrypted at rest and prevent the Terraform apply from proceeding if the buckets do not meet this requirement. This ensures that the security control is automatically and proactively enforced every time Terraform makes changes to the infrastructure.

upvoted 11 times

selvaraj133ece Most Recent 9 months, 4 weeks ago

Answer, B only. They want to keep the S3 bucket private. So, it will be a different state file.

upvoted 1 times

Rohit000003 10 months, 2 weeks ago

Selected Answer: A

As per terraform document

upvoted 1 times

Ni33 1 year ago

Selected Answer: A

AAAAAAAAAA

upvoted 1 times

Power123 1 year, 1 month ago

A is correct

upvoted 1 times

Nunyabiznes 1 year, 2 months ago

Selected Answer: A

import "tfplan"

Ensure all new S3 buckets are private and encrypted at rest
deny[msg] {
resources := tfplan.module_paths["aws_s3_bucket"]
not all_true([
for r in resources:
r.attributes.acl == "private" and
r.attributes.server_side_encryption_configuration.0.rule.0.apply_server_side_encryption_by_default.0.sse_algorithm == "AES256"
])
msg := "All new S3 buckets must be private and encrypted at rest"
}

upvoted 3 times

SilentMilli 1 year, 2 months ago

Selected Answer: A

Terraform Enterprise provides the ability to enforce security controls through Sentinel policies, which are a form of policy as code. Sentinel policies allow you to define and enforce organizational or regulatory policies by creating a set of rules that run before each Terraform operation.

upvoted 2 times

Ame2222 1 year, 3 months ago

A is correct

upvoted 1 times

Daro_ 1 year, 3 months ago

Selected Answer: A

yes A Corrct

upvoted 1 times

seif1993 1 year, 5 months ago

yes A Corrct

upvoted 1 times

RVivek 1 year, 8 months ago

Selected Answer: A

Sentinel policy is the best way to manage multiple workspaces

upvoted 1 times

bora4motion 1 year, 8 months ago

Selected Answer: A

I go with A.

upvoted 2 times

Burakko 1 year, 8 months ago

Selected Answer: A

With a Sentinel policy for sure.

upvoted 2 times

mav3r1ck 1 year, 8 months ago

A.

Reference:
https://docs.hashicorp.com/sentinel/intro/what
https://medium.com/hashicorp-engineering/enforcing-aws-s3-security-best-practice-using-terraform-sentinel-ddcd181ff4b7

upvoted 4 times

Question #105 *Topic 1*

Most Terraform providers interact with _____.

A. API

B. VCS Systems

C. Shell scripts

D. None of the above

Suggested Answer: *A*

Community vote distribution

A (100%)

Hizumi Highly Voted 1 year, 8 months ago

Answer is A.

Terraform relies on plugins called "providers" to interact with cloud providers, SaaS providers, and other APIs, as per:

https://www.terraform.io/language/providers

upvoted 5 times

Boomboy_420 Most Recent 7 months ago

Selected Answer: A

A is the right answer

upvoted 1 times

debabrata6983 9 months ago

Selected Answer: A

Provider plugin abstracts the API interaction with the cloud provider.

upvoted 3 times

SairamObili 9 months, 2 weeks ago

it s API

Answer is A

upvoted 1 times

IK912 11 months, 1 week ago

A is the answer

upvoted 1 times

Power123 1 year, 1 month ago

A is correct

upvoted 1 times

seif1993 1 year, 5 months ago

Selected Answer: A

yes for A

upvoted 1 times

donathon 1 year, 8 months ago

Selected Answer: A

A is the answer

upvoted 1 times

Question #106 *Topic 1*

terraform validate validates that your infrastructure matches the Terraform state file.

A. True

B. False

Suggested Answer: *A*

Community vote distribution

B (100%)

RVivek Highly Voted 1 year, 8 months ago

Selected Answer: B

Validate only checks the syntax

http://man.hubwiz.com/docset/Terraform.docset/Contents/Resources/Documents/docs/commands/validate.html

upvoted 11 times

babgad Most Recent 9 months ago

Selected Answer: B

False B

Validate runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state. It is thus primarily useful for general verification of reusable modules, including correctness of attribute names and value types.

It is safe to run this command automatically, for example as a post-save check in a text editor or as a test step for a re-usable module in a CI system

https://developer.hashicorp.com/terraform/cli/v1.1.x/commands/validate

upvoted 3 times

Busi57 10 months, 1 week ago

Selected Answer: B

false B Validate only checks the syntax

upvoted 2 times

Busi57 10 months, 1 week ago

False B

upvoted 1 times

Ni33 1 year ago

Selected Answer: B

B is the correct answer. Terraform plan validates desired state with current state configuration.

upvoted 2 times

Power123 1 year, 1 month ago

False , B is correct

upvoted 1 times

adouban 1 year, 5 months ago

Selected Answer: B

B is correct

upvoted 1 times

0ptimus 1 year, 8 months ago

Selected Answer: B

B because Terraform validate only checks for syntax error or so

upvoted 2 times

bora4motion 1 year, 8 months ago

Selected Answer: B

It's more of a syntax thing.

upvoted 2 times

Burakko 1 year, 8 months ago

Selected Answer: B

False, terraform validate has nothing to do with the state file.

upvoted 2 times

Uma10 1 year, 8 months ago

Selected Answer: B

The terraform validate command validates the configuration files in a directory, referring only to the configuration and not accessing any remote services such as remote state, provider APIs, etc.

Validate runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state. It is thus primarily useful for general verification of reusable modules, including correctness of attribute names and value types.

Source: https://www.terraform.io/cli/commands/validate

upvoted 3 times

Question #107 *Topic 1*

What does terraform import allow you to do?

A. Import a new Terraform module

B. Use a state file to import infrastructure to the cloud

C. Import provisioned infrastructure to your state file

D. Import an existing state file to a new Terraform workspace

Suggested Answer: *C*

Community vote distribution

C (100%)

Ni33 6 months, 2 weeks ago

Selected Answer: C

Yes, C is the correct answer.

upvoted 2 times

Power123 7 months, 4 weeks ago

C is correct

upvoted 1 times

bora4motion 1 year, 2 months ago

Selected Answer: C

I go with C

upvoted 4 times

Burakko 1 year, 2 months ago

Selected Answer: C

%100 true

upvoted 2 times

Question #108 *Topic 1*

FILL BLANK -

In the below configuration, how would you reference the module output vpc_id?

```
module "vpc" {
  source = "terraform-and-modules/vpc/aws"
  cidr = "10.0.0.0/16"
  name = "test-vpc"
}
```

Type your answer in the field provided. The text field is not case sensitive and all variations of the correct answer are accepted.

Suggested Answer: *output "outvpc_id"*

Hizumi Highly Voted 2 years, 8 months ago

module.vpc.vpc_id

Reference: https://cloudcasts.io/course/terraform/community-vpc-module

upvoted 81 times

shopkitty Highly Voted 2 years, 8 months ago

it should be module.vpc.vpc_id

upvoted 21 times

DataEngDP Most Recent 1 month, 2 weeks ago

module.vpc.vpc_id (type + name + id)

upvoted 2 times

michasacuer 7 months, 3 weeks ago

module.vpc.vpc_id

upvoted 2 times

Algol 8 months, 2 weeks ago

module.vpc.vpc_id

upvoted 3 times

nmnm22 9 months, 3 weeks ago

Module.vpc.vpc_id

upvoted 2 times

temor 1 year, 2 months ago

module.vpc.vpc_id

upvoted 3 times

5719d28 1 year, 2 months ago

module.vpc.vpc_id

upvoted 3 times

NashP 1 year, 3 months ago

To reference the module output `vpc_id` in the given configuration, you would use the following syntax:

```hcl
module.vpc.vpc_id
```

This assumes that the output variable `vpc_id` is declared in the "example" module. Replace `vpc` with the actual name you gave to your module. So, if your module is named "vpc," the reference would be:

```hcl
module.vpc.vpc_id
```

This syntax allows you to access the output value `vpc_id` from the specified module in your Terraform configuration.
upvoted 1 times

KingFvsher 1 year, 8 months ago
You can reference the vpc_id output of a module in Terraform using the module.<MODULE_NAME>.<OUTPUT_NAME> syntax. In this case, if the name of the module block is vpc, you would reference the vpc_id output as module.vpc.vpc_id.
upvoted 6 times

debabrata6983 1 year, 9 months ago
module.vpc.vpc_id
upvoted 2 times

arunrkaushik 1 year, 9 months ago
resource "aws_vpc" "my_vpc" {
cidr_block = "10.0.0.0/16"
...
}

output "vpc_id" {
value = aws_vpc.my_vpc.id
}
upvoted 1 times

joyboy23 1 year, 10 months ago
module.vpc.vpc_id
upvoted 2 times

abhi6199 1 year, 11 months ago
its module.vpc.vpc_id
upvoted 1 times

IK912 1 year, 11 months ago
module.vpc.vpc_id
upvoted 1 times

Ni33 2 years ago
module.vpc.vpc_id
upvoted 1 times

Nunyabiznes 2 years, 2 months ago
To reference the vpc_id output of the vpc module in the above configuration, you would use the following syntax:
module.vpc.vpc_id
This would allow you to reference the vpc_id output from other parts of your Terraform configuration, such as when creating resources that depend on the VPC. For example:
resource "aws_subnet" "subnet_1" {
vpc_id = module.vpc.vpc_id
cidr_block = "10.0.1.0/24"
}
In this example, the aws_subnet resource is referencing the vpc_id output of the vpc module to ensure that the subnet is created in the correct VPC.
upvoted 5 times

Question #109 *Topic 1*

How would you reference the Volume IDs associated with the ebs_block_device blocks in this configuration?

```
resource "aws_instance" "example" {
  ami = "ami-abc123"
  instance_type - "t2.micro"

ebs_block_device {
  device_name = "sda2"
  volume_size = 16
}

ebs_block_device {
  device_name = "sda3"
  volume_size = 20
 }
}
```

A. aws_instance.example.ebs_block_device.[*].volume_id

B. aws_instance.example.ebs_block_device.volume_id

C. aws_instance.example.ebs_block_device[sda2,sda3].volume_id

D. aws_instance.example.ebs_block_device.*.volume_id

Suggested Answer: *C*

Community vote distribution

D (85%) A (15%)

KJ_Rollings Highly Voted 1 year, 6 months ago

Selected Answer: D

It's D.

If you're using square brackets, then it should be like,

ebs_block_device[*].volume_id

upvoted 20 times

fedeX Highly Voted 1 year, 10 months ago

100% D.

aws_instance.example.ebs_block_device.*.volume_id and aws_instance.example.ebs_block_device[*].volume_id are equivalent and will both refer to the volume_id field for all of the EBS block devices that are attached to the aws_instance resource with the name example.

The * syntax is known as the "splat" operator and can be used to indicate that the preceding attribute should be repeated for each element in a list. It can be used with both the . and [] notation.

For example, suppose you have an aws_instance resource named example that has two EBS block devices attached to it, with volume_id values of vol-12345678 and vol-87654321. You could use the * operator like this:

aws_instance.example.ebs_block_device.*.volume_id
This will evaluate to ["vol-12345678", "vol-87654321"]

aws_instance.example.ebs_block_device[*].volume_id
This will also evaluate to ["vol-12345678", "vol-87654321"]

In both cases, the volume_id field is being accessed for each element in the ebs_block_device list, and the resulting value is a list of the volume_id values for each element.

upvoted 19 times

vl67 Most Recent 1 week, 5 days ago

Selected Answer: D

Reference:

Terraform Splat Expressions: https://www.terraform.io/docs/language/expressions/splat.html

Explains the .*. operator for accessing all elements of a list.

AWS Provider aws_instance Documentation:

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance#ebs_block_device

Confirms that ebs_block_device is a list of blocks, and attributes like volume_id are accessible.

Final Answer:

D. aws_instance.example.ebs_block_device.*.volume_id is the correct option to reference the volume IDs for all ebs_block_device blocks in the configuration.

upvoted 1 times

kennywuu 10 months, 4 weeks ago

it is D.

Just check it in terraform console, only D is passed :)

upvoted 4 times

TigerInTheCloud 11 months, 1 week ago

Selected Answer: D

A is almost right and better and more modern if there is no extra dot

You can add an output and validate each of the expressions:

output "ebs-volume-ids" {

value = aws_instance.example.ebs_block_device["sda2"].volume_id

}

upvoted 1 times

TigerInTheCloud 11 months, 1 week ago

C can rewrite as:

[for v in ["sda2", "sda3"] : aws_instance.example.ebs_block_device[v].volume_id]

upvoted 1 times

Tyler2023 1 year ago

All of them are invalid expression

Based on this https://developer.hashicorp.com/terraform/language/expressions/references

The arguments of the ebs_block_device nested blocks can be accessed using a splat expression. For example, to obtain a list of all of the device_name values, use aws_instance.example.ebs_block_device[*].device_name.

upvoted 3 times

MauroSoli 1 year, 1 month ago

The answer is D but it's a form deprecated

upvoted 1 times

amehim 1 year, 1 month ago

I with D: > aws_instance.app_server.ebs_block_device.*.volume_id

tolist([

"vol-0aa30805cf71780e4",

"vol-081f89b26f2fed5ac",

])

upvoted 1 times

vj_dhaksh 1 year, 5 months ago

answer is D.. <<<Tested >>>

upvoted 5 times

Nunyabiznes 1 year, 8 months ago

Selected Answer: D

D. aws_instance.example.ebs_block_device.*.volume_id

To reference the Volume IDs associated with the ebs_block_device blocks in the given aws_instance resource, you can use the .* syntax to reference all elements of the list. The ebs_block_device blocks are represented as a list in the aws_instance resource, and each block has a volume_id attribute that you can reference.

upvoted 4 times

lordoftheringsnewavatar 1 year, 8 months ago

output "splat" {
value = aws_instance.test-vm.ebs_block_device.*.volume_id
}

Outputs:

splat = tolist([
"vol-088ecdfa25d46bb49",
"vol-030bfe9d14a06ecd2",
])

Selected Answer: D

upvoted 1 times

alexsandroe 1 year, 9 months ago

Selected Answer: D

one valid expierience

resource "aws_instance" "example" {
ami = "ami-a10b4dc2"
instance_type = "t2.micro"

Connect the instance to the EBS Volumes attached
https://www.terraform.io/docs/providers/aws/r/instance.html#ebs_block_device
ebs_block_device {
device_name = "/dev/sda1"
volume_size = 10
volume_type = "gp2"
}
ebs_block_device {
device_name = "/dev/sdb1"
volume_size = 20
volume_type = "gp2"
}
}

output "splat" {
value = ["${aws_instance.example.*.id}", "${aws_ebs_volume.example.*.id}"]
}

upvoted 2 times

awsguys 1 year, 9 months ago

D => Right

upvoted 2 times

gekkehenk 1 year, 10 months ago

Selected Answer: D

The brackets do not work, asterisk only.

upvoted 4 times

resnef 1 year, 10 months ago

Selected Answer: D

Answer is D

> aws_instance.example.ebs_block_device.*.volume_id
tolist([
"vol-123457",
"vol-43437834743348",
])

> aws_instance.example.ebs_block_device.[*].volume_id

```
╷
│ Error: Invalid attribute name
│
│ on <console-input> line 1:
│ (source code not available)
│
│ An attribute name is required after a dot.
╵
```

upvoted 3 times

AB7088 1 year, 11 months ago

If remove the dot then the answer should be A. I think there has a typo.. D is not correct.

upvoted 2 times

BaburTurk 1 year, 11 months ago

aws_instance.example.ebs_block_device[*].volume_id
or
aws_instance.example.ebs_block_device.*.volume_id

then D is the answer

upvoted 2 times

Question #110 *Topic 1*

What does state locking accomplish?

A. Copies the state file from memory to disk

B. Encrypts any credentials stored within the state file

C. Blocks Terraform commands from modifying the state file

D. Prevents accidental deletion of the state file

Suggested Answer: *B*

Community vote distribution

C (100%)

Uma10 Highly Voted 1 year, 2 months ago

Selected Answer: C

If supported by your backend, Terraform will lock your state for all operations that could write state. This prevents others from acquiring the lock and potentially corrupting your state.

Source: https://www.terraform.io/language/state/locking

upvoted 8 times

chael88 Highly Voted 10 months ago

Just passed the exam. There was a similar question not listed here on ExamTopic:

What backend types supports state locking?

A. local

B. None

C. All

D. Some

I chose D. Some as according to Hashicorp document: "Not all backends support locking"

Source: https://developer.hashicorp.com/terraform/language/state/backends

upvoted 8 times

phidelics 8 months, 1 week ago

Not all backends supports state locking. you are right with option D

upvoted 2 times

Ni33 Most Recent 6 months, 2 weeks ago

Selected Answer: C

C is the right answer

upvoted 1 times

Power123 7 months, 4 weeks ago

C is correct

upvoted 1 times

adouban 11 months, 2 weeks ago

Selected Answer: C

C state locking is protect state file from modification at the same time

upvoted 2 times

barracouto 1 year, 1 month ago

Selected Answer: C

I really wonder if exam topics needs to display the incorrect answers for legality purposes.

It's C.

upvoted 3 times

secdaddy 11 months, 3 weeks ago

They're guessing at answers and then refining by croudsourcing answers from us
upvoted 1 times

asudhin 1 year, 1 month ago

Selected Answer: C

It's C
upvoted 1 times

legendary7 1 year, 1 month ago

C-is the correct answer
https://www.terraform.io/language/state/locking
upvoted 1 times

RVivek 1 year, 2 months ago

Selected Answer: C

Blocks Terraform commands from modifying the state file. Since already one command is doing a modification and mutiple simultaneous writes can make it inconsistant
upvoted 1 times

bora4motion 1 year, 2 months ago

Selected Answer: C

I go with C
upvoted 3 times

Burakko 1 year, 2 months ago

Selected Answer: C

It just locks the state file not to be modified.
upvoted 4 times

Question #111 *Topic 1*

You just upgraded the version of a provider in an existing Terraform project. What do you need to do to install the new provider?

A. Run terraform apply -upgrade

B. Run terraform init -upgrade

C. Run terraform refresh

D. Upgrade your version of Terraform

Suggested Answer: *B*

Community vote distribution

B (100%)

Hizumi Highly Voted 2 years, 2 months ago

Answer is B.

[-upgrade] - Opt to upgrade modules and plugins as part of their respective installation steps. See the sections below for more details.

Reference: https://www.terraform.io/cli/commands/init#upgrade

upvoted 16 times

Prince_devops Most Recent 10 months ago

Selected Answer: B

Answer is B

upvoted 1 times

Tyler2023 1 year ago

https://developer.hashicorp.com/terraform/tutorials/configuration-language/provider-versioning#upgrade-the-aws-provider-version

upvoted 1 times

IK912 1 year, 5 months ago

its B, I tried this

upvoted 1 times

Ni33 1 year, 6 months ago

Selected Answer: B

BBBBBBBB

upvoted 1 times

srcntpc 1 year, 10 months ago

Selected Answer: B

Answer is B.

upvoted 1 times

hectordj 2 years, 1 month ago

Selected Answer: B

Answer is B.

upvoted 1 times

Bortoloto 2 years, 1 month ago

Selected Answer: B

Answer is B.

upvoted 1 times

Question #112 *Topic 1*

A module can always refer to all variables declared in its parent module.

A. True

B. False

Suggested Answer: *A*

Community vote distribution



Burakko Highly Voted 1 year, 8 months ago

Selected Answer: B

Modules do not inherit variables from the parent module. All modules are self-contained units. So you have to explicitly define variables in the child module, and then explicit set these variables in the parent module, when you instantiate the child module.

upvoted 26 times

Burakko 1 year, 8 months ago

However you can refer them from the parent module, I guess it is A. The question is asking for referring not inheriting.

upvoted 14 times

kiran15789 Highly Voted 1 year ago

Selected Answer: B

A module in Terraform can only access variables that are explicitly passed as input variables or declared as outputs by the parent module. It cannot access variables declared within the parent module unless they are passed to the module as input variables. This is known as the "principle of least privilege," which means that modules should only have access to the information and resources that they need to perform their tasks, and no more.

So, to refer to a variable from a parent module, you must either pass it as an input variable to the child module or declare it as an output variable in the parent module and then reference it in the child module using the module.<name>.<output> syntax.

upvoted 8 times

Tyler2023 6 months, 3 weeks ago

The question is asking if we can always refer to all variables declared which is true,
It is not asking if it is automatically or manually

upvoted 1 times

Tyler2023 6 months, 3 weeks ago

Answer is A

upvoted 1 times

Demonik Most Recent 5 months, 4 weeks ago

Selected Answer: A

A for me

upvoted 4 times

Stargazer11 6 months, 3 weeks ago

Answer: B

A child module can refer to variables declared in its parent module, but it does so in a controlled and explicit manner. By default, child modules do not have automatic access to all variables in the parent module. To reference a variable from the parent module in a child module, you should declare an input variable with the same name in the child module and then explicitly pass the value from the parent module.

upvoted 2 times

milan92stankovic 11 months, 2 weeks ago

Selected Answer: A

The answer is A.
You CAN always REFER. You don't have to, but you can.

upvoted 5 times

Ni33 1 year ago

Selected Answer: B

B is the correct answer

upvoted 1 times

raf314 1 year, 1 month ago

Selected Answer: B

The question is whether a (child) module can always refer to ALL variables in the parent module. The answer is NO. The child module can "see" and refer to the variables defined in its own (child module's) definition. The child module has no visibility of the variables defined in the parent module. Thus it cannot refer to them. Answer is False (B)

upvoted 7 times

BennaniHaythem 1 year, 1 month ago

A module can refer to variables in its parent module only if those variables are explicitly passed to it as input variables. If a variable is not passed to a child module, then that child module cannot refer to it. So the correct answer is B - False.

upvoted 3 times

sylvergorilla 1 year, 1 month ago

Selected Answer: B

No, a module cannot always refer to all variables declared in its parent module. Only variables declared as public in the parent module can be referred to by a module.

upvoted 1 times

Sathisgm 1 year, 1 month ago

True we can refer parent module

upvoted 1 times

camps 1 year, 1 month ago

Selected Answer: B

False.

A module can refer to variables in its parent module only if they are explicitly passed to it using variable definitions or module blocks in the calling module.

upvoted 2 times

Nunyabiznes 1 year, 2 months ago

Selected Answer: B

B. False

A module can only refer to variables that are explicitly passed to it as inputs, either through variable declarations or module calls. It cannot automatically refer to all variables declared in its parent module, as it may not need or have access to all of them. If a variable is not passed to a module as an input, the module will not be able to use it directly.

upvoted 1 times

phidelics 1 year, 2 months ago

Selected Answer: A

refer to* key word i guess

upvoted 1 times

rotimislaw 1 year, 3 months ago

Selected Answer: A

Seems A to me

upvoted 1 times

agmesas 1 year, 3 months ago

Selected Answer: B

B you must declare the variable in the child module if you can use it. Also, the parent module must call the child module using inputs. First, the parent module declares variables, second, the child module declares its inputs, then, the parent module call to the child module using inputs that were declared in the variables.tf of the child module.

upvoted 3 times

Zeppoonstream 1 year, 4 months ago

A. True. A module in Terraform can always refer to all variables declared in its parent module, as long as the parent module is called before the child module in the configuration file.

upvoted 1 times

ssanjayt 1 year, 4 months ago

Selected Answer: A

A is correct !A module can always refer to all variables declared in its parent module

upvoted 2 times

Question #113 *Topic 1*

When you use a remote backend that needs authentication, HashiCorp recommends that you:

A. Use partial configuration to load the authentication credentials outside of the Terraform code

B. Push your Terraform configuration to an encrypted git repository

C. Write the authentication credentials in the Terraform configuration files

D. Keep the Terraform configuration files in a secret store

Suggested Answer: *B*

Community vote distribution

A (100%)

Hizumi Highly Voted 2 years, 8 months ago

Answer is A.

Note: We recommend omitting the token from the configuration, and instead using terraform login or manually configuring credentials in the CLI config file.

Reference: https://www.terraform.io/language/settings/backends/remote

upvoted 8 times

Mattt Most Recent 8 months, 1 week ago

Selected Answer: A

Answer is A

upvoted 1 times

Bere 1 year, 6 months ago

Selected Answer: A

Answer is A

This means that you should use environment variables to store these credentials. The partial configuration does not include sensitive data in the file but relies on environment variables that Terraform automatically detects.

Credentials and Sensitive Data

https://developer.hashicorp.com/terraform/language/settings/backends/configuration#credentials-and-sensitive-data

Warning: We recommend using environment variables to supply credentials and other sensitive data. If you use -backend-config or hardcode these values directly in your configuration, Terraform will include these values in both the .terraform subdirectory and in plan files. This can leak sensitive credentials.

upvoted 4 times

Tyler2023 1 year, 6 months ago

Partial Configuration

You do not need to specify every required argument in the backend configuration. Omitting certain arguments may be desirable if some arguments are provided automatically by an automation script running Terraform. When some or all of the arguments are omitted, we call this a partial configuration.

With a partial configuration, the remaining configuration arguments must be provided as part of the initialization process.

https://developer.hashicorp.com/terraform/language/settings/backends/configuration#partial-configuration

upvoted 1 times

Tyler2023 1 year, 6 months ago

Answer is A

upvoted 1 times

cruz95 1 year, 8 months ago

Selected Answer: A

definitely

upvoted 1 times

camps 2 years, 1 month ago

Selected Answer: A

it's a

upvoted 1 times

legendary7 2 years, 8 months ago

A is the correct answer

upvoted 2 times

RVivek 2 years, 8 months ago

Selected Answer: A

https://www.terraform.io/language/settings/backends/configuration#credentials-and-sensitive-data

upvoted 2 times

jjkcoins 2 years, 8 months ago

https://www.terraform.io/language/settings/backends/configuration#credentials-and-sensitive-data

upvoted 3 times

jjkcoins 2 years, 8 months ago

Warning: We recommend using environment variables to supply credentials and other sensitive data. If you use -backend-config or hardcode these values directly in your configuration, Terraform will include these values in both the .terraform subdirectory and in plan files. This can leak sensitive credentials.

upvoted 3 times

hkacholiya 2 years, 8 months ago

Answer is A

upvoted 1 times

shopkitty 2 years, 8 months ago

Selected Answer: A

vote for A

upvoted 1 times

Burakko 2 years, 8 months ago

Selected Answer: A

It should be A.

upvoted 3 times

Question #114 *Topic 1*

You have a simple Terraform configuration containing one virtual machine (VM) in a cloud provider. You run terraform apply and the VM is created successfully.
What will happen if you terraform apply again immediately afterwards without changing any Terraform code?

A. Terraform will terminate and recreate the VM

B. Terraform will create another duplicate VM

C. Terraform will apply the VM to the state file

D. Nothing

Suggested Answer: *C*

Community vote distribution

D (100%)

Bere Highly Voted 5 months, 4 weeks ago

Selected Answer: D

If you run terraform apply again immediately afterwards without changing any Terraform code, Terraform will compare the desired state with the actual state of the infrastructure as recorded in the state file. Since there are no changes, then Terraform will report that there are no differences and won't do anything.

upvoted 5 times

Ni33 Most Recent 1 year ago

Selected Answer: D

D-Nothing

upvoted 1 times

Power123 1 year, 1 month ago

Nothing. Answer is D

upvoted 2 times

InformationOverload 1 year, 4 months ago

Selected Answer: D

keyword: Idempotent

upvoted 3 times

adouban 1 year, 5 months ago

Selected Answer: D

Nothing

upvoted 1 times

legendary7 1 year, 8 months ago

D is the correct answer. Terraform would refresh and see that the current state and the desired state are the-same and hence would do nothing

upvoted 3 times

shopkitty 1 year, 8 months ago

Selected Answer: D

nothing change hence do nothing

upvoted 1 times

amanp 1 year, 8 months ago

Selected Answer: D

Nothing.

If the desired state in state file is same as current state, no changes will happen

upvoted 4 times

Burakko 1 year, 8 months ago

Selected Answer: D

Nothing without changing the configuration file, the state file or desired state.

upvoted 3 times

Question #115 *Topic 1*

A junior admin accidentally deleted some of your cloud instances. What does Terraform do when you run terraform apply?

A. Build a completely brand new set of infrastructure

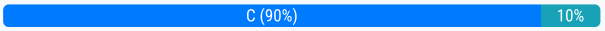
B. Tear down the entire workspace infrastructure and rebuild it

C. Rebuild only the instances that were deleted

D. Stop and generate an error message about the missing instances

Suggested Answer: *D*

Community vote distribution



legendary7 Highly Voted 2 years, 8 months ago

C

Terraform would refresh and check that the Current State in the state file is not thesame as the Desired State.

Terraform will check its configuration resource files and if it finds a resource that is present in the state file but not in the Desired state, it will create it

upvoted 7 times

chessmaster5000 Most Recent 2 months, 1 week ago

Selected Answer: C

C is the answer. Terraform will detect the missing instances and create them to match the tf config file.

upvoted 1 times

Mattt 8 months, 1 week ago

Selected Answer: C

C is correct

upvoted 1 times

090200f 10 months, 2 weeks ago

C is the correct ans

upvoted 1 times

recep38 1 year, 3 months ago

Selected Answer: D

it is very clear

upvoted 1 times

tamng 9 months, 2 weeks ago

stupid

upvoted 1 times

Ni33 2 years ago

Selected Answer: C

C is the correct answer !

upvoted 2 times

Power123 2 years, 1 month ago

C is correct

upvoted 2 times

adouban 2 years, 5 months ago

Deleting instance from cloud console will result terraform to create instance again to match what written in tf config files

upvoted 3 times

shopkitty 2 years, 8 months ago

Selected Answer: C

vote for C

upvoted 3 times

amanp 2 years, 8 months ago

Selected Answer: C

Answer is C.

upvoted 2 times

Burakko 2 years, 8 months ago

Selected Answer: C

the correct answer is C.

upvoted 2 times

duffyduck 2 years, 8 months ago

The Answer is C

upvoted 1 times

Question #116 *Topic 1*

You have created a main.tr Terraform configuration consisting of an application server, a database, and a load balancer. You ran terraform apply and all resources were created successfully. Now you realize that you do not actually need the load balancer so you run terraform destroy without any flags What will happen?

A. Terraform will destroy the application server because it is listed first in the code

B. Terraform will prompt you to confirm that you want to destroy all the infrastructure

C. Terraform will destroy the main.tf file

D. Terraform will prompt you to pick which resource you want to destroy

E. Terraform will immediately destroy all the infrastructure

Suggested Answer: *B*

Community vote distribution

B (100%)

SilentMilli Highly Voted 2 years, 2 months ago

Selected Answer: B

Terraform will prompt you to confirm that you want to destroy all the resources before proceeding. The prompt will ask you to enter "yes" or "no" to confirm or cancel the destruction of resources unless you add the "-auto-approve" flag to stop terraform from prompting you to confirm.

upvoted 6 times

090200f Most Recent 10 months, 2 weeks ago

B is correct ans. first it will ask you yes or no then if we enter yes , destroy all the resources

upvoted 1 times

ajbffkabf 1 year, 11 months ago

xxxxxxxxxxxxxxx

upvoted 1 times

ozbeyucel 2 years, 3 months ago

It is B, however, E is tricky there. can confuse

upvoted 3 times

bora4motion 2 years, 8 months ago

Selected Answer: B

I go with B

upvoted 3 times

Burakko 2 years, 8 months ago

Selected Answer: B

The answer is B.

upvoted 3 times

Question #117 *Topic 1*

Which type of block fetches or computes information for use elsewhere in a Terraform configuration?

A. provider

B. resource

C. local

D. data

Suggested Answer: *A*

Community vote distribution

D (100%)

Burakko Highly Voted 1 year, 2 months ago

Selected Answer: D

Data sources allow data to be fetched or computed for use elsewhere in Terraform configuration. Use of data sources allows a Terraform configuration to build on information defined outside of Terraform, or defined by another separate Terraform configuration.

upvoted 14 times

bora4motion Highly Voted 1 year, 2 months ago

Selected Answer: D

it's D as you're using infrastructure already created - that's when you use "data". With "resource" you create new infrastructure.

upvoted 13 times

Ni33 Most Recent 6 months, 2 weeks ago

Selected Answer: D

D is the correct answer !

upvoted 1 times

mkeology 7 months, 2 weeks ago

Selected Answer: D

D. data, a data source

upvoted 1 times

Ravi528 8 months ago

Selected Answer: D

Definitely not A, It's D as Infra is done already, Data is used to compute

upvoted 2 times

SilentMilli 8 months, 2 weeks ago

Selected Answer: D

The data block is used in Terraform to fetch or compute information from a given data source, and make that information available for use elsewhere in the Terraform configuration. This data can come from a variety of sources, including cloud provider APIs, databases, and other external systems.

upvoted 3 times

TechHero 10 months, 1 week ago

D. data

Data blocks in Terraform are used to fetch or compute information for use elsewhere in a Terraform configuration. They allow you to read data from external sources such as an API, a file, or a database and use it to populate variables or other parts of your Terraform configuration.

A. provider

A provider block is used to configure the provider plugin used in Terraform configuration, it's not related to data fetching or computation

B. resource

A resource block is used to create and manage infrastructure resources, it's not related to data fetching or computation

C. local

A local block is used to define variables that are only used within the module in which they are defined and not exposed as outputs. It's not related to

data fetching or computation.

It's important to understand that data blocks do not make changes to the infrastructure, they are only used to fetch or compute data.

upvoted 5 times

kprod 1 year, 2 months ago

Selected Answer: D

D - data

upvoted 5 times

Question #118 *Topic 1*

You have just developed a new Terraform configuration for two virtual machines with a cloud provider. You would like to create the infrastructure for the first time.
Which Terraform command should you run first?

A. terraform apply

B. terraform plan

C. terraform show

D. terraform init

Suggested Answer: *C*

Community vote distribution

D (100%)

Burakko Highly Voted 2 years, 8 months ago

Selected Answer: D

terraform init if you want to run it for the first time.

upvoted 6 times

090200f Most Recent 10 months, 2 weeks ago

keyword is for the first time: so terraform init is correct ans
option : D

upvoted 1 times

Pikopo 1 year, 7 months ago

D is correct

upvoted 1 times

Ni33 2 years ago

Selected Answer: D

If it is first time then Terraform init command.

upvoted 1 times

Ravi528 2 years, 2 months ago

Selected Answer: D

First we should run terraform init

upvoted 1 times

Manguu 2 years, 2 months ago

Selected Answer: D

This will initialize the working directory containing the TF config files

upvoted 1 times

adouban 2 years, 5 months ago

Selected Answer: D

Terraform init

upvoted 1 times

legendary7 2 years, 8 months ago

D
You need to initialize so all the provider plugin and configurations dependencies get initialized

upvoted 1 times

kprod 2 years, 8 months ago

Selected Answer: D

D - init

upvoted 3 times

bora4motion 2 years, 8 months ago

Selected Answer: D

I go with D. init

upvoted 4 times

Question #119 *Topic 1*

All modules published on the official Terraform Module Registry have been verified by HashiCorp.

A. True

B. False

Suggested Answer: *B*

Community vote distribution



Hizumi Highly Voted 2 years, 2 months ago

Answer is False.
Only modules considered "Verified Modules" are reviewed by Hashicorp, otherwise anyone can publish modules on the Terraform Registry.
Reference: https://www.terraform.io/registry/modules/verified
https://www.terraform.io/registry/modules/publish

upvoted 20 times

samimshaikh Most Recent 11 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/registry/modules/verified

upvoted 1 times

Tyler2023 1 year ago

False
Verified modules are expected to be actively maintained by HashiCorp partners. The verified badge isn't indicative of flexibility or feature support; very simple modules can be verified just because they're great examples of modules. Likewise, an unverified module could be extremely high quality and actively maintained. An unverified module shouldn't be assumed to be poor quality, it only means it hasn't been created by a HashiCorp partner.

https://developer.hashicorp.com/terraform/registry/modules/verified

upvoted 2 times

Pikopo 1 year, 1 month ago

B is correct

upvoted 1 times

NextBrand 1 year, 4 months ago

B. False
A verified badge will usually appear on modules that are verified by Hashicorp. However, you can use filters to view unverified modules.

upvoted 1 times

KJ_Rollings 1 year, 6 months ago

Selected Answer: B

https://developer.hashicorp.com/terraform/registry/modules/verified

upvoted 1 times

BennaniHaythem 1 year, 7 months ago

True because When you submit your module, HashiCorp will review it to ensure that it meets the quality standards for the Terraform Module Registry. This review process includes checks for compliance with Terraform's best practices, security vulnerabilities, and other potential issues that could impact the stability or security of the infrastructure being provisioned with Terraform. If your module meets these standards and passes the verification process, it can be listed on the official Terraform Module Registry with the HashiCorp Verified badge.

upvoted 1 times

Ravi528 1 year, 8 months ago

Selected Answer: B

False, It's not mandatory. If Hashicorp verified then it will show Blue tick at the end, Otherwise not. But it's not mandatory verification by hashicorp

upvoted 2 times

Nunyabiznes 1 year, 8 months ago

Selected Answer: B

B. False

While the Terraform Module Registry contains many useful modules, not all modules are verified by HashiCorp. Verified modules are indicated by a special badge on the module's page. These modules have been reviewed and approved by HashiCorp to ensure they follow best practices and are of high quality. Other modules in the registry are created and maintained by the community, and while they may be helpful, they have not gone through the same verification process. Always exercise caution and review the module's documentation and source code before using it in your infrastructure.

upvoted 1 times

rotimislaw 1 year, 9 months ago

Selected Answer: B

Seems B to me

upvoted 1 times

Abuu 1 year, 10 months ago

Selected Answer: A

The modules available on the official Terraform Module Registry have been reviewed by HashiCorp to ensure that they follow best practices and are compatible with Terraform. HashiCorp also provides support and guidance for users of the Module Registry.

upvoted 2 times

TechHero 1 year, 10 months ago

B. False

The Terraform Module Registry is a public repository of modules created and shared by the community, it's not maintained by Hashicorp.
These modules have not been verified by HashiCorp, and they may not have undergone any kind of security or quality review.
It's important to review the code and the documentation of the modules and also check if the author is trustworthy before using it in your infrastructure.
Although, Hashicorp provides a way to submit a module to the community and it goes through a review process to be added to the verified module list.
It's also important to check the module's version, release date, and the number of downloads before using it.
It's a good practice to use a module from a verified publisher or a module that is widely used and well-maintained.

It's also worth noting that you can use modules from other sources like GitHub, GitLab, or any other VCS. These modules may have been reviewed and validated by other organizations or communities, but they are not part of the official Terraform module registry.

upvoted 1 times

ssanjayt 1 year, 10 months ago

Selected Answer: A

A is the correct one coz here its saying about the official hasi corp

upvoted 2 times

ArizonaClassics 1 year, 11 months ago

A is the correct answer. Read: verified modules are reviewed by HashiCorp and actively maintained by contributors to stay up-to-date and compatible with Terraform and their respective providers.

The verified badge appears next to modules published by a verified source.
Verified modules are expected to be actively maintained by HashiCorp partners. The verified badge isn't indicative of flexibility or feature support; very simple modules can be verified just because they're great examples of modules. Likewise, an unverified module could be extremely high quality and actively maintained. An unverified module shouldn't be assumed to be poor quality, it only means it hasn't been created by a HashiCorp partner.
https://developer.hashicorp.com/terraform/registry/modules/verified

upvoted 1 times

ArizonaClassics 1 year, 11 months ago

I meant to SAY option B is the correct answer

upvoted 3 times

Bobby1977 2 years, 1 month ago

https://www.terraform.io/registry/modules/verified
A is the right answer

upvoted 2 times

RVivek 2 years, 2 months ago

Terraform public regstry has community modules, which are not verified by Hashicorp

upvoted 2 times

Pinky0289 2 years, 2 months ago

B. Not all modules published are verified by HashiCorp.

upvoted 2 times

Question #120 *Topic 1*

You have to initialize a Terraform backend before it can be configured.

A. True

B. False

Suggested Answer: *B*

Community vote distribution



yaza Highly Voted 2 years, 1 month ago

Selected Answer: B

you cannot initialize something that does not exist, so obviously you have to configure it first

upvoted 27 times

zyxphreez Highly Voted 2 years, 2 months ago

Selected Answer: B

you first configure the backed and then initialize ....therefore should be B

upvoted 10 times

RevSD Most Recent 10 months, 3 weeks ago

In case of local backend , just an init is sufficient and no config is needed. May be question should be more clearer to say if it is local or remote

upvoted 2 times

samimshaikh 11 months ago

Selected Answer: B

B

When you change a backend's configuration, you must run terraform init again to validate and configure the backend before you can perform any plans, applies, or state operations.

After you initialize, Terraform creates a .terraform/ directory locally. This directory contains the most recent backend configuration, including any authentication parameters you provided to the Terraform CLI. Do not check this directory into Git, as it may contain sensitive credentials for your remote backend.

upvoted 2 times

shortcut 1 year ago

Selected Answer: B

I guess, here "initialize" means "instantiate" i.e. the backend must be there For e.g.- a S3 bucket should be present first before we can use it (i.e. configure it in HCL). So, from that perspective, it should be "A". However, we can also have local backends, which is where the terraform configuration file themselves reside, and we don't have to separately instantiate those. Hence, the answer is "B"

upvoted 1 times

aanataliya 1 year, 2 months ago

Selected Answer: A

Configure doesnt mean writing configuration of "backend block" in this context. Hashicorp always use "block" if they want to mention writing configuration. example : backend block, terraform block or cloud block. If I say, I want to configure linux server doesnt mean writing HCL for linux server. Just writing "backend block" doesnt actually configure backend. You have to run/execute that HCL "backend block" to actually create backend configuration.

During actual backend configuration, Terraform writes the backend configuration in plain text in two separate files.
Check Ref: https://developer.hashicorp.com/terraform/language/settings/backends/configuration
And this cannot be done until we first execute init.

upvoted 2 times

3cc17f1 1 year, 1 month ago

That link you provided contradicts your answer:

"When you change a backend's configuration, you must run terraform init again to validate and configure the backend before you can perform any plans, applies, or state operations."

Clearly this suggests that configuration occurs PRIOR to initialisation.
upvoted 3 times

arunrkaushik 1 year, 3 months ago
Correct is B:
How sleepy are people who voted for an option A... like compiling the code before writing it.
upvoted 2 times

milan92stankovic 1 year, 5 months ago
Selected Answer: B
I vote for B
upvoted 2 times

Priyankakanna 1 year, 6 months ago
What I understand is who choose false confusing with terminology its config nor writting configuration ie before creating the resource ie apply we need to initialise the backend so we have to run init so that from the configuration files it will download the necessary plugins for the backend and then apply the configuration to configure the resources
upvoted 3 times

piezaS 1 year, 6 months ago
A. True

Before configuring and using a Terraform backend, you need to initialize it first. The initialization process sets up the backend configuration, such as the type of backend (e.g., S3, Azure Storage, etc.) and the connection details. This is done by running the terraform init command, which initializes the working directory and prepares it for Terraform operations, including the configuration of the backend.
upvoted 3 times

sdm13168 1 year, 6 months ago
Selected Answer: B
B you need to configure the backend first, then run terraform init
upvoted 1 times

kiran15789 1 year, 6 months ago
Selected Answer: A
Before configuring a backend in Terraform, it is necessary to initialize it. This is done by running the terraform init command, which sets up the working directory for the configuration, downloads any necessary plugins, and initializes the backend. Once the backend is initialized, you can configure it by editing the backend block in the Terraform configuration file.
upvoted 1 times

Ni33 1 year, 6 months ago
Selected Answer: B
B is the correct answer !
upvoted 1 times

Stanislav4907 1 year, 7 months ago
Selected Answer: A
Yes, before configuring a Terraform backend, you need to initialize it. Initializing a backend sets up the necessary resources and configurations to use a particular backend, such as creating storage containers or initializing remote state.

To initialize a backend, you can use the terraform init command. This command reads the configuration files in your working directory and installs any necessary plugins or modules, as well as initializes the backend you have specified in your configuration.

After initializing the backend, you can configure it by adding the appropriate configuration to your Terraform configuration files. The backend configuration typically includes details such as the backend type, the backend's credentials or access details, and any additional settings specific to the backend you're using.

Once you have initialized and configured your backend, you can then use Terraform to create and manage infrastructure resources in the backend.
upvoted 2 times

Nunyabiznes 1 year, 8 months ago
Selected Answer: B

B. False

The backend configuration is specified within the Terraform configuration files, typically in the main.tf file. You don't have to initialize a backend before configuring it; instead, you provide the backend configuration within the Terraform files, and then run terraform init. The initialization process will set up the backend as per the configuration provided. So, the backend configuration is done first, and then you initialize Terraform, which sets up the backend accordingly.

upvoted 4 times

sreya23 1 year, 8 months ago

Selected Answer: B

Without configuration (not writing code) you can't initialize it.

upvoted 1 times

khaled_razouk 1 year, 8 months ago

Selected Answer: B

you can't do that if it doesn't exist

upvoted 1 times

Question #121 *Topic 1*

Which of the following does terraform apply change after you approve the execution plan? (Choose two.)

A. Cloud infrastructure

B. The .terraform directory

C. The execution plan

D. State file

E. Terraform code

Suggested Answer: *C*

Community vote distribution

A (67%) D (33%)

faltu1985 Highly Voted 2 years, 2 months ago

Selected Answer: A

A and D

upvoted 33 times

camps Highly Voted 1 year, 7 months ago

After you approve the execution plan, terraform apply will make changes to both the cloud infrastructure and the state file. Therefore, options A and D are correct. The .terraform directory is a local directory used by Terraform to store plugins and other files, and is not modified by terraform apply. The execution plan is generated by Terraform to show you the changes that will be made, and is not modified by terraform apply. The Terraform code is also not modified by terraform apply; it simply tells Terraform what changes to make to the infrastructure.

upvoted 11 times

f2e2419 Most Recent 10 months, 1 week ago

Selected Answer: D

A and D

upvoted 4 times

[Removed] 11 months, 2 weeks ago

Selected Answer: D

A and D

upvoted 3 times

awsexams 11 months, 2 weeks ago

A and D

upvoted 3 times

Selva247 1 year ago

A. Cloud infrastructure: Terraform applies the changes to your cloud infrastructure based on the configuration described in your Terraform code.

D. State file: Terraform applies the changes and updates the state file to reflect the current state of the infrastructure.

upvoted 5 times

Pikopo 1 year, 1 month ago

A and D

upvoted 2 times

VSMu 1 year, 4 months ago

Selected Answer: A

A and D.

upvoted 1 times

abhi6199 1 year, 5 months ago

A and D are the correct options.

upvoted 1 times

March2023 1 year, 5 months ago

Selected Answer: A

A and D for sure.

upvoted 1 times

KJ_Rollings 1 year, 6 months ago

Selected Answer: D

A and D

upvoted 2 times

Ni33 1 year, 6 months ago

A is the correct answer.

upvoted 1 times

wojtec 1 year, 6 months ago

A and D

upvoted 2 times

Ravi528 1 year, 8 months ago

Selected Answer: A

Cloud infra will be updated/changed not the execution plan.

It will update the content/values in state file based on latest changes happened as a part of apply.

I will go with A

upvoted 1 times

SilentMilli 1 year, 8 months ago

Selected Answer: A

When you run terraform apply and approve the execution plan, Terraform will make changes to the cloud infrastructure based on the desired state defined in the Terraform code. Terraform will also update the state file to reflect the new state of the infrastructure after the changes have been made. Therefore, options A and D are correct.

upvoted 1 times

rotimislaw 1 year, 9 months ago

Selected Answer: D

A and D

upvoted 3 times

mpetbteam 1 year, 9 months ago

I think the question is bad formulated, not all providers are cloud-based

upvoted 2 times

Question #122 *Topic 1*

A Terraform backend determines how Terraform loads state and stores updates when you execute _____.

A. apply

B. taint

C. destroy

D. All of the above

E. None of the above

Suggested Answer: *E*

Community vote distribution

D (100%)

amanp Highly Voted 1 year, 8 months ago

Selected Answer: D

D is right

upvoted 7 times

imkhan Highly Voted 7 months, 1 week ago

D. All of the above

Explanation: A Terraform backend is responsible for managing the state of your infrastructure and how Terraform loads and stores updates for various operations, including apply, taint, and destroy, as well as other commands like plan, import, and more. The backend defines how and where the Terraform state is stored and retrieved, and it's used in various Terraform operations to ensure consistency and manage the state of your infrastructure.

upvoted 6 times

Pikopo Most Recent 7 months, 3 weeks ago

D is right

upvoted 1 times

Foram31 11 months, 2 weeks ago

Selected Answer: D

When you execute commands like terraform apply, terraform taint, or terraform destroy, Terraform needs to access and update the state file to perform the desired actions. The backend determines how Terraform interacts with the state file during these operations.

upvoted 3 times

Ni33 1 year ago

Selected Answer: D

D is correct

upvoted 1 times

BennaniHaythem 1 year, 1 month ago

Select Answer :A apply.

The taint and destroy commands do not involve loading or storing state, but can interact with state data depending on how they are used.

upvoted 2 times

Halimb 8 months, 3 weeks ago

Upvoted by mistake; this is wrong. Correct answer is C.

upvoted 1 times

Pinky0289 1 year, 8 months ago

D is the answer

upvoted 1 times

harshit101 1 year, 8 months ago

Selected Answer: D

D is correct

upvoted 1 times

bora4motion 1 year, 8 months ago

Selected Answer: D

D looks OK

upvoted 3 times

dc_98 1 year, 8 months ago

Selected Answer: D

Thats when state is used so the backend should be looked at

upvoted 4 times

Question #123 *Topic 1*

What does Terraform use .terraform.lock.hcl file for?

A. Tracking provider dependencies

B. There is no such file

C. Preventing Terraform runs from occurring

D. Storing references to workspaces which are locked

Suggested Answer: *D*

Community vote distribution

A (96%) 4%

[Removed] Highly Voted 1 year, 8 months ago

Selected Answer: A

A is correct

https://www.terraform.io/language/files/dependency-lock

upvoted 9 times

Nunyabiznes Highly Voted 1 year, 2 months ago

Selected Answer: A

A. Tracking provider dependencies

The .terraform.lock.hcl file is used to track provider dependencies and their exact versions. This file is automatically generated by Terraform when you run terraform init. It locks the versions of the providers used in your configuration, ensuring that subsequent Terraform runs use the same provider versions for consistency and reproducibility across environments. This file should be committed to your version control system to maintain consistency across team members and environments.

upvoted 9 times

Pikopo Most Recent 7 months, 3 weeks ago

A is correct

upvoted 1 times

Ni33 1 year ago

Selected Answer: A

A is the correct answer !

upvoted 1 times

Livvie 1 year, 6 months ago

Selected Answer: A

The right answer is A.

Based on this reference: https://developer.hashicorp.com/terraform/language/files/dependency-lock

upvoted 1 times

Anderson01 1 year, 7 months ago

"Terraform has made some changes to the "provider dependency" selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make."

-> I will go with A

upvoted 1 times

Atta33 1 year, 7 months ago

Selected Answer: A

A is correct

upvoted 1 times

shopkitty 1 year, 8 months ago

Selected Answer: D

I voted for D, this file for the purpose that now allow other user to perform init again the from different machine (workspace) if you work as a team.

upvoted 1 times

bora4motion 1 year, 8 months ago

Selected Answer: A

so if you google the first thing that shows up is A

upvoted 4 times

Burakko 1 year, 8 months ago

Selected Answer: A

I guess it is A.

"hcl , and this name is intended to signify that it is a lock file for various items that Terraform caches in the . terraform subdirectory of your working directory. Terraform automatically creates or updates the dependency lock file each time you run the terraform init command."

upvoted 2 times

Question #124 *Topic 1*

You've used Terraform to deploy a virtual machine and a database. You want to replace this virtual machine instance with an identical one without affecting the database. What is the best way to achieve this using Terraform?

A. Use the terraform state rm command to remove the VM from state file

B. Use the terraform taint command targeting the VMs then run terraform plan and terraform apply

C. Use the terraform apply command targeting the VM resources only

D. Delete the Terraform VM resources from your Terraform code then run terraform plan and terraform apply

Suggested Answer: *B*

Community vote distribution

B (100%)

KJ_Rollings Highly Voted 1 year ago

Selected Answer: B

B. for v0.15.2 and later, you can also use "terraform apply -replace=ADDRESS" to achieve the same result.

upvoted 8 times

keiffo2 Highly Voted 1 year, 8 months ago

terraform apply -target - just looks at applying part of the config.

In this case we'd need to taint the resource and then run an apply. Just to note taint is now deprecated - seems like it comes up in the exam mind, instead of using taint you should use terrform apply -replace=aws_instance.my_server

upvoted 5 times

Spandrop 6 months, 2 weeks ago

being taint deprecated, I would go with C.

upvoted 3 times

keiffo2 1 year, 8 months ago

So answer B

upvoted 2 times

alirasouli 1 year, 5 months ago

Exactly I was thinking about `terraform apply -replace=...`, so answer C can also be correct. Of course answer B is also correct and deprecated!

upvoted 2 times

fad3r 4 months ago

Under rated comment and 10/10 for the funnies

upvoted 1 times

junk4share Most Recent 10 months, 2 weeks ago

Selected Answer: B

ITS B.

upvoted 1 times

adouban 1 year, 5 months ago

B correct

upvoted 1 times

depal_dhir 1 year, 8 months ago

Selected Answer: B

https://www.terraform.io/cli/state/taint

upvoted 2 times

Question #125 *Topic 1*

How do you specify a module's version when publishing it to the public Terraform Module Registry?

A. The module's configuration page on the Terraform Module Registry

B. Terraform Module Registry does not support versioning modules

C. The release tags in the associated repo

D. The module's Terraform code

Suggested Answer: *C*

Community vote distribution

C (100%)

Burakko Highly Voted 1 year, 8 months ago

Selected Answer: C

The registry uses tags to identify module versions. Release tag names must be a semantic version, which can optionally be prefixed with a v . For example, v1. 0.4 and 0.9.

upvoted 10 times

Tyler2023 Most Recent 6 months, 3 weeks ago

answer is C:

Once a module is published, you can release a new version of a module by simply pushing a properly formed Git tag.

x.y.z tags for releases. The registry uses tags to identify module versions. Release tag names must be a semantic version, which can optionally be prefixed with a v. For example, v1.0.4 and 0.9.2. To publish a module initially, at least one release tag must be present. Tags that don't look like version numbers are ignored.

https://developer.hashicorp.com/terraform/registry/modules/publish

upvoted 3 times

Ni33 1 year ago

Selected Answer: C

C is the correct answer.

upvoted 1 times

SilentMilli 1 year, 2 months ago

Selected Answer: C

C. The release tags in the associated repo. When publishing a module to the Terraform Module Registry, the module version is typically specified using Git tags in the associated Git repository. Terraform retrieves the module from the registry using the provider's source argument, which includes the module's owner, name, and version. By specifying the version in the source argument of your module, Terraform will download that specific version of the module from the registry.

upvoted 1 times

Pinky0289 1 year, 8 months ago

A module's version is specified using the release tag names. So, the right answer is C.

upvoted 2 times

keiffo2 1 year, 8 months ago

x.y.z tags for releases. The registry uses tags to identify module versions. Release tag names must be a semantic version, which can optionally be prefixed with a v. For example, v1.0.4 and 0.9.2. To publish a module initially, at least one release tag must be present. Tags that don't look like version numbers are ignored.

ANSWER C

upvoted 3 times

dinesh198728 1 year, 8 months ago

Selected Answer: C

https://www.terraform.io/registry/modules/publish

upvoted 4 times

bora4motion 1 year, 8 months ago

Selected Answer: C

C makes sense to me

upvoted 2 times

Question #126 *Topic 1*

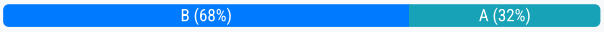
Terraform plan updates your state file.

A. True

B. False

Suggested Answer: *A*

Community vote distribution



Nunyabiznes Highly Voted 1 year, 8 months ago

Selected Answer: B

B. False

The terraform plan command does not update your state file. Instead, it generates an execution plan by comparing the desired state (defined in your Terraform configuration files) with the actual state (stored in the state file) of your infrastructure. The plan shows the changes that Terraform will make to the infrastructure without actually applying those changes. The state file is only updated when you run terraform apply, which applies the changes to your infrastructure and updates the state file to reflect the new state.

upvoted 17 times

Halimb 1 year, 2 months ago

A good explanation. If anyone is still doubting this; remember that the state file reflects your actual infrastructure. Since "terraform plan" does not actually change your infrastructure, it would make no sense for it to change your state file. That would make your state file invalid (state drift)!

upvoted 6 times

RVivek Highly Voted 2 years, 2 months ago

Selected Answer: A

Terraform plan always forst check the state of existing resources refreshes/updates that to state.
If there is configuration drift from last apply, then terraform will first refresh the state to respreset the current state of the resources, for example if any resouce is deleted or changed that will be upadted in the state and then the chnages to be doe will be displayed

upvoted 7 times

RonZhong 1 year, 3 months ago

Yes, everytime when I run Terraform Plan, my backend state file is also updated. I can see the last modified timestamp has been updated, even I didn't apply the change.

upvoted 2 times

55e2078 Most Recent 2 months, 2 weeks ago

Selected Answer: B

all of this questions same 1:1 as an exam?

upvoted 1 times

AkaAka4 7 months ago

Selected Answer: A

Refer to https://stackoverflow.com/questions/66240556/terraform-plan-not-updating-state-file

upvoted 1 times

AkaAka4 7 months ago

Check the 2nd last comment in the link, should be B. Apologies.

upvoted 1 times

Ajit18 8 months, 1 week ago

B. False

Terraform plan does not update the state file itself. It performs an in-memory refresh to reconcile the state file with the actual infrastructure, but it doesn't modify the state file on disk. The update to the state file happens during terraform apply if there are changes required.

upvoted 1 times

dizzy_monkey 8 months, 2 weeks ago

Selected Answer: A

"A" should be correct :
"Terraform automatically performs the same refreshing actions as a part of creating a plan in both the terraform plan and terraform apply commands."
https://developer.hashicorp.com/terraform/cli/commands/refresh
upvoted 1 times

91576b8 9 months, 2 weeks ago

Selected Answer: A

when you run terraform plan , it will refresh the state file automaticlly
upvoted 1 times

boapaulo 10 months ago

The terraform plan command does not update the state file. It performs a refresh of the state in-memory to generate an execution plan, but it does not persist these changes to the state file. The state file is updated when you run terraform apply.
upvoted 1 times

vipulchoubisa 10 months, 3 weeks ago

create one simple instance and change the tags from AWS console. After this run terraform plan command. As a result it wont update the state file with new tags. So answer is B
upvoted 1 times

Demonik 11 months, 3 weeks ago

Selected Answer: B

Nope.Plan does not change state file.But others like apply destroy after approve will :)
upvoted 1 times

pamfidelis 1 year, 1 month ago

Selected Answer: A

A. True
"You can also update your state file without making modifications to your infrastructure using the -refresh-only flag for plan and apply operations."
https://developer.hashicorp.com/terraform/tutorials/state/refresh
upvoted 1 times

TigerInTheCloud 11 months, 1 week ago

terraform plan -refresh-only only make the plan of changing state file.
upvoted 1 times

gofavad926 1 year, 1 month ago

Selected Answer: B

B, terraform plan doesn't update the state
upvoted 1 times

nahed 1 year, 2 months ago

Selected Answer: A

terraform state will UPDATE your state file in case you change the resources outside the terraform. IT will "sync" your state file with the actual infrastructure.
upvoted 1 times

kudakk 1 year, 3 months ago

Selected Answer: B

The terraform plan command does not change the state of your infrastructure. It creates an execution plan that lets you preview the changes that Terraform plans to make to your infrastructure
upvoted 1 times

foreverlearner 1 year, 4 months ago

"To determine whether state drift occurred, Terraform performs a refresh operation before it begins to build an execution plan. This refresh step pulls the actual state of all of the resources currently tracked in your state file. --> Terraform does not update your actual state file <--, but captures the refreshed state in the plan file." (https://developer.hashicorp.com/terraform/tutorials/cli/plan)
"You can also update your state file without making modifications to your infrastructure using the -refresh-only flag for plan and apply operations." (https://developer.hashicorp.com/terraform/tutorials/state/refresh)

So plan doesn't update the state file (just an in-memory sync) unless you specify the --refresh-only flag
upvoted 5 times

milan92stankovic 1 year, 5 months ago

Selected Answer: A

terraform state will UPDATE your state file in case you change the resources outside the terraform. IT will "sync" your state file with the actual infrastructure.

upvoted 4 times

Ni33 1 year, 6 months ago

Selected Answer: B

B is the correct answer

upvoted 1 times

Question #127 *Topic 1*

To check if all code in a Terraform configuration with multiple modules is properly formatted without making changes, what command should be run?

A. terraform fmt -check

B. terraform fmt -write-false

C. terraform fmt ı€"list -recursive

D. terraform fmt -check -recursive

Suggested Answer: *C*

Community vote distribution

D (100%)

RVivek Highly Voted 8 months ago

Selected Answer: D

terraform fmt takes followong options
Options:

-list=false Don't list files whose formatting differs
(always disabled if using STDIN)

-write=false Don't write to source files
(always disabled if using STDIN or -check)

-diff Display diffs of formatting changes

-check Check if the input is formatted. Exit status will be 0 if all
input is properly formatted and non-zero otherwise.

-no-color If specified, output won't contain any color.

-recursive Also process files in subdirectories. By default, only the
given directory (or current directory) is processed.

upvoted 12 times

jabbawockie12 Highly Voted 8 months, 3 weeks ago

Selected Answer: D

-check -recursive should check all formatting without modifying the configuration files.

upvoted 10 times

Anderson01 Most Recent 7 months ago

"fmt -> without making changes"
So, I will go with D (make sure)

upvoted 1 times

Pinky0289 8 months, 2 weeks ago

terraform fmt -check -recursive, recursively checks all modules and submodules for the format.

upvoted 2 times

Pinky0289 8 months, 2 weeks ago

The answer is C.

upvoted 1 times

ptR95 8 months, 1 week ago

The answer is D.

upvoted 2 times

zyxphreez 8 months, 3 weeks ago

Selected Answer: D

-check Check if the input is formatted. Exit status will be 0 if all input is properly formatted and non-zero otherwise.
-recursive Also process files in subdirectories. By default, only the given directory (or current directory) is processed.

answer for me is D

upvoted 5 times

Question #128 *Topic 1*

As a member of the operations team, you need to run a script on a virtual machine created by Terraform. Which provision is best to use in your Terraform code?

A. null-exIµ

B. local-exec

C. remote-exec

D. file

Suggested Answer: *B*

Community vote distribution

C (100%)

RVivek Highly Voted 1 year, 8 months ago

Selected Answer: C

remote-exec to run anything on the deployed virtual machine.
local exec is to run anything on the system where terraform is running

upvoted 25 times

bluee 1 year, 4 months ago

thanks mate

upvoted 4 times

Uma10 Highly Voted 1 year, 8 months ago

Selected Answer: C

The remote-exec provisioner invokes a script on a remote resource after it is created.

To invoke a local process, see the local-exec provisioner instead.

Source: https://www.terraform.io/language/resources/provisioners/remote-exec

upvoted 10 times

imkhan Most Recent 7 months, 1 week ago

C- remote-exec is the correct answer
If you need to run a script on a virtual machine created by Terraform, the remote-exec provisioner is the best choice. The remote-exec provisioner allows you to run commands or scripts on a remote machine, which is especially useful for configuring and provisioning resources after they have been created.

upvoted 1 times

Only5 1 year, 3 months ago

Selected Answer: C

Source: https://www.terraform.io/language/resources/provisioners/remote-exec

upvoted 1 times

legendary7 1 year, 8 months ago

c
is the correct answer

upvoted 1 times

keiffo2 1 year, 8 months ago

remote-exec all day

upvoted 3 times

bora4motion 1 year, 8 months ago

Selected Answer: C

VM is not local, I go with remote, C

upvoted 3 times

Burakko 1 year, 8 months ago

Selected Answer: C

On a VM , it is a remote-exec.

upvoted 4 times

Question #129 *Topic 1*

You are using a networking module in your Terraform configuration with the name label my_network. In your main configuration you have the following code:

```
output: "net_id" {
  value = module.my_network.vnet_id
}
```

When you run terraform validate, you get the following error:

```
Error: Reference to undeclared output value

  on main.tf line 12, in output "net_id":
  12:     value = module.my_network.vnet_id
```

What must you do to successfully retrieve this value from your networking module?

A. Define the attribute vnet_id as a variable in the networking module

B. Change the referenced value to module.my_network.outputs.vnet_id

C. Define the attribute vnet_id as an output in the networking module

D. Change the referenced value to my_network.outputs.vnet_id

Suggested Answer: *D*

Community vote distribution

C (91%) 9%

zyxphreez Highly Voted 2 years, 2 months ago

Selected Answer: C

In a parent module, outputs of child modules are available in expressions as module.<MODULE NAME>.<OUTPUT NAME>. For example, if a child module named web_server declared an output named instance_ip_addr, you could access that value as module.web_server.instance_ip_addr.

Answer is C

upvoted 21 times

RVivek Highly Voted 2 years, 2 months ago

Selected Answer: C

Create an out put in child module and resfrence that as module.<MODULE NAME>.<OUTPUT NAME>

upvoted 5 times

Bere Most Recent 10 months, 2 weeks ago

Selected Answer: A

Answer is A

In Terraform versions 0.11 and earlier, you would use the interpolation syntax with ${} to reference attributes:
Terraform 0.11 and earlier
output "instance_ip_addr" {
value = "${aws_instance.main.private_ip}"
}

Starting with Terraform 0.12, interpolation is not needed for simple references, so you can directly reference the attribute:
Terraform 0.12 and later
output "instance_ip_addr" {
value = aws_instance.main.private_ip
}

upvoted 4 times

Bere 10 months, 2 weeks ago

Selected Answer: C

Answer is C

Define the attribute vnet_id as an output in the networking module.

In the Child Module (my_network), if you had a resource defined as aws_vpc and named it example_vpc, you would reference its ID like this:

In your networking module's outputs.tf or main.tf file

output "vnet_id" {
description = "The ID of the created VNet"
value = aws_vpc.example_vpc.id # Replace example_vpc with the actual name of your VPC resource
}

In the Parent Module (main configuration):

output "net_id" {
value = module.my_network.vnet_id
}

upvoted 3 times

Jayanth 1 year, 4 months ago

C is the right answer

upvoted 1 times

Williamus 1 year, 8 months ago

Selected Answer: C

All looks good except that the output in the child module is missing.

upvoted 2 times

Question #130 *Topic 1*

You are writing a child Terraform module which provisions an AWS instance. You want to make use of the IP address returned in the root configuration. You name the instance resource "main".
Which of these is the correct way to define the output value using HCL2?
A.

```
output "instance_ip_addr" {
  value = "${aws_instance.main.private_ip}"
}
```

B.

```
output "instance_ip_addr" {
  return aws_instance.main.private_ip
}
```

Suggested Answer: *A*

keiffo2 Highly Voted 2 years, 2 months ago
Has to be Answer A. no such definition as "return"
upvoted 22 times

Bere Most Recent 10 months, 2 weeks ago
Answer is A

In Terraform versions 0.11 and earlier, you would use the interpolation syntax with ${} to reference attributes:
Terraform 0.11 and earlier
output "instance_ip_addr" {
value = "${aws_instance.main.private_ip}"
}

Starting with Terraform 0.12, interpolation is not needed for simple references, so you can directly reference the attribute:
Terraform 0.12 and later
output "instance_ip_addr" {
value = aws_instance.main.private_ip
}
upvoted 3 times

MauroSoli 1 year, 1 month ago
Answer is A but syntax is deprecated
upvoted 4 times

syam22587 1 year, 3 months ago
A is correct answer
upvoted 1 times

junk4share 1 year, 4 months ago
A is correct.
upvoted 1 times

phidelics 1 year, 8 months ago
The answer is A
upvoted 1 times

kennynelcon 1 year, 9 months ago
A is accurate
upvoted 1 times

Question #131 *Topic 1*

How can a ticket-based system slow down infrastructure provisioning and limit the ability to scale? (Choose two.)

A. A full audit trail of the request and fulfillment process is generated

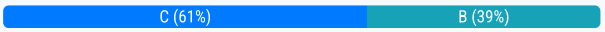
B. A request must be submitted for infrastructure changes

C. As additional resources are required, more tickets are submitted

D. A catalog of approved resources can be accessed from drop down lists in a request form

Suggested Answer: *B*

Community vote distribution



Darshan07 9 months ago

Selected Answer: C

Both B & C

upvoted 3 times

NashP 9 months, 4 weeks ago

B. A request must be submitted for infrastructure changes

C. As additional resources are required, more tickets are submitted

Explanation:

B. A request must be submitted for infrastructure changes: In a ticket-based system, users often need to raise a request or a ticket for any infrastructure changes. This process can introduce delays, especially when changes are needed urgently or frequently. It can slow down the provisioning process as it adds an additional layer of approval and manual intervention.

C. As additional resources are required, more tickets are submitted: In a ticket-based system, when scaling is required, users may need to submit additional tickets for each resource or instance they need. This manual process can become a bottleneck as the number of tickets increases, making it less efficient for scaling rapidly or handling dynamic workloads.

upvoted 4 times

[Removed] 11 months, 1 week ago

Selected Answer: C

B and C

upvoted 2 times

[Removed] 11 months, 2 weeks ago

Selected Answer: C

B and C. increase amount of C

upvoted 1 times

kennynelcon 1 year, 9 months ago

Selected Answer: C

B and C

lets populate C

upvoted 3 times

Edileimig 1 year, 10 months ago

Selected Answer: C

B and C

upvoted 1 times

robertninho 1 year, 10 months ago

B and C

upvoted 1 times

DawidB 1 year, 11 months ago

Selected Answer: C

B and C indeed.

upvoted 1 times

legendary7 2 years, 1 month ago

B and C are correct

upvoted 3 times

dani88ge 2 years, 2 months ago

Selected Answer: C

B and C

upvoted 3 times

Pinky0289 2 years, 2 months ago

B and C

upvoted 2 times

dinesh198728 2 years, 2 months ago

Selected Answer: B

B and C both ticket based

upvoted 3 times

kprod 2 years, 2 months ago

Selected Answer: B

Answer is BC

upvoted 4 times

Question #132 *Topic 1*

Which of the following statements about Terraform modules is not true?

A. Modules must be publicly accessible

B. Modules can be called multiple times

C. Module is a container for one or more resources

D. Modules can call other modules

Suggested Answer: *C*

Community vote distribution

A (100%)

Uma10 Highly Voted 2 years, 2 months ago

Selected Answer: A

A: In addition to modules from the local filesystem, Terraform can load modules from a public or private registry. Also, members of your organization might produce modules specifically crafted for your own infrastructure needs.

Source: https://www.terraform.io/language/modules

upvoted 16 times

ravi135 Most Recent 9 months, 3 weeks ago

A is right

upvoted 1 times

Daro_ 1 year, 10 months ago

Selected Answer: A

A is correct

upvoted 1 times

ruganesh 1 year, 10 months ago

A is the correct one.

upvoted 1 times

adouban 1 year, 11 months ago

Selected Answer: A

A is the correct

upvoted 1 times

RVivek 2 years, 2 months ago

Selected Answer: A

Modules can be from local disk , private registry in your company private network

upvoted 2 times

Pinky0289 2 years, 2 months ago

Besides being publicly accessible, modules can also be privately accessible only within the organization. So, the answer is option A.

upvoted 4 times

Burakko 2 years, 2 months ago

Selected Answer: A

It can be private modules too...

upvoted 4 times

Question #133 *Topic 1*

Which Terraform collection type should you use to store key/value pairs?

A. tuple

B. set

C. ma€

D. list

Suggested Answer: *C*

Community vote distribution

C (100%)

Uma10 Highly Voted 1 year, 8 months ago

The answer is correct.

Maps/objects are represented by a pair of curly braces containing a series of <KEY> = <VALUE> pairs

Source: https://www.terraform.io/language/expressions/types

upvoted 9 times

debabrata6983 Most Recent 9 months ago

Selected Answer: C

map collection

upvoted 1 times

camps 1 year, 1 month ago

Selected Answer: C

C. map is the Terraform collection type that should be used to store key/value pairs.

A map is an unordered collection of key/value pairs, where each key is unique. It can be used to store related data that doesn't have a specific order, and it's useful for storing configuration parameters or metadata associated with infrastructure resources. A tuple is an ordered collection of elements, a set is an unordered collection of unique elements, and a list is an ordered collection of elements that can include duplicates. While each of these collection types has its own uses, a map is the most appropriate type for storing key/value pairs.

upvoted 4 times

Pinky0289 1 year, 8 months ago

Elements stored in key=value pairs within curly braces is a map. So, the right option is C.

upvoted 1 times

depal_dhir 1 year, 8 months ago

Selected Answer: C

C - Map is the correct answer

https://www.terraform.io/language/expressions/types

upvoted 1 times

keiffo2 1 year, 8 months ago

map variable is key-value

upvoted 1 times

keiffo2 1 year, 8 months ago

so C is the right anwer

upvoted 1 times

Question #134 *Topic 1*

You have used Terraform to create an ephemeral development environment in the cloud and are now ready to destroy all the infrastructure described by your
Terraform configuration. To be safe, you would like to first see all the infrastructure that will be deleted by Terraform.
Which command should you use to show all of the resources that will be deleted? (Choose two.)

A. Run terraform plan -destroy

B. Run terraform show -destroy

C. Run terraform destroy and it will first output all the resources that will be deleted before prompting for approval

D. Run terraform show -destroy

Suggested Answer: *AD*

Community vote distribution

AC (100%)

bora4motion Highly Voted 1 year, 8 months ago

Selected Answer: AC

I go with AC

upvoted 9 times

imkhan Highly Voted 7 months, 1 week ago

AC are the correct options

When you want to see a preview of the resources that will be deleted by Terraform, you can use the terraform plan -destroy command to generate a destruction plan that shows what will be removed. This is a safe way to ensure you understand the impact of the destroy operation before proceeding.

Option C is also correct because when you run terraform destroy, Terraform will provide an execution plan that lists the resources to be deleted before it actually prompts you for approval to proceed with the destruction. This allows you to review the list of resources that will be affected before confirming the destroy operation.

upvoted 6 times

gofavad926 Most Recent 8 months ago

Selected Answer: AC

AC for sure

upvoted 1 times

Bilalglg93350 1 year, 2 months ago

A. Run terraform plan -destroy

D. Run terraform show -destroy

To see all the resources that will be deleted by Terraform, you should run the terraform plan -destroy command. This command will show a preview of all the changes that Terraform will make, including any resources that will be deleted.

After running the terraform plan -destroy command, you can also run the terraform show -destroy command to view a detailed summary of the resources that will be deleted.

Option C (Run terraform destroy and it will first output all the resources that will be deleted before prompting for approval) is not correct. Running the terraform destroy command will immediately destroy all the resources without showing a preview of what will be deleted. To be safe, it is recommended to run the terraform plan -destroy command first to see the resources that will be deleted, and then run terraform destroy to confirm the deletion.

upvoted 4 times

Halimb 8 months, 3 weeks ago

Nosense. There is no terraform show -destroy. Stop confusing people. Answer is A&C.

upvoted 4 times

Roytf 7 months, 2 weeks ago

don't paste chatGpt output..option D is invalid

upvoted 3 times

MrTee 10 months, 2 weeks ago

Terraform show -destroy is not a valid command

upvoted 4 times

VamsiPopuri 1 year, 3 months ago

Selected Answer: AC

A and C.

Aren't B and D the same ?

upvoted 4 times

rafpe 1 year, 5 months ago

Selected Answer: AC

Def A&C

upvoted 1 times

Atta33 1 year, 7 months ago

Selected Answer: AC

AC is correct

upvoted 3 times

RVivek 1 year, 8 months ago

Selected Answer: AC

A and C

upvoted 1 times

shopkitty 1 year, 8 months ago

Selected Answer: AC

vote for A & C

upvoted 1 times

Burakko 1 year, 8 months ago

Selected Answer: AC

It is AC. terraform show only shows the state file

upvoted 4 times

dc_98 1 year, 8 months ago

Selected Answer: AC

There is no terraform show -destroy

upvoted 3 times

Question #135 *Topic 1*

When do you need to explicitly execute terraform refresh?

A. Before every terraform plan

B. Before every terraform apply

C. Before every terraform import

D. None of the above

Suggested Answer: *D*

Community vote distribution

D (100%)

Uma10 Highly Voted 2 years, 2 months ago

The answer is correct.

Wherever possible, avoid using terraform refresh explicitly and instead rely on Terraform's behavior of automatically refreshing existing objects as part of creating a normal plan.

Source: https://www.terraform.io/cli/commands/refresh

upvoted 13 times

keiffo2 2 years, 2 months ago

so the answer is D

upvoted 2 times

Darshan07 Most Recent 9 months ago

Selected Answer: D

D is correct

upvoted 1 times

hajurbau 1 year, 2 months ago

Selected Answer: D

D is correct

upvoted 1 times

RVivek 2 years, 2 months ago

Selected Answer: D

As per Terraform documentation refresh command is not recommended, it is there only for backward compatibility

upvoted 2 times

Pinky0289 2 years, 2 months ago

D. None of the above.

Refresh is used to check backward compatibility to read the infrastructure objects, and update the state file. Terraform performs refresh by itself during every plan and apply commands. So, it is recommended that we not perform refresh often.

upvoted 3 times

depal_dhir 2 years, 2 months ago

Selected Answer: D

https://www.terraform.io/cli/commands/refresh

upvoted 1 times